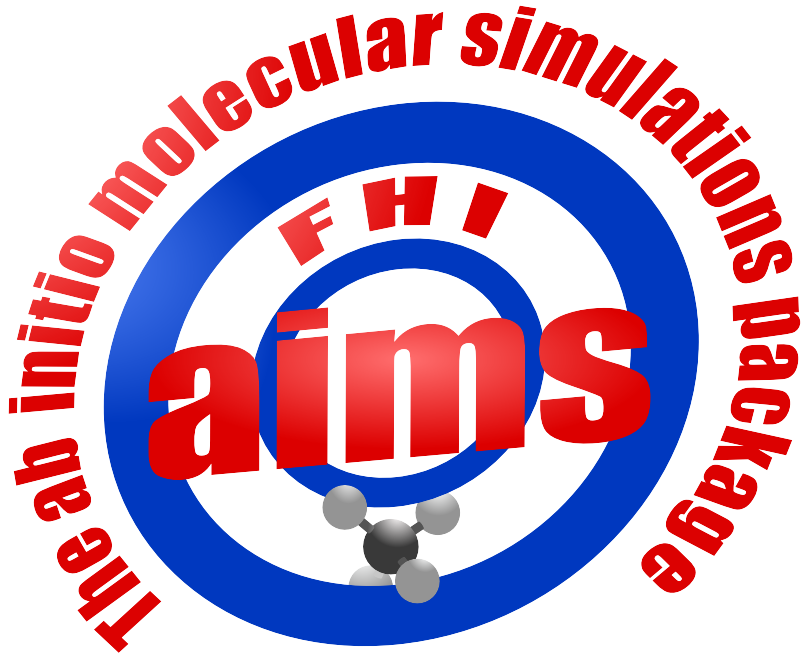
Fritz Haber Institute *ab initio* molecular simulations: FHI-aims



All-Electron Electronic Structure Theory
with Numeric Atom-Centered Basis Functions

A Users' Guide

FHI-aims team
Fritz-Haber-Institut der Max-Planck-Gesellschaft, Berlin
and many contributors around the world.
May 22, 2016

Contents

How to use this manual 7

Introduction 8

1 Getting started with FHI-aims 10

1.1 First step: Installation . . . 10
1.2 Building the code . . . 11
1.3 Running FHI-aims . . . 13
1.4 Shortcut: make.sys . . . 16
1.5 Cross-Compiling with a C Compiler . . . 17
1.6 Compilation options beyond the standard Makefile . . . 17
1.6.1 Alternative build process with the Waf build system . . . 19
1.7 Compiling faster versions of FHI-aims on specific platforms . . . 21

2 Input Files: Basic Handling 22

2.1 The mandatory input files: `control.in` and `geometry.in` . . . 23
2.2 Defaults for chemical elements: species_defaults . . . 26
2.3 A very quick guide to ensuring numerical convergence with FHI-aims . . . 29
2.3.1 Basis set . . . 29
2.3.2 Hartree potential . . . 31
2.3.3 Integration grid . . . 32
2.4 Why does my calculation take too long? . . . 35
2.5 Stopping a run: Files `abort_scf` and `abort_opt` . . . 38

3 The Full Monty: All Keywords and Capabilities 39

3.1 Usability (convenience) . . . 40

3.2 Physical model: Geometry, charge, spin, etc. . . . 43
3.3 Electronic structure: Exchange, correlation (incl. DFT+U), and excited states . . . 48
3.4 Specifying the basis (functions, empty sites, k-points, ...) . . . 63
3.5 Integration, grids, and partitioning . . . 76
3.6 Electron density update . . . 87
3.7 Electrostatic (Hartree) potential . . . 89
3.7.1 Non-periodic Ewald method . . . 90
3.8 Kinetic energy, scalar relativity, and spin-orbit coupling . . . 100
3.9 Eigenvalue solver and (fractional) occupation numbers . . . 108
3.10 SCF Cycle: Initialization, density mixing, preconditioning, convergence . . . 117
3.11 Energy derivatives (forces, stress) and geometry optimization . . . 135
3.12 Molecular dynamics . . . 150
3.13 Thermodynamic Integration . . . 160
3.14 Electronic constraints . . . 164
3.15 Embedding in external fields . . . 171
3.16 QM/MM Embedding . . . 175
3.17 C_6/R^6 corrections for long-range van der Waals interactions . . . 179
3.18 Many-Body Dispersion (MBD) energy . . . 183
3.19 Calculating nonlocal correlation energy within density functional approach 187
3.19.1 Monte Carlo integration based vdW-DF . . . 187
3.19.2 Analytic integration scheme for non-selfconsistent and self-consistent vdW-DF . . . 192
3.20 Hartree-Fock, hybrid functionals, GW, *et al.*: All the details . . . 194
3.21 Hartree-Fock and hybrid functionals, including periodic systems . . . 208
3.22 TDDFT - linear response . . . 212
3.23 MPE Implicit Solvent Model . . . 216
3.24 Linear macroscopic dielectric function and Kubo-Greenwood transport . . . 218
3.25 Electronic Transport . . . 224
3.26 ESP charges . . . 228
3.27 Large-scale, massively parallel: Memory use, sparsity, communication, etc. 236
3.28 Fragment molecular orbital DFT calculations . . . 243
3.29 Output options . . . 252

3.30 Deprecated keywords . . . 276

4 Running FHI-aims: Guides to specific tasks 283

4.1 Ground state DFT: Total energies and relaxation . . . 284
4.2 Heavy elements ($Z \gtrsim 30$): Modifications for scalar relativity . . . 290
4.3 k-point sampling in the Brillouin zone for semiconductors . . . 293
4.4 Plotting the band structure and density of states of a solid . . . 297
4.5 Visualizing charge densities and orbitals . . . 300
4.6 Calculation of vibrational and phonon frequencies . . . 303
4.7 Restarting FHI-aims calculations . . . 315
4.7.1 General restart procedure . . . 315
4.7.2 Mixing variants - the "force_single_restartfile" option . . . 316
4.7.3 Rotating the FHI-aims wavefunction . . . 316
4.8 Finding Transition States: the aimsChain . . . 320
4.8.1 Installation . . . 320
4.8.2 A Quick Start . . . 321
4.8.3 Configuration . . . 323
4.8.4 Preparation before running . . . 333
4.8.5 Running the script . . . 337
4.8.6 Tips & Guides On Running . . . 339
4.9 Transition state search: Nudged Elastic Band method . . . 341
4.9.1 Theory and methods . . . 341
4.9.2 Usage . . . 341
4.10 Plugin for free-energy calculations with molecular dynamics: PLUMED . . . 346
4.10.1 Usage . . . 346
4.11 Script based parallel tempering (a.k.a. replica exchange) . . . 348
4.11.1 Usage . . . 348
4.11.2 Output . . . 350
4.12 Formation energies of charged defects . . . 352

5 The AITRANSS package 354

5.1 Source code and supporting materials . . . 355
5.2 Compiling the AITRANSS module . . . 355

5.3 How to set-up and run transport calculations . . . 356
5.3.1 FHI-aims run: input and output . . . 356
5.3.2 What to be aware of before running AITRANSS module . . . 356
5.3.3 How to create a mandatory file `tcontrol` . . . 358
5.3.4 How to submit a transport calculation and its output . . . 360
5.3.5 Further option: local density of states . . . 361
5.4 Keywords of file `tcontrol` . . . 361

A Trouble-shooting 368
A.1 Format flags required by some compilers . . . 368
A.2 FHI-aims aborts with a segfault at the beginning of the first test run. . . . 369
A.3 Use of FHI-aims with multithreaded BLAS (e.g., Intel's mkl) . . . 370
A.4 Parallel runs across different file systems . . . 370
A.5 I'm running a calculation with a large system, and it is abruptly exiting. What's going on? . . . 371
A.6 What do I do if I run out of memory? . . . 371
A.7 Nearly singular basis sets: Strange results from small-unit-cell periodic calculation with many k-points . . . 371
A.8 No convergence of the s.c.f. cycle even after many iterations . . . 372

B Structure of the code 375
B.1 Flow of the program . . . 375
B.2 Commenting and style requests . . . 379

C Memory Tracing 380

D Debug Manager 381

E XML output 382

F Optional Libraries to be Linked into FHI-aims 383
F.1 Adding Optional Libraries into FHI-aims: Stubs . . . 383
F.2 Spglib . . . 384

G Multiple Instances of Aims 385

Bibliography **386**

Index **386**

How to use this manual

If you are reading this introduction, you are likely reading the manual for the first time. In that case, please read on. There is, however, a strategy to use this manual most effectively to find keywords used in the input files to FHI-aims. This is it:

- Open the manual (pdf)
- Go to the table of contents
- At the bottom of the table of contents, click on "Index"
- Find the keyword you are looking for in the index
- Click on it.

Using the manual in this way may greatly reduce the barrier to look up what a keyword actually does. And now, for the actual ...

Introduction

FHI-aims ("Fritz Haber Institute *ab initio* molecular simulations") is a computer program package for computational materials science based only on quantum-mechanical first principles. The main production method is density functional theory (DFT) [?, ?, ?] to compute the total energy and derived quantities of molecular or solid condensed matter in its electronic ground state. In addition, FHI-aims allows to describe electronic single-quasiparticle excitations in molecules using different self-energy formalisms (GW and MP2), and wave-function based molecular total energy calculation based on Hartree-Fock and many-body perturbation theory (MP2, RPA, SOSEX, or the more encompassing renormalized second-order perturbation theory, RPT2).

The basic physical algorithms in FHI-aims concerning ground state DFT and applications are described in

> Volker Blum, Ralf Gehrke, Felix Hanke, Paula Havu, Ville Havu, Xinguo Ren, Karsten Reuter, and Matthias Scheffler, Computer Physics Communications **180**, 2175-2196 (2009).

A copy of this paper can also be obtained from our web site:
http://www.fhi-berlin.mpg.de/aims/ .
Please cite this reference if you use FHI-aims.

When making use of / reference to scalability, please refer to and cite

> Ville Havu, Volker Blum, Paula Havu, and Matthias Scheffler, Journal of Computational Physics **228**, 8367-8379 (2009).

and also to the large-scale eigenvalue solver ELPA:

> A. Marek, V. Blum, R. Johanni, V. Havu, B. Lang, T. Auckenthaler, A. Heinecke, H.-J. Bungartz, and H. Lederer, The Journal of Physics: Condensed Matter **26**, 213201 (2014).

Any application making use of functionality beyond LDA, GGA, or mGGA – i.e., Hartree-Fock, hybrid functionals, MP2, RPA, GW, etc. – should please refer to and cite

> Xinguo Ren, Patrick Rinke, Volker Blum, Jürgen Wieferink, Alex Tkatchenko, Andrea Sanfilippo, Karsten Reuter, and Matthias Scheffler, New Journal of Physics 14, 053020 (2012).

There are several further methodological publications for specific methods in FHI-aims. Please take the time to go to our publications page:

https://aimsclub.fhi-berlin.mpg.de/aims_publications.php

and make sure that you give credit to those who worked to make this code a reality. As of version "072414", references for the implementation parts used in a particular run are also given at the end of the FHI-aims standard output. FHI-aims is a scientific code, written by and for scientists, whose primary recognition for their work is credit in the form of appropriate reference to their work.

In the present documentation, we do not repeat the basic physical algorithms; rather, the focus is on the actual *use* of the methods in FHI-aims for a given task, including a full description of all input and output possibilities.

The rest of this document is organized as follows:

- In Chapter 1, a "quickstart" description attempts to give you all the necessary (but not more) information to get FHI-aims up and running on your own computer system, up to the first test run.
- Chapter 2 explains the basic input files and input philosophy very briefly. Some important remarks on choosing the numerical accuracy are summarized here.
- Chapter 3 gets into the gory details, summarizing *all* available input keywords and their meaning, sorted roughly by their expected use.
- A large chapter 4 is dedicated to some frequently required "meta-tasks" of electronic structure theory: Not just setting up a specific set of input files for a given run, but actually extracting some of the frequently required information from those runs. For the more complex tasks (e.g., a transition state search), we attempt to provide scripts that perform a series of well-defined runs automatically, the use of an external visualization tool, etc.
- In chapter 5 we provide a description of the AITRANSS (*ab initio* transport simulations) package which is a project under continuous development at the Institute of Nanotechnology of the Karlsruhe Institute of Technology (KIT), Germany, since 2002. When combined with FHI-aims, AITRANSS provides a post-processor module that enables calculation of the electron transport characteristics of molecular junctions based on a Landauer formalism in a Green's function formulation.
- In the appendices, we suggest further reading, and also address frequently encountered issues ("troubleshooting") that are either beyond our control (operating-system related issues come to mind), or simply require some level of experience to address.

In any case, we hope that this layout will be helpful for your specific purposes. We welcome feedback, in particular regarding issues from production settings that we might not yet have thought of / experienced ourselves. In any event: Happy computing with FHI-aims!

Chapter 1

Getting started with FHI-aims

1.1 First step: Installation

FHI-aims comes as a gzipped tar archive that can be extracted in any directory of your choice, e.g., by typing

```
gzip -d fhi-aims.tar.gz
tar -xvf fhi-aims.tar
```

at the command line of any current Unix-like system.

> Before you ask: FHI-aims is designed to run on any current Unix-based or Unix-like system, such as Linux, IBM's aix, Sun's Solaris, or even Mac OS X. However, we do *not* support FHI-aims on Windows at this point. It is certainly possible to make it run on a Windows platform using the appropriate tools, but not simply out-of-the-box.

The full package then extracts itself into a directory *./fhi-aims*, with the following subdirectories:

- *bin/* : Location for any FHI-aims binaries built using the standard Makefile
- *doc/* : Contains possible further documentation.
- *species_defaults/* : Grids, basis sets and other defaults for chemical elements 1-102. These can be copy-pasted as "species" into the FHI-aims input file `control.in`. FHI-aims provides three levels of standard species defaults: "light", "tight", and "really_tight" (see Sec. 2.2). In addition, some further preconstructed special-purpose species defaults are provided in a "non-standard" subdirectory.
- *src/* : This directory, and its subdirectories, contain all FHI-aims source code files, as well as the Makefile needed to build this code.

- *testcases/* : Simple examples to test and illustrate the basic functioning of the code. The input files provided here may also be used as templates for any new electronic structure calculations, rather than assembling them from scratch.

- *utilities/* : Some simple scripts to extract basic information from the standard output of FHI-aims: Visualization of geometries using the .xyz format, extracting a series of geometries during relaxation as a movie, or extracting the development of energies and forces during relaxation. There is also some more sophisticated infrastructure here: Script-based ab initio replica exchange molecular dynamics (Luca Ghiringhelli) and a basin-hopping framework to predict the structure of small clusters from scratch (Ralf Gehrke).

- *regression/* : This directory contains a set of small standard test cases that can be run automatically using a script, `regression.py` – when run without any flags, it will provide its own self-documentation. We provide these regression tests purely for convenience. They will allow to check the compiled FHI-aims binary a little more extensively, but they are not strictly necessary to run FHI-aims. In particular, please do *not* view the input files of the regression tests as FHI-aims best practices. Follow the manual, not simply the regression tests. In many cases, they are not. Rather, what is tested may be a corner case that can be handled differently (better) in normal practical scenarios.

A `README` file in that directory contains some of the quickstart information given here in condensed format.

1.2 Building the code

To state this at the outset: The "aimsclub" Wiki is the appropriate place to look for detailed compiler settings for specific platforms. Do not be shy to add your own, and do not be shy to ask in the Forum when in doubt. The information given in this chapter is essential as it explains the process, but the platform specific remarks in the Wiki may help you save some time.

Since FHI-aims is distributed in source code form, the first task is to compile an executable program. For this, the following *mandatory prerequisites* are needed:

- A working Fortran 95 (or later) compiler. A good example for x86 type computers is Intel's `ifort` compiler. A free but significantly slower compiler for all platforms is `gfortran` from the GNU compiler collection (`http://gcc.gnu.org/fortran`) or the g95 compiler (`http://www.g95.org`). Do not underestimate this slowdown, though – a factor of three or so is possible.

- A compiled version of the `lapack` library, and a library providing optimized basic linear algebra subroutines (BLAS). Standard commercial libraries such as Intel's `mkl` or IBM's `essl` provide both `lapack` and BLAS support. `lapack` can also be found at `http://www.netlib.org/lapack/`.
  Having an optimized BLAS library for YOUR specific computer system(s) is critical

for the performance of FHI-aims. Very good free implementations include ATLAS (http://math-atlas.sourceforge.net/).

You should also have a version of the Gnu make command, as not all ancient Unix make commands support our Makefile. Typically, Gnu make will already be present on your system, either as make, or possibly as gmake.

The next two prerequisites are *optional*, but absolutely encouraged for any advanced use of FHI-aims: Support for parallel architectures, and (separately) support for fully parallel Scalapack-based eigenvalue solvers. In these cases, you will also need:

- A version of MPI libraries for parallel execution, often already present on a parallel system (if not, http://www.open-mpi.org/ provides one of several free implementations)
- Compiled versions of the scalapack library, and basic linear algebra *communication* subroutines (BLACS). Capable implementations can be found at http://www.netlib.org/, but are often provided already in the numerical libraries of many vendors (e.g., Intel MKL on Linux).

The creation of a complete, MPI- and scalapack-enabled binary is effort well spent. This should be the goal when compiling FHI-aims for any production purposes. This means that you should ultimately aim to build the Makefile target (see below) called "scalapack.mpi".

The "aimsclub" Wiki gives compilation hints for many platforms (including MPI, BLACS and Scalapack, which we strongly recommend on any platform with more than one CPU). Please help by adding your own working settings there, and ask us if there are questions.

Once all prerequisites are in place, change directory to the *src/* directory, and open the Makefile in a text editor.

> *You must adjust at least some system-specific portions of the* Makefile*—simply typing "make" and hoping that the problem will go away will not work.*

Usually, all you will have to do is to decide on one of the preconfigured make targets – "serial", "mpi", or "scalapack.mpi". Near the top of the Makefile, a number of *mandatory* settings are commented for each target. Uncomment *only* the block of settings relevant to your chosen make target, and fill in the correct values of each variable (FC, FFLAGS, LAPACKBLAS, ...) for your computer system. *Note* that the Makefile itself contains detailed instructions and explanations regarding the meaning of these variables. Often, simply adjusting the compiler name, the location of your libraries (LAPACKBLAS, possibly MPIFC or SCALAPACK) will be sufficient. In addition, we *strongly* recommend that you consult the documentation of your compiler, in order to find out which optimization options *beyond* the generic "-O3" optimization level suggested preset in the Makefile will make a difference on your computer.

Finally, this brings us to the key step of the build process: Building the code. After the Makefile is adjusted, type

```
make <target>
```

at the command line, where "⟨target⟩" should be replaced by the target of your choice: "serial", "mpi", or "scalapack.mpi".[1] If successful, this should build the desired FHI-aims binary (the compilation will take a while) and place it in the *bin/* directory mentioned above.

Building FHI-aims can take a while nowadays. If you have more than one processor on the machine for building FHI-aims, try

```
make -j <number_of_processors> <target>
```

This choice should speed up the process greatly.

You may also wish to keep your own copy of the `Makefile`, for instance to be able to work directly with the FHI-aims git repository without overwriting the general `Makefile`. In that case, just copy the standard `Makefile` to something like `Makefile.myname`, and use

```
make -j <number_of_processors> -f Makefile.myname <target>
```

Finally, we do note that there is some support for more sophisticated tasks, such as cross-platform builds (building binaries for different architectures from the same home directory) among the non-standard environment variables in the Makefile (see there).

1.3 Running FHI-aims

As a simple test run to establish the correct functioning of FHI-aims and also to familiarize yourself with the basic structure of the input and output files, we suggest you change directories to the *testcases/H2O-relaxation/* directory. The test run provided there relaxes a simple H_2O molecule from an initial (distorted) structure to the stable one, and computes total energies, eigenvalues etc. along the way. Notice that the key convergence settings (basis sets and grids) in this example are chosen to be fast. The results (particularly the relaxed geometry) are still trustworthy, but we encourage you already here to explore more stringent convergence settings later. In fact, *always* explore the impact of critical convergence settings on the accuracy of key results in your own project.

In the *testcases/H2O-relaxation/* directory, type

```
../../bin/aims.version.serial.x < /dev/null | tee H2O_test.own
```

at the command line. For "*version*", you must insert the code version stamp that was actually downloaded and built (for example, `010110`, `071711_5`, `081912`, `081213`,

[1] There is an additional target, `parser`, which builds an executable that stops after parsing the input. This binary can be used to check the validity of input files. (The `dry_run` keyword achieves the almost same effect with the full binary.)

071914, or whichever code version you are building).[2] Similarly, for make targets other than "serial", you should use the appropriate binary including the necessary mpi command instead. On many (but not all) platforms, that command will be `mpirun`, and will also require you to specify the number of processors to be used by a flag.

The result will be an output stream on your computer screen (created by "tee") which is also captured in an output file `H2O_test.own`. Any critical information regarding computational settings, results (total energies, forces, geometries, ...), errors etc. should be contained in this file, which we encourage you to look at (yes, it is *meant* to contain human-readable and useful explanations). Any *other* output files are only written if requested, and will be covered in the later sections of this text.

> *The standard output stream or file contains any and all output that FHI-aims writes by default. For later use, you must save this output stream to disk in some way, using standard Unix redirections such as the `tee` command above or a simple redirect.*

Apart from the first expression given above, such redirections might look like this:

```
../../bin/aims.version.serial.x < /dev/null > H2O_test.own
```

or even like this:

```
nohup ../../bin/aims.version.serial.x < /dev/null > H2O_test.own 2>&1 &
```

The latter version decouples the FHI-aims run completely from your current login, and additionally saves any system error messages to the standard output file as well. With the above command sequence, you may safely log out from the computer in question, the code should keep running in the background. Take care to monitor your running processes using the `ps` Unix command. For instance, it is highly unadvisable to run ten instances of FHI-aims at once in the background on a single CPU and expect any reasonable performance of the computer at all. The above hints are just examples of general Unix command-line sequences. For a complete treatment, we recommend that beginners read a separate Unix textbook, or—often feasible—learn by doing and Google.

If successful (otherwise, consider the warnings three paragraphs below), you may wish to compare your results to those contained in our own output from this run, which is contained in the file `H2O.reference.out`. You should obtain exactly the same total energies, forces, and geometries as given in this file. Any information regarding timing is, of course, specific to your computer environment, and not necessarily the same.

The directory *testcases/H2O-relaxation/* contains two more files, `control.in` and `geometry.in`. These are the sole two input files required by FHI-aims, and are the most important files to learn about in the rest of this documentation. In brief, `geometry.in` contains any information related directly to a system's geometry – normally, this will be atomic positions (in Å) and perhaps lattice vectors for periodic calculations, but no more. Any other, method-related, input information is part of `control.in`.

[2] The FHI-aims version stamp can be modified to whatever you wish in `Makefile.backend` in the *src/* directory.

In practice, we attempt to strike a balance between the information *needed* by `control.in`, and information set to safe defaults unless specified explicitly. For example, you *must* specify the level of theory (e.g., the fact that PBE exchange-correlation is used) and also the basis set employed. While it is highly useful to have this relevant information openly accessible, this would also create the need to personally edit a large amount of input before ever tackling the first run. For any information tied to the actual element (or "species"; arguably the most complex information required), we therefore provide ready-made template files for all elements (1-102) in the *species_defaults* directory. They are ready for copy-paste into `control.in`. These files will still benefit from some adjustment to your personal needs (for instance, the provided integration grids are set rather on the safe side, at the expense of more CPU time), but should greatly simplify the task.

Two final, important warnings regarding the execution of FHI-aims that are beyond our direct control:

- FHI-aims *requires* that the execution stack size available to you be large enough for some initial internal operations. Spare us the details (ample explanation of the meaning of the "stack" in Unix can be found elsewhere), but for reasons unbeknownst to us, some vendors limit the default user stack size to ≈5 MB in a time when typical available RAM per processor is 2 GB or more. If too little stack is available, your FHI-aims run will *segfault* shortly after the command was launched. In that case, type:

  ```
  ulimit -s unlimited
  ```

  (when using the bash shell or similar), or

  ```
  limit stacksize unlimited
  ```

  (when using the tcsh or similar).

  ```
  echo $SHELL
  ```

  will tell you which shell you are using. Ideally, this same setting should be specified in your .profile, .bashrc, or .cshrc login profiles. If "unlimited" does not work, try setting a large value instead, e.g., `ulimit -s 500000`.

- An important system settings for parallel execution is the environment variable

  ```
  export OMP_NUM_THREADS=1
  ```

  (the syntax is correct for the bash shell). When using Intel's mkl, you should additionally set `MKL_NUM_THREADS` to 1 and `MKL_DYNAMIC` to FALSE.

After startup, the first messages contain information about your computer's environment: Code version, compiler information, host names, environment variables which turned out to be useful and which should be set on your system (e.g. `OMP_NUM_THREADS`), etc. The complete input files `control.in` and `geometry.in` are also repeated verbatim. Any FHI-aims run should thus be completely reproducible based on the standard output stream alone.

Should you encounter further issues, consider also the troubleshooting information documented in Appendix A.

All this said, after successfully running the test run, you should now be ready to go with FHI-aims. The remainder of this document is about the details – available options, how to run aims most efficiently, etc. Happy computing!

1.4 Shortcut: make.sys

The initial set of builds described so far worked by directly editing the `Makefile` shipped with FHI-aims. This is useful because working examples of Makefile variables are included there, as well as the comments documenting the meaning of individual variables.

Once you are familiar with that process, there is a workaround that does not require to edit the Makefile itself at all. Instead, FHI-aims also allows a user to create a file `make.sys` in the *src* directory, containing only the Makefile variables that need to be set. For example, a `make.sys` file that builds a Fortran-only scalapack.mpi build could look like this:

```
FC = ifort
FFLAGS = -O3 -ip -module $(MODDIR)
F90MINFLAGS = -O0 -module $(MODDIR)
FMINFLAGS = -O0 -module $(MODDIR)
KNOWN_COMPILER_BUGS = ifort-14.0.1-O3
F90FLAGS = $(FFLAGS)
ARCHITECTURE = Generic
LAPACKBLAS = -L/opt/intel/composerxe/mkl/lib/intel64 \
  -I/opt/intel/Compiler/11.0/084/mkl/include/intel64/lp64 -lmkl_intel_lp64 \
  -lmkl_sequential -lmkl_core -lpthread
USE_MPI = yes
MPIFC = mpiifort
SCALAPACK = -L/opt/intel/composerxe/mkl/lib/intel64 \
  -I/opt/intel/Compiler/11.0/084/mkl/include/intel64/lp64 -lmkl_scalapack_lp64 \
  -lmkl_blacs_intelmpi_lp64
```

Note that this example of a `make.sys` file comes from a specific computer and holds only for a specific compiler version. Please do not just use it blindly – instead, please determine the compiler, libraries, compiler flags etc. that are appropriate for your computer system when creating your own file `make.sys`.

1.5 Cross-Compiling with a C Compiler

The standard build process of FHI-aims is intended to work with *only* a Fortran compiler and a minimal number of libraries, e.g., those needed towards the scalapack.mpi build target (see above). The idea is that this provides all the necessary infrastructure to run FHI-aims with optimal performance on a given platform, yet does not overburden a user with the need to balance an overwhelming set of potentially conflicting system or environment variables if they are not needed.

That said, FHI-aims does provide some functionality that can only be accessed by compiling part of the code base with a C compiler, in some cases simply because a system call in question is not available from a Fortran interface.

Cross-compiling with a C compiler can be simple *if* your Fortran and C compiler use compatible interfaces. (This may not be always the case.) In the simplest case, adding the following two variables to your `Makefile` or to your `make.sys` file will enable some basic functionality that can only be accessed with a C cross-compiled build:

```
USE_C_FILES = yes
CC = gcc
```

The CC variable should specify a C compiler compatible with your Fortran compiler on your specific computer system. This choice may not always be the *gcc* compiler. Again, C cross-compiling is not necessary, but if you do, please make sure that the right C compiler is chosen for your system.

An additional variable `CCFLAGS` can be set to specify C compiler flags along with the `CC` variable above and will add C compiler flags that might be necessary for your particular computer setup.

1.6 Compilation options beyond the standard Makefile

The build provided by the standard `Makefile` in FHI-aims is designed for minimal complexity to obtain the full functionality that most users should have. Separate "basic linear algebra subroutines" (BLACS), Lapack, parallel builds (if a parallel machine is available, nowadays almost always), and scalapack support are so performance-critical that every user should spend the time to investigate them in detail before doing serious production work with FHI-aims. These dependencies of FHI-aims on external libraries are therefore kept in the main `Makefile`.

In addition, FHI-aims provides further functionality that can be achieved by linking to other external libraries. However, this functionality will not be needed by all users and/or could seriously complicate the build process for everyone. Such functionality is therefore available through separate, amended versions of the `Makefile`. We encourage everyone to try these builds (they are not so difficult after all), but they should not become stumbling blocks.

In particular, several optional Makefile with additional functionality exist (and could even be combined):

- `Makefile.cuba` : Allows to compile in the separate "CUBA" Monte Carlo integration library, which enables the Langreth-Lundqvist van der Waals functional based on noloco as a post-processing step. See Section 3.19 for more details.
- `Makefile.meta` : *still experimental!* Allows to interface FHI-aims to the `PLUMED` library for free-energy calculations for molecular dynamics.
  (see `http://merlino.mi.infn.it/~plumed/PLUMED/Home.html` , the `PLUMED` project homepage). Currently, a copy of the `PLUMED` library is kept in the *external* directory of the FHI-aims source code, and must be compiled separately using a C compiler. Note that the token "-DFHIAIMS" must be included in the CCFLAGS (at present, this is already specified in `plumed.inc`). For compiling on IBM power machines, the flags -mpowerpc64 -maix64 should be included. In the future, this will be corrected by housing the respective FHI-aims plugin directly in the `PLUMED` library. We apologize that the linking process is not yet further documented, but if this functionality is of interest to you, please contact us.
  By selecting the correct target in `Makefile.meta`, the code can be compiled with `lapack` or `scalapack` libraries, or with shared memory support (see next item).
- `Makefile.ipi` : Allows to interface with the i-PI path integral molecular dynamics wrapper[?]. See the `use_pimd_wrapper` keyword for a few more details.
- `Makefile.shm` : Another example of a Makefile that cross-links C and Fortran based functionality, although the actual functionality that Makefile.shm provides – access to shared-memory arrays in the Hartree potential – is no longer needed at this point.
- `Makefile.amd64_SSE` : This Makefile contains an example how to use FHI-aims together with a version of the ELPA library which is especially tuned for SSE vector instructions. Note that the GNU C compiler must be installed in order to produce running code from the specific assembler files. As the name "amd64" indicates this optimization is only reasonable on processors which support the amd64 instruction set. Note that "ELPA_ST" is set, to use the single-threaded version of ELPA.
- `Makefile.amd64_SSE_mt` : The same as Makefile.amd64_SSE, but by setting "ELPA_MT" the hybrid MPI OpenMP version of ELPA is used.
- `Makefile.amd64_AVX` : This Makefile contains an example how to use FHI-aims together with a version of the ELPA library which is especially tuned for AVX vector instructions. Note that the GNU C and C++ compiler must be installed in order to produce running code from the specific ELPA files which contain gcc intrinsic functions. As the name "amd64" indicates this optimization is only reasonable on processors which support the amd64 instruction set and which already "understand" AVX, i.e. Intel Sandybridge or newer. Note that "ELPA_ST" is set, to use the single-threaded version of ELPA.
- `Makefile.amd64_AVX_mt` : The same as Makefile.amd64_AVX, but by setting "ELPA_MT" the hybrid MPI OpenMP version of ELPA is used.

1.6.1 Alternative build process with the Waf build system

still experimental!

FHI-aims can be alternatively built with the Waf build system (https://waf.io/). Waf is a modern build system written in Python which enables writing build scripts (also in Python) that are easier to maintain compared to makefiles. Another advantage specific to Fortran is that the dependencies between modules are recognized automatically by Waf. The actual build flow (such as recompiling only changed and depending files) and resulting binaries should be bit-for-bit identical to the standard build process with `make`.

The Waf executable (a Python script) is distributed together with FHI-aims and is located at `src/waf`. The Waf build script is at `src/wscript` and is not supposed to be changed under normal circumstances. The build script assumes that the file `src/Makefile` is unchanged from the distributed version and all user build environment variables (such as `FC` or `LAPACKBLAS`) are defined in `src/make.sys` (which is sourced by the standard makefiles). If this prerequisite is satisfied, building with Waf should be as easy as

```
cd src
./waf configure <options>
./waf install
```

where `<options>` can be any of the AIMS build extensions such as `--with-mpi` or `--with-scalapack`. For a full list of available extensions, see `./waf --help`. Both Waf calls can be joined to `./waf configure install <options>`. The build is done in directory `build` and installs the binaries into `bin`, identically to the standard build process. Subsequent builds can be run only with

```
./waf install
```

With the Waf build system, all build variants of AIMS (such as MPI, Scalapack, building AIMS as a shared library, any additional libraries) are managed in so-called build extensions defined in `wext_<name>.py` files. A build extension defines changes to the basic build process both when it is activated (via the command-line option) and when it is not activated (there is often a need for so-called stub files). To learn how to write an AIMS build extensions, see the `wext` module documentation below.

`wext` — AIMS build extensions

The `wext` module manages AIMS build extensions such as MPI or Scalapack.

To make a new build extension, create a file called `wext_<name>.py` in the `src` directory where `<name>` is the name of your extension. This Python file can be empty, in which case the extension has no effect besides adding an option to the Waf options (see `./waf --help`). The extension is then activated by calling Waf with this option. Functionality is added by defining variables and functions (all optional) in `wext_<name>.py`:

help A string with a help for the build extension which appears in `./waf --help`.

require A list of build extensions that the extension requires if activated.

stub(ctx) Function that takes a `Context` (see below) as an argument and applies actions (such as adding source files) that should happen if the extension is not active.

extension(ctx) Takes a `Context` as argument and applies actions that should happen when the extension is active.

For example, the Scalapack extension is defined as:

```
help = 'compile aims with Scalapack'
require = ['mpi']

def stub(ctx):
    ctx.add_files('fc', 'scalapack_stub.f90')

def extension(ctx):
    ctx.add_link_flag(ctx.env.SCALAPACK)
```

In common language, Scalapack requires MPI to work. The Scalapack stub adds an appropriate stub file (compiled with Fortran compiler – 'fc'). And finally if Scalapack is activated, a link flag is added from the `SCALAPACK` environment variable. For full list of avaialble actions, see documentation of `Context`.

class **`wext.Context`**(***wctx***) Provides interface to modify a build process.

`add_feature`(***feat***, ***files***) Add a feature to files.

`add_files`(***feat***, ***files***) Add files with features to the build process.

`add_link_flag`(***flag***) Add a link flag.

`change_ext`(***fext***) Change file name extension (including the dot) of the resulting binary.

`env` = ***None*** Provides access to the variable environment.

`replace_files`(***newfiles***, ***oldfiles***, ***feat=None***) Replace old files with new files.

Optionally add a feature. All old files have to have the same features.

`wctx` = ***None*** Provides access to the Waf build configuration context.

All inquiries, issues and general questions about the Waf build system should be directed at jhermann@fhi-berlin.mpg.de.

1.7 Compiling faster versions of FHI-aims on specific platforms

FHI-aims is intended to be a Fortran-only code, which – for most of the code – means that building the "fastest" version of FHI-aims on a given computer architecture is "only" a matter of finding the right Fortran compiler and compiler options for that processor. For some architectures, specific compiler options are collected in the FHI-aims club wiki – please check there and please add any useful information that you may find.

That said, one particular performance-critical area for large systems is the Kohn-Sham eigenvalue solver. In FHI-aims and on parallel computers, this problem is solved by the ELPA library. ELPA, in fact, allows its users to specify specific, platform-optimized so-called linear algebra "kernels." By default, FHI-aims uses a generic kernel which will compile with any Fortran compiler and will give reasonable speed. However, if one knows which specific computer chip one is using, it is possible to substitute this kernel with an architecture specific kernel and compile a faster version of ELPA into FHI-aims. This is possible, for example, for the BlueGene/P, BlueGene/Q, Intel AVX and several other Intel architectures. For standard Intel x86 chips, there is even an "assembler" based kernel that will get fast performance regardless of the Fortran compiler above.

Note that this choice can matter. For example, the "generic" ELPA kernel will produce fast code for the Intel Fortran compiler, but much slower code with certain versions of the PGI Fortran compiler (often found on Cray machines).

How to build FHI-aims with these special ELPA kernels?

In some cases, the necessary information already exists in the form of architecture-specific Makefiles in FHI-aims, e.g., `Makefile.amd64_SSE` or `Makefile.amd64_AVX` mentioned in the last section. In other cases (e.g., for the Intel Assembler kernel elpa2_kernels_asm_x86_64.s, found in the src directory) you may have to precompile with the Gnu assembler "as" before building the rest of FHI-aims.

Going away from the "generic" kernels can therefore require a little bit of extra work. The point of the present section is to clarify that this extra work is possible, and often worthwhile, if you already know that you are going to use FHI-aims for a long time on the same architecture.

Chapter 2

Input Files: Basic Handling

```
# Geometry for water -- needs to be relaxed as the water molecule
# described here has a 90degree bond angle and a
# 1 Angstrom bond distance ...
atom     0.00000000     0.00000000     0.00000000     O
atom     0.70700000    -0.70700000     0.00000000     H
atom    -0.70700000    -0.70700000     0.00000000     H
```

Figure 2.1: Example input file `geometry.in`, provided with the simple test case (relaxation of H_2O) described in Sec. 1.3.

2.1 The mandatory input files: `control.in` and `geometry.in`

As discussed in Sec. 1.3, FHI-aims requires exactly two input files—`control.in` and `geometry.in`—located in the same directory from which the FHI-aims binary is invoked. To start FHI-aims, no further input should be needed.[1]

Figures 2.1 and 2.2 show as examples the `geometry.in` and `control.in` files used for the simple test case (relaxation of a water molecule) described in Sec. 1.3. The philosophy of their separation is simple:

- `geometry.in` contains only information directly related to the atomic structure for a given calculation. This obviously includes atomic positions, with a description of the particulars of each element (or *species*) expected in `control.in`. In addition, lattice vectors may be defined if a periodic calculation is required. Any other information is only given here if it is *directly* tied to the atom in question, such as an initial charge, initial spin moment, relaxation constraint etc. The order of lines is irrelevant, except that information specific to a given atom must follow *after* the line specifying that atom, and *before* any following atom is specified.
- `control.in` contains all other runtime-specific information. Typically, this file consists of a *general* part, where, again, the particular order of lines is unimportant. In addition, this file contains *species* subtags that are references by `geometry.in`. Within the description of a given species, the order of lines is again unimportant, but *all* information concerning the same species must follow the initial *species* tag in one block.

In both files, the choice of units is Å for length parameters, and eV for energies; derived quantities are handled accordingly. Lines beginning with a # symbol are treated as comments, and empty lines are ignored. Finally, each non-comment line has the following, free-format structure:

```
keyword value <value> <value>
```

[1] A few specific keywords (e.g., a wave-function based restart of an existing calculation) may require additional input that simply can not be included in user-edited file. Such input files will be described with the appropriate tasks.

```
#########################################################################
#
#  Volker Blum, FHI 2009 : Test run input file control.in for simple H2O
#
#########################################################################
#
#  Physical model
#
  xc                 pbe
  spin               none
  relativistic       none
  charge             0.
#
#  SCF convergence
#
  occupation_type    gaussian 0.01
  mixer              pulay
    n_max_pulay        10
    charge_mix_param   0.5
  sc_accuracy_rho    1E-5
  sc_accuracy_eev    1E-3
  sc_accuracy_etot   1E-6
  sc_accuracy_forces 1E-4
  sc_iter_limit      100

[...]

#############################################################################
#
#  FHI-aims code project
#  VB, Fritz-Haber Institut, 2009
#
#  Suggested "light" defaults for H atom (to be pasted into control.in file)
#  Be sure to double-check any results obtained with these settings for post-processin
#  e.g., with the "tight" defaults and larger basis sets.
#
#############################################################################
  species        H
#     global species definitions
    nucleus             1
    mass                1.00794

[...]
```

Figure 2.2: Excerpts from the example input file `control.in`, provided with the simple test case (relaxation of H_2O) described in Sec. 1.3. A section of *general* (system-wide) run-time settings is separate from individual sections that describe settings specific to certain *species* (chemical elements).

Generally, all keywords and values are case sensitive: Do not expect FHI-aims to understand an "XC" keyword if the specified syntax is "xc".

It is the objective of the *next* chapter, Chapter 3, to list all legitimate keywords in FHI-aims, and to describe their function.

2.2 Defaults for chemical elements: species_defaults

FHI-aims requires exactly two input files, located in the same directory where a calculation is started: `control.in` and `geometry.in`. Both files can in principle be specified from scratch for every new calculation, using the keywords listed in Chapter 3. However, choosing the central computational settings consistently for series of calculations greatly enhances the accuracy of any resulting energy differences (error cancellation).

In FHI-aims, the key parameters regarding computational accuracy are actually subkeywords of the `species` keyword of `control.in`, controlling the basis set, all integration grids, and the accuracy of the Hartree potential. These settings should of course not be retyped from scratch for every single calculation; on the other hand, they should remain ovbious to the user, since these are the central handles to determine the accuracy and efficiency of a given calculation.

FHI-aims therefore provides preconstructed default definitions for the important subkeywords associated with different `species` (chemical elements) from Z=1-102 (H-Md). These can be found in the *species_defaults* subdirectory of the distribution, and are built for inclusion into a `control.in` file by simple copy-paste.

For all elements, FHI-aims offers three different levels of *species_defaults*:

- *light* : Out-of-the-box settings for fast prerelaxations, structure searches, etc. In our own work, no obvious geometry / convergence errors resulted from these settings, and we now recommend them for many household tasks. For "final" results (meV-level converged energy differences between large molecular structures etc), any results from the *light* level should be verified with more accurate post-processing calculations, e.g. *tight*.

- *tight* : Regarding the integration grids, Hartree potential, and basis cutoff potentials, the settings specified here are rather safe, intended to provide meV-level accurate energy *differences* also for large structures. In the *tight* settings, the basis set level is set to *tier* 2 for the light elements 1-10, a modified *tier* 1 for the slightly heavier Al, Si, P, S, Cl (the first *spdfgd* radial functions are enabled by default), and *tier* 1 for all other elements. This reflects the fact that, for heavy elements, *tier* 1 is sufficient for tightly converged ground state properties in DFT-LDA/GGA, but for the light elements (H-Ne), *tier* 2 is, e.g., required for meV-level converged energy differences. For convergence purposes, the specification of the basis set itself (*tier* 1, *tier* 2, etc.) may still be decreased / increased as needed. Note that especially for hybrid functionals, *tight* can already be very expensive and specific reductions of the number of radial functions may still provide essentially converged results at a much more affordable cost.

- *really_tight* : Same basis sets (radial functions and cutoff radii) as in the *tight* settings, but for the other numerical aspects (grids, Hartree potential), settings that are *strongly overconverged* settings for most purposes. The idea is that *really_tight* can be used for very specific, manual convergence tests of the basis set

and other settings – if really needed.
Note that the *"tight"* settings are intended to provide reliably accurate results for most DFT production purposes; and they are not cheap. The absolute total energies for *tight* and DFT are in practice converged to some tens of meV/atom for most elements. To go beyond, take the *really_tight* settings and increase the basis set or other numerical aspects step by step. (Radial function by radial function may often be a good strategy to go.) *We emphasize that the* really_tight *settings should only ever be needed for individual, specific tests. They should not be needed for any standard production tasks unless you have seriously too much CPU time to spend.*
Specific differences between *tight* and the unmodified *really_tight* settings: The `basis_dep_cutoff` keyword is set to zero, a prerequisite to approach the converged basis limit. Regarding the Hartree potential, `l_hartree` is set to 8, and the maximum number of angular grid points per radial integration shell is increased to 590.
Note that there can still be corner cases where you may want to test some numerical setting beyond *really_tight*. Mostly, these are custom scenarios or things beyond standard FHI-aims calculations of DFT total energies. Examples include: The confinement radius for surface work functions (should be checked), use of very extended or extremely tight Gaussian-type orbital basis functions (e.g., from very large Dunning-type basis sets – the density of the radial and angular grids should be checked), or RPA and MP2 calculations, which can need very different and often much larger basis sets (again, radial and angular grids should be checked).

A separate group of species defaults or light elements (H-Ar) is available especially for calculations involving explictly correlated methods (beyond semilocal and hybrid density functionals):

- *NAO-VCC-nZ* : NAO type basis sets for H-Ar by Igor Ying Zhang and coworkers [**?**]. These basis sets are constructed according to Dunning's "correlation consistent" recipe. Their intended application is for methods that invoke the continuum of unoccupied orbitals explicitly, for instance MP2, RPA or GW. Note that they were constructed for valence-only correlation (hence "VCC", valence correlation consistent), i.e., they work best in frozen-core correlated approaches following a full s.c.f. cycle (core and valence) to generate the orbitals. While NAO-VCC-nZ can be used for "normal" density functional theory (LDA, GGA, or hybrid functionals), the normal "light", "tight" and "really_tight" species defaults are more effective in those cases. The advantage of NAO-VCC-nZ over GTO basis sets such as the Dunning "cc" basis sets is that with NAOs, both the behaviour near the nucleus as well as that for the tails of orbitals far away from atoms is much more physical. This means that we can use more efficient integration grids than for GTO basis sets to obtain systematic convergence of the unoccupied state space.

In addition, the *species_defaults* directory contains a few more sets of species defaults for special purposes. These can be found in the *non-standard* subdirectory and include:

- *gaussian_tight_770* : Species defaults that allow to perform calculation with some standard published Gaussian-type orbital (GTO) basis sets for elements H-Ar

(including basis sets due to Pople, Dunning, Ahlrichs and their coworkers). These species defaults are meant to allow for exact benchmarks against GTO codes such as NWChem. The other numerical settings (especially grids) are thus much tighter than needed for "normal" NAO-type calculations. Note that FHI-aims is not optimized for GTO basis sets. We recommend NAO-type basis sets, not GTO basis sets, for production calculations – NAO-type basis sets are much easier to handle with our techniques and give better accuracy at lower cost. That said – the grid settings in the *gaussian_tight_770* species defaults are rather overconverged for benchmark purposes. One could create much more efficient species defaults for GTO basis sets – but GTOs still would not be as efficient as NAO basis sets (at the same level of accuracy).

- *light_194* : This is just an example of how to tune down the normal "light" basis sets of FHI-aims by reducing the integration grid even further. For things like fast molecular-dynamics type screening of many structures, this is a perfectly viable approach. Examples are provided for H-Ne. Obviously, do test the impact of such modifications for your own purposes.

- *LVL_intermediate* : Example of how to create much more manageable basis sets than "tight" especially for light elements and hybrid functionals (where the number of basis functions per atom plays an especially costly role); yet, these "intermediate" species defaults are significantly more accurate than our standard "light" basis sets for hybrid functional calculations.

For calculations that involve the excited state spectrum directly (this includes GW, MP2, or RPA, among others), the numerical settings from *tight* still perform rather well *if* a counterpoise correction is performed (i.e., for energy differences). Still, the basis set size and/or cutoff radii *must* be converged and carefully verified beyond the settings specified in *tight*.

To extrapolate the absolute total energy of methods which rely on the unoccupied state continuum explicitly, e.g., RPA or MP2, we recommend using the NAO-VCC-nZ basis sets. These basis sets are presently available for light elements (H-Ar). A popular completeness-basis-set extrapolation scheme is two-point extrapolation:

$$E[\infty] = \frac{E[n_1]n_1^3 - E[n_2]n_2^3}{n_1^3 - n_2^3}$$

where "n_1" and "n_2" are the indicies of NAO-VCC-nZ. This $1/n^3$ formula was originally proposed for the correlation energy, but was also used directly for the total energy.

2.3 A very quick guide to ensuring numerical convergence with FHI-aims

FHI-aims is programmed and set up to allow efficient all-electron calculations for any type of system. During the writing of FHI-aims, a key goal was to always ensure that such efficiency does not come at the price of some irretrievable accuracy loss. Results obtained by FHI-aims should be *the* answer to the physical question that you asked (provided that the functionality is there in FHI-aims) - not some arbitrary approximation.

The *species_default* levels provided by FHI-aims, *light*, *tight*, and (if ever needed!) *really_tight*, should provide such reliable accuracy as they come. The *NAO-VCC-nZ* basis sets provide additional functionality specifically for correlated methods (MP2, RPA, GW, etc.) and light elements. However, in all *species_default* files, all important accuracy choices are deliberately kept out in the open and available: They can—and sometimes should!— be explicitly tested by the user to check the convergence of a given calculation.

Such a convergence test may sometimes be geared at simply ensuring numerical convergence explicitly, but equally, it is possible that some default settings are too tight for a specific purpose, and can be relaxed in a controlled way to ensure faster calculations for some large problem.

In the following, we explain the most important species default settings explicitly, and comment on how to choose them. We use the *light* defaults for Oxygen as an example.

2.3.1 Basis set

The key physical choice to ensure converged results in FHI-aims is the list of radial functions (and their angular momenta) that are used in FHI-aims. Beyond the *minimal* basis of free-atom like radial functions, we always recommend to add at least a single set of further radial functions that are optimized to describe a chemical bond efficiently. These basis functions can be found as a list (line by line) at the end of each species defaults file. For Oxygen / light, the list reads like this:

```
#  "First tier" - improvements: -699.05 meV to -159.38 meV
     hydro 2 p 1.8
     hydro 3 d 7.6
     hydro 3 s 6.4
#  "Second tier" - improvements: -49.91 meV to -5.39 meV
#     hydro 4 f 11.6
#     hydro 3 p 6.2
#     hydro 3 d 5.6
#     hydro 5 g 17.6
#     hydro 1 s 0.75
#  "Third tier" - improvements: -2.83 meV to -0.50 meV
#     ionic 2 p auto
#     hydro 4 f 10.8
```

```
#     hydro 4 d 4.7
#     hydro 2 s 6.8
 [...]
```

Obviously, only a single set of radial functions (one for each angular momentum s, p, d) is active (not commented!) beyond the minimal basis. Since the minimal basis already contains one additional valence s and p function, this choice is often called "double numeric plus polarization" basis set in the literature (where d is a so-called polarization function as it does not appear as a valence angular momentum of the free atom). We call this level "tier 1".

In order to increase the accuracy of the basis, further radial functions may be added, simply by uncommenting more lines *in order*! We recommend to normally proceed in order of full "tiers", not function by function, but adding specific individual functions on their own can sometimes capture the essence of a problem at lower cost. For example, tier 2 may be added by uncommenting:

```
#  "First tier" - improvements: -699.05 meV to -159.38 meV
     hydro 2 p 1.8
     hydro 3 d 7.6
     hydro 3 s 6.4
#  "Second tier" - improvements: -49.91 meV to -5.39 meV
     hydro 4 f 11.6
     hydro 3 p 6.2
     hydro 3 d 5.6
     hydro 5 g 17.6
     hydro 1 s 0.75
#  "Third tier" - improvements: -2.83 meV to -0.50 meV
#     ionic 2 p auto
#     hydro 4 f 10.8
#     hydro 4 d 4.7
#     hydro 2 s 6.8
 [...]
```

tier 2 is the default choice of our *tight* settings for O.

Beyond the choice of the radial functions itself, a critical parameter is the choice of the *confinement* radius that all basis functions experience. Ensuring that each radial function goes to zero in a controlled way beyond a certain, given value is critical for numerical efficiency, but on the other hand, you do not want to reduce this confinement radius too much in order to preserve the *accuracy* of your basis set.

By default, the confinement radius of each potential is specified by the following line:

```
    cut_pot             3.5  1.5  1.0
```

This means (see also the CPC publication on FHI-aims) that each radial function is constructed with a confinement potential that begins 3.5 Å away from the nucleus, and

smoothly pushes the radial function to zero over a width of 1.5 Å. The full extent of each radial function is thus 5 Å.

Of course, this setting is chosen to give good total energy accuracy at the *light* level, but the convergence of the confinement potential must still be tested, especially in situations where a strong confinement may be unphysical. Such questions include:

- Accurate free atom calculations for reference purposes: choose 8 Å or higher for the onset of the confinement, or something similarly high—for a single free atom, the CPU time will not matter, and you will get all the tails of your radial functions right without much thinking.
- Surfaces— e.g., low electron densities above the surface for STM simulations must not be abbreviated by the onset of the confinement potential—even if the total energy is not affected by this confinement any more.
- Neutral alkali atoms, or any negatively charged ions. Those are tricky—the outermost electon shell may decay very slowly to zero with distance, and explicit convergence tests are required.

As the corresponding *tight* setting, we use:

```
cut_pot             4.0  2.0  1.0
```

Although the modification does not seem large, CPU times for periodic systems *are* significantly affected by this change of the full extent of each radial function from 5 Å to 6 Å. For example, in a densely packed solid, the *density* of basis functions per volume increases as R^3 with the full extent of each radial function, and thus the time to set up the Hamiltonian matrix should increase as R^6. Very often, the effect on the total energy is completely negligible, but again, explicit convergence tests are always possible to make sure.

Finally, there is the line

```
basis_dep_cutoff    1e-4
```

If this criterion is set above zero (10^{-4} in our *light* settings), all radial functions are individually checked, and their tails are cut off at a point where they are already near zero.

You should note that the `basis_dep_cutoff` criterion usually does not matter at all, but for very large systematic basis set convergence studies (going to tier 3, tier 4, etc, and/or testing the cutoff potential explicitly), this value should be set to zero—as is done in the *really_tight* settings, for example.

2.3.2 Hartree potential

The Hartree potential in FHI-aims is determined by a multipole decomposition of the electron density. The critical parameter here is the order (highest angular momentum) used in the expansion (all higher components are neglected). This value is chosen by:

```
l_hartree           4
```

Energy differences with this choice are usually sub-meV converged also for large systems, but total energy differences, vibrational frequencies at the cm^{-1} level etc may require more. Our *tight* settings,

```
l_hartree           6
```

provide sub-meV/atom converged *total* energies in all our tests, but you may simply wish to test for yourself ...

2.3.3 Integration grid

FHI-aims integrates its Hamiltonian matrix elements numerically, on a grid. However, this is an all-electron code: Performing integrations on an *even-spaced* grid (as is done in many pseudopotential codes) would provide terrible integration accuracy near the nucleus (sharply peaked, deep Coulomb potential and strongly oscillating basis functions).

Instead, we use what is a standard choice also in other codes: Each atom gets a series of radial spheres (*shells*) around it, and we distribute a certain number of actual grid points on each shell. Obviously, increasing the number of grid points ("angular" points) on each shell will improve the integration accuracy, but at the price of a linear increase in computational cost.

The fact that the integration spheres will overlap does not matter—we remedy this fact automatically by choosing appropriate integration weights (partitioning of unity, see CPC paper).

The number and basic location of radial shells is chosen by

```
radial_base         36 5.0
radial_multiplier   1
```

which means that we here choose 36 grid shells, and the outermost shell is located at 5 Å (this happens to be the outermost radius of each basis function, as dictated by the confinement potential).

The `radial_multiplier` tag allows to increase the radial grid density systematically by adding shells inbetween those specified in the `radial_base` line. For example, we choose

```
radial_multiplier   2
```

in our *tight* species defaults (for all practical purposes, this is converged), which means that we add one shell between the zero and the (former) first shell, one between the first and second, etc., and finally one between the (former) outermost shell and infinity ... two times 36 plus one shells total.

The distribution of actual grid points *on* these shells is done using so-called Lebedev grids, which are designed to integrate all angular momenta up to a certain order l exactly. They come with fixed numbers of grid points (50, 110, 194, etc). As a rule, fewer grid points will be needed in the inner grid shells, and more will be needed at the (more extended) faraway grid shells. We specify the increase the number of grid points per radial shell in steps, by writing:

```
     angular_grids specified
      division   0.2659   50
      division   0.4451  110
      division   0.6052  194
#      division   0.7543  302
#      division   0.8014  434
#      division   0.8507  590
#      division   0.8762  770
#      division   0.9023  974
#      division   1.2339 1202
#      outer_grid 974
      outer_grid 194
```

This example pertains to the *light_194* settings (and very light elements), and means that only 50 points will be used on all grid shells inside a radius of 0.2659 Å, 110 grid points are used on all shells within 0.4451 Å, 194 grid points will be used on all shells inside 0.6052 Å—and that's it! No more `division` tags are uncommented, and all shells outside 0.6052 Å also get 194 grid shells, as given by the uncommented `outer_grid` tag.

We note that the form of the *increase* of the number of points per radial shell near the nucleus, as well as the *maximum* number of angular grid points used outside a given radius are *critical* for the numerical accuracy in FHI-aims. When suspecting numerical noise anywhere in the calculations, the specification of the angular grid points should be checked first. This can be done by uncommenting further `division` tags with larger numbers of grid points, as well as a suitably increased `outer_grid` value. In particular, the choice of only 194 grid points max. per radial shell (only for the lightest elements!) is a rather aggressive choice, but in our experience still enables very reasonable geometry relaxations, structure searches or molecular dynamics for most purposes. However, the first thing to check in order to provide better convergence would be to set `outer_grid` to 302 (regular *light* settings). If this produces a noticeable change of the quantity you are calculating, be careful.

Of course, one can always introduce the denser grids provided in the *tight* settings, which (for reference) are

```
     angular_grids specified
      division   0.1817   50
      division   0.3417  110
      division   0.4949  194
      division   0.6251  302
```

```
      division   0.8014  434
#      division   0.8507  590
#      division   0.8762  770
#      division   0.9023  974
#      division   1.2339 1202
#      outer_grid 974
      outer_grid  434
```

These grids alone are roughly twice as expensive as the *light_194* ones above, and should provide reasonable accuracy for pretty much any purpose. Nonetheless, of course one can still go and check explicitly, simply by increasing the number of grid points per shell by hand.

2.4 Why does my calculation take too long?

This is, indeed, an excellent question to ask. Understanding what the code spends its time on, and why, is often the best first approach to understanding what is actually being calculated – and thus, to learn something about the scientific problem to be solved.

Many calculations take as long as they do, simply because getting an accurate result for many atoms can take some time.

That said, if calculations that seemed simple start taking excessive amounts of time, it may be a very good idea to question your input settings. It may also be a very good idea to actually read the output of the code. It tells you a lot about what the code does. Some suggestions for different scenarios:

- Do invest the time to compile a scalapack enabled binary and actually use scalapack. Most architectures today are parallel, and using those efficiently is perhaps the single biggest technical strength of FHI-aims. Never ask for the use of a lapack eigenvalue solver (the serial version, i.e., the eigenvalue solver that only uses a single CPU) explicitly unless you are testing. The code sets the correct default automatically if needed. But if you ask for the serial eigenvalue solver explicitly, you may find 999 of your 1000 CPU cores doing nothing. (See the keyword `KS_method` . Most importantly, never set this keyword explicitly if there is no reason to do so.)
- Look at the timing output at the end of each s.c.f. iteration – not(!) just the final timings. These timings summarize the time spent for each of the physical steps of your calculation, and can tell you a great deal about what is going on. Example:

```
End self-consistency iteration #     1       :  max(cpu_time)    wall_clock(cpu1)
| Time for this iteration                     :      219.302 s         219.900 s
| Charge density update                       :       16.121 s          16.162 s
| Density mixing & preconditioning            :        1.084 s           1.099 s
| Hartree multipole update                    :        0.088 s           0.090 s
| Hartree multipole summation                 :        4.440 s           4.489 s
| Integration                                 :        0.980 s           0.986 s
| Solution of K.-S. eqns.                     :      196.568 s         197.023 s
| Total energy evaluation                     :        0.004 s           0.016 s
```

The key times to look for here are the wall clock times. This is the physical time spent by the code on each task. The individual sub-timings of each task should roughly add up to the total time, which is given first.

The "CPU time", on the other hand, is measured internally, without accounting for times when the CPU is in fact idle. The CPU time is only given here since large deviations between wall clock time and internally measured CPU time are a good way to indicate an inefficient computer system setup. In case of doubt, however, wall clock time is the relevant measure for the real cost of the calculation

Typically, the time for the density update should be approximately the same (within a single-digit factor) as for the creation of the Hamiltonian integrals. (All these numerical

steps are explained in the FHI-aims CPC paper, Ref. [?].) The fact that this is not the case indicates some kind of a problem.

However, the bulk of the time is spent in what is called "Solution of K.-S. eqns.", which here means the simple solution of a matrix eigenvalue problem. This is simple linear algebra. This step scales formally as $O(N^3)$ with system size N, while all other steps scale roughly as $O(N)$.

This means that the eigenvalue problem should become the dominant part of the calculation time only for rather large systems (100s or 1,000s of atoms, depending on whether heavy or light elements are used, whether the system is non-periodic or periodic, etc.). The fact that the eigenvalue problem takes up so much time above warrants at least a question.

In the case shown above, a periodic calculation was conducted, with a total of 64 k-points, i.e., a total of 64 independent eigenvalue problems to be solved. Asking for many k-points is obviously a good reason why the solution of eigenvalue problems could dominate.

In the case considered here, however, the number of basis functions (the matrix dimension) was only a few thousand.[2] As a rule of thumb, this should not have been a problematic matrix size yet. (Several ten thousand basis functions or, perhaps, a few thousand k-points are typically what is needed to make a single eigenvalue solution become relevant, even on a large number of CPU cores.)

What happened above is that the calculation was in fact conducted in parallel on ≈500 CPU cores, but erroneously enforced a serial eigenvalue solver in the `control.in` file. This means that about 450 CPU cores idled while the eigenvalue problem was solved on only a few others.

The point of this example is: It helps to check and question the timing output. Another common problem is the fact that the calculation of forces costs far more than just the calculation of the electron density. Thus, the FHI-aims code by default only computes forces once the s.c.f. cycle is otherwise converged. If, however, your s.c.f. convergence criteria are set inadequately, you might see ten s.c.f. iterations per geometry step computing forces. The code has no way to foresee this, but as a user, you may be able to check after the fact, and prevent this behavior for the future.

- Mixing factor and occupation broadening. These are again values that decidedly depend on the system type to be computed, which is difficult to foresee from the perspective of the code. The default values for `charge_mix_param` (0.6) and `occupation_type` (0.01 eV) are set to be adequate for many molecules or semiconductors with a wide HOMO-LUMO gap. However, for metallic systems or for ferromagnetic systems, these settings can be very inadequate indeed. Even if the s.c.f. cycle does converge, more than just several tens of s.c.f. iterations will

[2] It is truly a system-dependent question what constitutes "many" k-points. For example, a metallic system with a single atom per unit cell should not have much trouble with, say, 24^3 k-points. On the other hand, a 1,000 atom supercell should probably not use more than a single-digit number of k-points. In the specific case considered above, 64 k-points did not happen to be a particularly large number.

be needed. For metallic systems, values such as `charge_mix_param` 0.05 and `occupation_type` 0.1 eV can be completely reasonable.

- There are (obviously) the numerical convergence parameters of the preceding section that should be heeded. For example, "tight" settings can be much more costly than "light" settings. Obviously, "tight" settings are needed for accurately converged final numerical results in many cases. However, this does not mean that, e.g., a long pre-relaxation has to be done with "tight" settings – prerelaxing with "light" settings and switching to "tight" settings only then is usually the way to go.

- Exchange-correlation methods beyond LDA and GGA typically take much more time. Here, the key bottleneck is the evaluation of the two-electron Coulomb Operator and its manipulations later. Even then, it pays to spend time learning about the respective settings, for instance, the `RI_method` to be used, the internal thresholds that go with it, or whether it is possible to reduce the number of s.c.f. steps in some other way.

2.5 Stopping a run: Files `abort_scf` and `abort_opt`

Sometimes, you may wish to stop a running FHI-aims calculation prematurely, but in an organized way.

Of course, with any running instance of FHI-aims, there is always the option to stop a run by invoking the Unix 'kill' command on every single running 'aims' process, and this will normally end the run right where it is.

To obtain a slightly more civilized stop (to allow the code to finish in a defined location and stop after writing some more output), you may instead create one of two specific files:

1. `abort_scf`
2. `abort_opt`

The code simply checks for the existence of either of these files periodically. No input is needed. Thus, simply change to the directory in which the code is running, and type (at the command line)

```
touch abort_scf
```

or

```
touch abort_opt
```

After a while, the run will stop.

The existence of `abort_scf` will stop the code after the current s.c.f. *iteration* is finished, i.e., the solution of the Kohn-Sham equations will not be self-consistent even for the present geometry.

The existence of `abort_opt` will stop the code after the current s.c.f. *cycle* is converged during a geometry relaxation, i.e., the electronic structure will be converged for the present geometry, but the forces will not be zero.

In either case, the stop of FHI-aims will not happen immediately. Depending on the nature of the run, it may take quite some time until the 'abort' takes effect, since the code needs to reach the appropriate state first. If you are interested in an immediate stop, the Unix 'kill' command is still your best bet.

One can also envision numerous refinements or alternative scenarios where an 'abort' file could be useful. If you really need such a case, please create the appropriate check where you need it. If it does the trick for you, we will be happy to incorporate the change into the mainline version of FHI-aims.

Chapter 3

The Full Monty: All Keywords and Capabilities

The present chapter aims to give a comprehensive overview and summary of all input options (keywords) that are available in FHI-aims: a full listing of keywords according to their intended use. In each of the following sections, keywords related to a given class of tasks are grouped together, and then listed according to whether they belong into `geometry.in`, the general section of `control.in`, or the species subsection(s) of `control.in`.

FHI-aims is a computer code under active development. Aside from established, stable and well-tested features, you may also find features that someone is still working on. Such features are marked as "experimental". If you are interested in using one or more of those features, contact us, and we will try to be of assistance as much as we can.

For the truly curious: All input and output options are managed by the subroutines `read_control.f90`, `read_geo.f90`, and `read_species_data.f90`. In cases of doubt, those subroutines are the ultimate place to determine a keyword's exact invocation and function.

3.1 Usability (convenience)

This section is only intended for functionality that fits none of the other categories (which are all scientifically / technically motivated). These files and keywords affect the general user convenience / experience for FHI-aims.

As an exception, this section also lists any *files* which may be used to interact with a *running* instance of FHI-aims. Currently, only two such files exist, but in principle, more could be envisioned.

Files that interacting with the running code:

Tag: `abort_scf` (file)

Usage: At the command line, use the Unix command `touch abort_scf` in the current working directory of a running instance of FHI-aims to trigger a controlled stop of the run later.

Purpose: If the file `abort_scf` is found in the current working directory of FHI-aims, the present run will be aborted after the next s.c.f. iteration is complete (but importantly without achieving self-consistency).

This functionality allows FHI-aims to stop in a controlled fashion, but not instantly. If you are interested in an instant stop, the Unix 'kill' command (or its equivalent in the queueing system of a production machine) is the best way to proceed. See Sec. 2.5 for some further remarks.

Tag: `abort_opt` (file)

Usage: At the command line, use the Unix command `touch abort_opt` in the current working directory of a running instance of FHI-aims to trigger a controlled stop of the run later.

Purpose: If the file `abort_opt` is found during a geometry relaxation in the current working directory of FHI-aims, the present run will be aborted after the next s.c.f. cycle is complete (i.e., afer achieving self-consistency for the present geometry, but without fully optimizing the structure).

This functionality allows FHI-aims to stop in a controlled fashion, but not instantly. If you are interested in an instant stop, the Unix 'kill' command (or its equivalent in the queueing system of a production machine) is the best way to proceed. See Sec. 2.5 for some further remarks.

Tags for general section of `control.in`:

Tag: `check_cpu_consistency` (control.in)

Usage: `check_cpu_consistency flag`

Purpose: In parallel runs, determines whether the consistency of geometry-related arrays is verified explicitly between different MPI tasks.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

This flag is introduced as default purely to monitor and possibly undo errors that should not happen. Theoretically, all MPI tasks of a given FHI-aims run should have the same atomic coordinates and lattice vectors. In practice, it appears that certain hardware and/or compilers/libraries introduce bit flips between different instances of what is formally the same variable on different CPUs.

If `check_cpu_consistency` is `.true.`, the code checks for deviations.

If the discrepancy is numerically negligible (below the value set by the tolerance parameter `cpu_consistency_threshold`, the code will work based on the assumption that the observed discrepancy is a platform-dependent artifact, will set all instances of the geometry to that stored on MPI task myid=0, and continue the run. Nonetheless, a warning will be printed in the output file and near the end of the output.

If the discrepancy is larger than the tolerance parameter `cpu_consistency_threshold`, the code will stop and inform the user.

This keyword is new functionality in 2016. The extent of the problem across different compilers is not yet known. Please inform us if this keyword produces an excessive amount of output so that we can tweak its behaviour.

Tag: `cpu_consistency_threshold` (control.in)

Usage: `cpu_consistency_threshold tolerance`

Purpose: In parallel runs, determines the degree to which inconsistencies of geometry-related arrays will be tolerated between different MPI tasks.

`tolerance` : A small real numerical value, positive or zero. Default: 10^{-12}.

See keyword `check_cpu_consistency`. If `check_cpu_consistency` is `.true.`, then keyword `cpu_consistency_threshold` determines the maximum value to which discrepancies of geometry-related quantities between different MPI tasks will be tolerated (they will, however, be set to identical values even if the run continues).

Tag: `dry_run` (control.in)

Usage: dry_run

Purpose: If set in control.in, the FHI-aims run will only pass through all preparatory work to ensure the basic consistency of all input files, but will stop before any real work is done.

This keyword is useful to check the consistency of input files with the same exact binary that may later be used in a series of (perhaps queued) production runs. If there are trivial errors in the input files, no need to wait for the queue. The same effect can be achieved by building a 'parser' binary, but this version saves the recompilation. The price is that one must not forget to comment out the dry_run option in the actual, queued input files.

Subtags for *species* tag in control.in:

---

species **sub-tag: cite_reference** (control.in)

Usage: cite_reference string

Purpose: Triggers the addition of a specific citation to the end of the FHI-aims standard output for a given run.

string is a string that identifies the reference in question.

This feature is useful, e.g., to make sure that the literature reference for a given basis set (encoded in the species_defaults input file) is available at the end of an FHI-aims run.

Each citation must, however, be coded into the FHI-aims source code in module applicable_citation to ensure that the requested output is actually available. Note that the practical format for such references can vary widely – from a simple string (explanation who did the work) all the way to the more usual case of a journal reference.

At the time of writing, species-related legitimate values of string are:

- NAO-VCC-2013 for reference [?], describing the NAO-VCC-nZ basis sets for valence-correlated calculations of elements H-Ar (useful for basis set extrapolation for many-body perturbation methods, e.g., MP2, RPA, RPT2, or GW).

3.2 Physical model: Geometry, charge, spin, etc.

The present section summarizes all keywords in FHI-aims that are directly concerned with the *physical model* of the problem to be tackled. Importantly, this includes some specific subtags that you *cannot* ignore, because they define the physical question that you are trying to address – and no one else but you can do that. The present section thus includes such things as atomic positions or unit cells, but also the level of theory to be used (exchange-correlation, relativistic treatment), or a potential charge of the system.

Tags for `geometry.in`:

Tag: `atom` (geometry.in)

Usage: `atom x y z species_name`

Purpose: Specifies the initial location and type of an atom.

`x`, `y`, `z` are real numbers (in Å) which specify the atomic position.
`species_name` is a string descriptor which names the element on this atomic position; it must match with one of the species descriptions given in `control.in`.

Tag: `atom_frac` (geometry.in)

Usage: `atom_frac` n_1 n_2 n_3 `species_name`

Purpose: Specifies the initial location and type of an atom in fractional coordinates.

n_i is a real multiple of lattice vector i. `species_name` is a string descriptor which names the element on this atomic position; it must match with one of the species descriptions given in `control.in`.

Fractional coordinates are only meaningful in peridic calculations.

Tag: `lattice_vector` (geometry.in)

Usage: `lattice_vector x y z`

Purpose: Specifies one lattice vector for periodic boundary conditions.

`x`, `y`, `z` are real numbers (in Å) which specify the direction and length of a unit cell vector.

If up to three lattice vectors are specified, FHI-aims automatically assumes periodic boundary conditions in those directions. *Note* that the order of latttice vectors matters, as the order of k space divisions (given in `control.in`) depends on it!

Tags for general section of `control.in`:

Tag: `charge` (control.in)

Usage: `charge q`

Purpose: If set, specifies an overall charge in the system.

`q` is a real number that specifies a positive or negative total charge in the system.

For most normal systems, this definition is unambiguous (sum of all nuclear charges in `geometry.in` minus number of electrons in the system). Note specifically that the same definition continues to hold also in systems with external embedding charges (specified bey keyword `multipole` in `geometry.in`). The charges of the external embedding charges are in addition to the `charge` keyword in `control.in`, and *not* included.

Tag: `fixed_spin_moment` (control.in)

Usage: `fixed_spin_moment value`

Purpose: If set, allows to enforce a fixed overall spin moment throughout the calculation.

`value` : real-valued number, specifies the difference of electrons between spin channels, $2S = N_{\text{up}} - N_{\text{down}}$.

Meaningful only in the spin-polarized case (`spin collinear` in `control.in`).

This keyword replaces the earlier keyword `multiplicity` . Note that the value that must be given for `fixed_spin_moment` is $2S$, which corresponds to a `multiplicity` $(2S + 1)$ —i.e., the values are not the same. Keyword `fixed_spin_moment` works for periodic and cluster systems alike, and uses two different chemical potentials (Fermi levels) for the spin channels.

Tag: `species` (control.in)

Usage: `species species_name`

Purpose: Defines the name of a species (element) for possible use with atoms in `geometry.in`

`species_name` is a string descriptor (e.g. C, N, O, Cu, Zn, Zn_tight, ...).

Every `species_name` used in an atom descriptor in `geometry.in` must correspond to a `species` given in `control.in`. Following the `species` tag, all sub-tags describing that species must follow in one block. (No particular order is enforced within that block). For example, the choice of the basis set, the atom-centered integration grid, or the multipole decomposition of the atom-centered Hartree potential are all specified per `species` .

Tag: `spin` (control.in)

Usage: `spin type`

Purpose: Specifies whether or not spin-polarization is used.

`type` is a string, either `none` or `collinear`, depending on whether an unpolarized (spin-restricted) or spin-polarized (unrestricted) calculation is performed.

In the `collinear` case, defining the moments used to create the initial spin density is required (see the beginning of Sec. 3.10 for an explanation). This means that an overall `default_initial_moment` (in `control.in`), or at least one individual `initial_moment` tag in `geometry.in`, or both, must be set. Else, the code will stop with a warning. (It is not necessary to specify `initial_moment` for every atom in `geometry.in`. A single one will do.) Choosing the right initial spin density can be performance-critical, and critical for the resulting physics. FHI-aims should not make this choice for you.

Warning: It is not a good idea to run each and every calculation with `spin collinear` just because that seems to be the more general case. In a system that will safely be non-magnetic, using something other than `spin none` *will* roughly double the CPU time needed in the best case, and it will most likely lead to much worse s.c.f. convergence (i.e., more iterations needed to find the self-consistent electronic solution). There is no fundamental problem with running `spin collinear`, but again: just doing this out of some sense of impartiality may not be a wise use of resources.

Subtags for *species* tag in `control.in`:

`species` **sub-tag: `mass`** (control.in)

Usage: `mass M`

Purpose: Atomic mass

`M` is a real number that specifies the atomic mass in atomic mass units (amu).

This tag is used only for molecular dynamics. The preconstructed `species_defaults` files supplied with FHI-aims contain the mass *average* over the different isotopes of each natural element.

`species` **sub-tag: `nucleus`** (control.in)

Usage: `nucleus Z`

Purpose: Via the nuclear charge defines the chemical element associated with the present species.

`Z` is a real number (the nuclear charge).

`Z` is usually an integer number. However, partial (non-integer) charges are also possible.

Fractional `Z` can be useful, for example, for a stoichiometric hydrogen-like termination of a compound semiconductor slab (e.g., in a III-V compound, the valence of the connecting element would be mimicked by a fractionally charged H of charge 0.75 or 1.25).

Fractional `Z` can also be useful to distribute a compensating charge for an electronically charged periodic supercell calculation. In electronic charged periodic systems, a compensating background charge is always implicit. This is often accomplished by introducing an implicit homegeneous charged background density. However, the choice of such a jellium background is often anything but ideal. For instance, in a surface slab calculation, part of this compensating charge will be located in the vacuum. In such cases, it may be better to place the compensating charge in the system explicitly and "by hand". One good way to do this is to place the compensating charges on certain nuclei.[?]

If you add a fractional `Z` to a species_default, you will have to take care to modify the `valence` tags to reflect the exact opposite charge, creating an overall neutral spherical free atom as far as the `valence` occupation numbers in the species definition go.

`species` **sub-tag: `element`** (control.in)

Usage: `element symb`

Purpose: Chemical element associated with the species.

`symb` is a string (max. 2 characters) that corresponds to the symbol of the chemical element. Default: see below.

The purpose of this tag is to specify the chemical identity of a species in the rare cases

when it cannot be determined from Z because it has been set to a non-integer value. In particular, when Z is more than 0.25 from the nearest integer number, the species element must be set with the `element` tag. Currently, this value is used only by the van der Waals routines, but the requirement above must be satisfied for any calculation.

3.3 Electronic structure: Exchange, correlation (incl. DFT+U), and excited states

A key choice *required* in every electronic structure calculation is the treatment of the required electronic structure: Exchange, correlation, and potentially quasiparticle energies, e.g., after a GW correction.

We here summarize the general options available regarding the choice of the electronic structure method. In addition, an important question is *which* electrons in the structure are treated at which level. For most practical purposes, FHI-aims treats all electrons in an equivalent way, but for some special cases, frozen-core treatments may be useful: at present, one may compute the correlation energy of only the *valence* but not the *core* electrons in second-order Møller-Plessett (MP2) perturbation theory.

For any method requiring the two-electron Coulomb operator explicitly (these include hybrid functionals, Hartree-Fock, MP2 or RPA perturbation theory, GW corrections, etc.) we note that an auxiliary basis is required to expand the Coulomb matrix (four basis functions $\equiv O(N^4)$ matrix elements) into a two-center Coulomb matrix, leading instead to $O(N^3)$ additional overlap matrix elements. The choice of this auxiliary basis ("product basis") is described in more detail in Sec. 3.20 and Ref. [?].

A note on "post-s.c.f" RPA-based methods

The algorithms for post-DFT methodologies as implemented in FHI-aims are detailed in Ref. [?]. Here we only briefly recapitulate the key ingredients behind the increasingly popular "RPA and beyond" methods as implemented in FHI-aims. The standard RPA total energy is computed as follows:

$$E_{\text{tot}}^{\text{RPA}} = E_{\text{tot}}^{\text{DFT}} - E_{\text{xc}}^{\text{DFT}} + E_{\text{x}}^{\text{EX}} + E_{\text{c}}^{\text{RPA}} \,. \tag{3.1}$$

$E_{\text{tot}}^{\text{DFT}}$ is a pre-computed self-consistent DFT total energy obtained from LDA, GGA, or hybrid functional calculations. $E_{\text{xc}}^{\text{DFT}}$ is the corresponding exchange-correlation contribution. E_{x}^{EX} and $E_{\text{c}}^{\text{RPA}}$ are the exact-exchange energy, and the RPA non-local correlation evaluated using the pre-determined Kohn-Sham or generalized Kohn-Sham eigenorbitals and eigenenergies.

Recently, several correction schemes to RPA have been proposed. FHI-aims currently provides the (renormalized) single excitation (SE) correction [?] and the second-order screened exchange (SOSEX) correction [?]. The renormalized SE (rSE) and SOSEX corrections can be combined. The combined scheme is called "renormalized 2nd-order perturbation theory" (rPT2) [?],

$$E_{\text{tot}}^{\text{rPT2}} = E_{\text{tot}}^{\text{RPA}} + E_{\text{c}}^{\text{SOSEX}} + E_{\text{c}}^{\text{rSE}} \,. \tag{3.2}$$

The "RPA+SE", "RPA+rSE", and "RPA+SOSEX" total energies can be computed similarly by combining the corresponding terms.

A note on "DFT plus U"

In the DFT method with local or semi-local approximations of the XC-functional, (LDA, GGA, etc.) strongly correlated systems like transition metal oxides are poorly described.

The "DFT plus U" method offers an ad hoc correction for strongly correlated systems at negligible computation cost [?].

The present implementation of "DFT plus U" in FHI-aims should be considered experimental, and is not complete in some respects. Please keep this in mind when using the method. That said, it should give physically sensible results. Simply take some care when using it, and please give us feedback if the method works for you (obviously, also if it does not for some reason).

- DFT+U total energies can be obtained in combination with any functional (typically LDA or GGA), by simply adding appropriate `plus_u` tags to the corresponding species.
- Total energy gradients ("forces") are not provided.
- The implementation does not yet offer self-consistent determination of the U parameter, so this needs to be supplied by hand.
- Finally, the orbitals on which we project are the somewhat extended free-atom like orbitals, defined with the usual cutoff potential of the remaining calculation. While somewhat arbitrary, it would be useful to be able to project onto more localized orbitals, but this option is not implemented yet.

Tags for general section of `control.in`:

Tag: `frozen_core` (control.in)

Usage: `frozen_core first_orbital`

Purpose: Allows to compute the MP2 correlation energy without the contribution arising from low-lying occupied orbitals.

`first_orbital` is the integer number of the first molecular orbital that is *included* in the computation of the MP2 correlation energy.

This keyword applies only to the calculation of the MP2 correlation energy (if requested). It does not imply a frozen-core treatment anywhere else.

In a nutshell, this is a simple way to exclude the large contribution from certain core electrons to the MP2 correlation energy. This contribution is mostly systematic, and therefore tends to cancel in energy differences. However, it is also the hardest to compute unless specialized basis sets are invoked that "know" about core correlation; it may thus be the source of a large systematic error that also cancels if excluded from the beginning. For consistency between different calculations, the number of excluded "core" orbitals must be readjusted between calculations with different numbers of atoms.

Tag: `frozen_core_postscf` (control.in)

Usage: `frozen_core_postscf valence_shell_number`

Purpose: Alternative way to specify the valence shells in the frozen-core algorithm for MP2, RPA and rPT2 methods. *This keyword is valid for elements from H (1) to Rn (86).*

`valence_shell_number` is the number of valence shells which are taken into account in the frozen-core MP2, RPA or rPT2 calculations.

Compared to the keyword `frozen_core`, `frozen_core_postscf` is more friendly, especially for large systems, as you don't need to count by hand which is the first valence orbital in the frozen-core algorithm.

For example, *valence_shell_number=2* means that at most two outer shells are taken as valence shells in the frozen-core calculations:

element	core shells	valence shells
H, He	–	$1s$
Li-Ne	–	$1s2s2p$
Na-Ar	$1s$	$2s2p3s3p$
K-Kr	$1s2s2p$	$3s3p3d4s4p$

Tag: `hybrid_xc_coeff` (control.in)

Usage: `hybrid_xc_coeff value`

Purpose: If set, will modify the (Hartree-Fock) exact exchange mixing parameter in a given hybrid XC functional. *No effect* if specified with a simple LDA / GGA type functional.

`value` is a real number (usually between zero and one) that specifies the degree of exact exchange admixture.

If (and only if) a hybrid functional is specified using the `xc` keyword, `hybrid_xc_coeff` allows to change the Hartree-Fock mixing parameter to a different, given `value`. For example, the mixing parameter in `pbe0` could be specified away from its literature value, α=0.25. No effect for `xc` functionals that do not have any Hartree-Fock exchange admixed in the first place.

Obviously, this option is only useful for test purposes and does change the definition of any functional away from its literature value. Handle with care.

Tag: `hse_unit` (control.in)

Usage: `hse_unit character`

Purpose: Required clarification of units for the `hse06` `xc` functional.

`value` is a character, either 'a' or 'A' (for Å^{-1}) or 'b' or 'B' (for [bohr radius]$^{-1}$).

The `hse06` functional comes with a screening parameter ω which must be specified

explicitly (see the xc keyword for a detailed explanation). Unfortunately, different codes and authors appear to have adopted different conventions for ω – either $Å^{-1}$ or [bohr radius]$^{-1}$. To avoid any possible confusion when using HSE06 in FHI-aims, we therefore only run hse06 if the unit has been explicitly specified, using the above keyword. We apologize for the inconvenience, but the risk of an innocent misunderstanding is rather high in the present case.

Tag: `plus_u_petukhov_mixing` (control.in)

Usage: `plus_u_petukhov_mixing mixing_factor`

Purpose: *Experimental—only for DFT+U.* Allows to fix the mixing factor between AMF and FLL contribution of the double counting correction [?].

`mixing_factor` is a floating point value, specifying the mixing ratio between 0.0 and 1.0. A value of 0.0 selects the Around Mean Field (AMF) contribution. A value of 1.0 selects the Fully Localized Limit (FLL). If unspecified, the value is determined self-consistently according to Ref. [?].

There are two common schemes for dealing with the double counting problem in DFT+U: The AMF method assumes that the effect of the DFT+U term on the actual occupations remains small, so that the occupations can be assumed to be equal within each shell for the purpose of the double counting correction. The FLL method, on the other hand, assums a maximal effect of the DFT+U term on the occupation numbers, handling double counting correctly in the case that all orbitals with in the shell are either fully occupied or empty. The self consistent mixing of both limits improves the handling of the intermediate range (see Ref. [?]).

Tag: `qpe_calc` (control.in)

Usage: `qpe_calc selfenergy-type`

Purpose: If set, specifies which self-energy should be used for a quasiparticle correction of single-particle eigenvalues.

`selfenergy-type` is a keyword (string) which specifies the selfenergy approximation used.

Note that quasiparticle corrections (GW, MP2) are currently possible only for cluster geometries (no periodic boundary conditions).

After the normal self-consistency cycle for a given exchange-correlation functional (set using the xc keyword) is complete, qpe_calc can be used to specify a perturbative quasiparticle correction to be applied as a post-processing step. Valid self-energy options `selfenergy-type` are:

- `gw` : Perturbative G_0W_0-type self-energy, where both the Green's function G_0 and the screened Coulomb interaction W_0 are computed only once, based on the self-consistent DFT or Hartree-Fock ground state eigenvalues and eigenfunctions.

- `ev_scgw` Perturbative G_0W_0-type self-energy, where self-energy is evaluated with partial self-consistency in the eigenvalues. Molecular orbitals are kept unchanged from the preliminary calculation. For true self-consistent GW, see the `sc_self_energy` further below.
- `mp2` : Perturbative MP2-type self-energy, based on the self-consistent DFT or Hartree-Fock ground state eigenvalues and eigenfunctions.

Tag: `sc_self_energy` (control.in)

Usage: `sc_self_energy self-consistent-scheme`

Purpose: If set, specifies the scheme adopted for the self-consistent calculation of the many-body self-energy.

`selfenergy-type` is a keyword (string) which specified the self-consistent approach used in the calculation.

Note that self-consistent GW calculation (sc-GW, sc-GW_0) are currently possible only for cluster geometries (no periodic boundary conditions).

After the normal self-consistency cycle for a given exchange-correlation functional (set using the `xc` keyword) is complete, `sc_self_energy` can be used to specify a self-consistent scheme for the calculation of the GW self-energy. The output consists of the total energy calculated from the Galitskii-Migdal formula, an output file (`spectrum_sc.dat` for spin unpolarized, `spectrum_sc_up.dat` and `spectrum_sc_do.dat` for spin up and down respectively in the case of spin polarized calculation) containing the spectral function calculated from the self-consistent Green's function. At the end of the calculation, the output include the dipole moment evaluated from the self-consistent density.

Currently implemented self-consistent methods are:

- `scgw` : Calculate the Green's function by solving until full self-consistency the Dyson's equation by using a self-energy in the GW approximation.
- `scgw0` : Solve self-consistently the Dyson's equation with the self-energy in the GW_0 approximation. Differently from fully self-consistent GW, in this case the screened Coulomb interaction is kept fixed at the RPA level.

Tag: `scgw_mix_param` (control.in)

Usage: `scgw_mix_param` α

Purpose: Define the linear mixing coefficient α, for the mixing of the Green function at each iteration of the self-consistent GW calculation. This keyword only produces an effect if `sc_self_energy` is set.

Tag: `scgw_it_limit` (control.in)

Usage: `scgw_it_limit` N

Purpose: Set the maximum number N of iteration of the Dyson equation in a self-consistent GW calculation. The default value is set to $N = 30$. This keyword only produces an effect if `sc_self_energy` is set.

Tag: `scgw_print_all_spectrum` (control.in)

Usage: `scgw_print_all_spectrum`

Purpose: Enables the print out of the spectral function each iteration of the self-consistent GW calculation. The spectrum is printed to the file `sp_ImG<N>.dat`, where <N> is number of iteration of the Dyson equation. This keyword only produces an effect if `sc_self_energy` is set.

Tag: `rpa_along_ac_path` (control.in)

Usage: `rpa_along_ac_path rpa_along_ac_path_grid`

Purpose: Calculate the RPA-approximated potentials along the adiabatic-connection path.

`rpa_along_ac_path_grid` is the number of potentials you want to sampling along the adiabatic-connection path.

The standard RPA method is a adiabatic-connection advanced DFT method, which integrates the contribution along the adiabatic-connection path analytically. This keyword `rpa_along_ac_path` allows you to unpack the adiabatic-connection path in the RPA approximation in detail.

Tag: `printout_dft_components` (control.in)

Usage: `printout_dft_component given_dft_method`

Purpose: Evaluate the XC contributions of a given DFT method based on SCF converged KS (or HF) orbitals.

`given_dft_method` is the name of the DFT method you want to investigate. At present, only two GGA methods (PBE and BLYP) are avaiable.

This keyword `printout_dft_components` is repeatable in the contril.in allowing to inspect several DFT methods in one task.

Tag: `scs_mp2_parameters` (control.in)

Usage: `scs_mp2_parameters pT pS`

Purpose: For MP2 correlation energies, allows to perform spin-component scaled MP2.

`pT` is the scaling parameter for the spin-up-spin-up (triplet) contribution.
`pS` is the scaling parameter for the spin-up-spin-down (singlet) contribution.

The MP2 correlation energy (total_energy_method mp2 or xc mp2) can be separated into a sum of triplet (spin-up-spin-up) and singlet (spin-up-spin-down) two-electron terms:

$$E_{\text{corr,MP2}} = E_T + E_S \,. \tag{3.3}$$

Grimme [?] pointed out that empirical scaling factors p_T and p_S can be introduced and fitted to improve the accuracy of MP2 results compared to quantum-chemical benchmark methods:

$$E_{\text{SCS,MP2}} = p_T E_T + p_S E_S \,. \tag{3.4}$$

For example, p_T=1/3 and p_S=6/5 are employed to obtain the reaction energies of Table I in Ref. [?].

Tag: `total_energy_method` (control.in)

Usage: total_energy_method type

Purpose: If set, specifies an exchange-correlation method *for post-processing only*, after the scf cycle is complete.

type is a keyword (string) which specifies the chosen post-processing exchange-correlation method.

After the regular scf cycle is complete for a given exchange-correlation method as given by the xc tag, the resulting Kohn-Sham orbitals and eigenvalues are used to recalculate *only* the exchange-correlation energy, and only once (i.e., perturbative post-processing). Valid post-processing options type are:

- C6_coef : Molecular C_6 dispersion coefficients at the MP2 / RPA level will be calculated after the s.c.f. calculation. (This functionality is somewhat experimental, be sure to check for consistency.)
- hf or HF: Calculate Hartree-Fock exchange on the given orbitals.
- ll_vdwdf : The nonlocal part of correlation energy is calculated using the van der Waals density functional proposed by M. Dion *et al.* [?] and the total correlation energy will be re-evaluated as proposed in their paper. For details about additional Tags needed for the calculation, please visit Sec. 3.19. Note that an alternative implementation by the Helsinki group is available as well, the present keyword is not your only option.
- m06-l or M06-L: Truhlar's optimized local meta-GGA of the "M06" suite of functionals. [?]
- m06 or M06: Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals; with 27% exact exchange. [?] Currently supported only for non-hybrid functionals specified through the xc tag.
- m06-2x or M06-2X: Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals, with double contribution (54%) from the hartree-fock exact exchange. [?] Currently supported only for non-hybrid functionals specified through the xc tag.

- `m06-hf` or `M06-HF`: Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals, with 100% exact exchage contribution. [?] Currently supported only for non-hybrid functionals specified through the `xc` tag.

- `m08-hx` or `M08-HX`: Truhlar's optimized hybrid meta-GGA of the "M08" suite of functionals, with 52.23% contribution from the hartree-fock exact exchange. [?] Currently supported only for non-hybrid functionals specified through the `xc` tag.

- `m08-so` or `M08-SO`: Truhlar's optimized hybrid meta-GGA of the "M08" suite of functionals, with 56.79% contribution from the hartree-fock exact exchange. [?] Currently supported only for non-hybrid functionals specified through the `xc` tag.

- `m11-l` or `M11-L`: Truhlar's optimized range-separated local meta-GGA of the "M11" suite of functionals. [?]

- `mp2` : The correlation energy is calculated in second-order Møller-Plesset perturbation theory (MP2), with Hartree-Fock added for the exchange part. *Note added in March 2016: A periodic implementation of MP2 is available but, at the time of writing, computationally extremely expensive. If you decide to use it, please keep in mind that the periodic version is included here as a matter of protocol but is not yet optimized to be fully usable in production calculations.*

- `pbe_vdw` : Evaluates the van der Waals density functional proposed by M. Dion *et al.* [?] with the methodology of Sec. 3.19.2. (Uses PBE exchange.) *This is* not *the Tkatchenko-Scheffler correction [?]. If you are looking for Tkatchenko-Scheffler, please use the keyword* *vdw_correction_hirshfeld* *instead.*

- `revpbe_vdw` : As pbe_vdw but uses revpbe instead of pbe for the exchange. *This is also* not *the Tkatchenko-Scheffler correction [?]. If you are looking for Tkatchenko-Scheffler, please use the keyword* *vdw_correction_hirshfeld* *instead.*

- `revtpss` : Meta-GGA revTPSS functional, thanks to E. Fabiano and F. Della Sala. [?, ?]

- `nlcorr` : Only the non-local correlation term of the pbe_vdw or revpbe_vdw is calculated and added to the total energy. *And this is still* not *the Tkatchenko-Scheffler correction [?]. If you are looking for Tkatchenko-Scheffler, please use the keyword* *vdw_correction_hirshfeld* *instead.*

- `rpa` : The RPA total energy as defined in Eq. (3.1) will be calculated. When this option is specified, the SE and rSE corrections to RPA are also evaluated. The total enegies computed with the RPA, RPA+SE, and RPA+rSE schemes are listed in items `"RPA total energy"`, `"PRA+SE total energy"`, and `"RPA+rSE (full) total energy"` respectively in the output file.

- `rpa+2ox` : Just RPA plus second-order exchange (not screened). Likely only useful for testing / benchmarking, use rpt2 for completeness.

- `rpa+sosex` : Just RPA plus second-order screened exchange. Likely only useful for testing / benchmarking, use rpt2 for completeness.
- `rpt2` : The rPT2 total energy as defined in Eq. (3.2) will be calculated. When this option is specified, the "RPA+SOSEX" total energy without the rSE correction will also be printed out in the output file.
- `scan` or `SCAN`: "Strongly Constrained and Appropriately Normed Semilocal Density Functional," i.e., the SCAN meta-GGA functional by Sun, Ruzsinszky, and Perdew.[?]
  Note: This functionality is very new at the time of the FHI-aims 2016 release in March, 2016. It has been tested, but please treat it with appropriate caution for a while.
- `tpss` : Meta-GGA TPSS functional, thanks to E. Fabiano and F. Della Sala. [?]
- `tpssloc` : Meta-GGA TPSSloc functional, thanks to E. Fabiano and F. Della Sala. Reference: L.A. Constantin, E. Fabiano, F.Della Sala, Phys. Rev. B. 86, 035130 (2012).
- `xyg3` : "XYG3" double-hybrid functional[?], which is defined only for a self-consistent B3LYP reference, i.e., `xc b3lyp` is mandatory. *Note* that double-hybrid functionals include MP2 components. When using the *tier* basis sets, you must use a counterpoise correction of energy differences to get reliable results.
- `xdh-pbe0` : "xDH-PBE0" double-hybrid functional[?], which is defined only for a self-consistent PBE0 reference, i.e., `xc pbe0` is mandatory. *Note* that double-hybrid functionals include MP2 components. When using the *tier* basis sets, you must use a counterpoise correction of energy differences to get reliable results.

Note that some of the correlation methods available here are only supported for cluster geometries at this time. *Note* also that when advanced correlation methods (e.g. rpa, rpt2, xyg3, xdh-pbe0 and mp2) are used for binding energy calculations, a **counterpoise correction** should always be performed with the default NAO basis sets in FHI-aims to get reliable results, since the basis set superposition error (BSSE) for these correlation methods is significant. For these advanced correlation methods, the sequence of NAO valence-correlation consistent basis sets (*NAO-VCC-nZ*[?]) is a better choice, which reduces the basis set incompleteness error, including BSSE, with increasing the basis size, and especially enables to approach the completeness-basis-set limit with the aid of extrapolation scheme.

Tag: `use_2d_corr` (control.in)

Usage: `use_2d_corr bool`

Purpose: Specifies whether to use the efficient 2D distribution of the MO based three index arrays where possible. Otherwise, stick to the old 1D distribution in all cases.

Default: `.true.`

Tag: `xc` (control.in)

Usage: `xc xc-type [value]`

Purpose: Specifies the exchange-correlation approach used for self-consistent DFT / Hartree-Fock.

Default: `pw-lda`

`xc-type` is a keyword (string) which specifies the chosen exchange-correlation functional.

`value` is a real parameter needed only for some functionals (e.g., `hse06`).

FHI-aims provides a wide range of current exchange-correlation options, ranging from local-density and generalized-gradient approximations (LDAs and GGAs) via hybrid functionals and Hartree-Fock to two-electron treatments of the correlated many-body system, such as second-order Møller-Plesset (MP2) theory and the random-phase approximation (RPA). The following choices for the `xc-type` option are currently available:

- Local-density approximation (different parameterizations):
  - `pw-lda` : Homogeneous electron gas based on Ceperley and Alder [?] as parameterized by Perdew and Wang 1992 [?]. *Recommended LDA parameterization.*
  - `pz-lda` : Homogeneous electron gas based on Ceperley and Alder [?], as parameterized by Perdew and Zunger 1981 [?].
  - `vwn` : LDA of Vosko, Wilk, and Nusair 1980 [?].
  - `vwn-gauss` : LDA of Vosko, Wilk, and Nusair 1980, *but based on the random phase approximation* [?]. Do not use this LDA unless for one specific reason: In the B3LYP implementation of the Gaussian code, this functional is allegedly used instead of the correct VWN functional. It is therefore now present in many reference results in the literature, and also available here for comparison.
- Generalized-gradient approximations:
  - `am05` : GGA functional designed to include surface effects in self-consistent density functional theory, according to Armiento and Mattsson [?]
  - `blyp` : The BLYP functional: Becke (1988) exchange [?] and Lee-Yang-Parr correlation [?].
  - `pbe` : GGA of Perdew, Burke and Ernzerhof 1997 [?].
  - `pbeint` : PBEint functional of Ref. [?]
  - `pbesol` : Modified PBE GGA according to Ref. [?].
  - `rpbe` : The RPBE modified PBE functional according to Ref. [?].
  - `revpbe` : The revPBE modified PBE GGA suggested in Ref. [?].

  - `pw91_gga` : GGA according to Perdew and Wang, usually referred to as "Perdew-Wang 1991 GGA". This GGA is most accessibly described in References 26 and 27 of Ref. [?]. Note that the often mis-quoted reference [?] does *not*(!) describe the Perdew-Wang GGA but instead only the correlation part of the local-density approximation described above.

- Meta-generalized gradient approximations:

  - `m06-l` : Truhlar's optimized meta-GGA of the "M06" suite of functionals. [?]
  - `m11-l` : Truhlar's optimized range-separated local meta-GGA of the "M11" suite of functionals. [?]
  - `revtpss` : Meta-GGA revTPSS functional of Ref. [?, ?].
  - `tpss` : Meta-GGA TPSS functional of Ref. [?]
  - `tpssloc` : Meta-GGA TPSSloc functional, thanks to E. Fabiano and F. Della Sala. L.A. Constantin, E. Fabiano, F.Della Sala, Ref. [?].
  - `scan` or `SCAN`: "Strongly Constrained and Appropriately Normed Semilocal Density Functional," i.e., the SCAN meta-GGA functional by Sun, Ruzsinszky, and Perdew.[?]
    Note: This functionality is very new at the time of the FHI-aims 2016 release in March, 2016. It has been tested, but please treat it with appropriate caution for a while.

- Hartree-Fock and hybrid functionals (including non-local exchange): *Please also see Secs. 3.20 and 3.21 for related keywords and technical hints.*

  - `b3lyp` : "B3LYP" hybrid functional as allegedly implemented in the Gaussian code (i.e., using the RPA version of the Vosk-Wilk-Nusair local-density approximation, see Refs. [?, ?] for details). Note that this is therefore *not* exactly the same B3LYP as originally described by Becke in 1993.
  - `hf` : Hartree-Fock exchange only.
  - `hse03` : Hybrid functional as used in Heyd, Scuseria and Ernzerhof [?, ?]. In this functional, 25 % of the exchange energy is split into a short-ranged, screened Hartree-Fock part, and a PBE GGA-like functional for the long-range part of exchange. The remaining 75 % exchange and full correlation energy are treated as in PBE. As clarified in Refs. [?, ?], two different screening parameters were used in the short-range exchange part and long-range exchange part of the original HSE functional, respectively:
    Screened Hartree-Fock exchange: $\omega_{\mathrm{HF}} = 0.15/\sqrt{2}$
    Screened PBE-like exchange: $\omega_{\mathrm{PBE}} = 0.15 \times 2^{1/3}$
    Following the notation of Ref. [?], the 'hse03' functional in FHI-aims reproduces these original values exactly.
  - `hse06` : Hybrid functional according to Heyd, Scuseria and Ernzerhof [?], following the naming convention suggested in Ref. [?]. In this case, the additional option `value` is needed, representing the single real, positive screening parameter `omega` (ω) as clarified in Ref. [?]. In this functional, 25 % of

the exchange energy is split into a short-ranged, screened Hartree-Fock part, and a PBE GGA-like functional for the long-range part of exchange. The remaining 75 % exchange and full correlation energy are treated as in PBE. *In the literature, the unit for ω is either $Å^{-1}$ or (bohr radius)$^{-1}$, depending on the code, authors, and their favorite convention. To avoid any confusion, a separate keyword* `hse_unit` *must be specified in* `control.in`*, specifying either $Å^{-1}$ ('A') or bohr^{-1} ('b'). The code will no longer run without this explicit clarification. A correct calling syntax example is therefore:*
`xc hse06 0.11`
`hse_unit bohr-1`[1]
or similar.
A few comments on typical choices for ω in the earlier literature:
The original value of 0.15 bohr^{-1} by Heyd, Scuseria and Ernzerhoff 2003 [?] was never true - see their 2006 erratum. In FHI-aims, the 'hse03' functional implements their actual choice.
Krukau, Vydrov, Izmaylov and Scuseria 2006 [?] clarify the distinction between 'hse03' and 'hse06' (in addition to the Erratum mentioned above). Their conclusion is that omega=0.11 bohr^{-1} is a reasonable choice.
Vydrov, Heyd, Krukau and Scuseria in 2006 [?] appear to favor omega=0.25 bohr^{-1}, but with a mixing parameter (keyword `hybrid_xc_coeff`) of 0.5 for the short-range exchange. (The default for `hybrid_xc_coeff` in FHI-aims is 0.25, i.e., only a quarter of HF-like exchange.)
You get the idea. As much as we would like to, we can not specify a single omega parameter for hse06 by default – the choice is up to you. Apologies for the inconvenience.

- `pbe0` : PBE0 hybrid functional [?], mixing 75 % GGA exchange with 25 % Hartree-Fock exchange.
- `pbesol0` : Hybrid functional in analogy to PBE0 [?], except that the PBEsol [?] GGA functionals are used, mixing 75 % GGA exchange with 25 % Hartree-Fock exchange.

- Hybrid Meta-generalized gradient functionals (including non-local exchange): *Please also see Secs. 3.20 and 3.21 for related keywords and technical hints. Currently the non-local exchange contribution is fixed in all implementations due to the parameterised nature of these density functionals.*

  - `m06` : Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals; with 27% exact exchange. [?]
  - `m06-2x` : Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals, with double contribution (54%) from the hartree-fock exact exchange. [?]
  - `m06-hf` : Truhlar's optimized hybrid meta-GGA of the "M06" suite of functionals, with 100% exact exchage contribution. [?]

[1] The `hse_unit` flag reads only the first character. Thus this is equivalent to `hse_unit b` (case insensitive).

  - `m08-hx` : Truhlar's optimized hybrid meta-GGA of the "M08" suite of functionals, with 52.23% contribution from the hartree-fock exact exchange. [?]
  - `m08-so` : Truhlar's optimized hybrid meta-GGA of the "M08" suite of functionals, with 56.79% contribution from the hartree-fock exact exchange. [?]
  - `m11` : Truhlar's optimized range-separated local meta-GGA of the "M11" suite of functionals [?]. The range-separation variable is also hardcoded in this implementation with $\omega = 0.25$ bohr^{-1}.

- Doule-hybrid functionals (including non-local exchange and correlation): Double-hybrid functionals are emerging quickly in the last decade. "*double-hybrid*" here means that the exchange functional mixes LDA(and/or GGA) exchange with "Hartree-Fock like exact exchange". Meanwhile, the correlation functional is composed of both conventional LDA(and/or GGA) correlation and second-order perturbation energy. Doubly-hybrid functionals are "*semi-empirical*", generally including several empirical parameters determined by optimizing against one or several well-chosen databases. Double-hybrid functionals show a remarkable improvement over conventional (hybrid-)GGAs in the description of heats of formation, bond dissociation enthalpies, reaction barrier heights and weak interactions of the main group elements. Doulbe-hybrid functionals have become new leading actors in the field of computational chemistry.

  - `xyg3` : Double-hybrid functional XYG3, containing 80.33% Hartree-Fock exchange and 32.11% second-order perturbation energy [?].
  - `xdh-pbe0` : Double-hybrid functional xDH-PBE0, containing 83.51% Hartree-Fock exchange and 52.42% opposite-spin second-order perturbation correlation [?].

- Some specific correlated methods: *Only a subset. For many correlated methods that can be used as non-selfconsistent perturbative post-processing methods after an initial s.c.f. calculation, see the* `total_energy_method` *keyword. Most of these are not available for periodic geometries, or, if at all, in a very experimental state.*

  - `mp2` : Self-consistent Hartree-Fock, followed by a second-order Møller-Plesset perturbative addition of the correlation energy. Note that the `frozen_core` keyword can be used to specify if and which low-lying states should be excluded from the correlation energy. For spin-component scaled MP2 [?], see keyword `scs_mp2_parameters` .

    Note that when mp2 is used for binding energy calculations, a **counterpoise correction** should always be performed to get reliable results, since the basis set superposition error (BSSE) for these correlation methods is significant. *Note added in March 2016: A periodic implementation of MP2 is available but, at the time of writing, computationally extremely expensive. If you decide to use it, please keep in mind that the periodic version is included here as a matter of protocol but is not yet optimized to be fully usable in production calculations.*

- `screx` : *experimental!* Self-consistent, screened Hartree-Fock exchange only. The Coulomb operator is screened as:

$$\frac{1}{r-r'} \rightarrow \frac{1}{\varepsilon(r,r')} \cdot \frac{1}{r-r'} \tag{3.5}$$

  $\varepsilon(r,r')$ is the non-local *microscopic* dielectric function, obtained in the $\omega \rightarrow 0$ frequency limit of the random-phase approximation (RPA). See Ref. [?] for details.

- `cohsex` : *experimental!* Self-consistent screened exchange plus Coulomb-hole (COH) correlation. See Ref. [?] for details.

- Method of non-local correlation using the "van der Waals density functional" (vdw-DF) as presented by Dion and coworkers in Ref. [?]. Two options are available for the exchange part:

  - `pbe_vdw` : the functional with pbe exchange
  - `revpbe_vdw` : the functional with revpbe exchange

  Note that this keyword is **not** the correction due to Tkatchenko and Scheffler 2009 [?]. To activate the Tkatchenko-Scheffler correction instead, use the `vdw_correction_hirshfeld` keyword. The functional by Dion *et al.* is a very different functional. As implemented here, it is also *much* more expensive than the Tkatchenko-Scheffler correction. To use the functional by Dion *et al.*, please review the numerical options described in Sec. 3.19.2.

Note that our version of the Coulomb operator (which is the basis for Hartree-Fock exchange also in hybrid functionals, as well as MP2 theory) is based on an auxiliary basis in what is known as *resolution of the identity* (Refs. [?, ?, ?, ?] and others). While our default settings should be safe, you may wish to consult Sec. 3.20 for particulars regarding this auxiliary basis.

Note also that some different *perturbative* exchange-correlation treatments *for post-processing* (after a self-consistent DFT or HF calculation is complete) may be invoked using the tag `total_energy_method`. Likewise, perturbative postprocessing for single-quasiparticle energies through a self-energy formalism (e.g., GW) is reached by specifying the `qpe_calc` tag and its options.

Right now, the correlated beyond-hybrid and beyond-meta methods are not implemented on top of the HSE03 or HSE06 functionals.

Subtags for *species* tag in `control.in`:

`species` **sub-tag:** `plus_u` (control.in)

Usage: `plus_u n l U`

Purpose: *Experimental—only for DFT+U.* Adds a +U term to one specific shell of this species.

`n` the (integer) radial quantum number of the selected shell.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...) of the selected shell.
`U` the value of the U parameter, specified in eV.

This implementation of DFT+U is based directly on the basis functions available within FHI-aims. This option selects one specific *atomic* shell of this species and adds the a rotationally invariant term with the specified fixed prefactor U to the Hamiltonian. The implementation follows the prescription in Ref. [?], based on the *dual* occupation numbers. The double counting term is handled through the mixed term proposed by Petukhov (see `plus_u_petukhov_mixing`).

3.4 Specifying the basis (functions, empty sites, k-points, ...)

Among the technical choices in FHI-aims, the choice of the basis set is by far the most important one, both regarding the efficiency and the desired accuracy of a calculation. The shape and details of the basis sets used are thus kept as obvious as possible to the user. At the same time, nobody should be required to type in an entire basis set plus additional specifications from scratch just to run a production calculation.

As described in Ref. [?], the basis functions of FHI-aims take the format

$$\phi(\boldsymbol{r}) = u(r)/r \cdot Y_{lm}(\theta, \phi) \tag{3.6}$$

in spherical coordinates (r, θ, ϕ) with respect to a given atomic center. Each radial function $u(r)$ is numerically tabulated on a dense logarithmic radial grid, and evaluated as a cubic spline function in other parts of the code. Finally, most radial function types are subject to a cutoff potential of radial with w, ensuring that $u(r)$=0 for $r > r_{\text{cut}} = r_{\text{onset}} + w$.

In periodic calculations, the full basis specification additionally includes the k-point grid for Bloch functions in the first Brillouin zone. Unlike in many other implementations, this is *not* a performance-critical setting in FHI-aims, and should be set to a well converged value if possible.

The recommended approach to basis sets in FHI-aims is twofold:

- *First*, obtain the basic description of each required element by copy-pasting one of the preconstructed `species_default` files into your `control.in` file. The preconstructed `species_default` files address all standard specifications associated with a single `species` , including the integration grids, the Hartree potential, and most importantly the basis set.
- *Second*, **edit the copy-pasted species_defaults file** to match your specific accuracy and efficiency requirements. For the basis set, this is done by adjusting the species-dependent keywords described below. Most importantly, *complete* basis sets are listed at the end of each `species_default` file. You can increase/decrease the basis set accuracy by successively uncommenting / commenting *tiers* of the basis set. `Note` that each higher *tier* must only be used if *all* lower *tiers* are active. For example, it does *not* make sense to use all tier 2 basis functions if the first tier is not used.

Tags for general section of `geometry.in`:

Tag: `empty` (geometry.in)

Usage: `empty x y z species_name`

Purpose: Specifies the initial location and type of a *site* where *only* the basis functions (but not the nucleus) of a given species are placed.

Restriction: Currently not functional with periodic boundary conditions. The use of this option should be avoided for physical reasons if a structure relaxation is requested.

`x`, `y`, `z` are real numbers (in Å) which specify the atomic position.
`species_name` is a string descriptor which names the element on this atomic position; it must match with one of the species descriptions given in `control.in`.

This allows to place extra basis functions at specified locations outside the actual atoms, e.g., allowing for a counterpoise correction of basis set superposition errors.

Tags for general section of `control.in`:

Tag: `hydro_cut` (control.in)

Usage: `hydro_cut flag`

Purpose: Determines whether or not hydrogenic functions are subject to a numerical cutoff potential.

`flag` is a logical expression, either `.true.` or `.false.` Default: `.true.`

This tag should be kept at the default value unless for testing purposes (e.g., comparing to other codes).

Tag: `k_grid` (control.in)

Usage: `k_grid n1 n2 n3`

Purpose: Sets up an evenly split k-points grid along the reciprocal lattice vectors of a periodic calculation

`n1`, `n2`, `n3` : integer numbers defining the number of k-point splits along the first, second and third reciprocal axis of the first Brillouin zone, respectively

Note that the order of `n1`, `n2`, `n3` must *directly* correspond to the order in which the `lattice_vector` s are listed in `geometry.in`, through the definition and order of the reciprocal lattice. By default, the resulting `k_grid` is centered around the Γ-point, but can be shifted using the `k_offset` keyword below.

The keyword `symmetry_reduced_k_grid` now allows to make use of time-reversal symmetry to reduce the number of k points by a factor of nearly two. This option is the default.

Tag: `k_offset` (control.in)

Usage: `k_offset f1 f2 f3`

Purpose: Defines a possible non-Γ offset for the k-point grid in periodic boundary conditions.

`f1`, `f2`, `f3` : Fractional coordinates (between zero and one) of a k-point offset , in units of the reciprocal lattice vectors. Default: (0., 0., 0.).

Can be used to shift the grid off-Γ for better k-space sampling. For example, (0.5, 0.5, 0.5) together with even $\mathtt{n}i$ for `k_grid` defines a Monkhorst-Pack [?] type grid. See Sec. 4.3 for details.

Tag: `k_points_external` (control.in)

Usage: `k_points_external`

Purpose: Instead of an internally specified k-point grid, allows to specify an externally read k-grid from a file `k_list.in` .

This option is useful to specify an uneven special k-point set [?], etc.

If specified, `k_points_external` expects a separate input file `k_list.in` to be provided in the same working directory as all other input files. The format of `k_list.in` is as follows:

line 1:	`n1 n2 n3`
line 2:	`Nk`
lines 3 - Nk+2:	`k1 k2 k3 weight`

All lines are read as free format. `n1, n2, n3` are integers, specifying the descriptors of a 3D k-point grid for reference only. `Nk` is the (integer) number of k-points following in the file. For each k-point, a separate line is expected, including via `k1, k2, k3` the coordinates of this point in units of the reciprocal lattice vectors, and via `weight` the integration weight of this k-point in all Brillouin zone integrals.

Tag: `symmetry_reduced_k_grid` (control.in)

Usage: `symmetry_reduced_k_grid flag`

Purpose: Determines whether or not to make use of time-reversal symmetry.

`flag` is a logical expression, either `.true.` or `.false.` Default: `.true.`

Tag: `onsite_accuracy_threshold` (control.in)

Usage: `onsite_accuracy_threshold threshold`

Purpose: Issues a warning if the onsite integral of a basis function on the 'radial' integration grid differs from the same onsite integral on the more accurate logarithmic grid by more than `threshold`.

`threshold` is given in eV. Default: 0.03 eV.

See keyword `output onsite_integrands` for the integrals that are actually evaluated.

For "light" settings, you can most likely ignore the associated warning. Possibly, treat it as a reminder to check final results with "tight" settings or similar, just in case.

FHI-aims writes the respective onsite integral values for all its radial functions side by side after the setup of all radial functions is complete. Too large deviations between the calculated values on the radial and logarithmic grids can indicate accuracy problems, which is a particular concern for high-accuracy benchmark calculations. In particular, Gaussian-type orbital basis sets of the Dunning type that are used for high-level reference calculations need grids that are far more accurate than normal NAO basis sets, due to the unphysical wiggles that contracted Gaussian functions with high exponents introduce

near the nucleus.

If the associated warning strikes, the grid accuracy can be improved either using the `radial` keyword or (simpler) the `radial_multiplier` keyword.

However, if `onsite_accuracy_threshold` triggers a warning for "light" settings, most likely you can safely ignore the warning. With light settings, the point may be to do a reduced-accuracy calculation, which should still be safe for its original purposes. Just check the radial grid when in doubt.

If the flag `override_integration_accuracy` is toggled to `.false.`, however, FHI-aims does stop whenever it encounters a radial function whose onsite integrals are not deemed accurate enough by the `onsite_accuracy_threshold` criterion.

Tag: `override_integration_accuracy` (control.in)

Usage: `override_integration_accuracy flag`

Purpose: If set to false, FHI-aims stops calculations for which the onsite integral of any radial function is less accurate than prescribed by keyword `onsite_accuracy_threshold` .

`flag` can be `.true.` or `.false.` Default: `.true.`

Tag: `wave_threshold` (control.in)

Usage: `wave_threshold threshold`

Purpose: Determines the outer radius beyond which a radial function is considered zero.

`threshold` is a small positive number. Default: 10^{-6}.

A radial function is considered zero (not evaluated) beyond the radius where $u(r)$ and its first and second derivatives become smaller than `threshold`. The default is chosen such as to not affect any results at all.

Tag: `calculate_atom_bsse` (control.in)

Usage: `calculate_atom_bsse flag`

Purpose: Allows calculation of the basis set superposition error (BSSE) corrected atomization energy.

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

The atomization BSSE correction for a molecular structure is defined as:

$$\Delta_{ac} = \sum_x [E^x(x) - E^x(sys)] \tag{3.7}$$

where $E^x(x)$ is the energy of the atom *x* calculated using only its basis set and $E^x(sys)$ is its energy calculated with the basis set of the whole structure. The BSSE corrected

total energy is then

$$E^{BSSE} = E^{sys}(sys) + \Delta_{ac} \tag{3.8}$$

The atomization BSSE correction is usually small in the case of the LDA/GGA functionals, but can become significant for methods with explicit correlation like RPA and MP2.

The BSSE corrected atomization energy implemented here, on the other hand, is

$$E_{at}^{BSSE} = E^{sys}(sys) - \sum_{x} E^{x}(sys) \tag{3.9}$$

In the case where relative energies between different conformations of the same molecule are needed, $(E_{at}^{BSSE})_{rel} \equiv (E^{BSSE})_{rel}$.

If the calculation of the full system is performed without spin polarization, the total energy of each atom will also be calculated without spin polarization (and vice versa). In this case, specially when performing the *scf* cycle with HF, symmetry breaking of the electronic configuration of certain atoms may occur, which might lead to wrong conclusions. Support for constraining/breaking the symmetry of the atoms in under development.

The keyword is currently implemented for use with cluster geometries only. Both RPA and MP2 as the `total_energy_method` are supported.

Subtags for *species* tag in `control.in`:

`species` **sub-tag:** `basis_acc` (control.in)

Usage: `basis_acc threshold`

Purpose: Technical cutoff criterion for on-site orthonormalization of radial functions

`threshold` is a small positive real threshold. Default: 10^{-4}.

Before any calculation, all radial functions for a single species are Gram-Schmidt orthonormalized. If the norm of the function after orthonormalization is smaller than `threshold`, that function is omitted.

`species` **sub-tag:** `basis_dep_cutoff` (control.in)

Usage: `basis_dep_cutoff threshold`

Purpose: Basis function dependent adjustment of the confinement potential for this species

`threshold` is either a positive real number, or can be explicitly set `.false.`. Default: 10^{-4}.

If not `.false.`, the onset of the basis confining potential (see `cut_pot` tag below) is adjusted separately for each basis function, such that the norm of this basis function *outside* r_{onset} is smaller that `threshold`. The *maximum* possible onset radius is still given by the value explicitly specified by the `cut_pot` tag.

`species` **sub-tag:** `confined` (control.in)

Usage: `confined n l radius`

Purpose: Adds a confined free-atom like radial function to the basis set.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).
`radius` is the onset radius of the confining potential (in atomic units, 1 a.u. = 0.529177 Å). If the word `auto` is specified *instead* of a numerical value, the default onset radius given in the `cut_pot` tag is used.

The defining potential for this basis function type consists of the non-spinpolarized, self-consistent spherical free-atom potential (possibly itself confined, using the `cut_free_atom` tag), and a confining potential. The shape of the confining potential is the same for all basis functions of a given species, and set using the `cutoff_type` and `cut_pot` subtags.

`species` **sub-tag:** `core` (control.in)

Usage: `core n l`

Purpose: Defines the top "core" shell of the species for this angular momentum.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).

Currently not needed for production calculations, but listed here because the "core" infrastructure is currently being reworked and may see useful additions in the near future. This flag defines which electrons of the species are considered "core" electrons, and which enter as explicit valence electrons.

---

species sub-tag: core_states (control.in)

Usage: `core_states number`

Purpose: Independent determination of the number of states that are core states in the current species.

`number` is an integer number (to be multiplied by 2 for the number of core electrons). Default: 0.

Experimental at present, not needed for any production purposes. See also the `core` keyword. The `core_states` keyword should interact with the `core` keyword, but does not yet, since any associated functionality is still under development. Both keywords are listed here for future reference only.

---

species sub-tag: cut_atomic_basis (control.in)

Usage: `cut_atomic_basis flag`

Purpose: **Only** relevant to decide whether the `basis_dep_cutoff` keyword also applies to atomic-type (minimal) radial functions.

`flag` is a logical expression, either `.true.` or `.false.` Default: `.false.`

This keyword applies only to the setting specified by keyword `basis_dep_cutoff` . Do **not** enable it routinely without thorough testing.

By default, the minimal basis functions in FHI-aims *are* subject to the cutoff potential with the fixed onset specified by the `cut_pot` keyword. However, the more restrictive `basis_dep_cutoff` keyword does *not* apply to the minimal basis by default.

This can be changed by setting `cut_atomic_basis` to `.true.`, but the associated total energy changes are significantly larger than for other basis functions. By default, we therefore do not recommend adding a tighter cutoff to the minimal basis functions at this time. It is, however, possible that this effect is mainly a systematic error between the core states, and after further testing, we may yet choose to enable this keyword *if* we can guaranteed that its use is safe.

---

species sub-tag: cut_pot (control.in)

Usage: `cut_pot onset width scale`

Purpose: Specifies the numerical parameters for the general (default) confinement potential $v_c(r)$ for all basis functions of this species.

`onset` specifies the default onset radius of the cutoff potential, in Å ($v_c(r)$=0 for $r < r_{\text{onset}}$).
`width` specifies the radial width w of the cutoff potential, in Å ($v_c(r)$=∞ for $r > r_{\text{onset}} + w$).
`scale` is a scaling parameter to increase or decrease the numerical value of v_c.

This tag is mandatory, since it specifies `onset`, a critical parameter that allows to tune the efficiency of a calculation for a given target accuracy. Unless reduced by the `basis_dep_cutoff` tag, `onset` is the default onset radius used to construct all valence (minimal) and hydrogen-like basis functions of this species. In addition, any confined free-atom or free-ion like radial functions use this onset radius if `auto` is used in their specification.

Notes: The functional form of $v_c(r)$ can be selected using the `cutoff_type` keyword, and `width` and `scale` apply to this shape. Modifying these latter parameters is usually not necessary for a production calculation, but the `onset` value should be verified at least as a quick numerical check.

`species` **sub-tag: `cutoff_type`** (control.in)

Usage: `cutoff_type identifier`

Purpose: Specifies the functional form of the confinement potential associated with this species.

`identifier` is a string that selects a given confinement potential shape as specified in the code. Default: `exp(1_x)_(1-x)2`.

All confinement potentials in FHI-aims are characterized by the rigorous boundaries $v_c(r)$=0 for $r < r_{\text{onset}}$ and $v_c(r)$=∞ for $r > r_{\text{cut}} = r_{\text{onset}} + w$, where r_{onset} may depend on the basis function, and w is the `width` specified by the `cut_pot` tag. In addition, each shape contains a scaling parameter s, also specified via the `cut_pot` tag.

Available confinement potential shapes (`identifier`) for $r_{\text{onset}} < r < r_{\text{cut}} = r_{\text{onset}} + w$ are:

- `exp(1_x)_(1-x)2` :

$$v_c(r) = \exp\left(\frac{w}{r - r_{\text{onset}}}\right) \cdot \frac{1}{(r - r_{\text{cut}})^2}$$

  (the default in FHI-aims)

- `junquera` :

$$v_c(r) = \exp\left(\frac{w}{r - r_{\text{onset}}}\right) \cdot \frac{1}{(r - r_{\text{cut}})}$$

  (the form originally suggested by Junquera et al. [?])

- `x2_(1-x2)`

$$v_c(r) = (r - r_{\text{onset}})^2 \cdot \frac{1}{(r - r_{\text{cut}})^2}$$

---

species sub-tag: gaussian (control.in)

Usage: `gaussian L N [alpha]`
[alpha_1 coeff_1]
[alpha_2 coeff_2]
[...]
[alpha_N coeff_N]

Purpose: Adds a Gaussian-based radial function to the basis set.

Restriction: This basis function type is not subject to a cutoff potential. It may therefore require a wider `radial_base` integration grid than the standard NAO's in FHI-aims.

`L` is an integer number, specifying the angular momentum
`N` is an integer number, specifying how many primitive Gaussians comprise the present radial function
`alpha` : *If* `N`=1, this is the exponent defining a primitive Gaussian function [in bohr^{-2}].
`alpha_i coeff_i` : *If* `N`>1, $i = 1, \ldots, N$ additional lines specify exponents α_i and expansion coefficients g_i for a non-primitive linear combination of Gaussians.

FHI-aims allows to use Gaussian-based radial functions to compare to existing popular Gaussian-based implementations of quantum chemistry. These functions can either be *primitive* Gaussians,

$$u(r) = \frac{1}{Norm} r^{L+1} \cdot \exp(-\alpha r^2), \tag{3.10}$$

or *non-primitive* linear combinations.

$$u(r) = \frac{1}{Norm} r^{L+1} \cdot \sum_{i=1}^{i=N} g_i \exp(-\alpha_i r^2). \tag{3.11}$$

In existing quantum chemistry codes, Gaussian basis functions can be defined either as *spherical* Gaussians [Eq. (3.10) above], or as *cartesian* Gaussians,

$$\phi(\boldsymbol{r}) = x^k y^m z^n \exp(-\alpha r^2), \quad \text{where} \quad k + m + n = L. \tag{3.12}$$

This behavior can be mimicked using the `pure_gauss` tag (see below). Finally, note that in order to use an *exclusively* Gaussian-based basis set, you must prevent the use of the minimal free-atom like NAO basis functions using the `include_min_basis` tag.

---

species sub-tag: hydro (control.in)

Usage: `hydro n l z_eff`

Purpose: Adds a hydrogen-like radial function to the basis set.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).
`z_eff` scales the radial function as an effective nuclear charge in the defining Coulomb potential z_{eff}/r.
By default, hydrogen-like basis functions in FHI-aims are subject to the numerical confinement potential given by `cutoff_type` and `cut_pot` . Optionally, analytical hydrogen-like functions (no confinement) can be requested using the global `hydro_cut` tag.

`species` **sub-tag:** `include_min_basis` (control.in)

Usage: `include_min_basis flag`

Purpose: Allows to exclude the minimal basis of free-atom valence functions from the basis set.

`flag` is a logical expression, either `.true.` or `.false.` Default: `.true.`

This flag is normally only useful to compare explicitly with basis sets from other methods, usually Gaussian basis sets.

`species` **sub-tag:** `ion_occ` (control.in)

Usage: `ion_occ n l occupation`

Purpose: Specifies the shell occupation of a radially symmetric, non-spinpolarized free ion that defines any ionic basis functions.

Restriction: Only one type of ion can be used to define ionic basis functions for a given species.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).
`occupation` is the number of electrons in the topmost occupied valence shell of this ion.

This tag defines an ionic configuration, but does not actually add any ionic functions to the basis used in the present calculation. Actual basis functions are added by the `ionic` keyword.

Only the topmost valence shell of each angular momentum channel of the ion is specified. All lower-lying shells are assumed to be completely filled for the self-consistent spherical free-ion calculation.

`species` **sub-tag:** `ionic` (control.in)

Usage: `ionic n l radius`

Purpose: Adds a free-ion like radial function to the basis set.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).
`radius` is the onset radius of the confining potential (in atomic units, 1 a.u. = 0.529177 Å). If the word `auto` is specified *instead* of a numerical value, the default onset radius given in the `cut_pot` tag is used.

`species` sub-tag: `pure_gauss` (control.in)

Usage: `pure_gauss flag`

Purpose: If `.true.`, any `gaussian` basis functions for this species will be purely spherical Gaussians.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

See keyword `gaussian` for the distinction between spherical and cartesian Gaussian functions. In short, cartesian and spherical Gaussian functions are equivalent *except* that for a given L, cartesian Gaussians add a so-called angular momentum contamination. If `pure_gauss` is specified, this angular momentum contamination is mimicked by FHI-aims.

Consider the simple (textbook!) three-dimensional harmonic oscillator in quantum mechanics. This can be solved either in cartesian coordinates, or in spherical coordinates. If solves in cartesian coordinates, you will find that there are six degenerate solutions for the principal quantum number 2, five of which correspond to l=2 (d channel), but one of which corresponds to l=0 (s channel). This is the exact angular momentum contamination exhibited by the cartesian definition of a Gaussian basis function.

`species` sub-tag: `valence` (control.in)

Usage: `valence n l occupation`

Purpose: Specifies the shell occupation of the radially symmetric, non-spinpolarized free atom that defines the minimal basis.

`n` is the (integer) radial quantum number.
`l` is a character, specifying the angular momentum (*s*, *p*, *d*, *f*, ...).
`occupation` is the number of electrons in the topmost occupied valence shell of this ion.

Only the topmost valence shell of each angular momentum channel of the atom is specified. All lower-lying shells are assumed to be completely filled for the self-consistent spherical free-atom calculation. The valence occupation must be defined explicitly for each `species` .

The self-consistent free-atom potential generated by this calculation is used to generate all minimal and `confined` basis functions used for this species, after the confining

potential is added.

The self-consistent free-atom calculation can itself be confined by a different confining potential, the onset of which is specified by the `cut_free_atom` keyword.

For DFT-LDA/GGA, the same `xc` functional that is used in the full three-dimensional calculation is also used to define the self-consistent free atom. For any methods involving Hartree-Fock exchange (e.g., hybrid functionals), the free atom is generated using the `pw-lda` LDA functional.

The self-consistent free atom density generated here is also used in the construction of partition functions for the Hamiltonian integrals and the Hartree potential, as well as to build the initial charge density (unless otherwise requested!) and the reference charge density subtracted before constructing the Hartree potential.

3.5 Integration, grids, and partitioning

The next single most important set of specifications required for FHI-aims are the settings regarding the numerical grids used in many contexts. Details regarding the shape and physical motivation behind these grids are given in Refs. [?, ?], and we do not repeat them here.

Notice that the actual required grids may depend on the context of the calculation, for example whether Hartree-Fock, hybrid functionals, and or GW calculations are required. In these cases, some specific settings may require tightening, and some defaults may automatically be chosen differently depending on whether or not those techniques are used.

Specifically, the present section deals with the following topics:

- the 1D logarithmic grid infrastructure required for atomic / free-atom like calculation
- radial and angular grids for all three-dimensional integrals
- shaping the partition functions used to split the full three-dimensional integrals into effective atom-per-atom pieces
- Splitting the grids into different batches for localization / parallelization efficiency

While many of the settings below take safe defaults for standard FHI-aims calculations and need not be modified, it is particularly important to verify the accuracy and efficiency of all three-dimensional integration grids (`radial_base` , `angular_grids` , and associated tags), since these determine the performance of the code. In the `species_defaults` files, (very) safe settings for DFT-LDA/GGA are provided, but for many tasks, may be reduced at very little accuracy loss.

Tags for general section of `control.in`:

Tag: `batch_size_limit` (control.in)

Usage: `batch_size_limit value`

Purpose: Hard upper bound to the number of points in an integration batch.

Restriction: Applies to the `maxmin` and `octree` `grid_partitioning_method` .

`value` is an integer number. Default: 200.

See `grid_partitioning_method` and Ref. [?] for details regarding integration batches.

Tag: `force_lebedev` (control.in)

Usage: `force_lebedev type`

Purpose: Allows to switch between Delley's [?] angular grids (17 digits) and the original angular grids tabulated by Lebedev and Laikov [?, ?, ?] (12 digits)

`type` is a keyword (string), either `original` or `Delley`. Default: `Delley`.

This option need not be changed (or invoked) in any normal runs, since there is no quantitative difference between integrals with Delley's and Lebedev's tabulated grids to our knowledge.

Lebedev's grids may be explicitly invoked when denser angular grids than 1202 points (already very dense!) per radial integration shell around each species are required. In detail, grids with the following numbers of grid points are provided:

- `Delley` : 6, 14, 26, 50, 110, 194, 302, 434, 590, 770, 974, 1202
- `Lebedev` : 6, 14, 26, 38, 50, 86, 110, 146, 170, 194, 302, 350, 434, 590, 770, 974, 1202, 1454, 1730, 2030, 2354, 2702, 3074, 3470, 3890, 4334, 4802, 5294, 5810

These numbers of grid points can be invoked in the subtags of the `angular_grids specified` description for fixed angular grids (the default in the preconstructed `species_defaults` files), and in further tags such as `angular` or `angular_min` .

Tag: `grid_partitioning_method` (control.in)

Usage: `grid_partitioning_method method`

Purpose: Allows to switch between different methods to partition the full (3D) integration grids into batches for individual operations.

`method` is a string, charactrizing one of the different methods outlined below. Default: serial–`maxmin`, parallel–`parallel_hash+maxmin`

Partitioning the integration grid properly can be performance-critical for the expensive grid-based Hamiltonian integration and charge density update steps. Details on these methods are given in Ref. [?]. In particular, we support:

- `maxmin` : The default for serial computations: the "grid-adapted cut-plane" method described in Ref. [?]
- `parallel_hash+maxmin` : The default for all parallel runs. We first hash the grid points to tasks by the geometric location and then run a maxmin algorithm locally in each task.
- `parallel_maxmin` : Memory-parallel implementation of the `maxmin` method. However, the exact implementations requires rather much communication, and has been superseded by `parallel_hash+maxmin`.
- `octree` : The "octree" method described in Ref. [?]
- `parallel_octree` Parallel version of the "octree" method described in Ref. [?]. Only useful for research purposes—superseded by `parallel_hash+maxmin` for all practical applications.
- `octant` Simple partitioning of the grid into "octants" of each radial integration shell arround each atom.

Note that additional parameters may be invoked to specify details for these methods, most importantly `batch_size_limit` and `points_in_batch` .

We mention for completeness that FHI-aims supports further, experimental grid batching methods, including the possibility to link to external libraries. The associated `method` strings are `tetgen+metis`, `qhull+metis`, `nearest+metis`, and `group`. As discussed in Ref. [?], the conceptually simpler `maxmin` method performs as well or even better than these "bottom-up" type approaches, and should be preferred.

Tag: **`min_batch_size`** (control.in)

Usage: `min_batch_size value`

Purpose: Sets the minimum number of points allowed in an integration batch.

Restriction: Affects the `octree` `grid_partitioning_method` method only.

`value` is an integer number. Default: 1.

No need to tweak for standard production calculations. See `grid_partitioning_method` for details regarding integration batches.

Tag: **`partition_acc`** (control.in)

Usage: `partition_acc threshold`

Purpose: If the partition function norm for 3D integrals and the Hartree potential is below `threshold`, that integration point is ignored.

`threshold` is a small positive real number. Default: 10^{-15}.

See Ref. [?] for details regarding "partitioning of unity" of the charge density in integrations and the Hartree potential. The partition functions $p_{\mathrm{at}}(\boldsymbol{r})$ are only calculated if their denominator (the norm; e.g. $\sum_{\mathrm{at}} \rho_{\mathrm{at}}/|\boldsymbol{r} - \boldsymbol{R}_{\mathrm{at}}|^2$ is greater than `value`, else that integration point is ignored.

Notice that this type of partitioning is strictly rigorous for integrands that extend no further than the free-atom like densities used to define our partition functions. This is always true for DFT-LDA/GGA, with NAO's but if you suspect (e.g., with very diffuse Gaussian basis functions) some kind of integration noise, reducing `threshold` may be a good first test.

Tag: `partition_type` (control.in)

Usage: `partition_type type`

Purpose: Specifies which kind of partition table is used for all three-dimensional integrations.

`type` : A string that specifies which kind of partition table is used. Default: `stratmann_smoother`

Usually, this tag need not be modified from the default. See the Computer Physics Communications description of FHI-aims for a description of the numerical integration technique used in FHI-aims.

In brief, each extended three-dimensional integrand is broken down into atom-centered pieces, using a set of localized, atom-centered partition functions:

$$p_{\mathrm{at}}(\boldsymbol{r}) = \frac{g_{\mathrm{at}}(\boldsymbol{r})}{\sum_{\mathrm{at}'} g_{\mathrm{at}'}(\boldsymbol{r})} \,. \tag{3.13}$$

where $g_{\mathrm{at}}(\boldsymbol{r})$ is an atom-centered weight function. The following options for `type` are available:

- `rho_r2` : $g_{\mathrm{at}}(\boldsymbol{r}) = n_{\mathrm{at}}^{\mathrm{free}}(r)/r^2$
  (first suggested by Delley [?]).
- `rho_r` : $g_{\mathrm{at}}(\boldsymbol{r}) = n_{\mathrm{at}}^{\mathrm{free}}(r)/r$
- `rho` : $g_{\mathrm{at}}(\boldsymbol{r}) = n_{\mathrm{at}}^{\mathrm{free}}(r)$
- `fermi` : *Deprecated—do not use.* A Fermi-function like approach, requires two additional parameters.
- `stratmann`: The shape suggested by Stratmann *et al.*, Ref. [?]. This saves ≈10-20 % of the numerical effort compared to `rho_r2`. More importantly, however, our

recent testing shows that `stratmann` is also significantly accurate in some corner cases where the effects of integration accuracy even make a difference. *Note the properly bounded "stratmann_smoother" default function below. Straight "stratmann" should not be used.*

- `stratmann_smooth`: Partial update to guarantee a smooth edge at the "outer radius" or atoms.

  `stratmann_smoother`: Corrected version of the `stratmann` partition table. The following explanation refers to the prescription given in Eqs. (8), (11), and (14) of Ref. [?]. The actual (normalized) partition function is given by Eq. (10). At each grid point, it depends on a product of cell functions Eq. (9) over potentially all atoms in the system—unless its cell function is equal to one, a faraway atom may contribute to the partition function at a given grid point. Through the definition of μ_{ik} in Eq. (4) of Ref. [?] and the limitation of the cell function to unity for $\mu_{ik} < a = 0.64$ through Eqs. (11) / (14), the distance from which an atom can contribute is restricted, but potentially to a very large radius indeed. This becomes a problem for periodic systems (in our case, a theoretical radius of 25 Å would have resulted even for light settings and the farthest grid points from each atom, set to 5 Å for light settings).
  To avoid an overly large volume of contributing atoms, we restrict the list of contributing atoms to only those whose free-atom charge density would not be zero at the integration point in question. To that end, Eq. (8) of Ref. [?] is additionally multiplied with a function that smoothly interpolates between the original s of Stratmann and coworkers and unity. The interpolation is done only for atom distances between 0.8 and 1.0 times their free-atom radius, and uses a $[1-\cos(2x)]$-like interpolating function. The bottom line is that we get the benefits of both the Stratmann table and a restricted atom list without any discontinuities or wiggles as a function of atomic positions or unit cell vectors — which is as it should be.

- `stratmann_sparse`: This version of the `Stratmann` partition table is the same as `stratmann_smoother`, but it stores the relevant interatomic distances in a memory saving form.

Note that the free atom electron density $n_{\text{at}}^{\text{free}}(r)$ still determines the extent of many partition function types. This is controlled by the `cut_free_atom` keyword. See also the `hartree_partition_type` keyword, which presently must have the same setting as the `partition_type` keyword.

Tag: `points_in_batch` (control.in)

Usage: `points_in_batch value`

Purpose: Target number of grid points per integration batch.

Restriction: Applies to the `maxmin` and `octree` `grid_partitioning_method`.

`value` is an integer number. Default: 100.

See `grid_partitioning_method` and Ref. [?] for details regarding integration batches.

Subtags for *species* tag in `control.in`:

---

species sub-tag: angular (control.in)

Usage: angular limit

Purpose: For *self-adapting* angular integration grids, the maximum allowed number of points per radial shell.

Restriction: This flag has no effect for species where angular_grids is explicitly specified (the default in our species_default files).

limit is the maximum allowed number of integration points per radial shell.

This option is only meaningful for self-adapting angular grids, which are *not* the recommended default for production calculations with FHI-aims – (i) because these grids are often rather dense, and (ii) because they are meaningful only for cluster-type geometries. In order to specify self-adapting angular grids anyway, you must also set the keywords angular_min and angular_acc .

The available values of integration points in given angular grids are listed with the keyword force_lebedev .

---

species sub-tag: angular_acc (control.in)

Usage: angular_acc threshold

Purpose: For *self-adapting* angular integration grids, specifies the desired integration accuracy for the initial Hamiltonian and overlap matrix elements.

Restriction: Use only for cluster-type geometries.

threshold is a small positive real number; if 0., no adaptation is performed. Default: 0.

If threshold is not zero, this option invokes the self-adaptation of all angular integration grids, within the limits given by angular_min and angular . The adaption criteria are the initial Hamiltonian / overlap matrix integrals.

In all preconstructed species_default files, we specify reliable angular integration grids for all elements for DFT. No adaptation is required. For the curious, our own grids are adapted for symmetric dimers at a tight bond distance, using threshold $= 10^{-8}$.

---

species sub-tag: angular_grids (control.in)

Usage: angular_grids method

Purpose: Indicates how the angular integration grids (in each radial integration shell) for this species are determined.

method is a string, either auto or specified.

The standard species_default files provided with FHI-aims provide specified angu-

lar grids (on the safe side, i.e., rather dense) for each radial_base integration shell around an atom. The line:

angular_grids specified

must be immediately followed by a series of lines with

division [...]
outer_grid [...]

tag(s). These contain the actual grid specification.

If method auto is given, appropriate specifications for self-adapting grids should be included in control.in (keywords angular , angular_min , angular_acc).

---

species **sub-tag: angular_min** (control.in)

Usage: angular_min value

Purpose: specifies the minimum number of angular grid points per radial integration shell

value is the minimum number of grid points per shell.

For specified angular_grids , acts as a lower bound for the number of points per radial shell (specified grids will be increased accordingly).

For self-adapting angular grids, use together with the angular and angular_acc keywords.

In practice, value will be reduced to the next-highest available Lebedev integration grid (see force_lebedev tag for possible values).

---

species **sub-tag: cut_free_atom** (control.in)

Usage: cut_free_atom type [radius]

Purpose: Adds a cutoff potential to the initial, non-spinpolarized free-atom calculation that yields free-atom densities and potentials for many basic tasks.

type : A string, either finite or infinite. Default: finite for DFT-LDA/GGA and for RI_method LVL; infinite for Hartree-Fock, hybrid functionals, GW, etc. if RI_method V is used.
radius : A real number, in Å: Onset radius for the cutoff potential, as defined in the cut_pot tag. Default: For DFT-LDA/GGA, r_{onset} as given by the onset parameter in cut_pot .

Although this is a technical parameter (ideally, no influence on self-consistent, converged results), it has important implications for a variety of numerical tasks in the code:

- It influences (slightly) the basis-defining potential for the minimal basis, and for confined basis functions.
- It limits the radius of the free-atom density, which in turn limits the extent of the default integration partition table. For DFT-LDA/GGA, this extent need must not

be smalled than the radius of the most extended basis function, but it also need not be larger, since all integrands are zero outside anyway. This is *not* the case for the two-electron Coulomb operator, which is needed for Hartree-Fock, hybrid functionals, GW, etc, in which case the default is currently `infinite` (no cutoff potential applied).

- It also limits the extent of the partition table used for the Hartree potential.Especially in periodic calculations, it is vital that the real-space part of the Hartree potential is kept small. In that case, it is thus critical to keep `radius` as small as possible.

Usually, the default specified in the code should be accurate for all requirements. If, however, you suspect some kind of integration noise which is not related to the grid, increasing the `cut_free_atom` value may be a good test.

`species` sub-tag: `division` (control.in)

Usage: `division radius points`

Purpose: For `specified` `angular_grids` , the number of angular points on all radial shells that are within `radius`, but not within another, *smaller* division.

Restrictions: Meaningful only in a block immediately following an `angular_grids specified` line.

`radius` : Outer radius (in Å) of this division.
`points` : Integer number of angular points requested in this division (see `force_lebedev` tag for possible values).

Use the `outer_grid` tag to specify the number of angular grid points used outside the outermost division `radius`.

`species` sub-tag: `innermost_max` (control.in)

Usage: `innermost_max number`

Purpose: Monitors the quality of the radial integration grid.

`number` is an integer number, corresponding to a radial grid shell. Default: 4.

If, after on-site orthonormalization, a radial function's innermost extremum is inside the radial grid shell `number`, counting from the nucleus, that radial function is rejected in order to prevent inaccurate integrations.

`species` sub-tag: `logarithmic` (control.in)

Usage: `logarithmic r_min r_max increment`

Purpose: Defines the dense one-dimensional "logarithmic" grid for the direct solution of all radial equations (free atom quantities, Hartree potential).

`r_min` is a real number (in bohr); the innermost point of the logarithmic grid is defined as $r(1)$=`r_min`$/Z$, where Z is the atomic number of the `nucleus` of the `species` . Default: 0.0001 bohr.
`r_max` is a real number (in bohr), the outermost point of the logarithmic grid, $r(N)$. Default: 100 bohr.
`increment` is a real number, the increment factor α between successive grid points, $r(i) = \alpha \cdot r(i-1)$. Default: 1.0123.

The number of logarithmic grid shells, N, is uniquely determined by `r_min`, `r_max`, and `increment`. Specifying a dense logarithmic grid is not performance-critical.

`species` sub-tag: `outer_grid` (control.in)

Usage: `outer_grid points`

Purpose: For `specified` `angular_grids` , the number of angular points on all radial shells outside the largest `division` .

Restrictions: Meaningful only in a block immediately following an `angular_grids` `specified` line.

`points` : Integer number of angular points (see `force_lebedev` tag for possible values).

Use the `division` tag to specify the number of angular grid points used for radial shells within specified radii.

`species` sub-tag: `radial_base` (control.in)

Usage: `radial_base number radius`

Purpose: Defines the basic grid of radial integration shells according to Ref. [?]

`number` is an integer number (the total number of grid points, N).
`radius` is a positive real number which specifies the outermost shell of the basic grid, r_{outer}, in Å.

The location of the `number` radial shells is given by

$$r(i) = r_{\text{outer}} \cdot \frac{\log\{1 - [i/(N+1)]^2\}}{\log\{1 - [N/(N+1)]^2\}} \tag{3.14}$$

With this prescription, shell i=0 would be located exactly at $r(i) = 0$, and shell i=$N+1$ would be located exactly at $r(i) = \infty$, i.e., this provides an exact mapping of the interval [0,∞].

The FHI-aims `species_default` files provide values for `number` according to the formula N=1.2×14(`nucleus` +2)$^{1/3}$, as determined empirically in Ref. [?]. These "basic" grids are can then be augmented by adding uniform subdivisions, using the

radial_multiplier keyword described below.

---

species sub-tag: radial_multiplier (control.in)

Usage: radial_multiplier number

Purpose: Systematically increases the radial integration grid density.

value is an integer, number specifying the number of added subdivisions per basic grid spacing. Default: value = 2

The basic grid of N radial shells (see radial_base definition) is evenly subdivided number-1 times, to yield number$\cdot(N+1)-1$ shells in the *actually used* integration grid. Thus, the radial_multiplier tag allows to systematically increase the number of radial shells (by factors). For example, number=2 (the default) would yield $2N+1$ shells total.

Note that some all-electron Gaussian basis sets contain either very high or very low exponents. If such basis sets are used for test purposes, it may be necessary to test the convergence of the radial integration grid directly by increasing the radial_multiplier .

3.6 Electron density update

In FHI-aims, the first step of a new iteration is the update of the electron density based on the output Kohn-Sham orbitals produced by a previous step.

The present section covers only the *actual* density update. Techniques relevant for the self-consistent *convergence* of the whole calculation (electron density mixing, preconditioning, etc.) are covered separately in Sec. 3.10.

Tags for general section of `control.in`:

Tag: `density_update_method` (control.in)

Usage: `density_update_method type`

Purpose: Governs the selection of the density update type.

Restriction: For periodic boundary conditions, only the density-matrix based electron density update is supported.

Default: Cluster case: `automatic`. Periodic case: `density_matrix`

Choices for `type`:

- `orbital` : Use Kohn-Sham orbitals based update
- `density_matrix` : Use density-matrix based update method. Required for periodic systems.
- `automatic` : Selects the best update method automatically, based on the expected amount of work.
- `split_update_methods` : Charge density is updated via Kohn-Sham orbitals and force is updated via density-matrix

If not specified, default for cluster geometries is the automatic selection of the density update method.

See Ref. [?] for details regarding density update mechanisms. In general, FHI-aims offers an electron density update based on Kohn-Sham orbitals [$O(N^2)$ with system size, but faster for finite systems up to ≈100-500 atoms depending on basis size], and an $O(N)$ rewrite based on the density matrix. This should be used for large systems, and is the default for periodic systems.

For the non-periodic case, the current code version determines the switching point between the orbital-based update and the density-matrix based update automatically through some heuristics. This procedure guarantees that accidental $O(N^2)$ calculations will not happen for very large systems, but the optimum cross-over point may not always be exactly found. If you are planning long runs of essentially the same geometry (molecular dynamics trajectories are a good example), you may save some time by performing some explicit benchmarks first. You can then specify the optimum density update method for *your* own case, instead of relying on our heuristics.

3.7 Electrostatic (Hartree) potential

This section describes the method used to compute the electrostatic (Hartree) potential in FHI-aims. For a more exhaustive description, please refer to Ref. [?].

Some central equations are repeated here in detail since, as a result, the calculation of the Hartree potential can be heavily customized by many analytically available accuracy / cutoff thresholds, given below.

For production calculations, it is important to note that *our standard accuracy thresholds in the Hartree potential are numerically sound, and usually do not require an explicit customization*. The only parameter which should be explicitly set is the angular momentum up to which the atom-centered partitioned charge density is expanded, `l_hartree` below.

As pointed out in Ref. [?], our experience is that energy differences are usually well converged for l_{hartree}=4, and total energy convergence at the level of a few meV is reached at l_{hartree}=6. Only in exceptional cases should different settings be required.

—

At the beginning of a calculation, we first compute the electrostatic potential associated with the initial superposition of free-atom densities, $\sum_{\text{at}} n_{\text{at}}^{\text{free}}(|\boldsymbol{r} - \boldsymbol{R}_{\text{at}}|)$:

$$v^{\text{es,free}}(\boldsymbol{r}) = \sum_{\text{at}} v_{\text{at}}^{\text{es,free}}(|\boldsymbol{r} - \boldsymbol{R}_{\text{at}}|) \tag{3.15}$$

This is the largest part of the Hartree potential, but is always accurately known from the solution of spherical free atoms on a dense logarithmic grid.

For a given electron density $n(r)$ *during* the s.c.f. cycle, we then only ever compute the electrostatic potential associated with the *difference* to the superposition of free atoms, $\delta v_{\text{es}}(\boldsymbol{r})$, based on

$$\delta n(\boldsymbol{r}) = n(r) - \sum_{\text{at}} n_{\text{at}}^{\text{free}}(|\boldsymbol{r} - \boldsymbol{R}_{\text{at}}|) \tag{3.16}$$

$\delta n(\boldsymbol{r})$ is first split up into a sum of *partitioned, atom-centered* charge multipoles,

$$\delta \tilde{n}_{\text{at},lm}(r) = \int_{r=|\boldsymbol{r}-\boldsymbol{R}_{\text{at}}|} d^2\Omega_{\text{at}} p_{\text{at}}(\boldsymbol{r}) \cdot \delta n(\boldsymbol{r}) \cdot Y_{lm}(\Omega_{\text{at}}) \tag{3.17}$$

(the sum of all partition functions at every point is always unity). Due to the finite extent of $\delta n(\boldsymbol{r})$ and $p_{\text{at}}(\boldsymbol{r})$ (both are controlled by the `cut_free_atom` keyword), the range of each component $\delta \tilde{n}_{\text{at},lm}(r)$ is also bounded.

The Hartree potential components $\delta \tilde{v}_{\text{at},lm}(r)$ are then determined on a dense, one-dimensional logarithmic grid, using classical electrostatics. The resulting $\delta \tilde{v}_{\text{at},lm}(r)$ are then numerically tabulated, and evaluated elsewhere using cubic spline interpolation.

For *cluster* systems, it is important to note that the finite extent of $\delta \tilde{n}_{\text{at},lm}(r)$ implies that the numerically tabulated part of $\delta \tilde{v}_{\text{at},lm}(r)$ can also be kept finite. Outside this "multipole radius", $\delta \tilde{n}_{\text{at},lm}(r)$=0, and $\delta \tilde{v}_{\text{at},lm}(r)$ falls off analytically as

$$\delta \tilde{v}_{\text{at},lm}(r) = m_{\text{at},lm}/r^{l+1} \quad . \tag{3.18}$$

Instead of a spline evaluation, faraway atoms can thus be analytically accounted for using tabulated, constant *multipole moments* $m_{\mathrm{at},lm}$. High-l components can be analytically cut off as they approach zero at large distances.

In this approach, the effort to create the complete Hartree potential on the entire grid is determined by tabulating the contribution from *every* atom on *every* grid point,

$$v_{\mathrm{es}}(\boldsymbol{r}) = v^{\mathrm{es,free}}(\boldsymbol{r}) + \sum_{\mathrm{at},lm} \delta\tilde{v}_{\mathrm{at},lm}(|\boldsymbol{r} - \boldsymbol{R}_{\mathrm{at}}|)Y_{lm}(\Omega_{\mathrm{at}}) \quad . \tag{3.19}$$

The scaling is thus close to $O(N^2)$ with system size, albeit reduced by high-l multipole components falling off towards large distances.

For *periodic* systems, essentially the same equations hold, except that the Hartree potentials associated with the atom-centered charge densities $\delta\tilde{n}_{\mathrm{at},lm}(r)$ are here additionally split into a short-ranged real-space part, and a smooth, long-ranged reciprocal-space part (Ewald's method), by splitting

$$\frac{1}{r} = \frac{\mathrm{erf}(r/r_0) + \mathrm{erfc}(r/r_0)}{r} \tag{3.20}$$

(and similar for components of higher angular momentum). The summation of long-range tails thus happens in reciprocal space, using Fourier transforms. As a result, the scaling of this effort is no longer $O(N^2)$, but rather approaches $O(N\ln N)$ in Fourier transforms.

The cutoff reciprocal space momentum for the Fourier part of the electrostatic potential, $|\boldsymbol{G}_{\mathrm{max}}|$, is estimated using a small threshold parameter η:

$$|\boldsymbol{G}_{\mathrm{max}}|^{l^{\mathrm{es}}_{\mathrm{max}}-2} \cdot \frac{1}{\boldsymbol{G}^2_{\mathrm{max}}} \cdot \exp(-\frac{r_0^2\boldsymbol{G}^2_{\mathrm{max}}}{4}) = \frac{\eta}{10 \cdot 4\pi} \, . \tag{3.21}$$

Our default choice for η (in atomic units, i.e., those used internally in the code) is $\eta = 5{\cdot}10^{-7}$, but this is somewhat overconverged, and a larger threshold value is probably sufficient for most situations. *Note* that Eq. (3.21) is slightly modified compared to the version given in Ref. [**?**].

3.7.1 Non-periodic Ewald method

For large, finite systems (more than 200 atoms) it is possible to use the so-called 'non-periodic Ewald method' in aims (keyword `use_hartree_non_periodic_ewald`). The basic idea of this method is to use interpolation to reduce the effort for calculting the Hartree term. Specifically, the method consists in computing the electrostatic potential not on the fine interpolation grid points but firstly on a coarse Cartesian grid. Subsequently, the values of the potential on the coarse grid are interpolated to the fine integration grid. If the Cartesian grid is suffiently coarse, time is saved because of the reduced number of potential computations.

We use an envenly spaced, Cartesian grid with a certain grid width. Due to this fixed grid width, special attention has to be paid to the near-atom regions where the electron

density and hence also the potential oscillates strongly. This problem can be solved by using the Ewald decomposition which was originally developed for periodic systems. Ewald's method aims at separating large and small scales by adding and subtracting charge spheres with Gaussian radial shape to a lattice of monopoles. In terms of the potential, this yields $\bar{q}/r = [\bar{q}/r - \Omega(r)] + \Omega(r)$ for each monopole, where $\bar{q} := q/(4\pi\epsilon_0)$ and q is the monopole charge. The function $\Omega(r) = \bar{q}\,\mathrm{erf}(r/r_0)/r$ is the potential of a Gaussian charge sphere with width parameter r_0. The first part of the decomposition $\bar{q}/r - \Omega(r)$ decays quickly with increasing r so that this part is calculated in real space, while the second part $\Omega(r)$ decays quickly in Fourier space so that it is calculated there. The two parts are often referred to as 'short range' and 'long range' part. However, this is somewhat misleading because the second part is actually defined in whole space. For this reason, we call the first part 'localized' and the second part 'extended'.

We can translate the classical Ewald decomposition to our case of a finite system by calculating the smooth extended part $\Omega(r)$ on the coarse Cartesian grid, with subsequent interpolation to the fine integration grid points. In addition, we have to calculate the localized part in the vicinity of the nuclei where we cannot save any computational time [actually some time is lost since we have to compute $\Omega(r)$ there, too].

In the classical Ewald method, Gaussian spheres are an excellent choice as auxiliary charges due to the quick convergence of both the localized part in real space and the extended part in Fourier space. However in our case, where we interpolate in real space, Gaussian spheres are not necessarily a proper choice. Therefore optimized charge distributions obtained from a variational method by W. Jürgens are used.

In order to reduce the number of grid points, we allow the Cartesian grid to have arbitrary orientation. More specifically, we are looking for a rectangular cuboid that covers all integration grid points but with minimum volume. This problem is solved approximately by using a common procedure that is based on principle component analysis.

Tags for general section of `control.in`:

Tag: `adaptive_hartree_radius_th` (control.in)

Usage: `adaptive_hartree_radius_th threshold`

Purpose: Determines the distance beyond which an analytical component $\delta\tilde{v}_{\mathrm{at},lm}(r)$ of the *periodic* (Ewald!) real-space Hartree potential for a given atom is considered zero.

`threshold` is a small positive real number. Default: 10^{-8}.

Usually, this tag need not be modified from the default. Long-range multipole components $\delta\tilde{v}_{\mathrm{at},lm}(r)$ of the real-space (Ewald!) Hartree potential are not evaluated for distances where $\delta\tilde{v}_{\mathrm{at},lm}(r) <$`threshold`. This tag provides similar functionality as the `multipole_threshold` tag for the cluster case (numerically different due to the absence of $\mathrm{erf}(r/r_0)$ in the cluster case).

Tag: `compensate_multipole_errors` (control.in)

Usage: `compensate_multipole_errors flag`

Purpose: If true, introduces a compensating normalization and density to eliminate the effects of small charge integration errors in the long-range Hartree potential.

`flag` is either `.true.` or `.false.`. Default: `.true.` if a surface dipole correction or work function calculation was requested, `.false.` otherwise.

This keyword is especially useful when assessing the electrostatic potential far away from a structure, e.g., when calculating a surface dipole correction (for asymmetric slabs) or work function. See `use_dipole_correction` or `evaluate_work_function` for details on these methods.

In general, keyword `compensate_multipole_errors` makes sure that the long-range charge components of the Hartree potential are exactly those expected from the calculated (and normalized) electron density. Any small spurious non-zero components that are solely due to integration errors on a finite integration grid.

It may indeed be reasonable to use this keyword as the default in any calculation, especially with "light" integration grids. It is not the general default because of a single test case where the harmonic vibrational frequencies of a large molecule deteriorated slightly compared to the standard (uncompensated) case. However, in general, total energies with `compensate_multipole_errors` appear to be slightly more accurate on balance than without.

Tag: `hartree_convergence_parameter` (control.in)

Usage: `hartree_convergence_parameter value`

Purpose: Governs the Ewald-type short-range / long-range splitting of the Coulomb potential in Eq. (3.20).

`value` : Distance parameter r_0 in Eq. (3.20) (in bohr). Default: 3.0 bohr.

Necessary for periodic boundary conditions only. May be changed from the default, but should not be set too small or too large (the compensating Gaussian charge density of the Ewald method must cancel the actual charge outside a radius that is still inside the partition table / integration grid for every atom.)

Tag: `hartree_fp_function_splines` (control.in)

Usage: `hartree_fp_function_splines .true. / .false.`

Purpose: Switches on the splining of the Greens functions for the long-range Hartree multipole decomposition in periodic boundary conditions. This accelerates the calculation of the Hartree potential in large unit cells.

Default: `.true.`

Tag: `hartree_fourier_part_th` (control.in)

Usage: `hartree_fourier_part_th threshold`

Purpose: *Implicitly* determines the required reciprocal space cutoff momentum $|\boldsymbol{G}_{\max}|$ for the Fourier summation of the long-range electrostatic potential (Ewald).

`threshold` is a real positive small number [η in Eq. (3.21)]. Default: $5{\cdot}10^{-7}$ (in atomic units) .

See Eq. (3.21). Usually, this tag need not be modified from the default. Necessary for periodic boundary conditions only.

Tag: `hartree_partition_type` (control.in)

Usage: `hartree_partition_type type`

Purpose: Specifies which kind of partition function $p_{\text{at}}(\boldsymbol{r})$ is used to split $\delta n(\boldsymbol{r})$ into atom-centered pieces.

Restriction: Presently, `type` should have the same value as specified for integration using the `partition_type` keyword.

`type` : A string that specifies which kind of partition table is used. Default: `stratmann`

Usually, this tag need not be modified from the default. The same options are available as for the `partition_type` keyword (partition functions for three-dimensional integrands). See `partition_type` for details.

Tag: `hartree_radius_threshold` (control.in)

Usage: `hartree_radius_threshold threshold`

Purpose: Technical criterion to ensure the inclusion of atoms with a potentially finite real-space Hartree potential component in periodic boundary conditions.

`threshold` is a small positive real number. Default: 10^{-10}.

Usually, this tag need not be modified from the default. Necessary for periodic boundary conditions only. For each atom, determines a safe real space outer radius based on $\mathrm{erf}(r_{\mathrm{outer}}/r_0) <$ `threshold`. This is then used to determine which atoms need be included in the second term (sum over atoms) of Eq. (3.19).

Tag: `legacy_monopole_extrapolation` (control.in)

Usage: `legacy_monopole_extrapolation flag`

Purpose: Specifies how the monopole ($l = 0$) part of the partitioned charge density is extrapolated to $r = 0$ before transforming to a logarithmic grid to integrate the radial Hartree potential. If `.true.`, use the legacy variant, and an improved extrapolation otherwise.

`flag` is a Boolean. Default: `.false.`.

The effect is generally very small, but for `light` grids, this can have some impact on total energies.

Tag: `l_hartree_far_distance` (control.in)

Usage: `l_hartree_far_distance value`

Purpose: Sets a maximum angular momentum beyond which the components of the analytic long-range Hartree potential will not be computed.

`value` is an integer number. Default: 10.

Usually, this tag need not be modified from the default. In Eq. (3.18), the multipole moments $m_{\mathrm{at},lm}$ are determined by an explicit integration of the finite real-space density component $\delta\tilde{n}_{\mathrm{at},lm}(r)$. However, for very high l, even spuriously small density components (10^{-10} or lower) may be artificially weighted up in $m_{\mathrm{at},lm}$; on a finite integration grid, $m_{\mathrm{at},lm}$ becomes prone to numerical noise. Capping the evaluation of such high-l components increases stability, but can be undone through `l_hartree_far_distance` if required.

Tag: `multip_moments_threshold` (control.in)

Usage: `multip_moments_threshold threshold`

Purpose: Implicitly defines the maximum angular momentum for which the analytical multipole components are non-zero at all.

`threshold` is a small positive real number. Default: 10^{-10}.

Usually, this tag need not be modified from the default. Used only in the periodic case. If $m_{\text{at},lm}/r_{\text{mp}}$ <`threshold` for all $l \geq l_{\text{thr}}$, all analytical components beyond l_{thr} are considered zero in the real-space and Fourier parts of the long-range potential. r_{mp} is the radius determined by `multip_radius_threshold` .

Tag: `multip_moments_rad_threshold` (control.in)

Usage: `multip_moments_rad_threshold threshold`

Purpose: Defines the outer radius of the density components $\delta\tilde{n}_{\text{at},lm}(r)$ for the purpose of determining the far-field moments $m_{\text{at},lm}$.

`threshold` is a small positive real number. Default: 10^{-10}.

Usually, this tag need not be modified from the default. The outer radius is set where $|\delta\tilde{n}_{\text{at},lm}(r)|$ <`threshold`. The actual $m_{\text{at},lm}$ are then determined by inward integration from this point, using the standard relations of classical electrostatics.

Tag: `multip_radius_free_threshold` (control.in)

Usage: `multip_radius_free_threshold threshold`

Purpose: Technical criterion to define the outermost charge radius of the spherical free atom density $n_{\text{at}}^{\text{free}}$

`threshold` is a small non-negative real number. Default: 0.0

Usually, this tag need not be modified from the default.

The free-atom radius inside the code is set to the radius where $n_{\text{at}}^{\text{free}}(r)$ becomes smaller than `threshold`. Note that the actual extent of the free-atom charge can be influenced by the `cut_free_atom` keyword, and has ramifications not just for the electrostatic potential, but also for the initial charge density, and the partition functions for all integrals.

Tag: `multip_radius_threshold` (control.in)

Usage: `multip_radius_threshold threshold`

Purpose: Determines the (per-atom) radius outside of which the analytical multipoles $m_{\text{at},lm}$ are used to construct the Hartree potential $v_{\text{es}}(\boldsymbol{r})$

`threshold` is a small positive real number. Default: 10^{-12}.

Usually, this tag need not be modified from the default. The outer radius is set where *all* $\delta\tilde{n}_{\text{at},lm}(r)$ <`threshold` for a given atom. At a given integration point $\boldsymbol{r}$, $v_{\text{es}}(\boldsymbol{r})$ is assembled by evaluating Eq. (3.19). The second part (sum over atoms) is evaluated separately

for each atom, and atoms outside the radius defined by multip_radius_threshold , the lm summation is performed using the analytical expression.

Tag: `multipole_threshold` (control.in)

Usage: multipole_threshold threshold

Purpose: Cluster case only – determines the distance beyond which an analytical component $\delta\tilde{v}_{\mathrm{at},lm}(r)$ of the real-space Hartree potential is considered zero.

threshold is a small positive real number. Default: 10^{-10}.

Usually, this tag need not be modified from the default. Long-range Hartree potential components $\delta\tilde{v}_{\mathrm{at},lm}(r)$ are not evaluated for distances where $\delta\tilde{v}_{\mathrm{at},lm}(r)$ <threshold. This tag provides similar functionality as the adaptive_hartree_radius_th tag for the periodic case (numerically different due to the absence of $\mathrm{erf}(r/r_0)$ in the cluster case).

Tag: `set_vacuum_level` (geometry.in)

Usage: set_vacuum_level z-coordinate

Purpose: Surface slab calculations only – defines a z-axis value that is deeply within the vacuum layer.

z-coordinate is a z coordinate value in the vacuum layer.

In the case of periodic surface slab calculations, this value defines the reference z coordinate that is used to define the work function (keyword evaluate_work_function) and/or the location of a dipole correction (electrostatic potential step) to offset a potential electrostatic dipole formed by a non-symmetric slab (keyword use_dipole_correction). As a requirement, the surface must be parallel to the xy plane. The chosen z-coordinate must be located deep in the vacuum, as far away as possible from any surface.

set_vacuum_level auto can be used instead to determine the vacuum level on its own.

If use_dipole_correction or evaluate_work_function are specified, omitting the keyword set_vacuum_level causes FHI-aims to automatically determine a suitable z.

However, a vacuum plane will only be determined if the nearest atom is more than 6 Å away from the vacuum level. Determining a surface dipole for distances for which basis functions or charge densities could overlap might lead to errors. Since FHI-aims does allow one to use rather large vacuum spacings at low (if any) computational overhead, the calculation will stop for too small vacuum spacings and alert the user.

Tag: `use_dipole_correction` (control.in)

Usage: use_dipole_correction

Purpose: Surface slab calculations only – compensates a potential dipole field of non-symmetric slabs by an electrostatic potential step in the vacuum region.

Restriction: When specified for a charged periodic system, this keyword is currently disabled (see below).

If set, this option introduces an electrostatic potential step in the vacuum region of a surface slab calculation, to compensate for a potential surface dipole. The surface must be parallel to the xy plane (perpendicular to the z direction). The z location of the surface dipole must be provided by hand, by specifying the set_vacuum_level keyword.

In practice, the dipole correction calculates the gradient of only the long-range Hartree potential term of the Ewald sum (which is evaluated in reciprocal space). If the gradients on both sides of the vacuum level do not agree to better than 10 % (i.e., the potential is not linear in this range), the dipole correction is not computed, and a warning is issued instead.

However, it must be possible to find a vacuum plane z, where the surface dipole is compensated, that is further than 6 Å away from the nearest atom. Otherwise, the calculation will stop and alert the user.

The reason is that a surface dipole cannot be safely determined for vacuum spacings for which basis functions or charge densities could overlap. This can lead to errors. Note that FHI-aims does allow one to use rather large vacuum spacings at low (if any) computational overhead.

If a dipole correction is requested, keyword compensate_multipole_errors is automatically switched on by default. This will change total energies slightly compared to the uncompensating case, and – we believe – even for the better.

Attention: This keyword is currently disabled for charged periodic systems. The Coulomb potential of a charged surface slab will reach far into the vacuum, apparently leading to a completely arbitrary dipole correction as a result. (The dipole correction will simply flatten out the potential wherever it is asked to do so, but for a charged surface, the residual Coulomb potential should not be flat.)

In order to alert user to the problem, the code presently stops with a warning. If you know what you are doing, the pertinent stop (one line) can always be commented out—if the code is recompiled, the method will be applied, even though the physical relevance of the result is uncertain. Charged periodic calculations with a vacuum region are physically questionable for very different reasons in any case; we recommend to find a different workaround with explicit charges whenever that is possible.

Tag: `use_hartree_non_periodic_ewald` (control.in)

Usage: `use_hartree_non_periodic_ewald .true.`
or: `use_hartree_non_periodic_ewald gridspacing value`
or: `use_hartree_non_periodic_ewald .false.`

Purpose: This option is *experimental* and applies only to non-periodic calculations. In this case, the Hartree potential is decomposed according to Ewald's method.

This method accelerates the calculation of the Hartree term in case of large systems (more than 200 atoms) by using Ewald's decomposition combined with spatial interpolation, see section 3.7.1. The method can be switched on by using option "`.true.`". In this case, a default grid spacing of 0.6 Å ($= 60$ pm) is used for the Cartesian grid. Other values for the grid spacing can be chosen with option "`gridspacing value`". If this option is used, the method is switched on and the grid spacing is set to `value` in Å ($= 100$ pm). Finally, the method can be switched off with option "`.false.`". However, since this is the default behaviour, it is not necessary to switch off the method explicitely.

Subtags for *species* tag in `control.in`:

`species` **sub-tag:** `l_hartree` (control.in)

Usage: `l_hartree value`

Purpose: For a given species, specifies the angular momentum expansion of the atom-centered charge density multipole for the electrostatic potential.

`value` is an integer number which gives the highest angular momentum component used in the multipole expansion of $\delta n(\boldsymbol{r})$ into $\delta\tilde{n}_{\mathrm{at},lm}(r)$ for the present species. *Must be specified.*

As pointed out in Ref. [?], our experience is that energy differences are usually well converged for l_{hartree}=4, and total energy convergence at the level of a few meV is reached at l_{hartree}=6. Only in exceptional cases should different settings be required.

3.8 Kinetic energy, scalar relativity, and spin-orbit coupling

For elements beyond approximately Z=30, relativistic effects near the nucleus cannot be neglected in an all-electron treatment—both for core, and for valence electrons. For the purposes of "everyday" matter, the full theory is given by Dirac's four-component Equation, but in the "practice" of materials physics and chemistry, we still tend to think in terms of Schrödinger-like objects. A true four-component treatment is not implemented in FHI-aims, but the following standard levels of approximation are available:

- Non-relativistic kinetic energy (one or two collinear `spin` components of the Kohn-Sham orbitals)
- Scalar-relativistic kinetic energy expression (one or two collinear spin components).
- Perturbative spin-orbit coupling, a single correction step to the Kohn-Sham eigenvalues based on the Kohn-Sham orbitals from a non-relativistic or scalar-relativistic s.c.f. cycle. *Perturbative spin-orbit coupling in FHI-aims is a new feature. Importantly, changes to the total energy beyond the sum-of-eigenvalues are not included, and total energy gradients (forces) are also unavailable.*

Scalar relativity

While the non-relativistic level of theory is exactly defined and will be the same in any first-principles implementation (at a complete basis set, all-electron level anyway), there are many different versions of scalar-relativistic approximations which can yield considerably different *total* energies for different systems. Their unifying feature is that any two scalar-relativistic methods should still yield the same energy *differences* for properties that concern valence electrons: Binding energies, valence eigenvalues, etc.

As outlined in Ref. [?], FHI-aims offers two reliable, effective one-component scalar relativistic treatments (by "effective one-component", we mean that no coupling exists between the two spin channels of a calculation with collinear spin): "atomic ZORA" and "scaled ZORA". (ZORA: "zero-order regular approximation".) The potential energy surfaces and valence eigenvalues obtained from either one are sound, with total energy *differences* (e.g., between different structures) of atomic ZORA matching benchmark results obtained by other scalar-relativistic methods somewhat more closely in the "Delta Project", https://molmod.ugent.be/deltacodesdft .

By construction, scaled ZORA (absolute) *total* energies and *core* eigenvalues are closer to their counterparts from other scalar-relativistic methods than atomic ZORA.

However, in practical calculations, atomic ZORA appears to be preferable:

- The computational overhead for scaled ZORA is significant
- Calculating energy gradients (forces) is much more involved than for atomic ZORA, which is why gradients are only implemented for atomic ZORA

- It appears (see above) that atomic ZORA based energy differences are slightly more reliable than scaled ZORA energy differences.

In practice, any valence electron related results from atomic ZORA, scaled ZORA, or other scalar relativistic methods in the literature should be expected to agree with one another up to benchmark accuracy level if all *other* computational tasks are numerically well converged. Only the broad comparison of materials and codes in the "Delta Project" (https://molmod.ugent.be/deltacodesdft) helped us distinguish what is otherwise a very subtle difference at best.

As illustrated in more detail in Sec. 4.2, we presently recommend the following strategy for the correct handling of scalar relativity for arbitrary structures in FHI-aims:

1. For energy differences, geometries, valence eigenvalues etc., simply rely on the *atomic* zero-order regular approximation approximation (ZORA). This has performed very well for us in a broad array of benchmarks.

2. If you wish for a "second opinion" or, for some reason, need a more credible (still not accurate!) handling of core eigenvalues, perform single-point calculations using the "scaled ZORA" [?] approximation.

Similarly, any other tasks involving forces should be performed using atomic ZORA, remembering that only energy *differences* between different structures are meaningful in this case.

Spin-Orbit Coupling

FHI-aims currently implements a treatment of the spin-orbit coupling operator which adds spin-orbit coupling (SOC) corrections to the Kohn-Sham eigenvalues, band structures, and densities of states in a single evaluation after a scalar-relativistic s.c.f. cycle without spin-orbit coupling is complete. In other words, this is a non-selfconsistent evaluation of spin-orbit coupling. The keyword

```
include_spin_orbit non_self_consistent
```

should be all that is needed for most production applications. This keyword should be *added* to a standard `control.in` file. The level of scalar relativity should still be specified as usual, for example,

```
relativistic atomic_zora scalar
```

to invoke the atomic ZORA kinetic energy operator.

Applying the SOC operator in this perturbative (non-selfconsistent) way is quantitatively accurate (to a few 0.01 eV for valence band structures) for elements below Xe (Z=54) when combined with atomic ZORA.

For heavy elements (approximately Au and beyond) this level of theory is only qualitatively accurate. It captures the majority of the SOC effect, but quantitative deviations

above 0.1 eV for band structures must be expected. Similarly, the corrections for any core levels would require one to go beyond non-selfconsistent SOC.

Since FHI-aims, by default, only treats a finite number of bands in a given calculation, increasing the number of bands using the keyword `empty_states` may be necessary to capture correctly the behavior of high-lying conduction bands.

Some Background

Spin-orbit coupling is a simple consequence of transforming Dirac's equation to a (two-component) Schrödinger-like form. This leads to an approximate Hamiltonian of the form

$$\hat{H} = \hat{t}_{SR} + \hat{v} + \hat{v}_{SOC}. \tag{3.22}$$

$\hat{t}_{SR}$ is the usual scalar relativistic kinetic energy operator (e.g., atomic ZORA). $\hat{v}$ is the local or non-local potential, and $\hat{v}_{SOC}$ is the spin-orbit coupling operator,

$$v_{SOC} = \frac{i}{4c^2}\boldsymbol{p}\tilde{v} \times \boldsymbol{p} \cdot \boldsymbol{\sigma}. \tag{3.23}$$

This operator also contains a potential $\tilde{v}$, and in principle, for Kohn-Sham theory (local potential), $\tilde{v}$ could be equal to $v(r)$. In practice, however, the combination of potential and gradients in v_{SOC} means that SOC arises essentially only very near the nucleus, where the potential is deep and dominated by the nuclear potential. In FHI-aims, we therefore only consider the sum of the nuclear and the electrostatic potentials, but not the Kohn-Sham exchange-correlation potential, for $\tilde{v}$ when constructing v_{SOC}.

We then treat spin-orbit coupling $\boldsymbol{\sigma} \times \boldsymbol{p}$ perturbatively , i.e., as a single-shot postprocessing step after an otherwise converged scalar-relativistic self-consistent field cycle. By default, we can simply include all Kohn-Sham states in the evaluation of v_{SOC} above and invert the resulting Hamiltonian once to obtain its eigenvalues and eigenvectors. This amounts to standard first-order perturbation theory and is the level of theory that we have found to be sufficient for accurate results.

It is possible to go beyond the first-order level by so-called quasi-degenerate (qd) perturbation theory in a second-variational approach. Some of this is implemented in FHI-aims and summarized below for completeness. However, we have not found any significant improvements compared to the first-order level. At present, it does not appear necessary to play with any of this except for development or research purposes.

Quasi-degenerate perturbation theory is based on Löwdin partitioning [?]. The spin-orbit interaction is seen as a perturbation to the scalar-relativistic Kohn-Sham-Hamiltonian $\hat{h}^{\mathsf{sr}}$:

$$\hat{h} = \hat{h}^{\mathsf{sr}} + \hat{h}^{\mathsf{soc}}\,. \tag{3.24}$$

The scalar-relativistic set of eigenstates $\{\psi^{\mathsf{sr}}\}$ is then divided into several subsets $\{\psi^{\mathsf{sr}}\}_k$ in the following way. Two scalar-relativistic states i and j belong to the same subset if

1. their eigenvalues are close OR
2. their matrix element $\left\langle \psi_i^{\mathsf{sr}} \middle| \hat{h}^{\mathsf{SOC}} \middle| \psi_j^{\mathsf{sr}} \right\rangle$ is comparable to the eigenvalue difference

which means that any two *different* subsets are weakly interacting in terms of h^{SOC} and separated in energy. For each subset the Hamiltonian 3.24 is then "block-diagonalized" to a form $\tilde{h}$ such that all matrix elements $\left\langle \psi_{\text{in subset}} \middle| \tilde{h} \middle| \psi_{\text{not in subset}} \right\rangle$ vanish up to a specified order in h_{SOC}. For details see reference [?].

Tags for general section of `control.in`:

Tag: `include_spin_orbit` (control.in)

Usage: `include_spin_orbit method`

Purpose: Calculates spin-orbit coupling corrected eigenvalues.

`method` The method for including spin-orbit coupling. At present, only `type` `non_self_consistent` is suitable for production-level calculations.

Note: While FHI-aims also prints out a corrected total-energy expression based on the SOC-corrected eigenvalues, please do not trust this total energy expression for anything. It is experimental.

Spin-orbit corrections can be added as a perturbative post-processed step after a non-relativistic, atomic ZORA or scaled ZORA scalar-relativistic calculation. Usually, just writing

`include_spin_orbit non_self_consistent`

is sufficient to obtain corrected eigenvalues and (if requested) band structures, densities of states, etc., in first-order perturbation theory to a scalar-relativistic calculation.

FHI-aims will write the results to standard output after the SCF cycle has converged and details to a separate output file `SOC_result.out`.

Since perturbative spin-orbit coupling operates on the space spanned by the calculated scalar-relativistic eigenvectors, for accurate high-lying bands one must include sufficiently many unoccupied states. If needed, this may be done by increasing the keyword `empty_states` .

For research purposes, FHI-aims also includes some level of theory beyond first-order perturbation theory. This is summarized below and gives rise to some additional keywords. However, we have not seen a large improvement from the approaches described below.

FHI-aims also enables the use of (quasi-)degenerate perturbation theory, for which there are three important parameters: `matrix_tolerance` and `ev_tolerance` which determine the assignment of eigenvalues to degenerate subspaces, and `soc_perturb_order` which determines the order of (quasi-)degenerate perturbation theory to perform.

`matrix_tolerance` places eigenstates in the same subspace if
$\left|\left\langle \psi_i^{\mathrm{sr}} \middle| \hat{h}^{\mathrm{soc}} \middle| \psi_j^{\mathrm{sr}} \right\rangle\right| \times$ `matrix_tolerance` $\geq |\epsilon_i - \epsilon_j|$
`ev_tolerance` [in eV] places eigenstates in the same subspace if $|\epsilon_i - \epsilon_j| \leq$ `ev_tolerance`

Standard first-order perturbation theory is a special case of the quasi-degenerate perturbation formalism, achievable by considering only first-order effects and setting a large enough value of `ev_tolerance` that all eigenstates are placed into a single subspace, and we have chosen to make this the default setting.

While we believe standard first-order perturbation theory gives reliable numbers as com-

pared to to quasi-degenerate perturbation theory and highly recommend the user stay with it, nevertheless if the user wishes to change these settings, they may do so. The default value for `matrix_tolerance` is 0.1. The default value for `ev_tolerance` is 1000000000 eV. To change these, use keyword `soc_subspace` . The default value for `soc_perturb_order` is 1. To change this, use keyword `soc_perturb_order` .

If the user wishes to play around with quasi-degenerate perturbation theory, we have found in the past that `soc_perturb_order` 2 and `ev_tolerance` 0.00001 eV are relatively safe starting values, and that `matrix_tolerance` is the parameter that should adjusted, but we give the user the ability to change all three. The characteristic signature of poorly chosen values are "wiggles" in the plotted band structure.

Tag: `soc_subspace` (control.in)

Usage: `soc_subspace parameter value`

Purpose: Used to change the tolerance parameters associated with subspace assignment for perturbative SOC.

`parameter` The parameter to change. Options are `matrix_tolerance` or `ev_tolerance`.

`value` The value assigned to the parameter chosen by `parameter`.

Default: `matrix_tolerance` 0.1, `ev_tolerance` 1000000000 eV.

This is an optional keyword. See `include_spin_orbit` for more information.

Tag: `soc_perturb_order` (control.in)

Usage: `soc_perturb_order order`

Purpose: Sets the order of quasi-degenerate perturbation theory to use for SOC calculations.

`order` The order of quasi-degenerate perturbation theory to use, an integer from 0 to 3.

Default: 1.

This is an optional keyword. See `include_spin_orbit` for more information.

Tag: `compute_kinetic` (control.in)

Usage: `compute_kinetic`

Purpose: *Experimental* - for test purposes, allows to compute the kinetic energy via the product of the kinetic energy matrix and the density matrix

This flag is presently kept for test purposes only (the electronic kinetic energy is separately computed and printed for each scf iteration anyway) but may be useful for some future modifications.

Tag: `override_relativity` (control.in)

Usage: `override_relativity flag`

Purpose: If explicitly set, allows to override the stop enforced by the code when physically questionable `relativistic` settings are used.

`flag` is a logical expression, either `.true.` or `.false.` Default: `.false.`

For example, this will allow you to run a physically incorrect calculation of heavy elements (think Au) with Schrödinger's expression for the kinetic energy, instead of a scalar relativistic treatment. The results will be wrong, so this flag should only be set for test purposes. When set, the code assumes that the user must know what they are doing.

Tag: `relativistic` (control.in)

Usage: `relativistic r-type s-type [threshold]`

Purpose: Specifies the level of relativistic treatment in the calculation.

`r-type` is a string, specifying the basic approximation made.
`s-type` is a string, specifying whether a scalar treatment is desired (currently, only the `scalar` option is supported).
`threshold` is a small positive real number, allowing to reduce some integration effort.

Detailed expressions for the scalar relativistic treatments available here are given in Ref. [?]. We here only repeat the salient options and expressions. Possible options for `r-type` are:

- `none` : Non-relativistic kinetic energy. In this case, `s-type` and `threshold` need not be provided.
- `atomic_zora` : Atomic ZORA approximation as described in Ref. [?]. `threshold` need not be provided. This is the currently recommended option for energy differences and valence and unoccupied eigenvalues.
- `zora` The ZORA approximation is used throughout the s.c.f. cycle, followed by a "scaled ZORA" [?] post-processing step (rescaling of all eigenvalues). **WARNING:** Do not rely on intermediate, simple ZORA total energies, but only on the final, rescaled total energies instead! ZORA (unscaled) is not the same as "atomic ZORA" and cannot be trusted.

Forces are only provided for `none` and `atomic_zora`.

Remember to never take energy differences between calculations performed with different "relativistic" settings.

In case of `zora`, the `threshold` option is required; it specifies the threshold value above which the difference between the sum-of-free-atoms ZORA expression and that for the actual potential during the s.c.f. cycle will be calculated. In areas of shallow potentials,

where both expressions are substantially similar, this saves the extra integration effort associated with ZORA. For (very!) safe settings, `threshold` may be set to 10^{-12}; in our experience, also 10^{-9} does not lead to any noticeable accuracy loss.

Default is `none` if all elements in the structure have Z <20 (no heavier than Ca). For all heavier elements, an explicit `relativistic` setting is required. For 11< Z <20, the setting `none` will be accepted, but a warning will be issued. For Z>20, choosing `none` will cause the code to stop with an error message in order to avoid accidental calculations with incorrect relativity. If a non-relativistic calculation is still desired, for example for test purposes, this "stop" can be disabled by setting the flag `override_relativity` —but use this only if you know what you are doing.

3.9 Eigenvalue solver and (fractional) occupation numbers

With an updated Hamiltonian matrix h_{ij} and overlap matrix s_{ij} available at the end of a s.c.f. iteration (i, j run over all basis functions), or in the post-processing step of the calculation, FHI-aims updates the Kohn-Sham orbitals l (wave function coefficients c_{jl}) by solving the following eigenvalue problem:

$$\sum_j h_{ij} c_{jl} = \epsilon_l \sum_j s_{ij} c_{jl} \quad . \tag{3.25}$$

In periodic boundary conditions, this eigenvalue problem is solved at every k-point, and $\boldsymbol{k}$ is implictly included in the eigenstate index l above.

Since the basis size needed even for meV-converged accuracy in FHI-aims is rather small, and this size determines the dimension of h_{ij} and s_{ij}, the recommended eigenvalue solver(s) in FHI-aims are customized conventional solvers, employing the same basic algorithms as Lapack or the parallel ScaLapack implementation, but with significant scalability enhancements. Although these solvers scale strictly as $O(N^3)$ with system size, their application becomes dominant only for systems above ≈1000 atoms (light elements) or ≈500 atoms (heavy elements, e.g. Au) in our experience.

The present section describes the available eigensolver options in FHI-aims, including the determination of a Fermi level and occupation numbers for all orbitals following the process.

FHI-aims also offers the possibility to solve a *constrained* eigenvalue problem, e.g., in order to restrict the number of spin-up or spin-down electrons in the basis functions of a given set of atoms. Since this functionality is experimental and for experienced users only, it is documented separately in Sec. 3.14.

Finally, we emphasize that the basis set in FHI-aims *is* non-orthogonal. For all practical production settings, this is not a problem, and in fact taken care of through the overlap matrix s_{ij} in Eq. (3.25) above. It is, however, still possible to generate an overcomplete, nearly ill-conditioned basis set in practical calculations, usually by specific, *deliberate* user action. The signature of such ill-conditioning are near-zero eigenvalues of s_{ij} (e.g., 10^{-5} and below). Possible reasons include: systematically constructed, deliberately overconverged basis sets for non-periodic calculations (not easy); excessively large cutoff radii in dense periodic structures together with very large basis sets (the density of non-zero basis functions per volume element increases as r_{cut}^3); or, badly integrated, very extended basis functions (diffuse Gaussian basis functions without increasing `radial_multiplier` appropriately).

FHI-aims does include a number of safeguards against an ill-conditioned overlap matrix, most importantly the `basis_threshold` keyword that projects out the eigenvectors of the overlap matrix that correspond to its smallest eigenvalues, usually enabling a meaningful calculation anyway. However, to alert every user to the fact that their chosen basis set may be ill-conditioned, the code now stops when it encounters an overlap matrix with too low eigenvalues—unless the keyword `override_illconditioning` is deliberately set, indicating that the user knows what they are doing and wishes to

continue regardless.

Tags for general section of `control.in`:

Tag: `basis_threshold` (control.in)

Usage: `basis_threshold threshold`

Purpose: Threshold to prevent any accidental ill-conditioning of the basis set.

`threshold` is a small positive threshold for the eigenvalues of the overlap matrix. Default: 10^{-5}.

Since NAO basis functions are situated at different atomic centers in a structure, they form a non-orthogonal basis set by construction. Usually, this is not a problem, since the non-orthogonality is naturally accounted for by inserting the overlap matrix s_{ij} into the Kohn-Sham eigenvalue problem, Eq. (3.25). For very large basis sets, this can lead to accidental ill-conditioning (some basis functions may be exactly expressable as linear combinations of some others).

This behavior is detected by directly inverting the overlap matrix, and computing its eigenvalues. If one or more eigenvalues are smaller that `threshold`, the corresponding eigenvectors are projected out of the basis before solving the Kohn-Sham eigenvalue problem, and the latter is solved after transforming to the reduced eigenbasis-set of the overlap matrix.

Important change: Even when `basis_threshold` is set, FHI-aims will automatically stop when a near-singular overlap matrix is detected. The user can still override this safeguard by setting the `override_illconditioning` keyword in `control.in` explicitly, but we do now do our best to alert the user to this condition.

Tag: `elpa_settings` (control.in)

Usage: `elpa_settings setting`

Purpose: Allows to determine the exact algorithm used in the ELPA eigensolver by hand.

`setting` is a descriptor (string) that selects certain aspects of ELPA. Default: `auto`

If the parallel ELPA eigensolver is used (see keyword `KS_method`), a number of choices are made automatically by default. The `elpa_settings` keyword allows to set some of these aspects by hand. Allowed choices for `setting` are:

- `auto` : The default. ELPA makes all its choices on the fly.
- `one_step_solver` : Only the one-step tridiagonalization (and corresponding back transformation) are used. This is usually the slower choice, but not always ...
- `two_step_solver` : Only the two-step tridiagonalization (and corresponding back transformation) are used. This is usually the faster choice, but not always ...

The elpa_settings keyword is particularly useful
(i) if you already know what the faster choice is, and you wish to eliminate the extra test of the slower solver from your calculations, or
(ii) if you suspect that one of the two solvers links to a buggy external(!) library. Lapack and BLAS implementations (still used in elpa) come from many vendors, they are often precompiled, and of course they *always* work—the computer vendor hopes so, after all. We have seen our share of bugs in external libraries (outside the control of FHI-aims), and sometimes, switching the algorithm to change the exact subroutines used can be a helpful backup check.

Tag: `empty_states` (control.in)

Usage: `empty_states number`

Purpose: Specifies how many Kohn-Sham states *beyond* the occupied levels are computed by the eigensolver.

`number` is the integer number of empty Kohn-Sham states *per atom* to be computed beyond the occupied levels.

For DFT-LDA/GGA, typically only a small (but non-zero) number of empty states is required to allow a complete determination of the Fermi level.

By default, $(l{+}1)^2{+}2$ states are added *for each* atom in the structure, where l is the maximum valence angular momentum in the valence of that atom ($l{=}0$ for hydrogen, but $l{=}3$ for f-electron atoms and beyond).

For correlated methods including excited states (MP2, RPA, GW, ...), *all* available states should be included. To achieve this, set `empty_states` to a large number (safely larger than your basis set).

Tag: `fermi_acc` (control.in)

Usage: `fermi_acc tolerance`

Purpose: The precision with which the Fermi level for occupation numbers will be determined.

`tolerance` : Tolerance parameter for the zero point search of the equation $\sum_i f_{\text{occ}}[\epsilon_F](\epsilon_i) - n_{\text{el}} = 0$. Default: 10^{-20}.

Usually, this tag need not be modified from the default. Within the (standard) Brent's method search for the Fermi level, `tolerance` has more than one function. Leave untouched unless problems arise.

Tag: `initial_ev_solutions` (control.in)

Usage: `initial_ev_solutions number`

Purpose: *Experimental!* Number of initial eigenvalue solutions using direct methods before switching on the lopcg-solver. Applies for both, Lapack and ScaLapack variants of the solver.

`number` is a positive integer. Default: 5.

Tag: `KS_method` (control.in)

Usage: `KS_method KS_type`

Purpose: Algorithm used to solve the generalized eigenvalue problem Eq. (3.25).

`KS_type` is a keyword (string) which specifies the update method used for the Kohn-Sham eigenvectors in each s.c.f. iteration. Default: `lapack_fast` or `scalapack_fast`, depending on available number of CPUs.

Available options for the eigensolver, `KS_type`, are:

- `lapack_fast` : **Preferred serial eigensolver implementation**. Lapack-based, and similar to the divide&conquer based standard solver provided by Lapack itself.
- `elpa` : **Preferred parallel eigensolver implementation. Uses the ELPA eigensolver library.** Same functionality as `scalapack_old`, however, substantially rewritten for an overall speedup and much improved scalability. Recommended on shared-memory platforms and distributed systems with any decent communication.
  See the `elpa_settings` keyword for some ELPA internals (usually determined automatically, but who knows).
  Note that you must set the shell variable `OMP_NUM_THREADS=1` prior to running FHI-aims on some platforms. (see Appendix A)
- `scalapack` : Synonymous with `elpa`.
- `scalapack_fast` : Synonymous with `elpa`.
- `lapack_old` : Expert solver provided by standard Lapack. This is stable and not a bottleneck in most standard situations.
- `lapack` : Synonymous with `lapack_fast`.
- `scalapack_old` : Fully memory-parallel implementation of the eigenvalue solver based on ScaLapack itself, scales much worse that our own `scalapack-fast`.
- `svd` : Effectively the same as `lapack_old`.
- `lopcg` : *Experimental – under development* Iterative, locally optimal preconditioned conjugate gradient eigensolver. Potentially useful for *very* large systems where `lapack` becomes a bottleneck. However, implementation without any serious testing—contact us if interested.

- `scalapack+lopcg` : *Experimental – under development*. Same as `lopcg`, but parallel with ScaLapack-type memory distribution.

The ScaLapack or ELPA eigensolvers are only available if ScaLapack support has been compiled into the FHI-aims binary—see the Makefile for more information.

In fact, the default parallel eigensolver in FHI-aims is the "ELPA" eigensolver, which uses some scalapack infrastructure but has been rewritten from the ground up for much improved parallel scalability.

On mildly parallel platforms (no more than tens of CPUs) and for normal system sizes (not significantly larger than ∼100 atoms), it is often acceptable to use the serial `lapack` eigensolver instead of the fully parallel `elpa`. Note that a (separate) parallelization over k-points will be performed in periodic systems in any case.

ELPA also allows calculations *without* explicitly collecting the resulting eigenvectors to each thread after the eigensolution is complete. This improves the memory efficiency especially in large-scale / massively parallel situations. For details, see keyword `collect_eigenvectors` .

Prior to the solution of Eq. (3.25) using `lapack` or `elpa`, the overlap matrix s_{ij} is checked for ill-conditioning (see `basis_threshold` keyword). For very large basis sets or periodic calculations with many k-points, this criterion may trigger. In that case, the Hamiltonian matrix is transformed to the "safe" set of eigenvectors of s_{ij}, and the transformed eigenvalue problem is solved. If you suspect ill-conditioning to be a problem, it may also be helpful to increase the density of the 3D integration grids in order to minimize any numerical noise in s_{ij} and h_{ij}. That said: In our experience, ill-conditioning is not a problem with accurate basis sets in standard calculations; see Appendix A for some additional comments.

Tag: `lopcg_adaptive_tolerance` (control.in)

Usage: `lopcg_adaptive_tolerance flag`

Purpose: *Experimental!* Allows the lopcg-algorithm to dynamically adjusts its convergence tolerance as $\max\{0.01\,|\delta n|,$ `lopcg_tolerance` $\}$ where δn is the change in the electron density as recorded in the self-consistency cycles.

`flag` is a logical expression. Default: `.false.`

Tag: `lopcg_block_size` (control.in)

Usage: `lopcg_block_size number`

Purpose: *Experimental!* The maximal size of a block in lopcg-iteration.

`number` is a positive integer. Default: 1.

Tag: `lopcg_auto_blocksize` (control.in)

Usage: `lopcg_auto_blocksize flag`

Purpose: *Experimental!* Selects if the lopcg algorithm tries to find automatically a better blocksize than the maximal one by grouping close eigenvalues together.

`flag` is a logical expression. Default: `.false.`

Tag: `lopcg_preconditioner` (control.in)

Usage: `lopcg_preconditioner type`

Purpose: *Experimental!* For `KS_method lopcg`, specifies the preconditioner used.

`type` is a string, either `diagonal` (diagonal preconditioning matrix) or `ovlp_inverse` (use inverse of the overlap matrix for preconditioning).

Tag: `lopcg_start_tolerance` (control.in)

Usage: `lopcg_start_tolerance tolerance`

Purpose: *Experimental!* Sets the tolerance for starting the lopcg-solver using the change in the sum of eigenvalues as a criterion. The lopcg-solver is activated as set in `initial_ev_solutions` latest, but `lopcg_start_tolerance` may trigger it earlier.

`tolerance` is a double precision real. Default: 0.0

Tag: `lopcg_tolerance` (control.in)

Usage: `lopcg_tolerance tolerance`

Purpose: *Experimental!* Sets the convergence tolerance for the lopcg-solver.

`tolerance` is a double precision real. Default: 10^{-6}.

Tag: `max_lopcg_iterations` (control.in)

Usage: `lopcg_tolerance number`

Purpose: *Experimental!* Sets the maximal number of iterations for one block in the the lopcg-solver.

`number` is an integer. Default: 100.

Tag: `max_zeroin` (control.in)

Usage: `max_zeroin number`

Purpose: Number of iterations allowed in Brent's method to find the Fermi level.

`number` is an integer number. Default: 200.

Usually, this tag need not be modified from the default. This limits the number of allowed iterations for the (standard) Brent's method search for the Femi level. Leave untouched unless problems arise. Note that changing the values given for `occupation_type` or `empty_states` may be the true fixes if the search for a Fermi level really ever fails.

Tag: `occupation_acc` (control.in)

Usage: `occupation_acc tolerance`

Purpose: Accuracy with which the sum of calculated occupation numbers for a given Fermi level reproduces the actual number of electrons in the system.

`tolerance` is a small positive real number. Default: 10^{-8}.

Usually, this tag need not be modified from the default. Determines the target accuracy for the Fermi level (calculated vs. actual number of electrons in the system) in the search using Brent's method. Note that changing the values given for `occupation_type` or `empty_states` may be the true fixes if the search for a Fermi level really ever fails.

Tag: `occupation_type` (control.in)

Usage: `occupation_type type width [order]`

Purpose: Determines the broadening scheme used to find the Fermi level and occupy the Kohn-Sham eigenstates.

`type` is a string which determines the desired broadening function. Default: `gaussian`
`width` specifies the width of the broadening function [in eV]. Default: 0.01 eV.
`order` is an integer, and only required to specify the order of `type` `methfessel-paxton`.

Based on the eigenvalues ϵ_l of each s.c.f. iteration, the selected `occupation_type` determines the Fermi level ϵ_F and occupies all Kohn-Sham states with fractional occupation numbers $f_l(\epsilon_F)$ for the following electron density update. Detailed discussions can be found in Ref. [?] or other standard literature [?]. We only briefly list the available options for the occupation `type` here:

- `gaussian` : Gaussian broadening function [?]

$$f_l = 0.5 \cdot [1 - \mathrm{erf}(\frac{\epsilon_l - \epsilon_F}{\texttt{width}})]$$

- `methfessel-paxton` : Generalized Gaussian-type distribution functions of Methfessel and Paxton (see Ref. [?] for details). In practice, any `order` beyond 1 is not recommended.

- `fermi` : Formally correct finite-temperature broadening scheme [?]

$$f_l = \frac{1}{1 + \exp[(\epsilon_l - \epsilon_F)/\texttt{width}]}$$

  However, to be useful in practice, `width` must take on values significantly greater than kT at room temperature, and therefore mostly loses its physical meaning. In practice, `fermi` broadening seems to lead to faster-increasing total energy inaccuracies than `gaussian` broadening, which is why the latter is preferred in FHI-aims.

For metallic systems / systems with small HOMO-LUMO gap, the availability of an occupation scheme with finite width (e.g., 0.1 eV) is critical to guarantee the stable convergence of the s.c.f. cycle. Especially for metallic systems, FHI-aims outputs an extrapolated total energy, which estimates the total energy for zero broadening based on the entropy of the electron gas [?, ?, ?]. This extrapolated total energy must only be used for metallic systems, not, e.g., for atoms with a decidedly discrete density of states.

For non-metallic systems / systems with appreciable HOMO-LUMO gap, the broadening width must be finite in order to guarantee the existence of a formal Fermi level, but not so large as to lead to any actual fractional occupation numbers. In our experience, the default width of 0.01 eV performs well for this purpose.

Tag: `override_illconditioning` (control.in)

Usage: `override_illconditioning flag`

Purpose: Allows to override a built-in stop and run with a nearly singular overlap matrix.

`flag` is a logical flag, either `.true.` or `.false.` Default: `.true.`

If the overlap matrix s_{ij} has an eigenvalue below `basis_threshold` or below 10^{-5} (whichever is larger), FHI-aims will stop and warn the user of a potentially ill-conditioned basis set. Usually this situation can still be resolved by setting an appropriate value of `basis_threshold`, but anyone relying on this functionality should first check whether their "ill-conditioning" condition is not also due to another, inadvertent choice, such as an insufficient integration grid for very extended functions, or an excessively large cutoff radius in dense periodic systems (is it really necessary?).

In other words: By all means, override if you wish, but check first whether all computational settings are actually intentional and appropriate.

3.10 SCF Cycle: Initialization, density mixing, preconditioning, convergence

The preceding tasks (charge density update, Hartree potential, Hamiltonian and eigenvalue solver) are all methodologically simple, with well-defined standard choices, since they all relate to the densities and potentials within a single s.c.f. iteration of the Kohn-Sham equations only.

However, in order to run a complete, self-consistent Kohn-Sham or generalized ground state calculation, many such cycles must be performed. Beginning with well-defined initial criteria, self-consistency of the charge density and orbitals must be reached, and must be reached within a rather finite number of iterations.

For many standard problems in electronic structure theory–especially systems with a large, well-defined HOMO-LUMO or band gap—reaching self-consistency today presents essentially no problem, and is achieved to great accuracy already within ≈10 or so iterations.

However, in cases where the band structure is metallic, different charge or spin states are close to one another or in competition, there may be several self-consistent solutions, depending on the exact chosen initialization. Even worse, in such cases reaching even a *single* one of potentially several different self-consistent solutions can be problematic.

The present section summarizes all available options in FHI-aims to facilitate the self-consistent solution of any given problem in FHI-aims in as few iterations as possible, including:

- Initialization of the s.c.f. cycle
- Criteria for the convergence of the self-consistency solution
- Electron density mixing
- Electron density *preconditioning*

Please refer to Ref. [?] for a more exhaustive discussion of the physics / mathematics behind the individual choices laid out below.

Important note: The following settings are made or required by default.

- *The initial spin density must be specified in a spin-polarized calculation.* In spin-polarized systems, the choice of a good initial spin density can be critical for good convergence. For example, for a free atom, you might wish for a high-spin initial density according to Hund's rules. In a ferromagnetic Fe crystal, you might want to use a `default_initial_moment` of 2 (far lower than the Hund's rule value) to obtain fast convergence. In an antiferromagnetic Cr crystal, a ferromagnetic default initialization might do no good at all. And in a molecule with a single magnetic atom enclosed, you might want a spin-polarized initial moment only for that atom, but not for the surrounding molecule. In short, FHI-aims can not and should not guess the spin initialization for you. The program will now stop

if no inital moments of any kind are provided by the user. Setting either an overall `default_initial_moment` (in `control.in`), or at least one individual `initial_moment` tags in `geometry.in`, or both will allow you to run as usual.

- *Use of the Kerker preconditioner for periodic systems.* This option can greatly improve the s.c.f. convergence especially of large periodic systems (see `preconditioner` for more details). At the very least, it does not appear to do much harm, and is therefore now used by default in any periodic calculation. Should you encounter any difficulties, either turn the `preconditioner` off by hand, or play with associated screening momentum, q_0 (default: q_0=2.0 bohr^{-1}).

Regarding options to converge the self-consistency cycle, note that one further important parameter is not covered here but instead in Sec. 3.9: The "broadening" of (fractional) occupation numbers around the Fermi level. Especially in metallic systems, this broadening must be large enough to prevent oscillations around the Fermi level, independent of the methods laid out below.

For further suggestions to improve s.c.f. convergence, see Sec. A.8.

Tags for geometry.in:

Tag: initial_charge (geometry.in)

Usage: initial_charge charge

Purpose: Allows to place an initial charge on an atom in file geometry.in.

charge is a real number. Default: 0.

The initial_charge keyword always applies to the last atom previously specified in input file geometry.in. The charge is introduced by using an ionic instead of neutral spherical free-atom density on that site in the initial superposition-of-free-atoms density. Note that initial charge densities are generated by the functional specified with xc for DFT-LDA/GGA, but refer to pw-lda densities for all other functionals (hybrid functionals, Hartree-Fock, ...).

Tag: initial_moment (geometry.in)

Usage: initial_moment moment

Purpose: Allows to place an initial spin moment on an atom in file geometry.in.

moment is a real number, referring to the electron difference $N^{\uparrow} - N^{\downarrow}$ on that site. Default: Hund's rules, unless default_initial_moment is set explicitly.

The initial_moment keyword always applies to the last atom previously specified in input file geometry.in. The moment is introduced by using a spin-polarized instead of an unpolarized spherical free-atom density on that site in the initial superposition-of-free-atoms density. Note that initial charge densities are generated by the functional specified with xc for DFT-LDA/GGA, but refer to pw-lda densities for all other functionals (hybrid functionals, Hartree-Fock, ...).

Note that Hund's rules may be ambiguous. We therefore specify the default state for certain spin-polarized atoms explicitly (for initialization only!). For example, Cu is $4s^1 3d^{10}$, but could also be set (less efficiently) to be $4s^2 3d^9$.

Tags for general section of `control.in`:

Tag: `charge_mix_param` (control.in)

Usage: `charge_mix_param value`

Purpose: Parameter for simple linear mixing of electron densities of previous and present s.c.f. iterations

`value` is a real number between 0. and 1. Default: Depends on chosen `mixer` algorithm.

See Ref. [?] for details regarding the available density mixing algorithms. In the simplest case of a `linear` `mixer`, `value` specifies a constant value $\hat{G}^1$ to mix the output density of the Kohn-Sham Equations in iteration number μ, $n_{\mathrm{KS}}^{(\mu)}$, with the (already mixed) *input* density that defined those equations, $n^{(\mu-1)}$:

$$n_{\mathrm{dmp}}^{(\mu)} = n^{(\mu-1)} + \hat{G}^1(n_{\mathrm{KS}}^{(\mu)} - n^{(\mu-1)})\,. \tag{3.26}$$

If a `preconditioner` is specified, `charge_mix_param` defines an additional linear factor to that preconditioner. In case of a `pulay` `mixer`, all density residuals are mixed with this factor.

In general, the best choice for `value` is system-dependent, and also depends on the chosen `mixer` algorithm. In general, please also see Appendix A.8, since several keywords can be used to alleviate s.c.f. mixing instabilities.

- In principle, a `linear` `mixer` will always converge with a sufficiently small `value`. In easy cases, we recommend `value`=0.1-0.2, but in difficult cases, "sufficiently small" can mean one to three orders of magnitude(!) lower, i.e., the process can be apallingly slow.
- For a straight `pulay` `mixer`, our default `value` is 0.6, which empirically works well for systems with a reasonable band gap / HOMO-LUMO gap. In metallic systems or systems that are otherwise problematic, values down to `value`=0.05 or even less are usually found to work. Note that this small value does not necessarily correspond to slow mixing since the Pulay mixer will learn over time and accelerate the mixing process.
- In metallic systems, density oscillations can occur from one iteration to the next (charge sloshing). This can be alleviated by a `preconditioner`. With a preconditioner and `pulay` `mixer` specified together, `value` is still important and may be chosen around 0.05 .

See also the `mixer` and `preconditioner` keywords. `charge_mix_param` sets a default for *all* possible mixing parameters in FHI-aims, but the following can be specified separately (additionally): `ini_linear_mix_param`, `ini_spin_mix_param`, `prec_mix_param`, and `spin_mix_param`. Note that it is not usually necessary to invoke those separate keywords unless the standard choices (`mixer` `pulay`, possibly with `preconditioner`) really fail.

Tag: `relative_fp_charge_mix` (control.in)

Usage: `relative_fp_charge_mix value`

Purpose: Parameter for under-relaxation of the fixed point part of s.c.f. cycle with the `broyden mixer`

`value` is a real number between 0. and 1. Default: 0.05

`relative_fp_charge_mix` determines the under-relaxation of the fixed point part of the Broyden mixer s.c.f. cycle together with `charge_mix_param`. `relative_fp_charge_mix` and `charge_mix_param` are multiplicative, and if no history is included the effective under-relaxation is `relative_fp_charge_mix` times `charge_mix_param`.

Tag: `default_initial_moment` (control.in)

Usage: `default_initial_moment moment`

Purpose: For spin-polarized calculations, sets the default initial moment of the spin-polarized atoms that make up the initial electron density.

`moment` is either a string or a number that defines the desired initial number of electrons, $N^{\uparrow} - N^{\downarrow}$. Default: `hund` for isolated atoms. Zero otherwise.

Sets the default initial spin moment for all atoms whose `initial_moment` s are not specified explicitly in `geometry.in`.

If there is at least a single `initial_moment` keyword specified in `geometry.in`, the `default_initial_moment` will be zero for all other atoms, for which no `initial_moment` is specified explicitly.

If no `initial_moment` is included in `geometry.in` at all, the `default_initial_moment` must be specified explicitly by the user for the code to run at all.

For most (bonded) systems, it is advisable to set the `default_initial_moment` to a numerical value close to what most atoms in the structure will do. For example, ferromagnetic Fe would be close to 2, whereas a large non-magnetic molecule would be better served with something close to zero.

For isolated free-atom calculations, `default_initial_moment hund` can be used. This will result in the usual high-spin atom initialization characteristic of free atoms.

Note that at least one moment in the system must be set to a non-zero value in order to reach any spin-polarized state at all. If the initial spin polarization is zero, the final s.c.f. result will also not be spin-polarized, no matter how magnetic the system is in reality.

Warning. It is not advisable to set a blanket `default_initial_moment hund` for any structures other than free atoms. The result can be enormous convergence difficulties because the calculation begins from a bad starting point – a high-spin state that is completely unrealistic for most bonded structures. A calculation will converge *much* better if the initial spin moment is realistic for *each* individual atom in the structure.

One more warning: It is *not* advisable to set a `default_initial_moment` other

than zero in structures in which only a few atoms actually carry spin (such as a large molecule with a few transition metal atoms). Rather, a good choice would be to set a zero `default_initial_moment` and then set finite `initial_moment` values for individual atoms in `geometry.in`.

And again: Setting `default_initial_moment` to zero and *not* specifying any `initial_moment` values in `geometry.in` will lead to a zero spin state in the final result, simply because the system is never given any indication which way to break its symmetry. So one does need to set a finite moment somewhere if a finite-spin converged solution is expected in the calculation.

The upshot is: It pays to think about the right spin initialization. There may be multiple different self-consistent solutions, and in spin-polarized systems, this can happen for very natural reasons (e.g., ferromagnetic vs. antiferromagnetic states). Similar to geometry optimization, starting from a very bad initial guess can cause problems. Conversely, a good starting point may greatly simplify a calculation.

Tag: `force_potential` (control.in)

Usage: `force_potential type`

Purpose: Determines how far / with which potential the Kohn-Sham equations are solved.

`type` is a string that determines the potential used. Default: `sc`

This option is not required under normal circumstances. It is mainly useful to produce / test a fast, non-selfconsistent solution for a superposition-of-atoms potential that yields only the sum of eigenvalues as a result. If a non-selfconsistent total energy is needed (correct only for the non-spinpolarized free-atom density!), running a normal calculation with `sc_iter_limit` =0 is the better way.

Options for `type` are:

- `sc`: Self-consistent Kohn-Sham potential in each s.c.f. iteration
- `superpos_pot`: Superposition of free-atom potentials, evaluation only once (no self-consistency cycle). *Restriction: This method works only for self-adapting* `angular_grids` *(i.e.,* `angular_acc` *not equal zero for at least one* `species` *). This also means that the option will not perform well in periodic boundary conditions. A fix is simple, contact us if needed.*
- `superpos_rho`: Superposition potential created from sum of free-atom densities; evaluation only once (no self-consistency cycle)
- `non-selfconsistent`: Same as `superpos_rho`.

Tag: `ini_linear_mixing` (control.in)

Usage: ini_linear_mixing number

Purpose: If mixer is pulay, specifies simple linear mixing for a number of initial iterations.

number is the integer number of iterations for which linear mixing is done. Default: 0 .

Try only if the standard / preconditioned pulay mixer definitely fails. Keywords ini_linear_mix_param , ini_spin_mix_param can be used to specify separate mixing parameters for the initial linear mixing.

Tag: ini_linear_mix_param (control.in)

Usage: ini_linear_mix_param value

Purpose: Separate parameter for simple linear mixing of electron densities for ini_linear_mixing .

value is a real number between 0. and 1. Default: same as charge_mix_param .

ini_linear_mixing should only be tried if the standard algorithms provably fail. In that case, value should be relatively small.

Tag: ini_spin_mix_param (control.in)

Usage: ini_spin_mix_param value

Purpose: For spin-polarized calculations, separate parameter to mix the *spin* density during ini_linear_mixing .

value is a real number between 0. and 1. Default: same as spin_mix_param

ini_spin_mix_param should only be tried if the standard algorithms provably fail. In that case, value should be relatively small.

Tag: mixer (control.in)

Usage: mixer type

Purpose: Specifies the electron density mixing algorithm used to achieve fast and stable convergence towards the self-consistent solution.

type specifies the density mixing algorithm used and can be set to either linear, pulay, or broyden. Default: pulay.

FHI-aims provides three mainstream density mixing algorithms across the s.c.f. cycle, type linear, pulay, and broyden. We here only give a brief summary of options, please see Ref. [?] for further references and for the exact mathematical details.

For most practical purposes (non-pathological systems), Pulay's DIIS mixing algorithm [?] is robust and fast, and should be the algorithm of choice. For this algorithm,

n_max_pulay n determines the number of past iterations $\mu-k$ (k=1,...,n) to be mixed with the Kohn-Sham output density of iteration μ. charge_mix_param determines an additional (system-dependent) linear factor that is multiplied with the output density change of the Pulay mixer. Normally, this (and perhaps a preconditioner) is all you need to do to ensure convergence.

In some pathological cases, reaching self-consistency is a more tricky problem. Broadly speaking, these are systems with a small HOMO-LUMO gap (band gap) and/or several competing possibilities for a self-consistent solution. Specifically, these difficult cases include:

- Large metallic systems (e.g., slabs), where charge may "slosh" from one end of the system to another before reaching self-consistency. In that case, the pulay mixer may be used together with a large charge_mix_param *and* a preconditioner (see that keyword) to dampen the resulting oscillations. Also make sure that occupation_type is set to a sufficiently large broadening of occupation numbers near the Fermi level in metallic systems.
- Spin-polarized systems with competing spin states. A classic. If problems arise, playing with ini_linear_mixing , the charge_mix_param and spin_mix_param and further options listed in this section may help. Likewise, setting a specific fixed_spin_moment may be helpful. Finally, different initial_moment settings may easily switch between different metastable self-consistent spin states (e.g., ferromagnetic vs. antiferromagnetic), and should be tested separately if different competing spin states are suspected.
- Systems near a level crossing (even dimers, if two or more Kohn-Sham levels of different symmetry come close for a given binding distance). Apart from the usual mixing mechanisms, keyword occupation_type with a larger broadening near the Fermi level may help alleviate this situation.
- Spin-polarized free atoms. The simplest conceivable systems may exhibit unexpected problems towards self-consistency, likely because the electron density can rotate between several fully equivalent spin states. In particularly nasty cases, breaking the symmetry explicitly (e.g., using a specified homogeneous_field may be useful.

In principle, even in the toughest cases a linear mixer will always converge with a sufficiently small charge_mix_param . Unfortunately, "sufficiently small" can mean a charge_mix_param of 10^{-2}-10^{-4}, i.e., the process can be apallingly slow. Playing with the pulay mixer settings is usually the better strategy, unless a proof of principle is sought.

The broyden mixer is an improvement on the linear mixer . The broyden mixer works by effectively dividing the s.c.f. cycle into two separte parts: the space in which we have local information gained by previous evaluations of the s.c.f. cycle, and the remaining space in which we do not have information. charge_mix_param determines the linear factor which under-relaxes the density predicted by the broyden mixer , n_max_broyden *n* controls the number of past iterations used to construct

the next estimate, and relative_fp_charge_mix is the additional (multiplicative) under-relaxation affecting only the step length in the space where we have no further information.

Note that a modification is needed when going beyond DFT-LDA/GGA (Hartree-Fock, hybrid functionals, ...). In that case, the density implicitly enters the two-electron exchange operator (via the density matrix, $\hat{n}_{ij} = \sum_l f_l c_{il} c_{jl}$, where i and j label basis functions, and l label the Kohn-Sham states), and should also be mixed.

By default, for `linear` mixing, we do not mix the exchange operator, unless keyword use_density_matrix_hf is enabled. The latter is the default if the `pulay` mixer is selected. Then, the density *matrix* is submitted to the same Pulay mixing factors as the normal charge density $n(r)$ before constructing the exchange operator. Note that we do not have a formal density matrix available that corresponds to the *initial* superposition of free-atom densities, making this form of mixing slightly less efficient than for normal Kohn-Sham DFT-LDA/GGA.

Tag: `mixer_threshold` (control.in)

Usage: mixer_threshold `keyword threshold`

Purpose: Allows to cap the density step between two iterations rigorously by setting an explicit threshold.

`keyword` is a string, indicating whether the following is the `charge` or `spin` density threshold.
`threshold` is a real number, the maximum allowed change in the density norm (in electrons). Default: no thresholds.

This option is perhaps useful when there are definite convergence problems with the standard mixing algorithms, but can otherwise safely be ignored.

Tag: `n_max_pulay` (control.in)

Usage: n_max_pulay `number`

Purpose: The number of past iterations that the `pulay` mixer uses for density mixing.

`number` is the number of stored iterations used by the mixer. Default: 8

A larger `number` of stored iterations can sometimes lead to a stabilization of the mixing process. Choosing `number` too large (e.g., 20 and above), though, may destabilize the Pulay matrix, which can become near-singular.

Note that the storage effort associated with Pulay mixing is significant on systems with few CPUs / low memory. Each additional stored iteration requires the storage of two charge density residuals, and two times three charge density gradient residual components. For large systems, low memory, and overall stable mixing, reducing n_max_pulay may be a way to get a given calculation below the most difficult memory barriers.

Tag: `n_max_broyden` (control.in)

Usage: `n_max_broyden number`

Purpose: The number of past iterations that the `broyden mixer` uses for density mixing.

`number` is the number of stored iterations used by the mixer. Default: 8

A larger `number` of stored iterations can sometimes lead to a stabilization of the mixing process. Choosing `number` too large (e.g., 20 and above), though, may destabilize the Broyden matrix, which can become near-singular.

Note that the storage effort associated with Broyden mixing is significant on systems with few CPUs / low memory. Each additional stored iteration requires the storage of two charge density residuals, and two times three charge density gradient residual components. For large systems, low memory, and overall stable mixing, reducing `n_max_broyden` may be a way to get a given calculation below the most difficult memory barriers.

Tag: `postprocess_anyway` (control.in)

Usage: `postprocess_anyway boolean`

Purpose: By default, FHI-aims simply stops if the SCF procedure does not converge. In particular, all desired postprocessing steps are skipped. If you do want postprocessing to be done anyway, set `boolean` to `.true.`.

`boolean` is either `.true.` or `.false.`. Default: `.false.`.

Tag: `prec_mix_param` (control.in)

Usage: `prec_mix_param value`

Purpose: Possible separate mixing parameter while the preconditioner is on.

`value` is a real number between 0. and 1. Default: same as `charge_mix_param`.

It's our tentative observation that a larger mixing parameter (0.5-0.8) is sometimes helpful with the `preconditioner`, but after the `preconditioner` is switched off, a smaller mixing parameter (as set by `charge_mix_param`) may be desirable. `prec_mix_param` can provide the needed separate setting, if desirable.

Tag: `preconditioner` (control.in)

Usage: preconditioner keyword [type] [value]

Purpose: "Master keyword" that precedes any information related to the preconditioner. May appear multiple times in control.in, in different contexts.

Restriction: Because it cannot simply be written in a density matrix formulation, the preconditioner has no effect on the density matrix entering the exchange operator for Hartree-Fock, hybrid functionals, etc.

keyword : A string, indicating the type of information following.
type : If required by keyword, a string with more details.
value : If required by keyword, a numerical value.

Default values:

- Non-periodic systems: preconditioner kerker off (no preconditioner).
- Periodic systems: preconditioner kerker 2.0 (Preconditioner with a momentum of q_0=2.0 bohr^{-1}).

See Ref. [?] regarding the mathematical definition of the Kerker-type [?, ?, ?] preconditioner, which is the currently implemented form.

See keyword precondition_max_l for the angular momentum cutoff specified for the kerker preconditioner .

keyword can have the following forms, controlling various aspects of the preconditioner:

- kerker :
  - if followed by off : No preconditioner used.
  - if followed by value : value is a real positive number, indicating the characteristic momentum q_0 associated with the Thomas-Fermi type screening assumed in the preconditioner (in bohr^{-1}). Typical values in the literature range around 1.0-2.0 bohr^{-1}, but larger values may be useful in small clusters.
- turnoff : To avoid any residual influence on the s.c.f. cycle, the preconditioner can be switched off at a given level of s.c.f. convergence, leaving the remaining convergence to a pure pulay mixer . Possible types of turnoff criteria are:
  - charge : value refers to the root-mean-square deviation between $n^{(\mu-1)}$ (the mixed and preconditioned *input* density to the Kohn-Sham Equations) and $n_{\mathrm{KS}}^{(\mu)}$ (the unmixed *output* density from the Kohn-Sham Equations). Default: sc_accuracy_rho .
  - energy : value refers to the total energy difference between two successive iterations [in eV].
  - sum_ev : value refers to the difference in the eigenvalue sums between two successive iterations [in eV].

  All requested convergence criteria for the preconditioner must be fulfilled. If *no* explicit turnoff criterion is set, the preconditioner turnoff charge,

energy and sum_ev defaults to the same values as sc_accuracy_rho , sc_accuracy_etot , and sc_accuracy_eev , respectively.

- none or off : No preconditioner used.

Tag: **precondition_max_l** (control.in)

Usage: precondition_max_l value

Purpose: Angular momentum cutoff used in the partitioned atom-centered real-space form of the kerker preconditioner .

value is an integer number, specifying an angular momentum. Default: 0.

We use a partitioned atom-centered multipole rewrite of the Kerker preconditioner in angular momentum space, similar in spirit to the Hartree potential (see Ref. [?] for details). In principle, precondition_max_l is thus the equivalent of the l_hartree angular momentum cutoff for the expansion. However, since we here precondition a density *difference* (which reduces to zero as we approach self-consistency), and since we are combatting charge sloshing across potentially faraway parts of the systems, preconditioning the atom-centered *monopole* component of the density difference (value=0) is often all that is needed for the preconditioner to work. This is also the numerically most efficient way of running the preconditioner.

Tag: **kerker_factor** (control.in)

Usage: kerker_factor value

Purpose: value is a real positive number. Additional empirical factor for the last step of the kerker precondioner. Best results were achieved by choosing value as the characteristic momentum $(q_0 - 0.5)$ associated with the Thomas-Fermi type screening assumed in the kerker preconditioner (kerker preconditioner).

This keyword is experimental and derived fully empirically. For notorious cases the keyword may help to achieve convergence (faster). For $q_0 = 1$ the method is identical to the standard kerker preconditioner.

Tag: **restart** (control.in)

Usage: restart file

Purpose: Saves and reads the final wave function of each scf-cycle to/from file.

file is a string, corresponding to the desired restart filename.

If file is not yet present, the calculation simply writes that file during the run. If file is already present, it is read and the wave function contained therein is used to restart the calculation, instead of a fresh superposition of free atoms initialization.

In parallel runs, there is one file for each process, numbered as fileXXX. See also

restart_read_only and restart_write_only . There are limited checks on whether or not the restart file provided is actually from the same system, but ensuring that a given restart file works is mainly the user's responsibility. See also restart_relaxations and MD_restart for more information on the separate restarting process for relaxations and molecular dynamics respectively.

Tag: restart_read_only (control.in)

Usage: restart_read_only file

Purpose: reads the final wave function of the last scf-cycle in a preceding calculation from file.

file is a string, corresponding to the desired restart filename.

Similar to keyword restart , but does not overwrite file at any time (this may facilitate another restart from the same file later on).

Tag: restart_write_only (control.in)

Usage: restart_write_only file

Purpose: writes the final wave function of the last scf-cycle to file for a later restart.

file is a string, corresponding to the desired restart filename.

Similar to keyword restart , but does not read the restart file in case it already exists.

Tag: restart_save_iterations (control.in)

Usage: restart_save_iterations number

Purpose: writes restart information every number scf-iterations or at the end of each cycle.

number is the integer number of s.c.f. iterations after which the restart information is rewritten.

See also restart .

Tag: force_single_restartfile (control.in)

Usage: force_single_restartfile .true.

Purpose: Forces FHI-aims to always write a single, wavefunction based restart file if possible.

For technical and efficiency reasons FHI-aims uses different types of restartfiles depending on the eigenproblem solver (see Section 4.7 for details). In cases where the wavefunction is needed (e.g. for external post-processing, . . .), the default restart handling might

make it difficult to do so.

This option only works for cluster (non-periodic) and periodic Γ-only calculations.

Tag: `calc_dens_superpos` (control.in)

Usage: `calc_dens_superpos` .true.

Purpose: When reinitializing the SCF cycle, fall back to the superposition of free atoms density for the initial guess (instead of using the guess from the previous converged cycle).

Tag: `sc_abandon_etot` (control.in)

Usage: `sc_abandon_etot iter threshold`

Purpose: If the s.c.f. cycle diverges, abort the s.c.f cycle after a specified number of iterations between which the total energy changed by more than a given threshold.

`iter` is an integer number of iterations after which the calculation is aborted. Default: 5 iterations.

`threshold` is a positive real number - if the total energy keeps changing by more than this number [in eV], the abort will be triggered. Default: 1000 eV.

This keyword allows to catch obviously ludicrous runs. If it triggers, something went seriously wrong during the mixing procedure and the settings for mixers, occupation broadening, preconditioner etc. should all be revisited very carefully.

The alternative setting `sc_abandon_etot never` switches the abort off, restoring the previous behaviour.

Tag: `sc_accuracy_eev` (control.in)

Usage: `sc_accuracy_eev value`

Purpose: Convergence criterion for the self-consistency cycle, based on the sum of eigenvalues.

`value` is a small positive real number [in eV], against which the difference of the eigenvalue sum between the present and previous s.c.f. iteration is checked.

Very sensitive criterion for s.c.f. convergence. Usually, `value`$=10^{-3}$ eV is enough to indicate a reliable total-energy and force convergence. If value is set to zero or not given, the sum of eigenvalues will not be used as a convergence criterion.

Tag: `sc_accuracy_etot` (control.in)

Usage: sc_accuracy_etot value

Purpose: Convergence criterion for the self-consistency cycle, based on the total energy.

value is a small positive real number [in eV], against which the difference of the total energy between the present and previous s.c.f. iteration is checked.

The Harris-Foulkes form of the functional is used as the total energy in FHI-aims (see Ref. [?] for a brief discussion). A typical tight convergence criterion is value=10^{-6} eV. If value is set to zero or not given, the total energy will not be used as a convergence criterion.

Tag: sc_accuracy_forces (control.in)

Usage: sc_accuracy_forces value

Purpose: Convergence criterion for the self-consistency cycle, based on energy derivatives ("forces").

value is EITHER a small positive real number [in eV/Å], against which the maximum difference of atomic forces between the present and previous s.c.f. iteration is checked, OR a string, 'not_checked'.

Attention: If keyword sc_accuracy_forces is set in control.in, forces are by default computed, regardless of whether or not they are later needed. The rationale is that the only way to check a requested force convergence criterion is to compute the necessary forces, despite the added numerical effort. For single-point calculations (no relaxation required), sc_accuracy_forces should therefore not be set unless explicitly needed for some reason.

The default for sc_accuracy_forces depends on the required type of calculation:

- If no forces are needed / requested, sc_accuracy_forces is by default not checked.
- If forces are needed, the default for value is 10^{-4} eV/Å. However, this is then still checked against the requested relax_geometry force accuracy below.
- If a geometry optimization is requested, the value for sc_accuracy_forces is checked against the requested convergence accuracy of keyword relax_geometry . If the requested accuracy for relax_geometry is less than 10 times the specified sc_accuracy_forces criterion, the latter value is set to 1/10 of the relax_geometry accuracy.

Calculating forces is relatively expensive, so this convergence criterion is only checked *after* the purely electronic / energetic criteria, sc_accuracy_eev , sc_accuracy_etot , sc_accuracy_rho , are all fulfilled. To avoid too many iterations with force computations before the forces are converged, it is important to set the other criteria (especially sc_accuracy_eev , which checks a non-variational quantity) to tight convergence, as indicated above. For sc_accuracy_forces itself, e.g., value=10^{-4} eV/Å is a reliable and robust criterion to avoid noise in geometry relaxations.

You can also simply set the keyword value to the string 'not_checked' . In this case, the forces are computed in only a single shot, their s.c.f. convergence will not be checked.

For simple structure relaxation, not_checked will often be good enough if the electronic criteria (especially sc_accuracy_eev) are set reasonably tightly.

For molecular dynamics, however, even slightly underconverged forces may lead to long-term energy drifts in (nominally) constant-energy runs. Here, it is a good idea to try out in explicit tests how tightly sc_accuracy_eev must be set to guarantee drift-free trajectories.

Tag: sc_accuracy_rho (control.in)

Usage: sc_accuracy_rho value

Purpose: Convergence criterion for the self-consistency cycle, based on the charge density.

value is a small positive real number [in electrons], against which the volume-integrated root-mean square change of the charge density between the present and previous s.c.f. iteration is checked.

By default, FHI-aims checks separately the convergence of the charge density and the spin density, using the same criterion. Specifically, the *unmixed* output density of the Kohn-Sham Equations, $n_{\mathrm{KS}}^{(\mu)}$, is checked against the input density $n^{(\mu-1)}$ to the same equations. A typical tight convergence criterion is value=10^{-5}. If value is set to zero or not given, the charge density will not be used as a convergence criterion.

Tag: sc_accuracy_stress (control.in)

Usage: sc_accuracy_stress value

Purpose: Convergence criterion for analytical stress.

value is EITHER a small positive real number [in eV/Å^3], against which the maximum difference of the analytical stress components between the present and previous s.c.f. iteration is checked. A negative number, or simply the string 'not_checked' results in no convergence check. Default: not_checked .

Warning. Setting sc_accuracy_stress to a finite non-negative value will result in a disproportionately large computational cost. In this case, the number of stress evaluations per relaxation step is at least doubled. This is normally by far the most expensive part of the calculation. Only set this to a finite value if you have a good reason to do so.

The default for value is not_checked, i.e. the convergence of the analytical stress will not be checked. *However*, you have to ensure that other convergence criteria (especially sc_accuracy_eev , which checks a non-variational quantity) are set to tight values, as indicated above.

Calculating the analytical stress is relatively expensive, so this convergence criterion is only checked *after* the purely electronic / energetic criteria, sc_accuracy_eev

, sc_accuracy_etot , sc_accuracy_rho , are all fulfilled. To avoid too many iterations with analytical stress computations before the analytical stress is converged, it is important to set the energetic criteria to tight convergence.

Tag: `sc_accuracy_potjump` (control.in)

Usage: sc_accuracy_potjump value

Purpose: Convergence criterion for the self-consistency cycle, based on the vacuum level potential shift.

value is a small positive real number [in eV], against which the difference of the dipole correction potential jump between the present and previous s.c.f. iteration is checked.

This keyword only makes sense (and is only accepted) for periodic slab calculations with the option use_dipole_correction set. If you are interested in the work function or vacuum level shifts explicitly, it is recommended to use this flag. A typical tight convergence criterion is value=10^{-4}.

Tag: `sc_iter_limit` (control.in)

Usage: sc_iter_limit number

Purpose: Maximum number of s.c.f. cycles before a calculation is considered and abandoned.

number is an integer number. Default: 1000

sc_iter_limit is a keyword that should be set in every run. Note: **You must ensure for every run that the self-consistency cycle was actually converged.** If this is not the case, a loud warning is issued in the standard output of FHI-aims at the end of the s.c.f. cycle, and relaxations, molecular dynamics, and postprocessing may continue anyway depending on the postprocess_anyway setting.

If the end of the s.c.f. cycle is reached in this way, forces are computed regardless of whether the electronic convergence was reached.

Tag: `spin_mix_param` (control.in)

Usage: spin_mix_param value

Purpose: Separate parameter to mix the spin density between different s.c.f. iterations.

value is a real number between 0. and 1. Default: same as charge_mix_param .

spin_mix_param may be different from charge_mix_param , but there is not usually a clear recipe *how* it should be different. This option is thus only needed if the standard algorithms provably fail.

Tag: `switch_external_pert` (control.in)

Usage: `switch_external_pert number type`

Purpose: May be used as a combined parameter to switch on an artificial perturbing `homogeneous_field` only for a given number of iterations.

`number` is the integer number of s.c.f. iterations before the external perturbation is switched off.
`type` is a string. If set to `safe`, specific settings for the `occupation_type` and for the `homogeneous_field` are enforced (see below).

This is an experimental keyword that allows to switch on an initial `homogeneous_field`, e.g., to lock in the symmetry of a free atom in order to enforce smoother s.c.f. convergence. The field is switched off after `number` iterations, before self-consistency is reached.

If `type` is not `safe`, the actual value of `homogeneous_field` and `occupation_type` should be set explicitly in `geometry.in` and `control.in`, respectively. The default homogeneous field is 10^{-3} eV/Å.

If `type` is set to safe, a homogeneous field of 10^{-3} eV/Å and a Gaussian occupation with very small (10^{-5} eV) broadening are automatically enforced.

This flag is mainly used to artificially break the symmetry of spin-polarized free atoms with open d and f shells, which are sometimes very hard to converge otherwise (see special cases listed for `mixer`). Remember that FHI-aims does not allow to enforce a given symmetry automatically.

Tag: `use_density_matrix_hf` (control.in)

Usage: `use_density_matrix_hf`

Purpose: Technical keyword that states that the density matrix is mixed prior to constructing the exchange matrix in hybrid functionals, Hartree-Fock, *et al.*

Default: When possible, `use_density_matrix_hf` is assumed.

3.11 Energy derivatives (forces, stress) and geometry optimization

With self-consistent Kohn-Sham orbitals, densities and total energies available, one of the primary tasks of electronic structure theory is to obtain energy *derivatives* with respect to the nuclear coordinates $\boldsymbol{R}_{\text{at}}$. First derivatives ("forces") allow to find the optimum structure and molecular dynamics on the Born-Oppenheimer surface. Based on the structure optimum, second derivatives then enable the calculation of the (Born-Oppenheimer) zero-point vibrational energy of the nuclei, and of vibrational excitations (phonons).

The present section deals with the options available in FHI-aims for the computation of forces in FHI-aims, and with algorithms related to geometry optimization. Specifics regarding first-principles molecular dynamics are given in Sec. 3.12, and the computation of second energy derivatives (vibrational frequencies, zero-point energies, and oscillator strengths) by a finite-difference technique is covered in Sec. 4.6.

Before coming to the full set of related keywords, we give some basic comments below. Please also consider using (and adding to) the wiki, especially if you encounter any kind of behavior that is obviously not intended. We would like to know about such cases.

One "most important" rule:

For efficient local structure optimizations, do not simply use "tight" settings from an arbitrary starting geometry and wait.

It is usually *much* more effective to use "light" settings first in a pre-relaxation step. Then, use "tight" settings only for post-processing, e.g., based on the "geometry.in.next_step" file that is now written out by the code by default.

Basic handling:

For most applications, simply specifying a convergence criterion `sc_accuracy_forces` will be enough to switch on the computation of forces itself. Reliable structure optimizations of atomic coordinates can then be obtained using the `relax_geometry` with the `bfgs` algorithm.

A note on naming conventions: *The default version of BFGS calls the trust radius enhanced variant of BFGS, also available as `trm`. A standard BFGS variant (sub-keyword `bfgs_textbook`) is also available as a fallback.*

Beginning with code version "012213", we have also formalized the handling of *restarting* relaxations more clearly. If `relax_geometry bfgs` is used, a file `geometry.in.next_step` is written out by default which contains both the geometry and the current estimate of the Hessian matrix of the system that are used by the BFGS algorithm. For an organized restart, simply copy this file to `geometry.in`, and the stored geometry and Hessian will be used to reinitialize the BFGS algorithm if the `control.in` file requests a structure relaxation.

There are also keywords `hessian_to_restart_geometry` and `write_restart_geometry` that can be set to ".false." to suppress the writing of the Hessian information, or even

of the `geometry.in.next_step` file as a whole, if that information is deliberately not wanted.

Unit cell shape optimization:

In periodic calculations, you may also want to relax the unit cell shape itself. In this case, an *additional* keyword `relax_unit_cell` must be set. Where it is available, an analytically computed stress tensor will be used, thanks to work by Viktor Atalla, Christian Carbogno, and Franz Knuth [?]. For other functionals, the unit cell itself can still be relaxed, but the stress tensor must be computed numerically from finite differences of total energies.

The numerical finite-difference stress tensor is always available as a fallback.

However, we also note that, in some cases, just optimizing the unit cell shape blindly may not be what you want. For example, in high-symmetry structures a fit to Murnaghan's Equation of state—see the utility provided in the *utilities* directory—will be more accurate and give you more information about the system (bulk moduli and, in case of more than one phase, also access to transition pressures).

Hybrid functionals:

For hybrid functionals, analytic energy gradients and the analytic stress tensor are only available together with the `RI_method LVL_fast` version of "resolution of identity" of the two-electron Coulomb operator. This is a localized, linear-scaling version of the more traditional "RI-V" method of quantum chemistry (see Ref. [?] for a comprehensive introduction of "RI-V").

At present, geometry optimizations and molecular dynamics using hybrid functionals work, but `RI_method LVL_fast` must be used.

You may wish to check any resulting total energies by single point calculations (no relaxation) using `RI_method V` afterwards.

Note also that `RI_method LVL_fast` itself becomes more accurate as one goes to larger basis sets. One may, in fact, augment only the auxiliary basis that is used to expand the Coulomb operator in order to increase the accuracy of the Coulomb operator itself even when a small orbital basis set is used. The keyword that allows one to do this is `for_aux` .

For perturbative methods (MP2 perturbation theory, RPA and beyond), analytical gradients are not yet available.

Tags for `geometry.in`:

Tag: `constrain_relaxation` (geometry.in)

Usage: `constrain_relaxation constraint`

Purpose: In `geometry.in`, fixes the position of the last specified `atom` / `lattice_vector` in a structure optimization.

`constraint` is a string, indicating what exactly will be constrained. Default: `.false.`

Allows to relax only parts of a structure, while keeping the rest at fixed positions. Currently, the following simple options for `constraint` are possible:

- `.true.`: All coordinates for this atom will be constrained.
- `.false.`: The relaxation of this atom will not be constrained.
- `x`: The x coordinate of this atom is not allowed to move.
- `y`: The y coordinate of this atom is not allowed to move.
- `z`: The z coordinate of this atom is not allowed to move.

Attention: If you wish to constrain more than one coordinate, the required constraints must be specified as separate lines, like this:

```
atom 0. 0. 0. Fe
  constrain_relaxation x
  constrain_relaxation y
```

In contrast, specifying two constraints in one line will *not* work. The second constraint would simply be ignored!

Tag: `hessian_block` (geometry.in)

Usage: `hessian_block i_atom j_atom block`

Purpose: In `geometry.in`, allows to specify a Hessian matrix explicitly, with one line for each 3×3 block. The option `block` consists of 9 numbers in column-first (Fortran) order. The 3×3 block corresponding to `j_atom`, `i_atom` is initialized by the transposed of `block`. The Hessian matrix is input in units of eV/Å^2.

If at least one `hessian_block` line is found in the file, the Hessian is constructed using this mechanism. So far there is no safe-guard from overriding Hessian blocks with subsequent lines with equal `i_atom`, `j_atom`.

There are two scripts in the `utilities` directory to automatically construct such Hessian matrix approximations. First, `conversions/thess2aims.py` converts a Tinker

generated Hessian matrix. Second, `Lindh.py` constructs a general purpose model matrix [?]. Please note that the Lindh model matrix is now also directly available with `init_hess Lindh`.

Tag: `hessian_block_lv` (geometry.in)

Usage / purpose: Like `hessian_block`, but for Hessian matrix elements between lattice vector degrees of freedom.

Tag: `hessian_block_lv_atom` (geometry.in)

Usage / purpose: Like `hessian_block`, but for Hessian matrix elements between lattice vector degrees of freedom.

Tag: `trust_radius` (geometry.in)

Usage: `trust_radius value`

Purpose: In `geometry.in`, allows to specify the initial trust radius value for the `trm` relaxation algorithm.

This keyword is a significant exception – it is the only algorithmic keyword found in the `geometry.in` file. Conceptually, it would belong into `control.in`, but since it is written out as part of the `geometry.in.next_step` file which can be used to restart a structure relaxation, we keep it here.

The keyword specifies the initial value of the "trust radius" used to limit structure relaxation steps determined by the `bfgs` (synonymous with `trm`) relaxation algorithm.

The default is set to 0.2 Å. It will be overridden by the default value of `max_atomic_move`, which can be set in `control.in`.

Tags for general section of `control.in`:

Tag: `bfgs_extrapolation_cap` (control.in)

Usage: `bfgs_extrapolation_cap value`

Purpose: If the `bfgs_textbook` algorithm of `relax_geometry` deviated from a line step of 1.0, sets a maximum for the line step during relaxation.

`value` is a real number $\geq$1.0 (dimensionless), the maximum allowed step when a variable line step used with BFGS. Default: 4.0

`value` is dimensionless, given in units specified by the BFGS Hessian matrix. `value`=1.0 corresponds to the default extrapolation of BFGS, assuming a perfectly quadratic energy surface.

Tag: `bfgs_step` (control.in)

Usage: `bfgs_step type [factor cap]`

Purpose: Can specify a BFGS extrapolation with an adaptive line step instead of the default linestep for purely quadratic extrapolation.

`type` is a string, either `quadratic` or `mix`. Default: `quadratic`.
`factor` : If `type` is not `quadratic`, specifies a mixing factor to interpolate between the line step actually used in the last relaxation step, and the *a posteriori* estimated optimum line step. Default: 0.1
`cap` : If `type` is not `quadratic`, specifies an upper limit for the allowed line step.

Tag: `calc_analytical_stress_symmetrized` (control.in)

Usage: `calc_analytical_stress_symmetrized flag`

Purpose: If `.false.`, calculates all 9 components of the analytical stress tensor. If `.true.` calculates only the upper triangle (6 components) of the tensor and copies the result to the lower triangle.

`flag` is a logical string, either `.true.` or `.false.` Default: `.true.`

Generally, it is sufficient to calculate the upper triangle of the tensor. This flag is mainly for debugging purposes.

Tag: `clean_forces` (control.in)

Usage: clean_forces type

Purpose: Can remove small unitary force components (rotation and translation of the whole structure due to residual numerical noise) in relaxations.

type is a string, specifying whether and how any overall rotations / translations are removed.

The default for type depends on the exact circumstances (see below). The following choices exist:

- none : No removal of residual rotations and translations. This is the default if any external embedding fields or charges are specified in geometry.in.
- sayvetz : Non-periodic structures: Removal of rotations and translations by a formal projection [?, ?]. In *periodic* systems, only translations are removed.
- fixed_plane : *experimental* Simple alternative algorithm by constraining three atoms into a plane (implicitly constraining all others).

Tag: compute_analytical_stress (control.in)

Usage: compute_analytical_stress flag

Purpose: If .true., switches on the computation of the analytical stress tensor.

flag is a logical string, either .true. or .false.

Default: .true. if a unit cell relaxation was requested and computation is possible. Otherwise, .false.

This flag allows to request an explicit analytical stress tensor computation for an otherwise explicit single-point calculation.

The calculation of the analytical stress is limited to LDA, GGA, Meta-GGA and hybrid functionals and is not possible with load_balancing . The vdW correction based on Hirshfeld partitioning (vdw_correction_hirshfeld) is included into the analytical stress tensor.

Tag: compute_forces (control.in)

Usage: compute_forces flag

Purpose: If .true., switches on the computation of forces.

flag is a logical string, either .true. or .false.

Default: .true. if a geometry optimization or molecular dynamics run was requested, or if the sc_accuracy_forces convergence criterion was set. Otherwise, .false.

This flag allows to request an explicit force computation for an otherwise explicit single-point calculation. This is necessary for use with external tools that require forces, such as a finite-difference calculation of vibrational frequencies (see Sec. 4.6) or a transition

state search (see Sec. 4.9). In these cases, keyword final_forces_cleaned should also be set.

Tag: compute_numerical_stress (control.in)

Usage: compute_numerical_stress flag

Purpose: If .true., switches on the computation of the numerical stress tensor based on central finite differences.

flag is a logical string, either .true. or .false.

Default: .true. if a unit cell relaxation was requested and the computation of the analytical stress is not possible. Otherwise, .false.

If not further specified (by delta_numerical_stress) the default value for the scaling factor delta is set to 10^{-4}.

Tag: delta_numerical_stress (control.in)

Usage: delta_numerical_stress value

Purpose: Specifies the scaling factor delta in the computation of the numerical stress tensor (compute_numerical_stress).

value is a dimensionless real number > 0. Default: 10^{-4}.

Tag: energy_tolerance (control.in)

Usage: energy_tolerance tolerance

Purpose: Sets the energy amount by which a relaxation step can move upwards and is still accepted.

tolerance is a small positive real number, in eV. Default: $2{\cdot}10^{-4}$ eV.

Small upward steps during relaxation may occur as a result of a slightly mis-guessed bfgs_textbook Hessian matrix somewhere along the path, or as a result of some residual numerical noise that leads to a discrepancy between energies and forces. In the present code version, such noise is always safely below the default energy_tolerance for reasonable settings. If exceeded, this may point to a different problem, simply increasing tolerance is not, in general, a good idea. For the trm optimizer, also see harmonic_length_scale .

Tag: external_force (geometry.in)

Usage: external_force x y z

Purpose: *Experimental* – Applies an external force to the atom previous to this keyword.

x,y,z are the force components in eV/Å applied to the atom in x, y, z direction.

When an external force is applied it is necessary to contstrain the relaxation of at least on other atom to avoid a constant shift of the geometry. Also the value has to be reasonably chosen. Tear apart geometries can result in very flat energy landscapes, which take a large amount of time to optimize. Typically this feature should be used in cases where a small external force, e.g. a STM-tip is applied on an atomic layer and the geomtry response of this external force is of interest.

This feature is experimental since no extensive testing was done for it.

Tag: `external_pressure` (control.in)

Usage: `external_pressure value`

Purpose: *Experimental* – Applies external pressure to the unit cell.

`value` is the pressure in eV/Å^3 applied to the unit cell.

In the periodic case, it is possible to apply hydrostatic pressure to the unit cell. To actually see the effect of the external pressure, a unit cell relaxation is required (see `relax_geometry` and `relax_unit_cell`). The crystal is then relaxed with the external pressure added to the stress tensor.

This feature is experimental since no extensive testing was done for it.

Tag: `final_forces_cleaned` (control.in)

Usage: `final_forces_cleaned flag`

Purpose: Decides whether spurious unitary transformations of the complete system (translations and rotations) are removed before the final output.

`flag` is a logical string, either `.true.` or `.false.` Default: `.true.`

This option affects directly the long-format (15 decimal) output of total energies and forces at the end of the s.c.f. cycle in the standard output file. If `flag` is `.true.`, the final output forces are "cleaned" using the `sayvetz` [?, ?] mechanism of keyword `clean_forces` (removal of translations and rotations for cluster geometries; only translations removed for periodic systems).

`final_forces_cleaned .true.` should be set for use with external tools that require forces, such as a finite-difference calculation of vibrational frequencies (see Sec. 4.6) or a transition state search (see Sec. 4.9).

Tag: `harmonic_length_scale` (control.in)

Usage: harmonic_length_scale length
Purpose: The trm/bfgs algorithm of relax_geometry judges a step by its energy gain. Usually, one simply uses the energy difference. For very short steps and rather light grids, however, it turns out that the qualitiy of the energy is inferior to the quality of the forces. For steps shorter than length, do not look at the energy but use the harmonic approximation $-\Delta\tilde{E} = (\boldsymbol{X}_2 - \boldsymbol{X}_1)\cdot(\boldsymbol{F}_2 + \boldsymbol{F}_1)/2$ as an estimate for the energy gain. If this procedure willfully accepts a step which increases the energy by more than energy_tolerance , the code stops to warn the user about the inconsistency between energy functional and forces. length is a length scale in Å. Default: 0.025

Effectively, this flag switches from a real energy minimizer to a search for a stable zero of the force field for short step lengths.

Tag: hessian_to_restart_geometry (control.in)

Usage: hessian_to_restart_geometry flag
Purpose: Exports the current approximation to the Hessian matrix to geometry.in.next_step during a relaxation restart using hessian_block . flag is a logical string, either .true. or .false. Default: .true.

Note: The geometry.in.next_step file is written out by default when the relax_geometry bfgs algorithm is used. For the older bfgs_textbook algorithm, this file is only written when combined with the restart_relaxations keyword.

Tag: init_hess_lv_diag (control.in)

Usage: init_hess_lv_diag value

Purpose: In a geometry relaxation with a unit cell optimization, allows to specify the initial Hessian matrix elements used to estimate relaxation steps that are associated with the lattice vector degrees of freedom.

length is a length scale in eV/Å^2. Default: 25 ev/Å^2

See the init_hess keyword for more information on the initial Hessian matrix used during a structure optimization.

Tag: init_hess (control.in)

Usage: init_hess diag [value] *or:* init_hess Lindh [add-value [thres]]

Purpose: Defines the initial Hessian matrix that is used by the structure relaxation algorithms bfgs_textbook and bfgs (the latter is synonymous with trm) of the relax_geometry keyword.

Default: init_hess Lindh 2. unless a specific Hessian is given in the geometry.in file.

If no explicit initial Hessian matrix is given in the geometry.in file or specified using the older `restart_relaxations` infrastructure, the keyword `init_hess` can be used to specify the initial Hessian matrix used in a structure optimization.

With the option `diag`, a diagonal initial Hessian matrix is assumed. Then, the number given by the `value` option sets the diagonal elements between all atomic coordinates directly. The default is 25 eV/Å^2. Larger values lead to a more conservative start, smaller values lead to more aggressive initial relaxation steps.

With `Lindh` the Lindh model matrix [?] is used to initialize the Hessian between all *atomic* coordinates (usually a very efficient guess). For stability reasons, `add-value` (in eV/Å^2, defaults to 2.0 eV/Å^2) is added to all matrix elements on the diagonal. The parameter `thres` (default: 15.0) can be used to specify the accuracy of the Lindh matrix; only terms estimated to be larger than $e^{-\texttt{thres}}$ are taken into account.

If a unit cell relaxation is additionally requested, all the Hessian rows and columns associated with the lattice vector degrees of freedom are set to be diagonal, both for the `diag` and `Lindh` versions. The default diagonal element is 25 eV/Å^2. Its value can be modified using the `init_hess_lv_diag` keyword. If you are planning a large number of complex unit cell optimizations of a similar type, we do recommend to check whether this default value is any good.

If the initial Hessian is specified explicitly by `hessian_block` in `geometry.in`, this explicit hessian overrides any information requested by the `init_hess` keyword.

Tag: `line_search_tol` (control.in)

Usage: `line_search_tol tolerance`

Purpose: During a `relax_geometry bfgs_textbook` geometry relaxation, rejects a relaxation step if the *a posteriori* estimate for the optimal relaxation step deviates by more than a given tolerance from the actually performed line step.

`tolerance` is a real positive relative tolerance (dimensionless in units of the BFGS Hessian). Default: 3.0

If keyword `relax_geometry bfgs_textbook` is used, `line_search_tol` will reject a relaxation step based on an *a posteriori* estimate of the true optimum step length (see Ref. [?] for details). In short, a step of length l in dimensionless units (units relative to the BFGS Hessian matrix) is rejected if

$$|l_{\textit{a posteriori}} - l_{\text{actual}}|/l_{\text{actual}} \geq \texttt{tolerance}\,. \tag{3.27}$$

Tag: `line_step_reduce` (control.in)

Usage: line_step_reduce value

Purpose: Determines the behaviour of FHI-aims if a relax_geometry bfgs_textbook relaxation step was rejected.

value is either a string descriptor, or a numerical value. Default: automatic .

During a structure optimization using the relax_geometry bfgs_textbook algorithm, FHI-aims checks if a performed relaxation step actually reduced the total energy compared to the previous step. The allowed upward tolerance is set by keyword energy_tolerance . If an upward step is encountered, FHI-aims first attempts to repeat the step with a smaller step length. If this smaller step also leads upwards, FHI-aims assumes that the BFGS Hessian has exceeded its validity limits, and reinitializes the Hessian before continuing.

If value is set to automatic, the BFGS step is repeated with an *a posteriori* estimated length l given by the previous (failed) step (see Ref. [?]).

Alternatively, value can be set to a real positive number <1, corresponding to the default factor by which the previous step length is reduced.

Tag: max_atomic_move (control.in)

Usage: max_atomic_move value

Purpose: Maximum allowed step length taken during relaxation.

value is a real positive upper bound for the maximum allowed change in single atomic coordinate, in Å. Default: 0.2 Å.

If the bfgs_textbook-predicted change in an atomic coordinate exceeds max_atomic_move , the length of the entire step (all coordinates) will be scaled down to not exceed the maximum allowed displacement.

Tag: max_relaxation_steps (control.in)

Usage: max_relaxation_steps number

Purpose: A structure optimization will be aborted after exceeding a prescribed maximum number of steps.

number is the prescribed maximum step number. Default: 1000 .

Tag: min_line_step (control.in)

Usage: min_line_step value

Purpose: Sets the minimum possible (dimensionless) BFGS linestep before the BFGS Hessian is reinitialized

value: A real positive number (between 0 and 1). Default: 0.1

See keyword line_step_reduce . If, after a rejected bfgs_textbook relaxation

step, the reduced, dimensionless BFGS line step decreases below `min_line_step`, the BFGS Hessian matrix estimate is considered inappropriate for the present point in space, and the Hessian is reinitialized.

Tag: `numerical_stress_save_scf` (control.in)

Usage: `numerical_stress_save_scf flag`

Purpose: Controls if `constrain_relaxation` directives are used to determine implicitly if a component of the numerical stress has to be calculated. This greatly accelerates unit cells with high symmetries (e.g. orthorhombic).

`flag` is a logical string, either `.true.` or `.false.` Default: `.true.`

Tag: `orthonormalize_eigenvectors` (control.in)

Usage: `orthonormalize_eigenvectors flag`

Purpose: Specifies whether or not the wave function coefficients from the previous geometry will be re-orthonormalized before initializing a new relaxation step.

`flag` is a logical string, either `.true.` or `.false.` Default: `.true.`

The `orthonormalize_eigenvectors` keyword allows to reorthnormalize the converged self-consistent Kohn-Sham orbitals c_{jl} after a relaxation step. These are then used to reinitialize the electron density for the next relaxation step.

Due to the change in atomic positions, the wave function coefficients c_{jl} for the earlier geometry are no longer orthonormal after the relaxation step. The consequence is an initial electron density which no longer satisfies the correct electron count (i.e., the system may appear to be charged immediately after a relaxation step, although a neutral system was requested). In principle, this does not matter for the outcome of a calculation, since the self-consistent solution will be independent of the starting point. In many cases, there is no clear benefit in terms of the s.c.f. convergence duration from orthonormalizing the c_{jl} prior to the reinitialization; however, some cases with unstable s.c.f. convergence may benefit significantly.

Tag: `relax_geometry` (control.in)

Usage: relax_geometry `type tolerance`

Purpose: Specifies if a structure optimization (geometry relaxation) is requested, and which.

`type` specifies the type of requested structure optimization. Default: `none`.
`tolerance`: Specifies the maximum residual force component per atom (in eV/Å) below which the geometry relaxation is considered converged.

Finds the nearest minimum of the Born-Oppenheimer potential energy surface for the nuclei.

For periodic calculations: If you are looking to relax not just atomic coordinates but also the unit cell shape (lattice vectors), you do need to specify an additional keyword: relax_unit_cell .

The presently supported options for type are none, bfgs_textbook and trm (which is now synonymous with bfgs).

- bfgs or trm uses a trust radius method enhanced version of the Broyden-Fletcher-Goldfarb-Shanno (BFGS) optimization algorithm (see Ref. [?], which was the basis for Jürgen Wieferink's code effort in this area). In our tests, this version appears to give the fastest convergence reliably.

- bfgs_textbook invokes a Cartesian BFGS optimization algorithm that adheres strictly to the textbook [?]; we note that our coded version is based on the concise description found at Wikipedia. See Ref. [?] for some additional details of our version.

The relaxation algorithm can be greatly sped up by using a somewhat intelligent guess for the Hessian matrix used in the initial step. By default, FHI-aims now sets the general purpose model matrix due to Lindh and coworkers [?] with a slight modification. For more information see the init_hess keyword above.

A reliable force convergence criterion tolerance for most structures is 10^{-2} eV/Å or $5{\cdot}10^{-3}$ eV/Å.

Going to a much smaller tolerance value can be necessary for some specific purposes, but do it with great care. If tolerance is set to a too small value by default, 80% or more of your CPU time may be spent groping around in the last meV of the structure optimization.

Use the 'get_relaxation_info.pl' utility script to monitor the progress of an ongoing or finished relaxation run.

To stop an ongoing structure relaxation in an organized way, create the abort_opt file in the respective directory.

For vibrations of large molecules or phonons in solids from finite differences, tighter settings of the tolerance parameter may be required, even down to 10^{-4} eV/Å. In that case, however, do not forget that rather tight s.c.f. convergence accuracy settings may also be required to get accurate gradients in the first place.

In other words, use the tolerance criterion for a structure relaxation run wisely – decide what is the physical quantity you are actually interested in, and then check which value of the tolerance criterion is safe but still efficient.

Tag: **relax_unit_cell** (control.in)

Usage: relax_unit_cell type

Purpose: Relaxes unit cell (lattice vectors) using the structure optimization as specified in relax_geometry .

type specifies the type of requested unit cell optimization. Presently supported options: none, full, fixed_angles. Default: none

Allows to optimize the lattice vectors of a periodic calculation, in addition to the normal atomic coordinates inside the unit cell. This keyword is not on by default, as automatically optimizing the unit cell of (say) a surface calculation could do a lot of unintended harm. Possible settings:

- none : Unit cell will be kept fixed, no optimization.
- full : All lattice_vector degrees of freedom will be relaxed, except those affectd by explicit constraints.
- fixed_angles : All angles between lattice vectors will be constrained (kept fixed), only the lengths of each lattice vector are varied. (This option used to called shape, but that is a misunderstandable term. The shape term will be removed in the future to avoid confusion.)

This keyword should be used only together with relax_geometry . Individual lattice vectors or its components can be constrained by using constrain_relaxation .

If the computation of the analytical stress is possible regarding the chosen computational settings, the analytical stress is used for the unit cell relaxation. Otherwise, the numerical stress is used. With stress_for_relaxation , one can explicitly choose either numerical or analytical stress for the unit cell relaxation.

If a unit cell relaxation produces strange results with the analytical stress, here are some potential remedies:

1. A very possible reason may be because the integration grids are not dense enough. This could especially well be the case for "light" settings. One remedy is to just use the integration grids from "tight" and the basis functions from "light".
2. Another possible remedy is to switch the way the integration weights are calculated to a slightly slower, non-default version. E.g., change the partition_type to a spherical one like rho_r2.
3. Finally, you may wish to set the convergence of the analytical stress with sc_accuracy_stress to an explicit, final value. **Only ever set this keyword for test purposes, though, not routinely. The cost for too many analytical stress evaluations can be disproportionately large.**

Tag: stress_for_relaxation (control.in)

Usage: stress_for_relaxation type

Purpose: Use either numerical or analytical stress for unit cell relaxations.

type can be either numerical or analytical. Default: Chosen automatically based on computational settings.

To perform an actual unit cell relaxation, one has to set relax_unit_cell . If one chooses analytical but the computation of the analytical stress is not possible, FHI-aims will abort.

Tag: restart_relaxations (control.in)

Usage: restart_relaxations flag

Purpose: To save all necessary data to restart a bfgs_textbook relaxation without needing to recompute the relaxation path in DFT.

flag is a logical string, either .true. of .false. Default: .false.

Note: The restart_relaxations keyword will work with both the bfgs (trm) or the bfgs_textbook versions of the relax_geometry keyword. However, it is only needed for the bfgs_textbook version, if a restart is desired. For the normal (and faster) bfgs (trm) version, all that is needed is the information contained in geometry.in.next_step.

In order to restart from an already known estimated Hessian, rather from unity (the default in the absence of any previous knowledge), the Hessian can be rebuilt from a series of stored atomic cordinates and forces.

This information is exported to a separate file relaxation_restart_file.FHIaims, which will be read and evaluated by a newly beginning FHI-aims calculation if the restart_relaxations keyword is set to true.

The flag also needs a geometry.in file that is an exact copy of the geometry.in.next_step file, which is updated during the structure optimization.

See also restart for keeping the wave functions as well.

Tag: write_restart_geometry (control.in)

Usage: write_restart_geometry flag
Purpose: During a structure optimization, exports the current geometrys and approximation to the Hessian matrix to a file geometry.in.next_step.
flag is a logical string, either .true. or .false. Default: .true.

Note: The geometry.in.next_step file is written out by default when the relax_geometry bfgs algorithm is used. For the older bfgs_textbook algorithm, this file is only written when combined with the restart_relaxations keyword.

3.12 Molecular dynamics

FHI-aims provides the capability to run Born-Oppenheimer molecular dynamics. The necessary keywords are described in this section. A brief description of the physical algorithms implemented in FHI-aims can be found in Ref. [?]. For a truly thorough explanation of the underlying concepts, please refer to the standard literature, e.g., Refs. [?, ?].

Wave function extrapolation

For molecular dynamics runs within the microcanonical ensemble ("NVE") or for deterministic thermostats, the wave function can be extrapolated in order to reduce the computational effort to reach self-consistency. This section specifies *what* quantity is actually extrapolated and *how*.

Following the approach of Kühne *et al.* [?], we extrapolate the contra-covariant density matrix $\mathbf{PS}$, the product of the ordinary (purely contravariant) one-particle density matrix $\mathbf{P}$ and the overlap matrix $\mathbf{S}$. The contra-covariant density matrix is then used to project new wave function coefficients from the last iteration $\mathbf{c}_{\mathrm{new}} = (\mathbf{PS})_{\mathrm{extra}}\mathbf{c}_{\mathbf{old}}$. As a last step, the one-particle coefficients are orthonormalized.

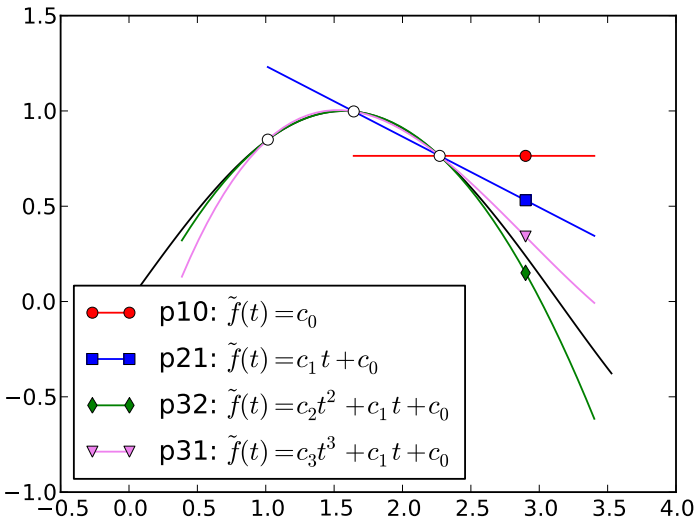


Figure 3.1: Different extrapolation schemes. The scheme "pno" refers to an n point scheme of order o, remaining orders used to fit odd components.

The extrapolation scheme we use is sketched in Fig. 3.1 for the example of an ordinary function $f(t)$ in a real variable t. For a pno scheme (chosen with `wf_extrapolation polynomial` n o) n old iterations $f(t_0 - k\Delta t)$, $k = 1, \ldots, n$ are stored. An ansatz function with n degrees of freedom is then used to fit all of these previous iterations and evaluated at t_0 and the value used to intialize the SCF procedure.

The choice of the fitting function should aim at two targets: accurate extrapolation and

time-reversal symmetry. The parameter o specifies the order of the extrapolation. The higher o, the better the extrapolation gets with decreasing time stemps Δt. Time-reversal symmetry is useful to avoid energetic drifts for not-so-tight SCF accuracy settings. By adding odd terms (odd with respect to $t_0 + t \leftrightarrow t_0 - t$), time-reversal symmetry is enhanced [?].

Therefore, in contrast to [?, ?], the $(n - o - 1)$ remaining degrees of freedom are not resolved within a least-squares fit but instead to add fitting functions $(t - t_0)^{2k+1}$ to enhance time-reversal symmetry and thus energy conversion. Please note, however, that time-reversal symmetry itself only enhances energy conservation and not necessarily dynamical properties of the trajectories.

If you are interested in energy conservation, "p31" is in general a good choice to start with. If a good initial guess for the SCF procedure is desired, you should give "p32" a try. Please note that deactivation of extrapolation (`wf_extrapolation none`) is actually the same as the "p10" extrapolation (see Fig. 3.1) and simply uses the one-particle coefficients of the last iteration are to initialize the SCF procedure.

Tags for `geometry.in`:

Tag: `velocity` (geometry.in)

Usage: `velocity vx vy vz`

Purpose: Specifies an initial velocity for the immediately preceding `atom` in file `geometry.in`.

`vx`, `vy`, `vz` : x, y, and z components of the initial velocities, in Å/ps.

In `geometry.in`, the line containing the velocity must follow the line containing the `atom` that the velocity refers to. Note that, as in the case of relaxations, the molecular dynamics section of the FHI-aims output stream contains this information in the proper format.

Tags for general section of `control.in`:

Tag: `MD_maxsteps` (control.in)

Usage: `MD_maxsteps N`

Purpose: Sets the maximal number of molecular dynamics steps.

`N` is an integer number. Default: -1 (infinite run).

A negative number signals that the ending criterion is not checked, in fact, the default setting is `N`=-1.

Tag: `check_MD_stop` (control.in)

Usage: `check_MD_stop .true. / .false.`

Purpose: if `.true.`, an MD calculation ist stopped when a file MD_stop is generated

Default: `.true.`

Tag: `MD_MB_init` (control.in)

Usage: `MD_MB_init Temperature`

Purpose: Initializes random velocities in a molecular dynamics calculation using a Maxwell-Boltzmann distribution.

`Temperature` : Initial temperature in K. Default: No initial velocities.

This keyword is for a rough initialization only, and is overridden by any successful calls of the MD restarting procedure through `MD_restart` . The default initialization for all velocities is zero.

Tag: `MD_clean_rotations` (control.in)

Usage: `MD_clean_rotations .true. / .false.`

Purpose: if `.true.`, uses Sayvetz conditions to clean initial velocities from rotations

Default: `.true.` for non-periodic systems, `.false.` for periodic systems or if relaxation_constraints are used.

This option is useful for non-periodic systems, allowing to weed out some residual numerical noise in the forces. However, seeming rotations of the unit cell can easily appear in periodic systems for completely normal, physical reasons. Since this led to some confusion, this option is now actively disabled (code stops) for periodic systems. If you have a good reason to use this option in periodic systems, it can be re-introduced by hand.

Tag: `MD_thermostat_units` (control.in)

Usage: `MD_thermostat_units units`

Purpose: To allow user specification of the effective thermostat mass outside of the internal units.

`unit` : Unit of the effective mass of the Nosé-Hoover and Nosé-Poincaré thermostats – either `amu*bohr^2` (mass-oriented specification) or `cm^-1` (frequency oriented specification, to allow to connect the thermostat mass to characteristic frequencies of the system). Default: `amu*bohr^2` .

Tag: `MD_restart` (control.in)

Usage: `MD_restart option`

Purpose: controls the MD initialization from a molecular dynamics restart file.

Restriction: At present, this keyword does not produce proper results when switching ensembles between runs.

`option` is a string (see below). Default: `.false.`

Possible values for `option` are `.true.`, `.false.`, `time_restart`, or `filename`. The default is `.false.`.

The data for the molecular dynamics integrator is always written to a file `aims_MD_restart.dat` after each time step (regardless of whether or not keyword `MD_restart` is used). If `MD_restart` is set, this keyword controls the reading of such data from a previous run (if that exists), to either

- continue a previous run (`.true.`), or
- to use previous position / velocity information but reset all timings (`time_restart`), or
- to begin a new run from scratch (`.false.`)

The default file name is used unless the user specifies a specific file `filename` from where the input is to be read.

Important: If the starting structure and velocities are taken from the restart file, any exact positions noted in `geometry.in` are ignored. However, the species identification, possible initial charges, and other per-atom information in `geometry.in` remain valid and must be present as always. If the option `.true.` is set, the molecular dynamics clock is also read from file, in all other restarts the clock is reset to zero.

Tag: `MD_restart_binary` (control.in)

Usage: MD_restart_binary option

Purpose: controls the format used for the molecular dynamics restart file.

option is a string (see below). Default: .true.

Possible values for option are .true. and .false.. The default is .true.. Use this flag to switch to the ASCII format in the MD restart file.

Tag: MD_run (control.in)

Usage: MD_run time ensemble [further specifications]

Purpose: Central controls of the physical parameters of a Born-Oppenheimer molecular dynamics run.

time : Requested MD time in ps.
ensemble : Ensemble specifications for MD_run , listed as subkeywords to MD_run in a separate section below.
further specifications: See ensemble subkeywords below.

This keyword is the key control for all molecular dynamics. The runtimes time are specified in ps. There are five different possible ensembles, each of which require different options - as described in a separate subsection below.

When a *schedule* of different temperatures, thermostats etc is required within the same run (e.g., initialization followed by NVE, change of temperature, etc.), use the MD_schedule keyword instead of MD_run .

Tag: MD_schedule (control.in)

Usage: MD_schedule

Purpose: Must be followed by specific segments of a Born-Oppenheimer molecular dynamics run.

Must be followed by an arbitrary number of lines MD_segment , where different temperatures, thermostats, etc. may be specified for each segment of the run.

Tag: MD_segment (control.in)

Usage: MD_segment time ensemble [further specifications]

Purpose: Central controls of the physical parameters of a Born-Oppenheimer molecular dynamics run.

time : Requested MD time in ps.
ensemble : Ensemble specifications for MD_run , listed as subkeywords to MD_run in a separate section below.
further specifications: See ensemble subkeywords below.

Keyword MD_segment must only appear in consecutive lines after an MD_schedule keyword. Instead of a single set of MD thermostats, temperatures etc. throughout the

simulation, this keyword allows to set specific values for only a segment of the full run.

Tag: `MD_time_step` (control.in)

Usage: `MD_time_step deltat`

Purpose: Set the time step for a molecular dynamics run, in ps.

Default: 0.001 (this is 1 fs)

Tag: `wf_extrapolation` (control.in)

Usage: `wf_extrapolation extrapolation_type`

Purpose: Used to specify the wave function extrapolation. Options are `polynomial` n o (an n-point polynomial extrapolation of order o, where the remaining degrees of freedom are used to enhance time reversibility), `niklasson06` n (an n-point extrapolation as specified by Niklasson *et al.* [?]), `none` (same as `polynomial 1 0`), `linear` (`polynomial 2 1`), and `quadratic` (`polynomial 3 2`).

Default: `polynomial 3 1` for NVE, `none` otherwise.

The option `polynomial 3 1` strongly reduces a possible energy drift in `NVE` runs even for moderate force accuracy settings and additionally lowers the number of SCF cycles per time step.

Warning: This feature is experimental. It most probably will not enhance convergence of metallic systems and is not parallelized. The `niklasson06` schemes are not suitable for long runs of nontrivial physical systems because a regular reinitialization would be necessary.

Tag: `wf_func` (control.in)

Usage: `wf_func specifications`

Purpose: Used to activate the wave function extrapolation with fine grained control over the basis functions used for extrapolation. Each `wf_func` line adds one iteration to the extrapolation scheme and one fitting function. Options are "`constant`" (1), "`linear`" (t), "`quadratic`" (t^2), "`cubic`" (t^3), "`polynomial` n" (t^n), "`sin` ω `unit`" ($\sin \omega t$), and "`cos` ω `unit`" ($\cos \omega t$). The unit "`unit`" is either "`cm^-1`" or "`fs`". Additionally, "`none`" can be used to add one degree of freedom which is used to stabilize things in a least squares manner.

The same warning as for `wf_extrapolation` applies.

Ensemble specification options for the MD_run and MD_segment keywords:

When used with a molecular dynamics schedule of time segments with different thermostats, the following subkeywords should appear behind an *MD_segment* *keyword instead of* *MD_run* .

MD_run sub-tag: NVE (control.in)

Usage: MD_run time NVE

Purpose: Performs molecular dynamics in the microcanonical ensemble.

MD_run sub-tag: NVE_4th_order (control.in)

Usage: MD_run time NVE_4th_order

Purpose: Performs molecular dynamics in the microcanonical ensemble, using a fourth-order integration method. The integrator used is called SI4 in Ref. [?]. This method is sometimes useful for longer time steps and for very accurate MD. Note that there are five force evaluations per time step, instead of the usual single calculation.

MD_run sub-tag: NVE_damped (control.in)

Usage: MD_run time NVE_damped damping_factor

Purpose: Performs microcanonical molecular dynamics, but dampens each velocity by a factor damping_factor after each time step.

damping_factor is the damping factor between different MD steps.

This option is a useful addition to the structural relaxation, which can be done in principle with a molecular dynamics run as well. In almost all cases, however, the BFGS algorithm as called with the relax_geometry keyword is preferable.

MD_run sub-tag: NVT_andersen (control.in)

Usage: MD_run time NVT_andersen Temperature nu

Purpose: Run molecular dynamics with an Andersen stochastic thermostat.

Temperature is the simulation temperature in K.
nu : Probability that a given atom's velocity will be "reset" to a Maxwell-Boltzmann distributed value, per picosecond!

Andersen's [?] simple definition of a thermostat that randomly resets the velocity of individual atoms to a Maxwell-Boltzmann distributed value with a given frequency. This

is an example of a rather harsh thermostat.

`MD_run` **sub-tag:** `NVT_berendsen` (control.in)

Usage: `MD_run time NVT_berendsen Temperature tau`

Purpose: Molecular dynamics run using the Berendsen thermostat.

`Temperature` is the simulation temperature in K.
`tau` is a relaxation time of the thermostat, in ps.

Note that the Berendsen thermostat does NOT resemble any physical ensemble, but that it is a useful standard to initialize a molecular dynamics simulation. The relaxation time `tau` must be chosen by the user, there is no default. Remember that the special case $\tau = \Delta t$ reproduces the correct temperature exactly at every time step and can be used for a very rough initialization.

`MD_run` **sub-tag:** `NVT_parrinello` (control.in)

Usage: `MD_run time NVT_parrinello Temperature tau`

Purpose: Molecular dynamics run using the Bussi-Donadio-Parrinello thermostat. [?]

`Temperature` is the simulation temperature in K.
`tau` is a relaxation time of the thermostat, in ps.

This is the Bussi-Donadio-Parrinello thermostat, as described in Ref. [?] It a) properly samples the canonical distribution and b) preserves time correlations. The relaxation time `tau` (in ps) must be chosen by the user, there is no default. The performance of the thermostat is claimed in Ref. [?] to be practically independent of the value of τ, for condensed phases. For clusters, though, a proper tuning of `tau` might be necessary. When this ensemble is chosen, the conserved pseudo-Hamiltonian (as described in Ref. [?]) is printed in the output.

`MD_run` **sub-tag:** `GLE_thermostat` (control.in)

Usage: `MD_run time GLE_thermostat Temperature Number_of_aux_DOF`

Purpose: Molecular dynamics run using the colored-noise thermostats based on the Generalized Langevin Equation, proposed by M. Ceriotti and coworkers [?, ?, ?].

`Temperature` is the simulation temperature in K.

`Number_of_aux_DOF` (integer) is the number of auxiliary degrees of freedom for this thermostat, that specifies the dimensions of the input matrices

This is a flexible thermostat based on the Generalized Langevin Equation, that adds extra

degrees of freedom to the equations of motion and has a frequency dependent memory kernel, so that one can adjust the performance of the thermostat for different degrees of freedom in the system. It requires also matrices as inputs, for which we refer the reader to the keywords MD_gle_A and MD_gle_C . Please read references [?, ?, ?] and references therein before using this. It can be used to sample the canonical ensemble or to break detailed balance and simulate other conditions, including approximate quantum effects. It has a conserved quantity in the same spirit of the BDP thermostat.

MD_run sub-tag: NVT_nose-hoover (control.in)

Usage: MD_run time NVT_nose-hoover Temperature Q

Purpose: Run molecular dynamics with a Nosé-Hoover thermostat.

Temperature is the simulation temperature in K.
Q : Effective mass specification of the thermostat in units as specified by MD_thermostat_units .

Probably the most popular thermostat in the literature.

MD_run sub-tag: NVT_nose-poincare (control.in)

Usage: MD_run time NVT_nose-poincare Temperature Q

Purpose: Run molecular dynamics with a Nosé-Poincaré thermostat.

Temperature is the simulation temperature in K.
Q : Effective mass specification of the thermostat in units as specified by MD_thermostat_units .

Due to numerical issues, this thermostat has been disabled for the time being.

Tag: MD_gle_A (control.in)

Usage: MD_gle_A entries

Purpose: Required input for GLE_thermostat

entries contains all entries (Number_of_aux_DOF + 1) of one row of the matrix. One must repeat this flag for each row of the matrix, in order.

Units should be 1/ps, and the matrix can be downloaded from http://epfl-cosmo.github.io/gle4md/. If you wish to model different dynamics (quantum effects, or thermalization of PIMD), one can also specify a C matrix with the keyword MD_gle_C .

Tag: MD_gle_C (control.in)

Usage: MD_gle_C entries

Purpose: Optional input for GLE_thermostat

entries contains all entries (Number_of_aux_DOF + 1) of one row of the matrix. One must repeat this flag for each row of the matrix, in order.

This matrix is optional for the usage of the GLE thermostats. If sampling the canonical ensemble it is not needed. Otherwise, it is. Units should be K, and the matrix can be downloaded from http://epfl-cosmo.github.io/gle4md/.

Tag: use_pimd_wrapper (control.in)

Usage: use_pimd_wrapper hostaddress portnumber

Purpose: Interfaces FHI-aims to an internet socket based python wrapper code that does path integral molecular dynamics.

hostaddress accepts a host name or an IP address. If you want to use UNIX sockets, add 'UNIX:' before the host address.

portnumber should contain the number of the port that the wrapper is listening to.

FHI-aims is compatible with the i-PI python PIMD wrapper developed by M. Ceriotti, J. More, and D. Manolopoulos [?]. Use 'Makefile.ipi' to compile the code with the necessary routines. Additional documentation on how to use FHI-aims as a driver for this wrapper can be downloaded with the wrapper itself. Please see http://epfl-cosmo.github.io/gle4md/index.html?page=ipi.

Currently, the total dipole of the molecule (or the bead images) can be passed to the wrapper if the flag ' output dipole' is set.

NPT simulations are currently supported only for functionals where the analytical stress tensor is available. If you wish to perform an NPT simulation, please add also ' compute_analytical_stress .true.' to your control.in file.

3.13 Thermodynamic Integration

Note added to the present manual: Albeit functional, the thermodynamic integration routines are still classified as "experimental". Please contact carbogno@fhi-berlin.mpg.de, if you encounter any problems or if you have suggestions.

FHI-aims provides the capability to compute the *anharmonic contributions* to the *Helmholtz free energy* of a system with the so called "thermodynamic integration" technique. For a truly thorough explanation of the underlying concepts, please refer to the standard literature, e.g., Refs [?, ?], since only a basic overview that sheds some light on the required input is given here.

Theory

In this brief introduction, we will focus on a perfect, periodic crystal, the *Helmholtz free energy* $F(T,V)$ of which can be decomposed in three contributions:

$$F(T,V) = F^{el}(V) + F^{nu}(T,V) = F^{el}(V) + F^{qh}(T,V) + F^{ah}(T,V)\ . \tag{3.28}$$

$F^{el}(T,V)$ is the free energy of the electronic system, which can be assessed by *Mermin's canonical generalization* of DFT [?]. For large band gap insulators, the electronic contribution is approximatively temperature independent and thus equal to the free energy of the electronic system at zero Kelvin (see `occupation_type`). $F^{nu}(T,V)$ is the free energy associated to the nuclear motion on the Born-Oppenheimer energy surface $V^{nu}(\vec{R})$. In the limit of low temperatures, this contribution can be described within the quasi-harmonic model (see Sec. 4.6), i.e., by only accounting for small elongations $\vec{U}$ from the equibrium positions $\vec{R}_0$ on the approximative harmonic potential

$$V^{qh}(\vec{R}) \approx V^{nu}(\vec{R}_0) + \frac{1}{2} \sum_{\substack{L,\alpha \\ N,M,\beta}} \left. \frac{\partial^2 V^{nu}(\vec{R})}{\partial \left(R_{0,L}\right)_\alpha \ \partial \left(R_{N,M}\right)_\beta} \right|_{\vec{R}=\vec{R}_0} (U_{0,L})_\alpha (U_{N,M})_\beta\ . \tag{3.29}$$

The free energy $F^{qh}(T,V)$ associated to the motion on such a potential can be computed with the FHI-aims code, as discussed in Sec. 4.6. At large temperatures, however, the quasi-harmonic approximation is no longer justified, since the deviations from the equilibrium are not minute. In this case, the "thermodynamic integration" technique can be employed to compute the *anharmonic contributions* to the free energy

$$F^{ah}(T,V) = F^{nu}(T,V) - F^{qh}(T,V)\ . \tag{3.30}$$

For this purpose, the dynamics of the *hybrid* system that is characterized by the potential

$$V^{\lambda}(\vec{R},\lambda) = \lambda\ V^{nu}(\vec{R}) + (1-\lambda)\ V^{qh}(\vec{R}) \tag{3.31}$$

is inspected. The parameter λ appearing therein describes the linear interpolation between the full Born-Oppenheimer potential $V^{nu}(\vec{R})$ and the quasi-harmonic potential $V^{qh}(\vec{R})$. The free energy $F^{\lambda}(T,V,\lambda)$ associated to the motion on this *hybrid* potential is directly related to the *anharmonic contributions* via

$$F^{ah}(T,V) = \int_0^1 d\lambda \left(\frac{\partial F^{\lambda}(T,V,\lambda)}{\partial \lambda} \right)\ , \tag{3.32}$$

as the fundamental theorem of differential and integral calculus shows. The relation [?]

$$\frac{\partial F^{\lambda}(T,V,\lambda)}{\partial \lambda} = \left\langle \frac{\partial}{\partial \lambda} V^{\lambda}(\vec{R},\lambda) \right\rangle_{V_\lambda} \tag{3.33}$$

allows to replace the integrand in Eq. (3.32) with a *canonical ensemble average* $\langle\cdot\rangle_{V_\lambda}$. If an ergodic thermostat is used (see Sec. 3.12), this ensemble average can then be substituted with a time average, so that the *anharmonic contributions* can be eventually expressed as [?]

$$F^{ah}(T,V) = \int_0^t dt' \, \frac{d\lambda}{dt'} \left(\frac{\partial}{\partial \lambda} V^{\lambda}(\vec{R},\lambda) \right) \ . \tag{3.34}$$

Within this approach it is thus possible to determine the *anharmonic contributions* to the free energy from an *ab initio* MD simulation, in which the parameter λ is adiabatically varied from zero to unity and/or vice versa.

Tags for MD_schedule section of control.in:

Tag: `thermodynamic_integration` (control.in)

Usage: `thermodynamic_integration` λ_{start} λ_{end} `QH_filename V0`

Purpose: Specifies the thermodynamic integration parameters for the immediately preceding `MD_segment` in file `control.in`.

λ_{start}, λ_{start} : Initial and final value for λ in the specific `MD_segment` .

`QH_filename` : Name of the file containing the parametrization of the quasi-harmonic potential $V^{qh}(\vec{R})$.

`V0` : Value of the Born-Oppenheimer potential $V^{nu}(\vec{R}_0)$ in equilibrium $\vec{R}_0$.

In `control.in`, the line containing the parameters for the thermodynamic integration must follow the line containing the `MD_segment` that the thermodynamic integration refers to. Note that a `thermodynamic_integration` line must be provided for all segments (or none) within an `MD_schedule` section, as shown in the following example:

```
MD_schedule
  # Equilibrate the system for 100 fs at 800 K with lambda = 0
  MD_segment   0.1 NVT_parrinello 800 0.0010
    thermodynamic_integration 0.0 0.0 FC_file.dat  -0.328E+07

  # Perform the thermodynamic integration over 10 ps
  MD_segment  10.0 NVT_parrinello 800 0.0010
    thermodynamic_integration 0.0 1.0 FC_file.dat  -0.328E+07
```

Tags for `QH_filename`:

Note that the file `QH_filename` can be automatically generated with the methods discussed in Sec. 4.6. As a reference, the syntax of the file is given here in spite of that.

Tag: `lattice_vector` (QH_filename)

Usage: `lattice_vector x y z latt_index`

Purpose: Specifies one lattice vector for periodic boundary conditions.

`x`, `y`, `z` are real numbers (in Å) which specify the direction and length of a unit cell vector.

`latt_index` : Sequential integer number identifying the lattice vector.

Lattice vectors associated with the equilibrium geometry. Please note that this input has to be equal to the specifications in `geometry.in`.

Tag: `atom` (QH_filename)

Usage: `atom x y z species_name atom_index`

Purpose: Specifies the equilibrium location and type of an atom.

`x`, `y`, `z` are real numbers (in Å) which specify the atomic position.

`species_name` is a string descriptor which names the element on this atomic position; it must match with one of the species descriptions given in `control.in`.

`atom_index` : Sequential integer number identifying the atom.

Equilibrium atom positions. Please note that this input has to be consistent with the specification in `geometry.in` (same number of atoms, same order, same species).

Tag: `force_constants` (QH_filename)

Usage: `force_constants FC_x FC_y FC_z atom_j direction atom_i`

Purpose: Specifies the force constants, i.e., the change in the forces that occur if one atom is displaced in the unit cell.

`FC_x`, `FC_y`, `FC_z` are the change in the forces in the respective cartesian coordinates.

`atom_j` : is the index of the atom that is displaced.

`direction` : is the cartesian direction in which atom_j is displaced.

`atom_i` : is the index of the atom the forces refer to.

Equilibrium atom positions. Please note that this input has to be consistent with the rest of the specification in `QH_filename`.

Example for a very basic file QH_filename:

```
lattice_vector    3.987    3.987    0.000    1
lattice_vector    0.000    3.987    3.987    2
lattice_vector    3.987    0.000    3.987    3

atom              0.000    0.000    0.000    Al   1
atom              1.993    1.993    0.000    Al   2
atom              0.000    1.993    1.993    Al   3
atom              1.993    3.987    1.993    Al   4
atom              1.993    0.000    1.993    Al   5
atom              3.987    1.993    1.993    Al   6
atom              1.993    1.993    3.987    Al   7
atom              3.987    3.987    3.987    Al   8

# displace atom 1 ------------------------------------------
force_constants   5.739e+00  -7.069e-16   6.805e-16    1 1 1
force_constants  -1.909e+00  -1.455e+00  -1.324e-16    1 1 2
force_constants   3.692e-01   2.618e-16   3.813e-16    1 1 3
force_constants  -1.909e+00  -1.398e-16   1.201e+00    1 1 4
force_constants  -1.909e+00   2.040e-16  -1.201e+00    1 1 5
force_constants   3.692e-01   1.423e-16  -7.205e-16    1 1 6
force_constants  -1.909e+00   1.455e+00   9.268e-17    1 1 7
force_constants  -3.934e-02   4.931e-18  -4.373e-18    1 1 8
force_constants  -2.979e-17   5.004e+00   1.698e-15    1 2 1
force_constants  -1.016e+00  -1.430e+00  -9.899e-17    1 2 2
.........................................................
force_constants   1.675e-19  -3.460e-02  -1.160e-17    1 2 8
force_constants   3.498e-17   2.663e-16   5.373e+00    1 3 1
.........................................................
force_constants  -2.894e-16   3.914e-16   3.969e-01    1 3 8
# end atom 1 -----------------------------------------------
# displace atom 2 ------------------------------------------
force_constants  -1.909e+00  -1.455e+00   3.466e-17    2 1 1
.........................................................
force_constants  -8.873e-17   1.914e-16   3.969e-01    2 3 8
# end atom 2 -----------------------------------------------
.........................................................
.........................................................
# end atom 7 -----------------------------------------------
# displace atom 8 ------------------------------------------
force_constants  -3.934e-02   4.931e-18  -4.373e-18    8 1 1
.........................................................
force_constants   3.498e-17   2.663e-16   5.373e+00    8 3 8
#--------------------------------------------------------
```

3.14 Electronic constraints

Most production calculations only require the converged ground state of a calculation, but in some cases, a deliberate deviation from the Born-Oppenheimer surface is desired. For example it may be desirable to fix the spin state of a spin-polarized calculation using a defined `multiplicity` .

More generally, intuitive chemical concepts may suggest the localization of a fixed given spin moment or number of electrons in one part of a system, and a different spin moment or number in another part. Examples include enforcing definite charges on ions in a system, or a desired spin moment on one particular atom.

The latter idea of partitioning different numbers of electrons or spin moments in *space* thought of as divided into different atoms is inherently ambiguous. Nonetheless, this is a classic intuitive picture of chemistry, and, at least in the limit of well-separated system parts, becomes exact.

For *non-perodic* geometries, FHI-aims implements the possibility of constrained calculations to enforce predefined electron numbers in different regions and/or spin channels. We follow the prescription of Behler et al. [?, ?], which assigns electrons to different atoms according to a Mulliken-like analysis, i.e., by partitioning the occupied Kohn-Sham eigenstates according to the occupation of different atom-centered basis functions.

For details regarding the method, we refer to the original references, but we add here a clear word of caution. The implemented partitioning is well-defined when the relevant parts of the system are far apart, and/or when relatively small, confined basis sets are employed. For large, overlapping basis sets, electrons may be *spatially* located at one atom even though they are *numerically* assigned to the basis functions of a different atom. In other words, the procedure becomes more ambiguous as the basis size and completeness increase. Contrary to the usual paradigm of electronic structure theory, it is not meaningful to converge constrained DFT calculations to the basis set limit *if* the individual pieces of the system are not very well separated.

To set up an electronic constraint, the only keywords normally required are `constraint_region` in `geometry.in` and `constraint_electrons` in `control.in`. The remaining keywords documented below are normally required only for experimental purposes or troubleshooting.

For the purpose of simulating electronic excitations, either from electronic core levels (XPS) or from valence levels (UPS), the keywords `force_occupation_basis` and `force_occupation_projector` enforce electron occupations of specific atomic basis states or Kohn-Sham states, respectively.

Tags for `geometry.in`:

Tag: `constraint_region` (geometry.in)

Usage: `constraint_region number`

Purpose: Assigns the immediately preceding `atom` to the region labelled `number`.

`number` is the integer number of a spatial region, which must correspond to a region defined by keyword `constraint_electrons` in file `control.in`. Default: 1.

To divide up space into regions for the purpose of enforcing an electronic constraint, each atom in the structure is included in a `constraint_region` .

Simple example of an Na-Cl dimer (`geometry.in`):

```
atom 0. 0. 0. Na
  constraint_region 1
atom 0. 0. 3. Cl
  constraint_region 2
```

assigns Na to the first region and Cl to the second region of a constrained calculation.

The special case of only one region (e.g., for a fixed spin moment calculation) needs no explicit `constraint_region` labels. Note that, apart from an explicit setup by keyword `constraint_electrons` , the case of an *integer* fixed spin moment for the *whole* system (all atoms) can also be called by the shortcut `multiplicity` .

Tags for general section of `control.in`:

Tag: `constraint_debug` (control.in)

Usage: `constraint_debug flag`

Purpose: If set, provides extra output that monitors the convergence of the local constraint potentials used to enforce the requested constraint.

`flag` is a logical string, either `.true.` or `.false.` Default: `.false.`

Tag: `constraint_electrons` (control.in)

Usage: `constraint_electrons region n_1 [n_2]`

Purpose: Fixes the number of electrons to `n_1` (in the spin-polarized case, to `n_1` in the spin-up channel and `n_2` in the spin-down channel) for a given `region`.

`region` is an integer number, corresponding to one of the regions listed as `constraint_region` in `geometry.in`.
`n_1` is the number of electrons in the corresponding `region` (the number of spin-up electrons in the case of a spin-polarized calculation).
`n_2` is the number of spin-down electrons in the corresponding `region` (only needed in the spin-polarized case).

This is the central keyword that can be used to define a strict constraint on the electron numbers associated (i) with the orbitals of a given subset of atoms ("region") and / or (ii) with a given spin channel. See the `multiplicity` keyword for a shortcut for fixed spin moment calculations with an integer spin multiplicity.

Tag: `constraint_it_lim` (control.in)

Usage: `constraint_it_lim number`

Purpose: For the determination of the constraint potentials in different `constraint_region` s, sets the maximum number of internal iterations before the search for a converged value is aborted.

`number` is an integer number. Default: 200.

The method to determine the constraint potentials that enforce the local electron / spin constraint is set by `constraint_mix` ; for more than one active `constraint_region` , this determination is always iterative. Keyword `constraint_it_lim` sets the maximum number of iterations before this search is aborted in case of failed convergence (or too ambitious accuracy requirements from keyword `constraint_precision`).

Tag: `constraint_precision` (control.in)

Usage: constraint_precision tolerance

Purpose: Sets the precision with which each requested local constraint on the elctron count must be fulfilled.

tolerance is a small positive real number. Default: 10^{-6}.

Tag: constraint_mix (control.in)

Usage: constraint_mix factor1 [factor2]

Purpose: Mixing factors for the iteratively determined constraint potentials.

factor1 is a mixing factor for the iteratively determined constraint potentials (for spin-polarized calculations, the spin-up mixing factor).
factor2 is the mixing factor for spin-down constraint potentials in the case of spin polarized calculations.

Only meaningful for non-standard settings of mixer_constraint , irrelevant for the standard bfgs case.

Tag: ini_linear_mixing_constraint (control.in)

Usage: ini_linear_mixing_constraint number

Purpose: If keyword mixer_constraint is a Pulay mixer, initial linear mixing for a few iterations can be requested first.

number is the number of initial linear iterations.

Only meaningful for non-standard settings of mixer_constraint , irrelevant for the standard bfgs case.

Tag: mixer_constraint (control.in)

Usage: mixer_constraint type

Purpose: Sets the iterative algorithm to determine constraint potentials.

type is a string. Default: bfgs

This flag has nothing to do with electron density mixing or the electronic self-consistency loop. Instead, this defines the process to determine constraint potentials that enforce the requested electron number constraints, *if* the keyword constraint_electrons was invoked. This process happens in each s.c.f. iteration after the Hamiltonian matrix is known, in the course of solving the Kohn-Sham eigenvalue problem.

If more than one constraint regions are requested, determining the constraint potentials to enforce the local constraint on the electron numbers is an iterative process. Technically, FHI-aims supports three different algorithms for type :

- bfgs : The default. A BFGS algorithm that optimizes the constraint potentials to their nearest local optimum. Nothing else should be used unless for experimental

purposes.

- `linear` : Linear mixing algorithm to determine the constraint potentials.
- `pulay` : Pulay-type mixing algorithm to determine the constraint potentials.

Under normal circumstances, the `mixer_constraint` keyword should not be needed explicitly.

Tag: `n_max_pulay_constraint` (control.in)

Usage: `n_max_pulay_constraint number`

Purpose: If the `pulay` mixer is selected for `mixer_constraint` , sets the number of iterations to be mixed.

`number` is the number of mixed iterations. Default: 8.

Only meaningful for non-standard settings of `mixer_constraint` , irrelevant for the standard `bfgs` case.

Tag: `force_occupation_basis` (control.in)

Usage: `force_occupation_basis i_atom spin basis_type basis_n basis_l basis_m occ_number max_KS_state`

Purpose: Flag originally programmed to compute core-hole spectroscopy simulations. In practice, it constrains the occupation of a specific energy level of an specific atom, being also able to "break the symmetry" of an atom.

`i_atom` is the number of the atom on which the occupancy is constrained, as it is listed in the geometry.in file.

`spin` is the spin channel (e.g., 1 if only one spin channel).

`basis_type` is the type of basis which is used to force the occupation of the orbital (set it to `atomic`).

`basis_n` is the main quantum number for the state of interest.

`basis_l` is the orbital momentum quantum number for the state of interest.

`basis_m` is the projection of the orbital momentum onto the z-axis (-1, 0, or 1 for a p state).

`occ_number` is the occupation constraint for the chosen state.

`max_KS_state` is the number of the highest energy Kohn-Sham state in the system that will be included in the constraint.

Example:

```
force_occupation_basis 1 1 atomic 2 1 1 1.3333 6
```

This choice will constrain the overall occupation of a given basis function (not Kohn-Sham state!) in the system to 1.3333 electrons.

The basis function in question resides on the first atom (number 1) as listed in `geometry.in`. The first spin channel is constrained.

Since we are interested in constraining an atomic-like orbital, we choose one that is part of the minimal basis (type "atomic").

In fact, we let the constraint act on a $2p$ level, m=1 (defined by the sequence "2 1 1").

Only Kohn-Sham orbitals up to the 6th state (counted in the overall system) will be included in the constraint. In general, it is a good idea to constrain the orbital in question out of the occupied space, i.e., choose a value for `max_KS_state` that indicates a state above the Fermi level.

There is a small problem here: We need to define the occupation of a "basis function," but really, we here have a non-orthogonal basis. Strictly speaking, only the Kohn-Sham states in the system have a well-defined occupation. What to do?

One thing we could use (and we do) are Mulliken occupation numbers of the basis functions. These are formally always well defined. In practice, however, as the overall basis set becomes larger and larger and approaches (over-)completeness, Mulliken occupations become less and less meaningful because other basis functions are not exactly orthogonal to the one used in our projection, and can "restore" the component that was originally constrained away.

Either way, we use Mulliken occupations, assuming that the atomic core basis functions are sufficiently compact and practically orthogonal to everything else.

This assumption will work well for localized basis functions such as the $1s$ levels of most elements. As a rule, the constraint is expected to be less and less unique if applied to more delocalized basis functions – there can even be multiple different self-consistent constrained solutions for the same formal constraint. For instance, Si $2p$ basis functions can exhibit this problem if the basis sets on the surrounding atoms – which overlap with the Si atom – become too large. Here, a smaller basis set (tier 1) can indeed be the way to keep a qualitatively meaningful Mulliken-type constraint.

Tag: **`force_occupation_projector`** (control.in)

Usage: `force_occupation_projector KS_state spin occ KS_start KS_stop`

Purpose: This keyword enforces the occupation `occ` in `KS_state` of spin channel `spin`. Between different SCF steps the overlap of this state with states `KS_start` to `KS_stop` is beeing checked and the constraint is changed correspondingly if the main character of the state changes.

Example:

```
force_occupation_projector 8 1 0.0 6 10
force_occupation_projector 9 2 1.0 6 10
```

This enforces 0.0 occupation in state 8 of spin channel 1 and 1.0 occupation for state 9

of spin channel 2. KS states between 6 and 10 are checked for overlap with state 8 and 9 of previous SCF steps. If 8/1 was occupied and 9/2 was unoccupied in the ground state, this corresponds to a triplet excitation 8→9.

Tag: `force_occupation_smearing` (control.in)

Usage: `force_occupation_smearing smearing_width`

Purpose: If keyword is set, the occupation constraints are enforced in form of gaussians with a width of `smearing_width` instead of delta peaks. This applies to orbitals within an energy range of -+ 3*`smearing_width`

Default: No Smearing at all.

Example:

```
force_occupation_smearing 0.05
```

This keyword helps to converge systems with state degeneracies, which are constrained by force_occupation_projector. Specifically when calculating electronic excited states in the frontier orbital regime, many state degeneracies can occur. If one of two degenerate states is constrained to a different than the ground state occupation, convergence can be hindered. If this happens, this keyword can enable convergence for the price of a minimally different final occupation and therefore also a small error in excitation energy. One should be very careful with this keyword and only employ it if convergence can not be reached without.

WARNING: It is very easy to generate reandom numbers when using this keyword. The smearing value should never exceed 0.15

3.15 Embedding in external fields

To simulate the effect of external field (for instance, to connect to a QM/MM embedding formalism), FHI-aims allows to add the effect of a homogeneous electrical field and/or point charges surrounding the molecule in question.

Note that these embedding charges are *in addition* to any `charge` specified in `control.in`, and *not* already included there. `charge` should equal only to the sum of charges of all nuclei in `geometry.in` minus the overall number of electrons in the system, but does not count any embedding charges specified by keyword `multipole`.

This functionality is not yet available for periodic systems.

Warning: When using a multipole, e.g., an external charge with no basis functions etc., you are creating a Coulomb singularity. If this singularity is inside the radius of a basis function of another atom, it will lead to numerical noise in integrals, up to near-infinities.

To test and/or overcome this problem, all you need to do is to place an integration grid on any multipole that is within the radius of a basis function of any atom. This radius is given by the cutoff radius plus width in the `cut_pot` keyword of each `species`.

Such a grid can be placed by creating an `empty` site with no basis functions (`include_min_basis` .false.) and placing this empty site on the same site as the multipole in question in `geometry.in`. Simply taking the species defaults for a H atom (light settings) and adjusting them to have no basis functions should create the necessary definition of the empty site in question (see Fig.3.2 for an example).

Tags for `geometry.in`:

Tag: `homogeneous_field` (geometry.in)

Usage: `homogeneous_field E_x E_y E_z`

Purpose: Allows to perform a calculation for a system in a homogeneous electrical field $\boldsymbol{E}$.

`E_x` is a real number, the x component of $\boldsymbol{E}$ in V/Å.
`E_y` is a real number, the y component of $\boldsymbol{E}$ in V/Å.
`E_z` is a real number, the z component of $\boldsymbol{E}$ in V/Å.

Please note: The electrical field is usually defined to point in the direction of a force exerted on a *positive* probe charge. Historically grown, FHIaims uses the opposite sign convention. Although this behaviour might be considered a bug, we decided to leave it this way in order not to break any scripts people are already using.

Tag: `multipole` (geometry.in)

Usage: `multipole x y z order charge`

Purpose: Places the center of an electrostatic multipole field at a specified location, to simulate an embedding potential.

`x` : x coordinate of the multipole.
`y` : y coordinate of the multipole.
`z` : z coordinate of the multipole.
`order` : Integer number, specifies the order of the multipole (0 or 1 $\equiv$ monopole or dipole).
`charge` : Real number, specifies the charge associated with the multipole.

If the order of the multipole is greater than zero (presently, only monopoles or dipoles are supported), a dipole moment must be specified in addition to the data provided with the `multipole` tag itself. To that end, a line must immediately follow the original `multipole` line, adhering to the following format:

`data m_x m_y m_z`

Here, `m_x`, `m_y`, `m_z` are the x, y, and z components of the dipole moment, in e·Å.

Warning: Note that monopoles amount to Coulomb singularities. When inside the basis function radius of any atom, such monopoles should be covered with an integration grid in `geometry.in`, as explained in the beginning of this section.

Tags for general section of `control.in`:

Tag: `full_embedding` (control.in)

Usage: `full_embedding flag`

Purpose: Allows to switch between embedding of the full electronic structure (affecting the Kohn-Sham equations) or the embedding of an electronic density that is calculated without knowledge of the embedding potential.

`flag` is a logical string, `.true.` or `.false.` Default: `.true.`

For most purposes, embedding into an external potential will involve a change to the electronic structure of the structure which is embedded. However, in some instances one may wish to embed a given charge density *non-selfconsistently*, i.e., by calculating the electron density without an external field and then computing the energy of that unperturbed electron density in the external field.

This feature is useful if multipoles are located too close to the quantum-mechanical region of the calculation. These act as Coulomb-like potentials, just like any other potential in the systems. If there are basis functions that cover the location of that potential, some electronic charge may artificially become trapped there, creating a bad approximation to the core / valence electrons of an atom with a nucleus of charge `charge`. In most cases, this is clearly undesirable behaviour, and apart from that will create unwanted numerical noise since the electronic structure near the Coulomb-like singularity of the multipole will be represented solely by basis functions that are inadequate for this purpose in the first place.

Tag: `qmmm` (control.in)

Usage: `qmmm`

Purpose: Allows to compute Hellmann-Feynman like forces *from* the quantum-mechanical part of a structure *exerted on* the multipoles on an external embedding field.

Restriction: Works only for external monopole potentials.

For quantum-mechanics / molecular-mechanics (QM/MM) "hybrid" molecular dynamics simulations, one must evolve *both* the quantum and classical subsystems with time. In that case, it is necessary to know the derivatives of the quantum-mechanical total energy with respect to the positions of the *classical* multipoles (see keyword `multipole`), i.e., the forces on the multipoles that originate from the quantum-mechanical region.

The computation of these forces is switched on by adding the `qmmm` keyword to `control.in`. The actual QM/MM simulation must still be performed using an outside framework, for example ChemShell [?] that uses the energies and forces provided by FHI-aims as a "plugin".

```
[...]
  species        empty_site
#     global species definitions
    nucleus             1
    mass                1.00794
#
    l_hartree           4
#
    cut_pot             3.5  1.5  1.0
    basis_dep_cutoff    1e-4
#
    include_min_basis .false.

    radial_base         24 5.0
    radial_multiplier   1
    angular_grids       specified
      division   0.2421   50
      division   0.3822  110
      division   0.4799  194
      division   0.5341  302
      outer_grid  302
###############################################################################
#
#  Definition of "minimal" basis
#
###############################################################################
#     valence basis states
    valence      1  s   1.
#     ion occupancy
    ion_occ      1  s   0.5
###############################################################################




[...]
```

Figure 3.2: Species data for what could be used as an empty site on top of monopole.

3.16 QM/MM Embedding

Please continue to consider this functionality experimental and contact the maintainers before using it.

When simulating nanostructured surfaces, it may be favorable to avoid the standard supercell approach and rather make use of a QM/MM embedding approach. E.g. with clusters or molecules adsorbed, extensive supercells are required to avoid spurious interaction between the nanostructure and its periodic copies. This makes computations tedious.

In the QM/MM approach, one embeds a quantum mechanical (QM) region in an extended monopole field. To avoid spurious charge leakage out of the QM region positively charged monopoles in the vicinity are replaced by norm-conserving pseudopotentials [?]. Those pseudopotential files can be either generated manually with the open source program package FHI98PP [?] or downloaded from http://www.abinit.org/downloads/psp-links/lda_fhi or http://www.abinit.org/downloads/psp-links/gga_fhi.

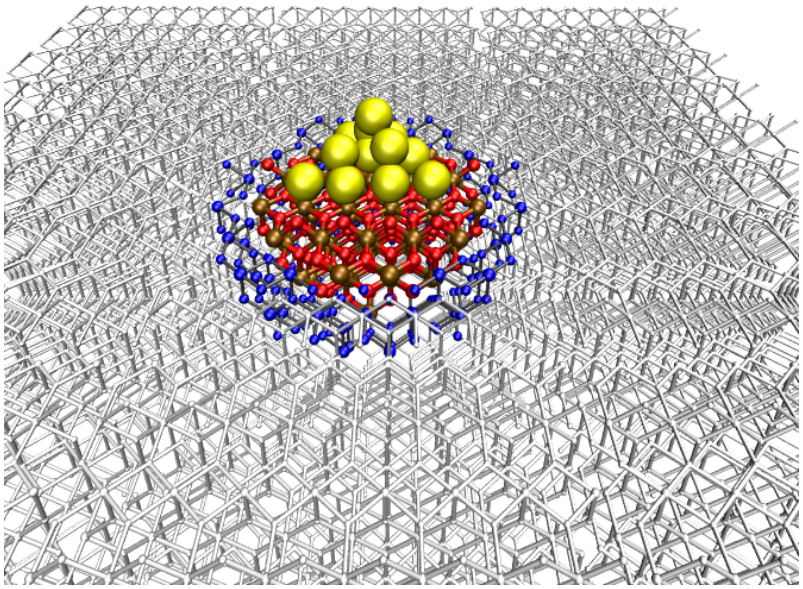


Figure 3.3: Example for QM/MM setup: $Au_n@TiO_2$. The adsorbed cluster and direct substrate vicinity defines the QM-region. The far field surrounding (grey particles) pictures a monopole field with formal charges (4+ for Ti and 2- for O). In the blue region, oxygen particles are still represented as monopoles, however Ti-cations are described with ionic pseudopotentials.

The QM/MM approach has the huge advantage ultimately also being capable to efficiently deal with **charged** systems, which will be a fundamental asset for the description of surface electrochemistry or photo-induced catalysis.

QM/MM embedding is not applicable for metal substrates for physical reasons.

Tags for `geometry.in`:

Tag: `pseudocore` (geometry.in)

Usage: `pseudocore x y z species`

Purpose: Places the center of a pseudopotential at a specified location.

`x` : x coordinate of the pseudocore.
`y` : y coordinate of the pseudocore.
`z` : z coordinate of the pseudocore.
`species` : string defining the name of the pseudoized species; this needs to correspond to name specified in control.in .

The keyword `pseudocore` should be used for those particles replaced by pseudopotentials, so cations. Anions are to be treated simply as monopoles, employing the `multipole` infrastructure.

Tags for general section of `control.in`:

Only species data concerning the pseudoized species mentioned in `geometry.in` need to be appended in the `control.in`. Accept for some mandatory changes (listed below) those species data are essentially the same as you can find them in the species_default folder. E.g. if you want to pseudoize for example titanium, take the Ti default file as a template. However, **the pseudoized species must not have any basis functions accept for the minimal basis**. The minimal basis in needed to construct the integration weights, however in order to exclude the minimal basis from the actual quantum chemical calculation, the flag `include_min_basis` needs to be set to `.false.`.

Although nomenclature is misleading as it is chosen at the moment, you do NOT need the `qmmm` in order to make QM/MM embedding work.

Similar to all-electron `atom` , FHI-aims expects all atom specifications like `mass` , `nucleus` , information for the integration grid etc. Some additional flags need to be set that FHI-aims is able to realize them as pseudoized species.

`species` **sub-tag:** `pseudo` (control.in)

Usage: `pseudo string`

Purpose: Parses the name of file the Kleinman-Bylander pseudopotential is written in

`string` name of file

FHI-aims expects the pseudopotential file to be in a specific formatting, namely the output format *.cpi of the generator program FHI98PP [?]. FHI98PP expects this file to be in the same folder as control.in and geometry.in.

---

species sub-tag: pp_charge (control.in)

Usage: pp_charge value

Purpose: Specifies the charge of the pseudoized ion.

value: any real value is allowed

pp_charge must be the charge which has been set in the generation of the pseudopotential and equals the number of pseudoized valence electrons. This parameter is needed for the far field extrapolation of the pseudopotential.

---

species sub-tag: pp_local_component (control.in)

Usage: pp_local_component value

Purpose: Specifies which l-channel of the pseudopotential should act as the local component. Find a detailed theoretical background in [?].

value: integer value

The choice which l-channel should be the local component is essential for the performance of the pseudopotentials. Again, read [?] for further help.

---

species sub-tag: nonlinear_core (control.in)

Usage: nonlinear_core flag

Purpose: when .true. FHI-aims expects and reads in a partial core density (and partial core density gradient) from the pseudopotential input file to take account of nonlocal core correction [?].

flag is a logical expression, either .true. or .false. Default: .false.

```
[...]
  species        Ti_pseudo
#     global species definitions
    nucleus             22
    mass                47.867

    pseudo              Ti.cpi
    pp_charge           4.
    pp_local_component     1
    nonlinear_core      .false.

    include_min_basis .false.


[...]
```

Figure 3.4: Species data for a pseudoized titanium atom. Starting from the default species files only a few flags need to be added and the basis functions (accept the minimal basis) need to be removed.

3.17 C_6/R^6 corrections for long-range van der Waals interactions

The correction improves the description of van der Waals (vdW) interactions in DFT. It is based on the leading-order C_6/R^6 term for the interaction energy between two atoms. Both energy and analytic forces are implemented.

Two flavors of the correction are implemented:

(1) The C_6 coefficients and vdW radii are obtained directly from Hirshfeld partitioning of the DFT electron density. This scheme only requires a single damping parameter, which is fitted to binding energies of small organic molecules and hardwired in the code for PBE, PBE0, revPBE, AM05, BLYP and B3LYP functionals. For more information and citation see Ref. [**?**]. Both cluster and periodic cases are implemented.

(2) The empirical C_6 coefficients and vdW radii must be specified directly. This scheme is coded for maintaining compatibility with empirical C_6 approaches. In actual applications, the usage of scheme (1) is advised, since it is significantly more accurate and less empirical.

Tags for general section of `control.in`:

Tag: `vdw_convergence_threshold` (control.in)

Usage: `vdw_convergence_threshold value`

Purpose: When using the vdW correction based on Hirshfeld partitioning of the electron density (as described in Tkatchenko and Scheffler 2009, Ref. [?]) in a periodic system, this sets the energy convergence threshold for the supercell sum over the TS components.
`value`: A small positive number (in eV). Default: For unit cells with less than 100 atoms: 10^{-6} eV. For structures with unit cell sizes above 100 atoms, the default is adjusted to $n_{\text{atoms}} \cdot 10^{-8}$ eV.

Note that the vdw part of the forces may be separately converged to (if set) `sc_accuracy_forces`.

Tag: `vdw_correction_hirshfeld` (control.in)

Usage: `vdw_correction_hirshfeld`

Purpose: Enables the vdW correction based on Hirshfeld partitioning of the electron density (as described in Tkatchenko and Scheffler 2009, Ref. [?]). If this keyword is set in a periodic calculation, the sum over atom pairs is done over successively larger supercells, until the energy is converged to the level set by `sc_accuracy_etot` or `vdw_convergence_threshold` and the forces (if requested) are converged to within `sc_accuracy_forces` .
No other input required.

This method is commonly referred to as the Tkatchenko-Scheffler method. The procedure is as follows. *First*, the normal self-consistency cycle is completed for a semilocal or hybrid density functional, most commonly PBE or PBE0. *Second*, the resulting self-consistent electron density is used to create interatomic (pairwise) C_6 coefficients. A simple pairwise van der Waals term is then added once to the self-consistent total energy from the preceding semilocal or hybrid functional. In other words, the Tkatchenko-Scheffler method is normally employed as a post-processing term in a non-self-consistent way, not during the self-consistency cycle. Since it needs to be combined with a different density functional, you would normally use it like this (example for "PBE+vdW"):

```
xc pbe
vdw_correction_hirshfeld
```

Three more caveats: (1) Do not use this method together with the local-density approximation (LDA) unless you know exactly what you are doing. The LDA already contains a spurious interaction term that will lead to very strange results if added to a pairwise van der Waals term. (2) Do not simply apply this method to a metallic system unless you know what you are doing. (3) This is also not the (very different) functional commonly known as the Langreth-Lundqvist or vdw-DF functional. [?] FHI-aims contains at least

two implementations of vdw-DF for those who are interested, but either implementation is much slower than the Tkatchenko-Scheffler pairwise interatomic sum.

Tag: `vdw_correction_hirshfeld_sc` (control.in)

Usage: `vdw_correction_hirshfeld_sc`

Purpose: Enables the self-consistent version of the vdW correction based on Hirshfeld partitioning of the electron density (see Tkatchenko and Scheffler 2009, Ref. [?]). In a periodic calculation, the energy is converged with the same criteria of the *a posteriori* approach: `vdw_correction_hirshfeld` .

This flag adds the Tkatchenko-Scheffler vdW functional as a part of the given exchange-correlation (XC) functional. In a self-consistent scheme, the contribution of the vdW potential, $v_{\mathrm{vdW}}[n(\mathbf{r})] = \delta E_{\mathrm{vdW}}[n(\mathbf{r})]/\delta n(\mathbf{r})$, is added to the XC potential to form the total effective potential in the Kohn-Sham equations. As a result, the van der Waals interatomic contributions affect the total electron density and are computed at each self-consistent cycle, until convergence is reached. In this way, it is possible to evaluate the effects of vdW interactions on the electron density and electronic properties, going beyond the vdW *a posteriori* correction of the total DFT energy.

Note: Do not use this self-consistent flag during a relaxation. The self-consistent forces are not implemented (yet) for the Tkatchenko-Scheffler vdW functional and this will lead to inconsistency errors.

Tag: `vdw_correction` (control.in)

Usage: `vdw_correction`

Purpose: Enables the empirical C_6/R^6 correction with the C_6 coefficients and vdW radii specified by the user.

The user needs to specify the interaction parameters for *all* atomic pairs in the system (i.e. for CNOH, there are 10 atomic pairs). This is done by putting "vdw_pairs N", where N is the number of pairs. This should be followed by N lines of "vdw_coeff atom$_i$ atom$_j$ C_{6ij} R^0_{ij} d", where C_{6ij} is the C_6 coefficient for the interaction between atom$_i$ and atom$_j$, R^0_{ij} is the corresponding vdW radius and d is the damping function parameter. A choice d=20 is suggested for all atomic pairs. An example for C-C interaction is: "vdw_coeff C C 30.00 5.59 20.0".

Tag: `vdw_pair_ignore` (control.in)

Usage: `vdw_pair_ignore species1 species2`

Purpose: excludes the interaction between `species1` and `species2` from any C6-correction, eg. such that metallic slabs are not affected internally by introducing C6-interactions.

Subtags for *species* tag in `control.in`:

---

species sub-tag: `hirshfeld_param` (control.in)

Usage: `hirshfeld_param C6 alpha R0`

Default: the values outlined in Ref. [?]
Purpose: To explicitly allow setting the parameters for the Tkatchenko-Scheffler van der Waals correction.

3.18 Many-Body Dispersion (MBD) energy

The non-local many-body dispersion (MBD) or correlation energy is computed using a system of coupled atomic response functions [?, ?]. The MBD method implemented in

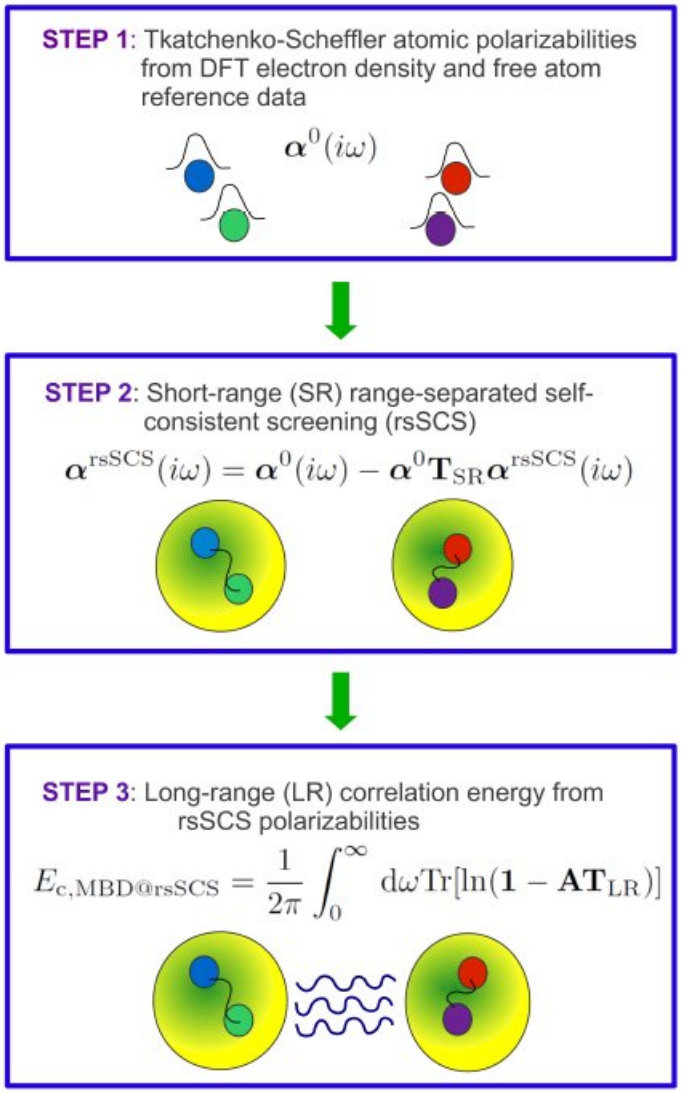


Figure 3.5: Schematic description of the MBD@rsSCS method.

FHI-aims is the so-called range-separated MBD@rsSCS method [?, ?]. The dynamic polarizability $\alpha(i\omega)$ as a function of imaginary frequency for a molecule or extended system is obtained as a solution to the self-consistent screening equation using a regularized dipole potential. The standard output of FHI-aims contains the upper triangular part of the dynamic polarizability tensor:

```
| Many-Body Dispersion (MBD@rsSCS) energy
| Dynamic polarizability of unit cell alpha(iw)(bohr^3)
|---------------------------------------------------------------------------
```

```
| omega(Ha)    XX          YY          ZZ          YZ          XZ          XY
| 0.000000  0.717E+04   0.724E+04   0.609E+04   0.180E+01   0.752E+01   0.224E+00
| 0.039290  0.704E+04   0.711E+04   0.599E+04   0.175E+01   0.762E+01   0.222E+00
| 0.118358  0.616E+04   0.623E+04   0.529E+04   0.141E+01   0.816E+01   0.201E+00
```

Furthermore, the atom projected static polarizability $\alpha(0)$ and vdW C_6 coefficient for each atom in a system are given as follows:

```
|--------------------------------------------------------------------------------
| Partitioned C6 (hartree bohr^6) | alpha(bohr^3)  of atom in unit cell
|--------------------------------------------------------------------------------
| ATOM    1   Zn   189.850551          30.849981
| ATOM    2   Zn   194.746383          31.378482
| ATOM    3   O    10.645313           2.411834
| ATOM    4   O    10.522485           2.335995
```

Please note that the above values are derived using a range-separated dipole potential. For more technical details see Refs. [?, ?].

The forces or gradients for MBD@rsSCS method are available via numerical finite differences. They work for both cluster as well as periodic geometries.

Tags for general section of `control.in`:

Tag: `many_body_dispersion` (control.in)

Usage: `many_body_dispersion`

Purpose: Calculates the MBD@rsSCS energy for a given xc functional (available for PBE, PBE0, and HSE functionals).

Tag: `mbd_scs_vacuum_axis` (control.in)

Usage: `mbd_scs_vacuum_axis flag flag flag`

Purpose: This keyword specifies direction of the vacuum region in case of a low-dimensional system.
Default: No vacuum `mbd_scs_vacuum_axis .false. .false. .false.`
For example, in the case of *periodic* slab along xy directions the keyword `mbd_scs_vacuum_axis .false. .false. .true.` is needed to specify z as vacuum direction in MBD/SCS calculation.

Tag: `mbd_cfdm_dip_cutoff` (control.in)

Usage: `mbd_cfdm_dip_cutoff value`

Purpose: Radius used to integrate the dipole field in a *periodic* MBD calculation (internal default value =100.0 Angstrom)

Tag: `mbd_scs_dip_cutoff` (control.in)

Usage: `mbd_scs_dip_cutoff value`

Purpose: Radius used to integrate the dipole field in a *periodic* SCS calculation (internal default value =120.0 Angstrom)

Tag: `mbd_supercell_cutoff` (control.in)

Usage: `mbd_supercell_cutoff value`

Purpose: Radius used to construct the supercell in *periodic* MBD@rsSCS calculations. (internal default value = 25 Angstrom) NOTE: The convergence with respect to the value of `mbd_supercell_cutoff` must be carefully tested for periodic systems with low symmetry.

Tag: `mbd_inversion_method` (control.in)

Usage: `mbd_inversion_method type`

Purpose: Algorithm used to solve the self-consistent screening (rsSCS) problem.

`type` is a keyword (string) which specifies the matrix inversion solver used. Default: `lapack` for serial or mpi runs with or without scaLAPACK support.

Available options for the matrix inversion, `type`, are:

- `lapack` : Standard solver provided by Lapack.
- `scalapack` : Fully memory-parallel implementation of the matrix inversion solver based on scaLAPACK.

Tag: `mbd_eigensolver` (control.in)

Usage: `mbd_eigensolver type`

Purpose: Algorithm used to solve the MBD eigenvalue problem.

`type` is a keyword (string) which specifies the eigenvalue solver used. Default: `lapack` for serial or mpi runs without scaLAPACK support. `scalapack` for mpi runs with available scaLAPACK library.

Available options for the eigensolver, `type`, are:

- `lapack` : **Preferred serial eigensolver implementation**. Lapack-based, and similar to the divide&conquer based standard solver provided by Lapack.
- `scalapack` : Fully memory-parallel implementation of the eigenvalue solver based on scaLAPACK.

- `elpa` : **Preferred parallel eigensolver implementation. Uses the ELPA eigensolver library.** Same functionality as `scalapack`, however, substantially rewritten for an overall speedup and much improved scalability. Recommended on shared-memory platforms and distributed systems with any decent communication. Note that you must set the shell variable `OMP_NUM_THREADS=1` prior to running FHI-aims on some platforms. (see Appendix A)

The scaLAPACK or ELPA eigensolvers are only available if scaLAPACK support has been compiled into the FHI-aims binary—see the Makefile for more information.

In fact, the default parallel eigensolver in FHI-aims is the "ELPA" eigensolver, which uses some scalapack infrastructure but has been rewritten from the ground up for much improved parallel scalability.

On mildly parallel platforms (no more than tens of CPUs) and for normal system sizes (not significantly larger than $\sim$100 atoms), it is often acceptable to use the serial `lapack` eigensolver instead of the fully parallel `elpa` or `scalapack`.

NOTE: Care must be taken for parallel MBD calculations if `lapack` is chosen: MBD uses a supercell that is often larger than the specified unit cell. The current implementation allocates a $3N$x$3N$ matrix on every thread. If there is a memory problem it is worth to compile FHI-aims with scaLAPACK support and choose `elpa` or `scalapack`.

3.19 Calculating nonlocal correlation energy within density functional approach

Warning: This functionality is available in FHI-aims, but has not seen extensive testing by ourselves. It should therefore be treated as experimental. If you intend to use this functionality, by all means ensure that literature results obtained using this functional are reproducible using the implementation presented here.

There are currently **two** separate working implementations of van der Waals DF in FHI-aims:

- Sec. 3.19.1: One version that allows post-processing only (i.e., compute the self-consistent density by another XC functional, the evaluate only the vdw-DF energy term again after the fact), using a Monte Carlo integration scheme. This version works for non-periodic as well as periodic systems. The original code was developed in the group of Claudia Ambrosch-Draxl at University of Leoben, Austria.
- Sec. 3.19.2: A second version that relies on an analytical integration scheme developed by Simiam Ghan, Andris Gulans and Ville Havu at Aalto University in Helsinki. This version allows non-self-consistent *and* self-consistent usage, as well as gradients (forces). It also has a number of numerical convergence parameters that can be adjusted.

3.19.1 Monte Carlo integration based vdW-DF

As a postprocessing step after a self-consistent calculation, the nonlocal part of the correlation energy can be calculated using the van der Waals density functional proposed by M. Dion et al. [?]. This task follows exactly the recipe presented in the original paper [?]. This calculation can be performed by choosing `ll_vdwdf` as a `total_energy_method` (please see Section 3.3).

Important acknowledgment: The Langreth-Lundqvist functional and basic Monte Carlo integration scheme used here was made available to us by the group of Claudia Ambrosch-Draxl and coworkers at University of Leoben, Austria. If you use this functionality successfully, please cite their work (currently, Ref. [?] and "unpublished").

In order to perform the calculation, one should define an even spacing grid (followed by the `vdwdf` tag explained below), where the total charge densities of a system obtained after the scf cycle are projected.

In order to effectively solve the nonlocal correlation energy part, presented in Equation(1) of [?], the Monte Carlo integration scheme of Divonne integration method of CUBA library (Please visit http://www.feynarts.de/cuba for details).

This means that you should (yourself) compile the CUBA library as an external dependency to FHI-aims. The alternative Makefile `Makefile.cuba` the contains examples of how to link FHI-aims to the CUBA library, and enable the vdW-LL functional

Parameters for tuning the performance of Monte Carlo integration are defined under

the mc_int tag explained below. Also, the kernel values are tabulated in terms of the parameters in the formula of Equation(14) of [?]. The aims package comes with the tabulated kernel data (a file called kernel.dat) and the name of the file should be included in control.in.

Necessary input file: In your FHI-aims distribution, a version of the kernel.dat file as well as an example control.in file should be located in subdirectory *src/ll_vdwdf* . For an actual FHI-aims program run, the kernel.dat file (currently, kernel_my.dat is provided) must be copied to your working directory and must be referenced in your control.in file, using the kernel_data sub-keyword of the mc_int keyword (see below).

Tags for general section of `control.in`:

Tag: `mc_int` (control.in)

Usage: `mc_int subkeyword(s)`

Purpose: A line that begins with `vdwdf` is associated with the Monte Carlo integration performed to evaluate the non-local Langreth-Lundqvist functional. The `mc_int` keyword must be followed *on the same line* by a subkeyword that indicates the specific setting made here.

`subkeyword(s)` are one or more subkeywords or data for the Monte Carlo integration.

To use the Monte Carlo integrated Langreth-Lundqvist functional, more than one subkeyword for `mc_int` must be specified in the `control.in` file. See below for valid / necessary subkeywords. You may also want to check the documentation for the CUBA Monte Carlo integration library (http://www.feynarts.de/cuba) that you must have built and linked to the FHI-aims code in order to use the Langreth Lundqvist functional.

Tag: `vdwdf` (control.in)

Usage: `vdwdf subkeyword(s)`

Purpose: A line that begins with `vdwdf` is associated with the non-local Langreth-Lundqvist functional. The `vdwdf` keyword must be followed *on the same line* by a subkeyword that indicates the specific setting made here.

`subkeyword(s)` are one or more subkeywords or data for the Langreth-Lundqvist functional.

To use the Langreth-Lundqvist functional, more than one subkeyword for `vdwdf` must be specified in the `control.in` file. See below for valid / necessary subkeywords.

Subtags for *vdwdf* tag in `control.in`:

`vdwdf` **sub-tag:** `cell_origin` (control.in)

Usage: `cell_origin` x y z

x y z indicates the cartesian coordinates (in Å) of the origin of an even-spacing cubic cell. Default: `0.0 0.0 0.0`.

This option is only valid for a cluster calculation. For the case of periodic system, a cell origin is automatically determined at the center of a supercell.

`vdwdf` **sub-tag:** `cell_edge_steps` (control.in)

Usage: `cell_edge_steps` N_x N_y N_z

Purpose: the total number of grids in each direction are defined by integer numbers, x y z.

---

vdwdf sub-tag: `cell_edge_units` (control.in)

Usage: `cell_edge_units` d_x d_y d_z.

Purpose: The real numbers d_x d_y d_z (in Å) define the length of grid units in each direction. Therefore, the full grid length is ($N_x d_x$,$N_y d_y$, $N_z d_z$).

---

vdwdf sub-tag: `cell_size` (control.in)

Usage: `cell_size` L_x L_y L_z

Purpose: this defines number of interacting cells in x, y, z directions for van der Waals interactions. This option is meaningful for periodic calculation.

L_x L_y L_z are integer. Default: `0 0 0`.

Note: As a temporary restriction, FHI-aims currently supports only grids with vectors aligning along x, y, and z axes.

Calculations for periodic systems: In defining even spacing grid of a periodic system, only information of cell_edge_steps and cell_size (if needed) is necessary and other parameters will be automatically determined from that.

Subtags for *mc_int* tag in `control.in`:

`mc_int` **sub-tag:** `kernel_data` (control.in)

Usage: `kernel_data kernel.dat`

Purpose: the name of the tabulated kernel file.

`mc_int` **sub-tag:** `output_flag` (control.in)

Usage: `output_flag flag`

Purpose: this controls output of Monte Carlo integration process, level 0 for no output, level 1 for ŚreasonableŠ, and level 3 prints further the subregion results(if applicable). `flag` is an integer number. Default: `0`

`mc_int` **sub-tag:** `number_of_MC` (control.in)

Usage: `number_of_MC N`

Purpose: the total number of Monte-Carlo integration steps.

`N` is an integer number. Default: `5E5`

`mc_int` **sub-tag:** `relative_accuracy` (control.in)

Usage: `relative_accuracy` E_{acc}

Purpose: control the accuracy of Monte-Carlo integration performed by Cuba library.

E_{acc} is a real number. Default: `1E-16`

`mc_int` **sub-tag:** `absolute_accuracy` (control.in)

Usage: `absolute_accuracy` E_{abs}

Purpose: control the error bar of nonlocal correlation energy.

E_{abs} is a real number (in the unit of Hartree). Default: `1E-2`

3.19.2 Analytic integration scheme for non-selfconsistent and self-consistent vdW-DF

This method calculates the non-local part of the correlation energy as described in [?] allowing for both non-self-consistent and self-consistent treatment. It works for both cluster and periodic geometries and can be used to compute forces. The implementation as well as the kernel function are from [?]. At each scf-cycle the following steps are performed:

- An octree is built to interpolate the current electron density to the new integration grid below.
- To each grid point of the main integration grid another grid of similar form is attached. The non-local correlation is then integrated on this grid using density and its gradient interpolated from the octree. In each node of the tree a tricubic interpolation is used.

The first step of building the octree is not parallel but the second step of integration is MPI-parallel the usual way.

Tags for general section of `control.in`:

Tag: `nlcorr_nrad` (control.in)

Usage: `nlcorr_nrad number`

Purpose: Sets the number of radial shells used in the integration of the non-local correlation potential and energy. Default: `10`

Tag: `nlcorr_i_leb` (control.in)

Usage: `nlcorr_i_leb number`

Purpose: Sets the index of angular Lebedev grid used in the integration of the non-local correlation potential and energy. Maximum value available is `15`. Default: `7`

Tag: `vdw_method` (control.in)

Usage: `vdw_method type accuracy`

Purpose: Sets the method for density interpolation for the integration of the non-local correlation potential and energy.

`type` is the method selected, either `octree`, `mixed`, or `multipoles`. Default: `octree`

`accuracy` applies to methods `octree` and `mixed`. It is the targer accuracy of the interpolation. In case of `multipoles` all available multipoles are used. Default: `1E-6`

The point of providing three different options for `type` is simply that any prospective user should test which one is fastest for a given problem. The difference is simply the style of integration of the non-local part. Ideally, the results should be the same. However, as always, please check in case of doubt.

3.20 Hartree-Fock, hybrid functionals, GW, *et al.*: All the details

The basic keywords to invoke different exchange-correlation methods (ground state and excited states) are given described in Sec. 3.3. Usually, invoking the relevant keyword together with the normal infrastructure required to run FHI-aims should be sufficient to produce a correct, converged result.

For Hartree-Fock and hybrid functionals, particularly for their periodic implementations, see also the dedicated next section, Sec. 3.21.

For methods that rely explicitly on a two-electron Coulomb operator (Hartree-Fock, hybrid functionals, GW, MP2, RPA, etc.) and/or a frequency-dependent response function (GW, RPA, ...), some considerable numerical trickery enters the computation in order to keep it efficient yet manageable for practical purposes. We hope to provide resilient, system-independent default settings, but there is a lot of freedom beyond those defaults to either tighten up things or speed up calculations (at the price of reduced accuracy).

The present section describes all numerical settings for the aforementioned exchange-correlation treatments.

Specifically, we describe:

- All settings that relate to the setup of the (over-)complete *auxiliary basis* that expands the products of pairs of basis functions into a separate basis to represent the Coulomb operator
- All settings that rely to the frequency grid and analytic continuation from the imaginary to the real axis in GW related methods.

Even if you do not know what this is all about, you *should* know that the "auxiliary basis" is determined as an overcomplete basis, and superfluous basis functions are then reduced out by a threshold criterion, using singular value decomposition. This threshold is a value to be tested in case something unexpected happens.

There are three key references that provide the technical background for these sections:

1. Principle of how we calculate the two-electron Coulomb operator by so-called resolution of identity: Xinguo Ren *et al.* (2012), New J. Phys. 14, 053020, Ref. [?]. The approach summarized below and implemented in Keyword `RI_method V` is still the default for any non-periodic many-body perturbation calculations *beyond* DFT (i.e., for MP2, RPA, GW, etc.)
2. Localized resolution of identity (Keyword `RI_method LVL`), which is used by default for all Hartree-Fock and hybrid functional calculations, and which is the only option for any periodic calculations including the two-electron Coulomb operator: Arvid Ihrig *et al.* (2015), New J. Phys. 17, 093020, Ref. [?].
3. Linear-scaling and periodic implementation of periodic Hartree-Fock and hybrid functionals based on `RI_method LVL`, described in Sergey Levchenko *et al.* (2015), Comput. Phys. Commun. 192, 60-69, Ref. [?].

Mathematical background:

Any feature beyond standard DFT (e.g., HF, hybrid functional, MP2, GW, etc) requires the two-electron Coulomb repulsion integrals, and in FHI-aims an additional auxiliary basis set is introduced to deal with them. By utilizing the auxiliary basis functions the N_{basis}^4 many 4-center integrals are reduced to $N_{\text{basis}}^2 \cdot N_{\text{aux}}$ many 3-center integrals and N_{aux}^2 many 2-center integrals (where N_{basis} and N_{aux} are the numbers of regular basis functions and auxiliary basis functions, respectively). There are different ways to do so, and here we describe two versions of these, namely, the "V" and "SVS" [?] versions which have been implemented in this code. In the "V" version, the 4-center integrals are approximated by

$$(ij|i'j') \approx \sum_{\mu\nu}(ij|\mu)V_{\mu\nu}^{-1}(\nu|i'j'), \tag{3.35}$$

and in the "SVS" one,

$$(ij|i'j') \approx \sum_{\mu\nu}\sum_{\mu'\nu'}(ij\mu)S_{\mu\mu'}^{-1}V_{\mu'\nu'}S_{\nu'\nu}^{-1}(\nu i'j'), \tag{3.36}$$

where $i, j, i', j', \ldots$ denote the regular basis functions and $\mu, \nu, \ldots$ denote the auxiliary basis functions. Here $V_{\mu\nu}$ is the Coulomb repulsion integral between two auxiliary basis functions, and $S_{\mu\nu}$ is the corresponding overlap integral. $(ij\mu)$ and $(ij|\mu)$ are the overlap and Coulomb repulsion between the regular basis orbital product $\phi_i\phi_j$ and the auxiliary basis function P_μ respectively. Eq. (3.35) and (3.36) are often refered to as resolution of identity (Refs. [?, ?, ?, ?] and others). In practice satisfactory accuracy can be gained with an auxiliary basis size N_{aux} of 4-5 times of N_{basis}. In addition, we implement a modified localized version of RI-V known as "RI-LVL", described in Ref. [?] and also in Sec. 3.21.

How is the auxiliary basis functions constructed? In FHI-aims, it is built up as the "on-site" pair products of the regular basis functions (hence the auxiliary basis is also called the "product basis" in this context). These products are then orthonormalized at each atom using the gram-Schmidt method. These auxiliary basis functions are hence atom-centered numeric functions with a radial function times spherical harmonics $P_\mu(\mathbf{r}) = \frac{\xi(r)_{\text{at},n,l}}{r}Y_{lm}(\vartheta,\varphi)$. The radial part of the auxiliary basis function is formally linked to that of the regular basis functions by

$$\{\xi_{\text{at},n,l}(r)\} = \{u_{\text{at},n_1,l_1}(r)u_{\text{at},n_2,l_2}, |l_1 - l_2| \le l \le |l_1 + l_2|\}. \tag{3.37}$$

In Eq. (3.37) we make it clear that the set of auxiliary basis functions centered on certain atom originates from the pair products of regular basis functions centered on the the same atom. The angular momentum of the auxiliary basis and those of the two constituent regular basis satisfy the triangular true. The number of auxiliary basis function for a give l (enumerated by n) is controlled by the allowed pairs of regular basis functions, and the accuracy threshold in the Gram-Schmidt orthormalization. The process is described in Ref. [?] and in more detail with an illustrating figure in Ref. [?] (open access). The parameters that controls the construction of the auxiliary basis functions can be found below.

For self-energy calculatons a proper frequency grid is needed. For instance, in the GW approximation, the self-energy is given by

$$\Sigma(\mathbf{r},\mathbf{r}';i\omega) = \frac{i}{2\pi}\int d\omega' G(\mathbf{r},\mathbf{r}';i\omega+i\omega')W(\mathbf{r},\mathbf{r}';i\omega') \tag{3.38}$$

In FHI-aims, the self-energy is first calculated in the imaginary frequency grid, and then analytically continued to the real frequency grid. One popular way of performing the analytical continuation is to model the self-energy with a multi-pole expression [?], namely,

$$\Sigma(i\omega) \approx \sum_n \frac{A_n}{i\omega - B_n}, \tag{3.39}$$

where n is the number of poles, and A_n and B_n are complex numbers. Eq. (3.39) is used to fitted the calculated self-energy on imagninary axis using the non-linear least square fitting algorithm. Once the the the parameters A_n and B_n are obtained that give the best fitting, the self-energy on the real frequency axis can be obtained by

$$\Sigma(\omega) \approx \sum_n \frac{A_n}{\omega - B_n}. \tag{3.40}$$

In practice $n = 2$ (the so-called two-pole fitting) is often found to give good performance. A Pade approximation based variant with more poles is also implemented in FHI-aims.

Tags for general section of `control.in`:

Tag: `anacon_type` (control.in)

Usage: `anacon_type value`

Purpose: If set, specifies type of analytical continuation for the self-energy (we calculate the self-energy on the imaginary frequency axis, and hence need to continue it to the real axis)

- `value` = 0 : The normal two-pole fitting
- `value` = 1 : Pade approximation

Default: `value`=0

Note that the `anacon_type` only makes sense if `qpe_calc` or `sc_self_energy` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `auxil_basis` (control.in)

Usage: `auxil_basis type`

Purpose: Specifies the type of auxiliary basis used in the "beyond-DFT" calculation.

`type` is a string, which can be set either as `full` or `opt`. Default: `full`. Here is a brief explanation.

- `full` : The auxiliary basis is constructed as the "on-site" pair products of the regular basis functions. The allowed pair products are controlled by the parameters `max_n_prodbas` and `max_l_prodbas` (see later). These pair products are then orthonormalized using Gram-Schmidt procedure for each atom.
- `opt` : The auxiliary basis is obtained from an optimization procedure, and must be specified by hand in `control.in` – in the same spirit as the basis sets used in standard Gaussian-bases RI-MP2 calculations.

Tag: `default_prodbas_acc` (control.in)

Usage: `default_prodbas_acc threshold`

Purpose: Specifies the default for `prodbas_acc`

`threshold` is a real value, defining the onsite threshold for the auxiliary basis construction.
Default: 10^{-4} for `RI_method lvl`, depends on species (`species_z`) otherwise.

See `prodbas_acc` for more details. Default settings are:

- 10^{-2} for $Z \leq 10$ (light elements)
- 10^{-3} for $10 < Z \leq 18$
- 10^{-4} for $Z > 18$ (all heavier elements)

The old default (version 042811 and earlier) was simply 10^{-2} for all elements. For light elements, this setting produces accurate total energies and energy differences to the sub-meV level in all our tests. For heavier elements, significant inaccuracies could happen in atomic total energies. These inaccuracies would cancel out in energy differences; to guarantee total energy accuracy as well, we now set significantly tighter defaults for `prodbas_acc` in this range (alas, also more expensive, both in time and memory use).

Tag: `default_max_l_prodbas` (control.in)

Usage: `default_max_l_prodbas value`

Purpose: Specifies the default for `max_l_prodbas`

Default: 20 for `RI_method lvl`, 5 for `RI_method V` and nuclear charge $Z <= 54$, and 6 for `RI_method V` and $Z > 54$.

Tag: `default_max_n_prodbas` (control.in)

Usage: `default_max_n_prodbas value`

Purpose: Specifies the default for `max_n_prodbas`

Default: 20 for `RI_method lvl`, 6 otherwise.

Tag: `frequency_points` (control.in)

Usage: `frequency_points value`

Purpose: If set, specifies the number of (imaginary) frequency points for the self-energy calculation.

Default: `value`=80.

Note that the `frequency_points` only makes sense if `qpe_calc` or `sc_self_energy` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `hf_version` (control.in)

Usage: `hf_version version`

Purpose: Allows to switch between the standard density-matrix based setup of the Fock operator, and an orbital based exchange operator.

`version` is a number, either 0 (alias `density_matrix`) or 1 (aliases `eigencoefficients` and `overlap`). Default: 0

The exchange operator can be constructed either by summing over all states *first* to construct the density matrix (`version 0`), of by a straightforward setup of orbitals and computation of the exchange operator only then (`version 1`).

Both versions are pure matrix algebra. For small systems (few states), `version 1` is vastly more efficient, but towards large systems (many states) an efficiency crossover clearly favors the density matrx based update.

By default, FHI-aims relies on the density-matrix based update always, because this is more memory efficient, but for small systems (atoms) with lots of basis functions, this choice should be reconsidered. Please note that both versions should work seamlessly with Pulay mixing.

Tag: `maximum_frequency` (control.in)

Usage: `maximum_frequency value`

Purpose: If set, specifies the maximal (imaginary) frequency value for the self-energy calculation. The unit for `value` here is Hartree.

Note that the `maximum_frequency` only makes sense if `qpe_calc` or `sc_self_energy` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `maximum_time` (control.in)

Usage: `maximum_time value`

Purpose: If set, specifies the maximal (imaginary) time value for the self-consistent self-energy calculation. The unit for `value` here is Hartree^{-1}.

Note that the `maximum_time` only makes sense if `sc_self_energy` is set, i.e., the post-processing-type self-consistent self-energy calculation is required.

Tag: `n_poles` (control.in)

Usage: `n_poles value`

Purpose: If set, specifies the number of poles (i.e. the number of functions of the form $f_i(\omega) = 1/(b_i + i\omega)$) adopted in the pole-based computation of the Fourier transform in self-consistent GW-type calculations.

Note that the `n_poles` only makes sense if `sc_self_energy` is set, i.e., the post-processing-type self-consistent self-energy calculation is required.

Tag: `pole_max` (control.in)

Usage: pole_max value

Purpose: If set, specifies the position in the (imaginary) frequency axis of the largest poles (i.e. the largest b_i coefficient in $f_i(\omega) = 1/(b_i + i\omega)$) used in computation of the Fourier transform in self-consistent GW-type calculations. The unit for value here is Hartree.

Note that the pole_max only makes sense if sc_self_energy is set, i.e., the post-processing-type self-consistent self-energy calculation is required.

Tag: pole_min (control.in)

Usage: pole_min value

Purpose: If set, specifies the position in the (imaginary) frequency axis of the smallest poles (i.e. the smallest b_i coefficient in $f_i(\omega) = 1/(b_i + i\omega)$) used in computation of the Fourier transform in self-consistent GW-type calculations. The unit for value here is Hartree.

Note that the pole_min only makes sense if sc_self_energy is set, i.e., the post-processing-type self-consistent self-energy calculation is required.

Tag: n_anacon_par (control.in)

Usage: n_anacon_par value

Purpose: If set, specifies the number of (complex) parameters used for analytical continuation.

Note that the n_anacon_par only makes sense if qpe_calc or sc_self_energy is set, i.e., the post-processing-type self-energy calculation is required.

Tag: prodbas_nb (control.in)

Usage: prodbas_nb nb

Purpose: For very large scale beyond-GGA calculations, the distribution of auxiliary basis functions among the CPUs becomes problematic because each CPU only gets very few. The default Scalapack distribution is more taylored to efficient calculations and distributes these functions in chunks of finite size. In massively parallel runs, often each CPU gets either one or two of these chunks, leading to bad memory distribution. This can be circumvented, possibly sacrificing some performance, by setting the chunk size nb to "1".

nb is the chunk size of auxiliary basis functions.
Default: $\min(16, \lfloor N_{\mathrm{aux}}/N_{\mathrm{proc}} \rfloor)$.

Tag: prodbas_threshold (control.in)

Usage: `prodbas_threshold threshold`

Purpose: Prevent the possible ill-conditioning of the auxiliary basis, similar to `basis_threshold` for the regular basis.

`threshold` is a small positive real number for the eigenvalue of the Coulomb matrix for the auxiliary basis. `Default:` 10^{-5}.

The the auxiliary basis functions centered on different atoms are nonorthogonal, and the possible linear dependence (with certain accuracy) between them and the resultant behavior has to be carefully eliminated. This is achieved by setting the cutoff threshold for the auxiliary basis `prodbas_threshold`. From many test calculations, it is found that the reliable value for `threshold` here are between 10^{-5} to 10^{-3}. Within this window, the total energy may still have some noticable change, but the energy difference is usually negligible. It is suggested that the user should play with this value if he/she is not sure about his/her result.

Tag: `RI_method` (control.in)

Usage: `RI_method type`

Purpose: Specifies the version of the resolution of identity used in the beyond-DFT calculations. Here `type` is a string, with possible options listed below.

Default: Non-periodic: `type=lvl` for Hartree-Fock and hybrid functional calculations, which can be used for geometry relaxations. `type=v` for MP2, RPA and GW etc. calculations that require unoccupied states. This gives better accuracy for a given auxiliary basis, but is usually more expensive and provides no geometry relaxation at present. Periodic: `type=LVL_fast`, which provides the necessary reduction to what is essentially an $O(N)$ framework.

Different options for the `type` option include:

- `svs` (for the "SVS" version, i.e., Eq. (3.36))
- `v` (for the "V" version, i.e. Eq. (3.35))
- `lvl` (for the "LVL" version)
- `LVL`, `LVL_fast`, or `lvl_fast` are all synonymous with option `lvl`
- `lvl_full` implements a non-linear scaling version of the "LVL" approach for total energy evaluations only
- `lvl_2nd` implements the so-called "robust Dunlap correction" for total energies only, which essentially follows up an s.c.f. calculation with an additional RI-V-like step. In our experience, similar results are better accomplished without this correction (see also Ref. [?]).

Rules of thumb:

- `type=V` is the preferred method for non-periodic calculations. For formal reasons, this is clearly the most accurate version. However, neither a periodic version nor gradients (forces and relaxations) are implemented at present.
- `type=LVl_fast` is the preferred version for periodic calculations, as well as for non-periodic Hartree-Fock and hybrid functional calculations with forces and relaxation. It scales as $O(N)$ and greatly limits the memory use compared to RI-V. `RI_method LVL_fast` localizes the expansion of the Coulomb potential of basis function products to two centers, as described in Sec. 3.21 and in Refs. [?, ?]. It also relies extensively on screening near-zero elements of the density matrix and of the Coulomb operator. On the other hand, the localized version has slightly larger errors than RI-V for hybrid functionals, and *significantly* larger errors for correlated methods like MP2 or RPA. These can be remedied by adding a few extra functions to the construction of the auxiliary basis set using the `for_aux` keyword, as described in detail in Ref. [?].
- At present, *do not use RI-LVL for MP2 and RPA* unless you know what you are doing. It is possible to repair their accuracy in RI-LVL, by increasing the size of the auxiliary basis set using the `for_aux` keyword, as described in detail in Ref. [?], but please see the figures and benchmarks in that reference before trying.
- `type=LVL_full` is a slower, non-screened version of LVL for non-periodic systems. Very useful as a reference to make sure.
- `type=SVS` is here only for testing purposes. This is the naive, purely overlap based version of RI and should not be used in production.

There is an additional option for non-periodic systems, `lvl_2nd`, which adds a correction term to the nonlocal exchange energy to make it "robust" in the Dunlap sense [?], that is, the error in the energy is quadratic in the error in the product expansion.

Tag: **`sbtgrid_lnrange`** (control.in)

Usage: `sbtgrid_lnrange lnrange`

Purpose: for `use_logsbt` , set the range of the logarithmic grid (in logarithmic units). Default is 45, which corresponds to nearly twenty orders of magnitude. Please note that the range should be larger than intuitively guessed because the range is the same both in real and reciprocal space (for algorithmic reasons) and the tails in reciprocal space have to be captured.

Tag: **`sbtgrid_lnr0`** (control.in)

Usage: `sbtgrid_lnr0 lnr0`

Purpose: for `use_logsbt` ,set the onset of the logarithmic grid in real space. The default is -38.

Tag: **`sbtgrid_lnk0`** (control.in)

Usage: `sbtgrid_lnk0 lnk0`

Purpose: for `use_logsbt`, set the onset of the logarithmic grid in Fourier space. The default is -25.

Tag: `sbtgrid_N` (control.in)

Usage: `sbtgrid_N N`

Purpose: for `use_logsbt`, set the number of logarithmic grid points both in real and Fourier space. The default is 4096.

For the accuracy, the density of points $N/$`lnrange` is relevant. Additionally, one has to make sure that the tails in real and Fourier space are properly included.

Tag: `state_lower_limit` (control.in)

Usage: `state_lower_limit value`

Purpose: If set, specifies the lowest single-particle eigenstate to be included for the quasiparticle calculation.

Note that the `state_lower_limit` only makes sense if `qpe_calc` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `state_upper_limit` (control.in)

Usage: `state_upper_limit value`

Purpose: If set, specifies the highest single-particle eigenstate to be included for the quasiparticle calculation.

Note that the `state_upper_limit` only makes sense if `qpe_calc` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `time_points` (control.in)

Usage: `time_points value`

Purpose: If set, specifies the number (imaginary) time points for the self-energy calculation.

Default: `value`=80.

Note that the `frequency_points` only makes sense if `qpe_calc` or `sc_self_energy` is set, i.e., the post-processing-type self-energy calculation is required.

Tag: `use_logsbt` (control.in)

Usage: use_logsbt bool

Purpose: If set, the two-center integrals are calculated by one-dimensional integrations in Fourier-space. In the case of RI_method LVL, also the three-center integrals are computed by this method.

Default: .true.

use_logsbt .true. is faster and more accurate than the alternative.

The algorithm for the overlap and Coulomb integrals is described by Talman in [?]. It uses an efficient spherical Bessel transform on a logarithmic radial grid [?, ?, ?] to obtain the Fourier transform of the auxiliary basis functions and calculates the integrals in Fourier space.

Tag: use_ovlp_swap (control.in)

Usage: use_ovlp_swap

Purpose: if set, the atomic orbital (AO) based 3-index overlap matrix ("ovlp_3fn" in the source code) is written to the disk before transforming to the molecular orbital (MO) based 3-index overlap matrix ("O_2bs1HF" for HF calculations and "ovlp_3KS" for GW calculations in the source code). This avoids the double allocation of both the AO-based 3-index integral matrix and MO-based 3-index integral matrix at the same time and thus reduces the memory cost by about a factor of two.

Subtags for *species* tag in `control.in`:

---

species sub-tag: aux_gaussian (control.in)

Usage: aux_gaussian L N [alpha]
[alpha_1 coeff_1]
[alpha_2 coeff_2]
[...]
[alpha_N coeff_N]

Purpose: For auxil_basis opt, adds a Gaussian basis function to the auxiliary basis set for the Coulomb operator.

L is an integer number, specifying the angular momentum
N is an integer number, specifying how many primitive Gaussians comprise the present radial function
alpha : *If* N=1, this is the exponent defining a primitive Gaussian function [in bohr^{-2}].
alpha_i coeff_i : *If* N>1, $i = 1, \ldots, N$ additional lines specify exponents α_i and expansion coefficients g_i for a non-primitive linear combination of Gaussians.

See the description of gaussian basis functions; Gaussian basis functions in the *auxiliary* basis use essentially the same infrastructure.

---

species sub-tag: for_aux (control.in)

Usage: for_aux basis options

Purpose: Add a extra basis function to constructor of auxiliary basis function.

Basis is a either hydro or ionic basis function keyword and options are options for that basis function.

Adds extra basis function ONLY to the construction of the auxiliary basis set used to expand the Coulomb operator (resolution of identity, see keyword RI_method). In particular, the accuracy of RI_method LVL can be increased by adding extra high-angular momentum radial functions to the auxiliary basis set. The improvement becomes less and less relevant as the orbital basis set itself increases. For instance, there may be a noticeable change for *tier 1*, but much less or not at all for *tier 2*. Currently supports only hydro and ionic basis functions.

For a detailed description with benchmarks, please see Ref. [?].

Here is an example for the use of the for_aux , which was obtained by altering the "light" settings of the C atom:

```
[...]
#  "First tier" - improvements: -1214.57 meV to -155.61 meV
     hydro 2 p 1.7
     hydro 3 d 6
     hydro 2 s 4.9
```

```
#  "Second tier" - improvements: -67.75 meV to -5.23 meV
  for_aux     hydro 4 f 9.8
#     hydro 3 p 5.2
#     hydro 3 s 4.3
  for_aux     hydro 5 g 14.4
#     hydro 3 d 6.2
[...]
```

Adding those two high-angular momentum functions does improve the quality of the RI_method LVL noticeably, at the price of more CPU time. On the other hand ... "light" settings are specifically chosen because not everything is completely converged. In fact, the orbital basis set error itself may be larger than the accuracy gained by amending the two-electron Coulomb operator expansion.

---

species sub-tag: prodbas_acc (control.in)

Usage: prodbas_acc threshold

Purpose: Technical cutoff criterion for on-site orthonormalization of auxiliary radial functions. Here threshold is a small positive real value. Default: See below.

To construct the set of auxiliary basis functions, the radial functions for a single species are "on-site" Gram-Schmidt orthonormalized. If the norm of the function after orthonormalization is smaller than threshold, that function is omitted.

The present default values are:

- 10^{-2} for $Z \leq 10$ (light elements)
- 10^{-3} for $10 < Z \leq 18$
- 10^{-4} for $Z > 18$ (all heavier elements)

The old default (version 042811 and earlier) was simply 10^{-2} for all elements. For light elements, this setting produces accurate total energies and energy differences to the sub-meV level in all our tests. For heavier elements, significant inaccuracies could happen in atomic total energies. These inaccuracies would cancel out in energy differences; to guarantee total energy accuracy as well, we now set significantly tighter defaults for prodbas_acc in this range (alas, also more expensive, both in time and memory use).

For simplicity, it is also possible to use default_prodbas_acc to set a global value of prodbas_acc across all elements.

Note that the prodbas_acc should not be confused with the prodbas_threshold. The former is used when constructing the auxiliary basis functions for each species, whereas the latter is used to deal with the ill-conditioning behavior of the Coulomb repulsion and/or the overlap matrix between the set of auxiliary basis functions for the whole systems.

---

species sub-tag: max_n_prodbas (control.in)

Usage: `max_n_prodbas value`

Purpose: Specifies the maximal principal quantum number for the *regular* basis function to be included in the *auxiliary (product)* basis construction.

`value` is a positive integer number here.

Note that `max_n_prodbas` has an effect only when `auxil_basis` is setted to `full`.

`species` sub-tag: `max_l_prodbas` (control.in)

Usage: `max_l_prodbas value`

Purpose: Specifies the maximal angular quantum number for the *auxiliary (product)* basis function. Any possible auxiliary basis with an angular momentum higher than `max_l_prodbas` is excluded.

`value` is a positive integer number here.

Note that `max_l_prodbas` controls the *auxiliary* basis whereas `max_n_prodbas` controls only directly the *regular* basis. `max_l_prodbas` has an effect regardless whether `auxil_basis` is setted to `full` or `opt`.

3.21 Hartree-Fock and hybrid functionals, including periodic systems

Periodic versions of the Hartree-Fock method and of and hybrid density functionals are implemented in FHI-aims. Generally, our experiences are very good. The implementation is stable and seems to scale well towards large systems. However, we still ask you to exercise some care. If you encounter any unexpected difficulties, consult the developers.

Periodic versions of MP2, RPA and GW are not yet part of the main FHI-aims distribution, as they are in various stages of development. Please feel free to ask at aimsclub regarding these methods for periodic systems. They are very important to us, and we will be happy to share them as they become ready and more usable.

Complete descriptions of the material described below, as well as extensive benchmarks, are summarized in Ref. [?], as well as Ref. [?] (for the non-periodic implementation of the "LVL" approach). Forces and the stress tensor are also implemented, with a clear description of the stress tensor given in Ref. [?].

There are two specific issues from a usability point of view which should be considered:

- The `RI_method` "LVL" described below is slightly less accurate (for formal reasons) than the standard for non-periodic systems, RI-V. Please bear this in mind. For standard solids and hybrid functionals, the effects seem very small, but for larger molecules and small basis sets, the effects become measurable.
- For band structure output, the `exx_band_structure_version` keyword allows to toggle between a slow but safe, reciprocal-space based version, and a faster real-space version that works only when a relatively dense k-space grid has been used during the regular s.c.f. cycle. The underlying reason is that the real-space Born-von Karman cell of the regular s.c.f. cycle may become too small to accommodate some k-vectors that are not exact reciprocal lattice vectors of the Born-von Karman cell.

In periodic Hartree-Fock (and hybrid functional) implementations, the key quantity that needs to be evaluated is the exact-exchange matrix,

$$K_{ij}(\mathbf{k}) = \sum_{kl,\mathbf{q}} D_{kl}(\mathbf{q}) \iint d\mathbf{r} d\mathbf{r}' \frac{\phi^*_{i\mathbf{k}}(\mathbf{r})\phi_{k\mathbf{q}}(\mathbf{r})\phi^*_{l\mathbf{q}}(\mathbf{r}')\phi_{j\mathbf{k}}(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} \tag{3.41}$$

where $\mathbf{k}, \mathbf{q}$ are the Bloch vectors, $\phi_{i\mathbf{k}}(\mathbf{r})$ is the Bloch summation of the i-th atomic orbital $\phi_i(\mathbf{r}-\mathbf{R})$ living in the unit cell $\mathbf{R}$, and $D_{kl}(\mathbf{q})$ is the density matrix. $K_{ij}(\mathbf{k})$ can be obtained from its Fourier transform,

$$K_{ij}(\mathbf{k}) = \sum_{\mathbf{R}} e^{i\mathbf{k}\mathbf{R}} X_{ij}(\mathbf{R}) \,, \tag{3.42}$$

where

$$X_{ij}(\mathbf{R}) = \sum_{kl} \sum_{\mathbf{R}'} D_{kl}(\mathbf{R}') \sum_{\mathbf{R}''} \iint d\mathbf{r} d\mathbf{r}' \frac{\phi_i(\mathbf{r})\phi_k(\mathbf{r}+\mathbf{R}'')\phi_j(\mathbf{r}'+\mathbf{R})\phi_l(\mathbf{r}'+\mathbf{R}'+\mathbf{R}'')}{|\mathbf{r}-\mathbf{r}'|} \tag{3.43}$$

In FHI-aims, periodic Hartree-Fock and hybrid density functionals are implemented in two different ways. One implementation is based on the "k-space" formulation, where one computes $K_{ij}(\mathbf{k})$ directly from Eq. (3.41). An alternative, and more efficient implementation is based on the"real-space" formulation, where one first computes $X_{ij}(\mathbf{R})$ from Eq. (3.43), and then Fourier transform it to $K_{ij}(\mathbf{k})$. The "real-space" implementation is used in the code by default, and the "k-space" implementation is only used for crossing-check purposes.

Both implementations are based on a localized resolution-of-identity approximation, which we termed as "RI-LVL", in analogy to "RI-SVS" and "RI-V" introduced in Sec. 3.20. Under "RI-LVL", the products of two normal basis functions (i, j) centering at atoms A_i and A_j are expanded only in terms of auxiliary functions centering on these two atoms. Possible contributions of auxiliary functions from a third center are excluded in this approximation, in contrast to "RI-V". Specifically, one has

$$\phi_i(\mathbf{r}-A_i)\phi_j(\mathbf{r}-A_j) = \sum_{\mu} C^{\mu(A_i)}_{i(A_i),j(A_j)} P_\mu(\mathbf{r}-A_i) + \sum_{\nu} C^{\nu(A_j)}_{i(A_i),j(A_j)} P_\nu(\mathbf{r}-A_j)\,, \quad (3.44)$$

where μ and ν enumerate the auxiliary basis functions centering on atom A_i and A_j respectively. This approximation has been extensively benchmarked with respect to the more accurate "RI-V" approximation for finite systems, and with respect to other independent implementations for molecular systems. The achieved accuracy is remarkable and should be sufficiently good for production calculations.

Periodic Hartree-Fock and hybrid-functional calculations can be run in the same manner as the periodic LDA and GGA cases, by setting the keyword `xc` to `hf` or desired hybrid functionals, and setting the `k_grid` mesh to appropriate values. As mentioned above, by default the "real-space" periodic Hartree-Fock implementation will be invoked. There are two thresholding parameters (detailed below) which control the balance between the computational load and accuracy in the calculation. One may also switch to the "k-space" implementation of periodic Hartree-Fock and hybrid functionals for testing or comparison purposes by setting the keyword `use_hf_kspace` to be true (see below). The thresholding parameters do not apply to the "k-space" implementation, however.

Tags for general section of `control.in`:

Tag: `coulomb_threshold` (control.in)

Usage: `periodic_hf` `coulomb_threshold` `value`

Purpose: This sets a threshold `value` for a key ingredient in the construction of the exact-exchange matrix – the Coulomb matrix. The Coulomb matrix elements below the specified threshold `value` are discarded in the calculation. Suggested values are between 10^{-6} and 0. The default value is 10^{-10}.

Tag: `exx_band_structure_version` (control.in)

Usage: `exx_band_structure_version value`

Purpose: A periodic band structure calculation can be performed either using a real-space version (`value`=1) or a reciprocal-space version (`value`=2). No default – user must decide.

`value` is an integer, either 1 or 2.

If `output band` is requested for a periodic Hartree-Fock or hybrid functional calculation, adhere to the following rules:

- Do not use excessively many k points in each band segment, for instance no more than 20.
- The real-space band structure version `value`=1 is fast and accurate IF a reasonably dense `k_grid` has been used during the s.c.f. calculation. We recommend to test to be sure.
- If the plotted band structure from the real-space version `value`=1 has obvious numerical problems, please switch to either a denser `k_grid` during s.c.f., or to `exx_band_structure_version` 2. The latter will always work, as the critical part of the work is handled in reciprocal space. As a consequence, though, sparsity in real space can no longer be exploited, and the band structure calculation becomes much slower than the real-space version.
- In case of doubt, the band structure ONLY at k-points used during the s.c.f. cycle itself can also be printed along certain directions by using the `output band_during_scf` keyword, which ensures that only the information that went into the s.c.f. cycle is actually used. This is mainly useful for debugging purposes but since the functionality in question is new, it may be helpful.

We apologize that this decision process is still a bit rough around the edges and leaves an essential decision up to the user. However, consider this: The functionality in question is new, and the above procedure provides a simple way to make band structure output work, and work safely. It is now generally not a problem to produce band structures with hybrid functionals in FHI-aims, if one follows the above hints.

Tag: `screening_threshold` (control.in)

Usage: `periodic_hf screening_threshold value`

Purpose: This sets a screening parameter in a periodic Hartree-Fock (or hybrid functional) calculation. The real-space exact-exchange matrix elements below the specified threshold `value` are neglected in the calculation. Suggested values are between 10^{-6} and 0. Smaller values mean better accuracy but heavier computational loads. The default value is 10^{-7}.

Tag: `use_hf_kspace` (control.in)

Usage: `use_hf_kspace flag`

Purpose: The "k-space" periodic HF implementation can be invoked by setting `flag` to be `.true.`

3.22 TDDFT - linear response

These routines are not completed yet. For now it is only possible do use f_{xc} kernels from LDA. The development goes on and more funcionality will be added. When publishing results obtained from this routine, please do cite me, Jan Kloppenburg as the author, as well as of course the usual people in the aims references. When problems, questions or suggestions arise, feel free to contact me at `kloppenburg@fhi-berlin.mpg.de`. Only use there routines if you know what you are doing!

Theory

The goal is to calculate excitation energies ω_I and corresponding oscillator strengths f_I from

$$\mathbf{\Omega}\mathbf{F}_I = \omega_I^2 \mathbf{F}_I \ . \tag{3.45}$$

Linear response theory (see [?]) is the basis for this calculation. We construct

$$\mathbf{\Omega}_{ias,jbt} = \delta_{i,j}\delta_{a,b}\delta_{s,t}(\epsilon_a - \epsilon_i)^2 + 2\sqrt{\epsilon_{as} - \epsilon_{is}}\mathbf{K}_{ias,jbt}\sqrt{\epsilon_{bt} - \epsilon_{jt}} \tag{3.46}$$

with the coupling kernel

$$\mathbf{K}_{ias,jbt} = \int\int \varphi_i^*(\mathbf{r})\varphi_a(\mathbf{r})\left[\frac{1}{|\mathbf{r}-\mathbf{r}'|} + f_{xc}(\mathbf{r},\mathbf{r}')\right]\varphi_j^*(\mathbf{r}')\varphi_b(\mathbf{r}')d\mathbf{r}d\mathbf{r}' \ . \tag{3.47}$$

In this notation I refer with the indices i, j to occupied and with a, b to virtual orbitals, while s and t denote the spin. The input energies ϵ are obtained from either ground state *Hartree-Fock* or DFT calculations. Going by this rule we contruct the matrix $\mathbf{\Omega}$ which then is solved for eigenvalues and eigenvectors. The excitation energies ω_I then follow from From the eigenvectors $\mathbf{F}_I$ the oscillator strengths f_I can be obtained from

$$f_I = \frac{2}{3}\omega_I\left[\left|\left\langle\Psi_0\middle|\hat{\mathbf{X}}\middle|\Psi_I\right\rangle\right|^2 + \left|\left\langle\Psi_0\middle|\hat{\mathbf{Y}}\middle|\Psi_I\right\rangle\right|^2 + \left|\left\langle\Psi_0\middle|\hat{\mathbf{Z}}\middle|\Psi_I\right\rangle\right|^2\right], \tag{3.48}$$

with the $\hat{\mathbf{X}}$ being the spatial operator for the X direction and the others respectively with Ψ_0 being the all electron ground state wave function and Ψ_I being the all electron excited state wave function for the state I with excitation energy ω_I.

For the TDHF calculation mode, the kernel $\mathbf{K}_{ias,jbt}$ is modified to become

$$\mathbf{K}_{ias,jbt} = (ias|jbt) + \delta_{s,t}(ij|ab) \tag{3.49}$$

that has only the bare *Coulomb* part $(ia|jb)$ and the exact exchange part $(ij|ab)$ from *Hartree-Fock* theory. This *Hartree-Fock* Kernel creates a non-Hermitian matrix $\mathbf{\Omega}$. Please not that this calculation mode is as yet only available for single processor runs due to the lack of a non-Hermitian parallel eigenvalue solver.

Available Kernels and *libxc*

As of now, there is only the `pw-lda` f_{xc} kernel available for the TDDFT calculation in aims. If the user wants to make use of additional f_{xc} kernels he is requested to install *libxc*. *libxc* is a library of exchange-correlation functionals for density-functional theory (see [?]) available free under the LGPL license v3 from the internet. [2] Additionally, for the full TDDFT calculation it is possible to choose functionals at will that are available from this *libxc*. It is not required to have the DFT level calculation that generates the input energies ϵ for 3.47 using the same XC functional as the TDDFT calculation. You should really know what you are doing when you choose to experiment with different functionals and always keep in mind that the results might be unpredictable and not necessarily have any physical meaning at all. Nevertheless it might come in handy to be able to mix different hybrid functionals or do a DFT ground state calculation with a functional that does not provide and f_{xc} and still be able to do a TDDFT calculation on top of that when switching to functionals that do provide an f_{xc}.

Tags for general section of `control.in`

Tag: `neutral_excitation` (control.in)

Usage: `neutral_excitation type`
Purpose: Triggers the calculation of neutral excitations.
`type`: String that defines the type of calculation to be performed.

- tddft: Full TDDFT calculation
- tdhf: Full TDHF calculation (Kernel from 3.49, serial CPU only)
- rpa: random phase approximation only (set $f_{xc} = 0$ in 3.47)

With the keyword `neutral_excitation` the user can specify the calculation mode for the linear response theory.
Also the keyword `empty_states` should be set to 1000 to make sure the code generates all possible empty states provided from the basis set. This number will also be reduced automatically by the code to the maximum number that can be generated from the basis set.

Tag: `tddft_kernel` (control.in)

[2] http://www.tddft.org/programs/octopus/wiki/index.php/Libxc

Usage: tddft_kernel string
Purpose: Specify the origin of the TDDFT kernel for 3.47
string: pw-lda/pz-lda or libxc
Both pw-lda or pz-lda are built-in options in FHI-aims. They are equivalent to the keywords defined in xc .
When using libxc, one must specify the desired kernels through keywords tddft_x and tddft_c . Note that libxc is only possible when the user has compiled aims with libxc binding.

Tag: `tddft_x` (control.in)

Usage: tddft_x string
Purpose: Set the desired exchange kernel to use from libxc. The definition is from libxc's manual and can be found at the libxc[a] website.
string: The name of the selected exchange functional, i.e. XC_LDA_X

[a] http://www.tddft.org/programs/octopus/wiki/index.php/Libxc:manual

---

Tag: `tddft_c` (control.in)

Usage: tddft_c string
Purpose: Set the desired correlation kernel to use from libxc. The definition is from libxc's manual and can be found at the libxc website as well.
string: The name of the selected correlation functional, i.e. XC_LDA_C_PW

Tag: `excited_mode` (control.in)

Usage: excited_mode string
Purpose: Select which excitations will be calculated.
string: one of {singlet|triplet|both}. To calculate both singlets and triplets is set as the default when this keyword is omitted.

Tag: `excited_states` (control.in)

Usage: excited_states n
Purpose: Specify the number of excited state energies and oscillator strengths to be printed.
n: Integer number $n \in \mathbb{N},\ n \geq 0$

With the keyword excited_states the user can specify the number of excited states that will be printed in the output. The default for this is 50 if there are that many. Normal production runs will generally have many more (depending on the basis set and the number of electrons involved) that can easily reach beyond 10000. So to avoid a really huge output from this routine this defaut is set rather low. Feel free to choose any number of your liking, if it should be too large it will automatically be defaulting to all excited states. At the end of a calculation the file TDDFT_LR_Spectrum_(singlet/triplet).dat will be written into the directory the FHI-aims program was run in. It contains all

computed excitation energies and the corresponding oscillator strengths.

Tag: `casida_reduce_matrix` (control.in)

Usage: `casida_reduce_matrix boolean`
Purpose: Set to .true. if you want to reduce the energy range for the Kohn-Sham eigenvalues to be included in the computation.
`boolean`: .true. or .false.

Tag: `casida_reduce_occ` (control.in)

Usage: `casida_reduce_occ x`
Purpose: Specify the energy in Hartree below which the occupied states are cut off.
`x`: Cutoff energy in Hartree

Tag: `casida_reduce_unocc` (control.in)

Usage: `casida_reduce_unocc x`
Purpose: Specify the energy in Hartree above which the virtual states are cut off.
`x`: Cutoff energy in Hartree

3.23 MPE Implicit Solvent Model

This is an experimental feature which is still in development. Do not rely on properties calculated by this method!

The simulation of a solvent in a quantum mechanical calculation can, in principle, be done in two ways. One way is to include explicit solvent molecules in the calculation. This straightforward approach usually requires molecular dynamics (MD) simulations in order to yield thermodynamically meaningful observables as e.g. solvation energies.

The second way is to average the effect of the solvent and treat it as a continuum which responds to the electrostatic potential created by the solute, i.e. the entity that is to be solvated. There are several flavors to this comparatively inexpensive approximation, e.g. the polarizable continuum model (PCM) by Mennucci and coworkers [?] or the conductor like screening model (COSMO) by Klamt and Schüürmann [?] to name two of the most popular ones.

The method presented here called multipole expansion (MPE) implicit solvent model is somewhat similar to the dielectric PCM model. It uses the multipole expansion of the electronic density to calculate the so-called reaction field, i.e. the electrostatic response of the polarizable continuum.

This functionality is not yet available for periodic systems.

Tags for general section of `control.in`:

Tag: `solvent` (control.in)

Usage: `solvent method cavity periodicity d`

Purpose: Specifies the desired implicit solvent model including the method which is used to create the cavity.

`method` is a string which specifies the implicit solvent method. Currently, only "mpe" is supported.

`cavity` is a string which specifies the method used to create the cavity. Currently, only the isodensity surface cavity ("isodens") based on the evaluation of the density on a Cartesian grid is supported.

`periodicity` is an integer which specifies the periodicity of the cavity. Currently, only cluster calculations (periodicity 0) are supported.

`d` is a real number (unit unknown). This keyword defines the distance of grid points in the Cartesian grid. Usually, a value of approximately 0.08 is enough to create a sufficiently large amount of points on the cavity surface (at least 1500).

Tag: `cavity_surface_position` (control.in)

Usage: `cavity_surface_position isovalue`

Purpose: Specifies the isodensity value for the creation of the isodensity cavity.

`isovalue` is a real number (unit unknown) and specifies the isodensity value. For small organic molecules a value of approximately 0.006 usually yields reasonable results. Be aware that this number can have a drastic influence on the calculated results. Thus, it does not have a default value but has to be provided by the user when the keyword `solvent` is present.

Tag: `solvent_permittivity` (control.in)

Usage: `solvent_permittivity epsilon`

Purpose: Specifies the macroscopic dielectric constant of the solvent.

`isovalue` is a real number equal to the macroscopic dielectric constant of the solvent. This keyword does not have a default value and has to be provided by the user when the keyword `solvent` is present.

Tag: `cavity_rebuild` (control.in)

Usage: `cavity_rebuild n`

Purpose: Specifies after how many scf steps the cavity is rebuilt.

`n` is an integer. Default: 10000

3.24 Linear macroscopic dielectric function and Kubo-Greenwood transport

The linear macroscopic dielectric tensor $\epsilon_{ij}(\omega)$, the ratio between the average of the total potential in one unit cell and the external field, within the RPA is calculated. For derivation from the microscopic dielectric function $\epsilon(\vec{r},t;\vec{r'},t')$ and references see [?] (Chapter 1, 2 and appendix). From the complex frequency dependent dieletric function all other optical constants can be determined, e.g. optical conductivity, Loss function, Reflectivity, etc. Also see Appendix K in Ashcroft/Mermin: solid state physics.
The frequency dependent real and imaginary part of the inter- (eq. 3.51) and intra- (eq. 3.50) band contribution to the linear dielectric tensor is calculated as post-processing after convergence of the SCF-cycle from (atomic units):

$$\epsilon_{ij}(\omega) = \delta_{i,j} - \frac{4\pi}{V_{cell}\omega^2}\sum_{n,\mathbf{k}}\left(-\frac{\partial f_0}{\partial \epsilon}\right)_{\epsilon_{n,\mathbf{k}}} p_{i;n,n,\mathbf{k}}p_{j;n,n,\mathbf{k}} \tag{3.50}$$

$$+\frac{4\pi}{V_{cell}}\sum_{\mathbf{k}}\sum_{c,v}\frac{p_{i;c,v,\mathbf{k}}p_{j;c,v,\mathbf{k}}}{\left(\epsilon_{c,\mathbf{k}}-\epsilon_{v,\mathbf{k}}-\omega\right)\left(\epsilon_{c,\mathbf{k}}-\epsilon_{v,\mathbf{k}}\right)^2} \tag{3.51}$$

f is the Fermi-function and V_{cell} the unit cell volume. The intra band part is singular at $\omega = 0$. Here the plasma frequency $\omega_{pl;ij}$ is calulated:

$$\omega^2_{pl;ij} = \frac{1}{\pi}\sum_n \int_{\vec{k}} p_{i;n,n,\vec{k}}p_{j;n,n,\vec{k}}\delta(\epsilon_{n,\vec{k}}-\epsilon_F) \tag{3.52}$$

By adopting a Drude-like shape for the intra-band contributions a lifetime broadening Γ is introduced and $\epsilon^{inter}_{ij}(\omega)$ becomes:

$$\mathrm{Im}(\epsilon^{intra}_{ij}(\omega)) = \frac{\Gamma\omega^2_{pl;ij}}{\omega(\omega^2+\Gamma^2)} \tag{3.53}$$

$$\mathrm{Re}(\epsilon^{intra}_{ij}(\omega)) = 1 - \frac{\omega^2_{pl;ij}}{\omega^2+\Gamma^2} \tag{3.54}$$

In the case of a spin unpolarized calulation eq. 3.51 and eq. 3.50 have to be multiplied by 2 to yield the correct occupation.
The following quantities are needed/calculated:

- $\epsilon_{n,\vec{k}}$ - the eigenvalue of the Kohn-Sham eigenstate $(n,\vec{k})$.
- $\delta(\epsilon_{n',\vec{k}}-\epsilon_{n,\vec{k}}-\omega) = \frac{1}{\sqrt{2\pi}width}exp\left(-\frac{1}{2}\frac{(\epsilon_{n',\vec{k}}-\epsilon_{n,\vec{k}}-\omega)^2}{\Gamma^2}\right)$ - Gauss function with width Γ=$width$ for calulating the plasma frequency. In eq 3.51 ω is replaced by $\omega+\mathrm{i}\Gamma$, introducing Lorentzian broadenig.
- $p_{j;n',n,\vec{k}} = \left\langle\psi_{n'\vec{k}}|\nabla_j|\psi_{n\vec{k}}\right\rangle$ - the momentum matrix elements calculated form the real space basis functions in k-space and KS-eigenstate basis

$$\left\langle\psi_{n'\vec{k}}|\nabla_j|\psi_{n\vec{k}}\right\rangle = \sum_{ij} c^{*\vec{k}}_{in'}c^{\vec{k}}_{jn}\sum_{\vec{N},\vec{M}}\mathrm{e}^{\mathrm{i}\vec{k}[\vec{T}(\vec{N})-\vec{T}(\vec{M})]}\left\langle\phi_{i\vec{M}}|\nabla_j|\phi_{j\vec{N}}\right\rangle \tag{3.55}$$

$$\langle \phi_{i\vec{M}} | \nabla | \phi_{i\vec{N}} \rangle = \int_{\text{unit cell}} \mathrm{d}^3 r \phi_{i,\vec{M}} \vec{\nabla} \phi_{j,\vec{N}} \tag{3.56}$$

with the real space functions $\phi_i(\vec{r})$ centered in unit cells shifted by $\vec{T}\left(\vec{N}\right)$, $\vec{N} = (N_1, N_2, N_3)$, see [?] for details.

Building up on the expressions for the dielectric function also the corresponding Kubo-Greenwood transport properties expressed via the Onsager coefficients L_{ij}:

$$L_{ij}(\omega) = \frac{2\pi e^{4-i-j}}{3Vm_e^2\omega} \sum_{\boldsymbol{k},m,n} |\langle \Psi_m | \hat{\boldsymbol{p}} | \Psi_n \rangle|^2 \cdot \left(\frac{\epsilon_{\boldsymbol{k}m} + \epsilon_{\boldsymbol{k}n}}{2} - \epsilon_{Fermi} \right)^{i+j-2} \cdot (f_{\boldsymbol{k}m} - f_{\boldsymbol{k}n}) \, \delta \left(\epsilon_{\boldsymbol{k}n} - \epsilon_{\boldsymbol{k}m} - \hbar\omega \right) \tag{3.57}$$

In this representation L_{11} corresonds to the optical conductivity $\sigma(\omega)$. Furthermore the (frequency dependent) Seebeck coefficient is easily obtainable:

$$S = \frac{L_{12}}{TL_{11}} \tag{3.58}$$

Tags for general section of `control.in`:

Tag: `compute_dielectric` (control.in)

Usage: `compute_dielectric width` E_{min} E_{max} ω_{min} ω_{max} n_ω `i j`

Purpose: Calculate and output the `i j` component (choices: x, y or z) of the imaginary/real part of the inter-band and intra-band contribution to the linear dielectric tensor within RPA. The momentum matrix elements in the energy window [E_{min},E_{max}] (in eV) relative to the internal zero are summed up. The Delta-distribution is represented by a Gaussian function with width `width` (in eV) for n_ω ω-values in the interval [ω_{min}, ω_{max}] (in eV) are used. The output file is named *dielectric_i_j.out*. The plasma frequency is written to the standard aims output.
Good choices are: `width`=$0.1eV$, usually it is enough to include only a few states below and above the fermi level in the energy window [E_{min},E_{max}].
The calculation is very sensitive to the k-point grid, an extremely high number of k-points might be needed for convergence, especially for metals.

Tag: `compute_absorption` (control.in)

Usage: `compute_absorption width` E_{min} E_{max} ω_{min} ω_{max} n_ω `i use_gauss`

Purpose: Calculate and output the `i` component (choices: x, y or z) of the linear absorption.

$$\alpha_i(\omega)) = \frac{1}{\pi\omega} \sum_{n,n'} \int_{\vec{k}} \left| p_{i;n',n,\vec{k}} \right|^2 \delta(\epsilon_{n',\vec{k}} - \epsilon_{n,\vec{k}} - \omega) \mathrm{d}\vec{k} \tag{3.59}$$

The momentum matrix elements in the energy window [E_{min},E_{max}] (in eV) relative to the internal zero are summed up. The Delta-distribution is represented by a Gaussian function (`use_gauss` = .true.) or a Lorentz function (`use_gauss` = .false.) with width `width` (in eV) for n_ω ω-values in the interval [ω_{min}, ω_{max}] (in eV). The output file is named *absorption_i.out*. A good choice is: `width`=$0.1eV$, usually it is enough to include only a few states below and above the fermi level in the energy window [E_{min},E_{max}]. The unit is a_0^{-3}.
The calculation is very sensitive to the k-point grid, an extremely high number of k-points might be needed for convergence, especially for metals.

Tag: `compute_momentummatrix` (control.in)

Usage: `compute_momentummatrix` E_{min} E_{max} `k-point`

Purpose: Calculate and output the momentum matrix elements $\left\langle \psi_{n'\vec{k}} | \nabla_x | \psi_{n\vec{k}} \right\rangle$, $\left\langle \psi_{n'\vec{k}} | \nabla_y | \psi_{n\vec{k}} \right\rangle$, $\left\langle \psi_{n'\vec{k}} | \nabla_z | \psi_{n\vec{k}} \right\rangle$ for the k-point with the number `k-point` for KS-eigenstates that are within the energy window [E_{min},E_{max}] (in eV) relative to the internal zero. The output file is named *element_k_*`k-point`*.dat*. The unit is a_0^{-1} (bohr^{-1}).
If you are conducting a cluster calculation (no periodic boundary conditions) make sure to set `k-point` to 1.
Setting `k-point` to 0 will output the momentum matrix elements for all k-points into one container file (mommat.h5). To use this functionality, FHI-aims has to be compiled with the external hdf5 module (see www.hdf5group.org). The module provides functions and routines to efficently make use of paralell writting and reading of data (mpi-io). In the source folder of FHI-aims an additional Makefile is provided, *Makefile.hdf5*. The usual compiler options have to be set in the same way as in the normal Makefie. Additionally the path to your installation of hdf5 must be provided. If the environment variable *HDF5_HOME* is already set on your system nothing else has to be changed otherwise it has to be set in *Makefile.hdf5*.

Tag: `compute_dipolematrix` (control.in)

Usage: `compute_dipolematrix` E_{min} E_{max} `k-point`

Purpose: Calculate and output the dipole matrix elements $\left\langle \psi_{n'\vec{k}} | x | \psi_{n\vec{k}} \right\rangle$, $\left\langle \psi_{n'\vec{k}} | y | \psi_{n\vec{k}} \right\rangle$, $\left\langle \psi_{n'\vec{k}} | z | \psi_{n\vec{k}} \right\rangle$ (position operator!) for the k-point with the number `k-point` for KS-eigenstates that are within the energy window [E_{min},E_{max}] (in eV) relative to the internal zero. The output file is named *element_k_*`k-point`*.dat*. The unit is a_0 (bohr).
If you are conducting a cluster calculation (no periodic boundary conditions) make sure to set `k-point` to 1.
Setting `k-point` to 0 will output the dipole matrix elements for all k-points into one container file (dipmat.h5). To use this functionality, FHI-aims has to be compiled with the external hdf5 module (see www.hdf5group.org). The module provides functions and routines to efficently make use of paralell writting and reading of data (mpi-io). In the source folder of FHI-aims an additional Makefile is provided, *Makefile.hdf5*. The usual compiler options have to be set in the same way as in the normal Makefie. Additionally the path to your installation of hdf5 must be provided. If the environment variable *HDF5_HOME* is already set on your system nothing else has to be changed otherwise it has to be set in *Makefile.hdf5*.

Tag: `compute_dipolematrix_k_k` (control.in)

Usage: compute_dipolematrix_k_k E_{min} E_{max} k_k_method

Purpose: Calculate and output the dipole matrix elements $\langle\psi_{n'\vec{k}'}|x|\psi_{n\vec{k}}\rangle$, $\langle\psi_{n'\vec{k}'}|y|\psi_{n\vec{k}}\rangle$, $\langle\psi_{n'\vec{k}'}|z|\psi_{n\vec{k}}\rangle$ (position operator!) for all (k, k')-points for KS-eigenstates that are within the energy window [E_{min},E_{max}] (in eV) relative to the internal zero. The output file is named *dipmat_k.h5*. The unit is a_0 (bohr). With the option k_k_method (choises: 1, 2, 3) you can chose the method for calculating the matrix elements. *1* will first calculate the matrix elements (k, k')!!! for the atomic basis before transforming to the KS-basis (biggest array size: n_basis*n_k_points*n_k_points). *2* will calculate the matrix elements (k) for the atomic basis and transform to the KS-basis for all (k') (n_k_points times) (biggest array size: n_basis*n_k_points). *3* will calculate the matrix elements in the atomic basis and transform to the KS-basis for all (k, k') (n_k_points*n_k_points times) (biggest array size: n_basis). *1* needs the most memory, *3* does the most (repetitive) calculations.
The dipole matrix elements for all (k,k')-points are written into one container file (*dipmat_k.h5*). To use this functionality, FHI-aims has to be compiled with the external hdf5 module (see www.hdf5group.org). The module provides functions and routines to efficently make use of paralell writting and reading of data (mpi-io). In the source folder of FHI-aims an additional Makefile is provided, *Makefile.hdf5*. The usual compiler options have to be set in the same way as in the normal Makefie. Additionally the path to your installation of hdf5 must be provided. If the environment variable *HDF5_HOME* is already set on your system nothing else has to be changed otherwise it has to be set in *Makefile.hdf5*.

Tag: compute_kubo_greenwood (control.in)

Usage: compute_kubo_greenwood width FD_Temp E_{min} E_{max} ω_{min} ω_{max} n_ω i j

Purpose: Calculate and output the i j component (choices: (x, y or z) or (a a)) of the the Kubo-Greenwood transport properties optical conductivity $\sigma(\omega)$ and Seebeck coefficient S in units $(\Omega \cdot cm)^{-1}$ and $\mu V/K$, respectively. The (a a) choice triggers the calculation of the average of the three diagonal elements $L_{ij,xx}$, $\sigma_{ij,yy}$ and $\sigma_{ij,zz}$. As before, [E_{min},E_{max}] (in eV) determine the energy window (relative to the internal zero) within which the momentum-matrix elements are considered. The Delta-distribution is represented by a Gaussian function with width width (in eV) and the Fermi occupations are calculated at an electronic temperature FD_Temp (in eV). The spectra contain n_ω ω-values in the interval [ω_{min}, ω_{max}] (in eV). The output file is named *dielectric_*i_j*<Fermi-level>.out* and consists of the dielectric function, the optical conductivity and the Seebeck coefficient. Usually it is enough to include only a few states below and above the fermi level in the energy window [E_{min},E_{max}].
The calculation is very sensitive to the k-point grid, an extremely high number of k-points might be needed for convergence, especially for metals.

Tag: `greenwood_method` (control.in)

Usage: `greenwood_method method`

Purpose: Switches between the use of sparse or full transition matrices. (Choices: *sparse* or *full*). As of now, the keyword has to be set in case the `compute_kubo_greenwood` is used. The use of *sparse* matrices is recommended. (Keyword will be deprecated in the near future)[1.0ex]

3.25 Electronic Transport

Electronic transport in FHI-aims can be computed either using the built-in routine described in this chapter or with the help of aitranss-package (see Chapter 5.1). The built-in routine computes the electronic transport through a nanostructure connected to two, three or four semi-infinite leads using the Landauer-Büttiker formalism. Only calculations within the zero-bias limit are supported. All transport related actions are initiated with the keyword `transport` that is followed by the action required.

A typical work-flow of a transport calculation is as follows

1. Information for the semi-infinite leads are created. To this end the action `lead_calculation` should be used. Separate periodic calculation is required for each lead and the third lattice vector must point away from the nanostructure, i.e., into the semi-infinite lead. The region used to model the semi-infinite lead should be large enough so that the basis functions from the nanostructure region do not extend beyond the lead region.
2. Once all the leads are calculated transport through the nanostructure is calculated using the action `transport_calculation`. This produces the tunneling information through the nanostructure for each pair of the leads. The nanostructure should be large enough so that the basis functions from different leads do not overlap. In this calculation the lead atoms need to be included in the file `geometry.in` and their positions must be exactly the same as in the calculation for the lead information. In particular, they should not be relaxed when the geometry of the nanostructure is optimized. This calculation should also be run as a periodic calculation using a sufficient amount of k-points. The transport part of the calculation is done using only one k-point but in order to ensure converged density and potential of the nanostructure region more k-points are usually needed.

Tags for general section of `control.in`:

Tag: `transport` (control.in)

Usage: `transport action [further options]`

Purpose: This is the keyword that is used to control the built-in transport routines.

`action` is a string that specifies the kind of requested action in the transport calculation; any further needed options depend on `action`.

Specific actions `transport` keyword:

`transport` **sub-tag:** `lead_calculation` (control.in)

Usage: transport lead_calculation

Purpose: Computes the lead information for the given lead. Only one lead can be calculated at at time and each lead needs to be calculated separately. The calculation is periodic and the third lattice vector must point into the lead.

---

transport sub-tag: transport_calculation (control.in)

Usage: transport transport_calculation

Purpose: Performs the transport calculation. Using this keyword requires that the information for all the semi-infinite leads is already calculated.

---

transport sub-tag: tunneling_file_name (control.in)

Usage: transport tunneling_file_name filename

Purpose: Specifies the name of the file where the tunneling information is written. Each pair of the leads is written to a separate column in plain text format.

---

transport sub-tag: energy_range (control.in)

Usage: transport energy_range E_{min} E_{max} n_{steps}

Purpose: Sets the energy range for which the tunneling information is calculated. The energy range $E_{\mathrm{max}} - E_{\mathrm{min}}$ is divided into n_{steps} steps and the tunneling information is calculated for each step. The energy zero is referenced at the chemical potential of the nanostructure.

---

transport sub-tag: lead_i (control.in)

Usage: transport lead_i atom_index filename

Purpose: Tells the transport calculation where the information on the semi-infinite leads is stored. One line for each lead is required and the index i is substituted by the lead index (1, 2, 3, or 4). The option atom_index is set to the index of the first atom of the lead in question in the file geometry.in and the coordinates of the lead atoms must be the same used when calculating the lead information. The option filename provides the name of the file where the lead information is stored. Note that for a successful transport calculation information for at least two leads is needed. Hence, this keyword must be invoked at least twice. The maximum number of leads supported is currently four.

The Green's function for the semi-infinite leads needs to be solved iteratively for each energy step. The iteration is regularized adding a small complex part to the energy of the step being computed. At each iteration the magnitude of the regularizing part is decreased until either convergence of lower bound of the parameter is reached. The iteration can be controlled by several keywords.

---

transport sub-tag: number_of_boundary_iterations (control.in)

Usage: transport number_of_boundary_iterations number

Purpose: Sets the maximum number of iterations to solve the Green's function for each lead. Default: 10

---

transport sub-tag: boundary_treshold (control.in)

Usage: transport boundary_treshold number

Purpose: Sets the convergence criterion for the iteration of the Green's functions. The metric used is the maximum absolute value change in the matrices for the Green's functions of the leads. Default: 1.0

---

transport sub-tag: boundary_mix (control.in)

Usage: transport boundary_mix number

Purpose: Mixing value for the linear mixer in the iteration of the Green's function. Values below one correspond to under-relaxation. Default: 0.7

---

transport sub-tag: epsilon_start (control.in)

Usage: transport epsilon_start number

Purpose: Starting value for the regularizing complex parameter in the iteration for the Green's function for the semi-infinite lead. Only the magnitude of the parameter should be given with this keyword. Default: $0.02i$

---

transport sub-tag: epsilon_end (control.in)

Usage: transport epsilon_end number

Purpose: Ending value for the regularizing complex parameter in the iteration for the Green's function for the semi-infinite lead. Only the magnitude of the parameter should be given with this keyword. Default: $0.0001i$

---

transport sub-tag: fermi_level_fix (control.in)

Usage: `transport fermi_level_fix`

Purpose: Set the potential reference of the leads to their Fermi level. Since in the transport calculations the leads are calculated first and only after that incorporated into the system there is a question of a potential reference. Currently two options are available. The default option is to use the average of the energy levels of the lowest orbitals of the lead atoms. The second option invoked by this keyword is to align the Fermi levels of the leads between the lead calculations and the transport calculation.
Note that since the calculation of the lead is separate from the transport calculation in general the distance from the energy of the lowest lying orbital to the Fermi level of the calculation is not necessarily the same. This means that the leads in the lead calculation are not in the same environment as in the transport calculation and bringing them together introduces an alignment problem for the potential. From the transport calculation point of view this implies that a gate voltage is introduced into the system. In the transport calculation the value of this gate voltage is printed. In the tunneling results the energy is always referenced to the Fermi level of the transport calculation. Default: `.false.`

3.26 ESP charges

This sections describes how to calculate partial charges by fitting to the electro static potential (ESP). These charges are widely used in the context of force fields ([?], [?], [?], [?], CHELP: [?], CHELPG: [?], RESP-charges: [?], CHELP-BOW/CHELMO: [?], ESP-charge from multi-pole-moments: [?]). FHI-aims implements a simple method for cluster calculation (molecules) as well as two methods for solids (periodic boundary conditions) [?], [?].
The starting point for these methods is the calculation of the electro static potential at a sufficiently high number of grid points outside the vdw radius of the atoms. To define a space region for the grid two parameters are necessary: a minimal radius and a maximal radius around the atoms. These radii are defined as multiples of the vdw-radius of the atoms, see figure 3.6 for details. The values for the vdw radii of most atoms in the periodic table have been taken from "http://de.wikipedia.org/wiki/Van-der-Waals-Radius" ([?], [?], [?]). For the generation of the points cubic grids are used. For cluster calculations points within a cube encapsulating the spheres with the maximal radius (multiple of the vdw radius) around all atoms are generated. For periodic boundary conditions the provided unitcell is used. The points within the superposition of the spheres with the minimal radius (minimal multiple of the vdw radius) are excluded. The atom-centered radial grids are also available, mainly for test purposes. Here, all points lie on N spheres with radii between the minimal and maximal multiple of the vdw radius. The spacing between the radii of the N spheres is equidistant or logarithmic.
For the cluster case the function to fit to is a sum of Coulomb potentials with charges

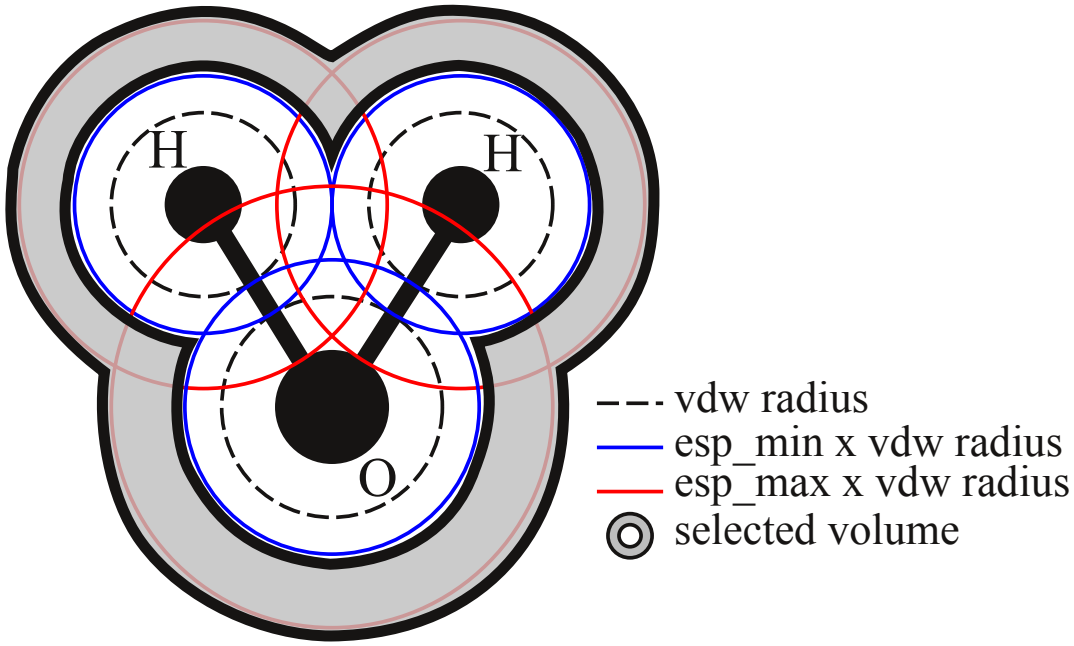


Figure 3.6: Definition of the volume used for the creation of grid points at which the potential is evaluated.

q_i, the ESP-charges, at the atomic position $\mathbf{R}_i$:

$$V_{ESP}(\mathbf{r}) = \sum_{i=1}^{N_{at}} \frac{q_i}{|\mathbf{r} - \mathbf{R}_i|} \tag{3.60}$$

The q_i are calculated by a least squares fit with the additional constraint of constant total charge $q_{tot} = \sum_{i=1}^{N_{at}} q_i$. We use the method of Lagrange multipliers to minimize the function:

$$F = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) - V_{ESP}(\mathbf{r_k})\right)^2 - \lambda \left(q_{tot} - \sum_{i=1}^{N_{at}} q_i\right)^2. \tag{3.61}$$

This can be translated into a system of linear equations:

$$\hat{\mathbf{A}}\mathbf{q} = \mathbf{B}. \tag{3.62}$$

with the N_{at+1} x N_{at+1} matrix $\hat{\mathbf{A}}$:

$$A_{ij} = \sum_{k=1}^{N_{grid}} \frac{1}{|\mathbf{r_k} - \mathbf{R}_i|} \frac{1}{|\mathbf{r_k} - \mathbf{R}_j|} \quad \text{with} \quad i,j \leq N_{at} \tag{3.63}$$

$$A_{i=N_{at}+1,j} = A_{i,j=N_{at}+1} = 1; \qquad A_{i=N_{at}+1,j=N_{at}+1} = 0$$

and the $N_{at} + 1$ vector $\mathbf{B}$:

$$B_i = \sum_{k=1}^{N_{grid}} \frac{V_{DFT}(\mathbf{r_k})}{|\mathbf{r_k} - \mathbf{R}_i|} \quad \text{with} \quad i \leq N_{at} \tag{3.64}$$

$$B_{i=N_{at}+1} = q_{tot} \tag{3.65}$$

$\mathbf{q}$ are the N_{at} charges.
For solids (periodic boundary conditions) the situation is more complicated because all charges are repeated infinitely and the potential is only defined up-to an arbitrary offset. The methods to solve this problem are based on Ewald summation. Details on method 1 can be found here [?] and on method two here [?]. The function for the potential generated by the ESP charges centered at the atoms of the unit-cell now reads:

$$V_{ESP}(\mathbf{r}) = \sum_{i=1,\mathbf{T}}^{N_{at}} q_i \frac{\mathrm{erfc}(\alpha|\mathbf{r} - \mathbf{R_{i,T}}|)}{|\mathbf{r} - \mathbf{R_{i,T}}|} + \frac{4\pi}{V_{cell}} \sum_{i=1,\mathbf{k}}^{N_{at}} q_i \mathrm{cos}(\mathbf{k}(\mathbf{r} - \mathbf{R}_i)) \frac{\exp^{-\frac{k^2}{4\alpha^2}}}{k^2} \tag{3.66}$$

with $\mathbf{T} = n_1\mathbf{a}_1 + n_2\mathbf{a}_2 + n_3\mathbf{a}_3$ mapping the lattice positions. $\mathbf{a_i}$ the lattice vectors, $n_i \in \mathbb{Z}$. $\mathbf{k} = m_1\mathbf{b}_1 + m_2\mathbf{b}_2 + m_3\mathbf{b}_3$ mapping the reciprocal space. $\mathbf{b_i}$ the reciprocal lattice vectors, $m_i \in \mathbb{Z}$. V_{cell} is the volume of the unit-cell. The parameter α is defined as $\alpha = \frac{\sqrt{\pi}}{R_c}$ with R_c the cutoff radius of the Ewald summation. For the second method Wolf summation is implemented as well:

$$V_{ESP}(\mathbf{r}) = \sum_{i=1}^{N_{at}} q_i \left[\frac{\mathrm{erfc}(\sqrt{\alpha}|\mathbf{r} - \mathbf{R_i}|)}{|\mathbf{r} - \mathbf{R_i}|} - \frac{\mathrm{erfc}(\sqrt{\alpha}R_c)}{R_c} + \left(\frac{\mathrm{erfc}(\sqrt{\alpha}R_c)}{R_c^2} + \frac{2\sqrt{\alpha}}{\sqrt{\pi}} \frac{\exp(\alpha R_c^2)}{R_c}\right)\right] \tag{3.67}$$

$$\mathrm{x}(|\mathbf{r} - \mathbf{R_i}| - R_c)$$

The function to minimize for method 1 is:

$$F_1^{PBC} = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) - V_{ESP}(\mathbf{r_k}) + \frac{1}{N_{grid}} \sum_{j=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_j}) - V_{ESP}(\mathbf{r_j}) \right) \right)^2 \quad (3.68)$$
$$- \lambda \left(q_{tot} - \sum_{i=1}^{N_{at}} q_i \right) + \sum_{i=1}^{N_{at}} w_i \left(E_i^0 + \chi_i q_i + \frac{1}{2} J_i^{00} q_i^2 \right).$$

The χ_i and J_i^{00} are electro-negativity and self-Coulomb interaction of the respective elements, which can be used to constrain the desired charges. w_i are weighting factors for the constraints. This results in $\hat{A}$ and $\mathbf{B}$ for the linear equations system:

$$A_{ij} = \sum_{k=1}^{N_{grid}} \left(\frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_i} - \frac{1}{N_{grid}} \sum_{j=1}^{N_{grid}} \frac{\partial V_{ESP}(\mathbf{r_j})}{\partial q_i} \right) \text{ x} \quad (3.69)$$
$$\left(\frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_j} - \frac{1}{N_{grid}} \sum_{m=1}^{N_{grid}} \frac{\partial V_{ESP}(\mathbf{r_m})}{\partial q_j} \right) + \frac{w_i}{2} J_i^{00} \delta_{ij} \quad ; \quad i,j \leq N_{at}$$
$$A_{i=N_{at}+1,j} = A_{i,j=N_{at}+1} = 1; \qquad A_{i=N_{at}+1,j=N_{at}+1} = 0$$
$$B_i = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) - \frac{1}{N_{grid}} \sum_{j=1}^{N_{grid}} V_{DFT}(\mathbf{r_j}) \right) \text{ x}$$
$$\left(\frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_i} - \frac{1}{N_{grid}} \sum_{m=1}^{N_{grid}} \frac{\partial V_{ESP}(\mathbf{r_m})}{\partial q_i} \right) - w_m \frac{\chi_m}{2} \quad ; \quad i \leq N_{at}$$
$$B_{i=N_{at}+1} = q_{tot}$$

The function to minimize for method 2 is:

$$F_2^{PBC} = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) - \left(V_{ESP}(\mathbf{r_k}) + V_{DFT}^{offset} \right) \right)^2 \quad (3.70)$$
$$- \lambda \left(q_{tot} - \sum_{i=1}^{N_{at}} q_i \right) + \beta \sum_{i=1}^{N_{at}} \left(q_i - q_{i0} \right)^2.$$

The constraint charges q_{i0} can be determined with other methods (e.g. Mulliken charge

analysis [?]), β is the weighing factor. This gives $\hat{A}$ and $\mathbf{B}$:

$$A_{ij} = \sum_{k=1}^{N_{grid}} \left(\frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_i} \frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_j} \right) + \beta \delta_{ij} \quad ; \quad i,j \leq N_{at} \tag{3.71}$$

$$A_{i=N_{at}+1,j} = A_{i,j=N_{at}+1} = \sum_{k=1}^{N_{grid}} \frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_j} \quad j \leq N_{at}$$

$$A_{i=N_{at}+2,j} = A_{i,j=N_{at}+2} = 1$$

$$A_{i=N_{at}+2,j=N_{at}+2} = A_{i=N_{at}+1,j=N_{at}+2} = A_{i=N_{at}+2,j=N_{at}+1} = 0$$

$$B_i = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) \frac{\partial V_{ESP}(\mathbf{r_k})}{\partial q_i} \right) - \beta q_{0i} \quad ; \quad i \leq N_{at}$$

$$B_{i=N_{at}+1} = \sum_{j=1}^{N_{grid}} V_{DFT}(\mathbf{r_j})$$

$$B_{i=N_{at}+2} = q_{tot}$$

Here the arbitrary offset of the potential V_{offset} is an additional fitting parameter, the matrix $\hat{A}$ is of dimension N_{at+2} x N_{at+2} and $\mathbf{B}$ of dimension N_{at+2}. For method 1 V_{offset} can be calculated as:

$$V_{offset} = \sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) - V_{ESP}(\mathbf{r_k}) \right) \tag{3.72}$$

from the fitted charges q_i. As a measure for the quality of the fit the root-mean-square ($RRMS$) is defined as:

$$RRMS = \left\{ \frac{\sum_{k=1}^{N_{grid}} \left((V_{ESP}(\mathbf{r_k}) + V_{offset}) - V_{DFT}(\mathbf{r_k}) \right)^2}{\sum_{k=1}^{N_{grid}} \left(V_{DFT}(\mathbf{r_k}) \right)^2} \right\}^2 \tag{3.73}$$

The current implementation is quite sensitive to the points chosen for the calculation of the electrostatic potential. Thorough studies regarding the parameters for the grid are advised! For periodic boundary conditions the ESP-charges calculated for transition densities are experimental, caution!! The ESP-charges from transition densities can be benchmarked against the dipole-moments calculated with `compute_dipolematrix`. Example for a periodic system:

```
output esp
esp n_radius 10
esp radius 1.0 2.
esp pbc_method 1
esp R_c 10
output esp
esp n_radius 10
esp radius 1.0 2.
esp pbc_method 2
esp R_c 30
output esp
```

```
esp radius 1.0 2.
esp pbc_method 2
esp R_c 20
esp equal_grid_n 10 10 10
```

The ESP-charges for the full potential will be calculated for a periodic system. At first with points within once the vdw radius and twice the vdw radius of the atoms, with 10 shells in between and a cutoff radius of 10Å for the Ewald summation, method 1 (Ewald summation) is used. Secondly with points within once the vdw radius and twice the vdw radius of the atoms, with 10 shells in between and a cutoff radius of 30Å, method 2 (Ewald summation) is used. Thirdly with points within once the vdw radius and twice the vdw radius of the atoms, with $10\times10\times10$ initial points on a cubic grid, method 2 is used.

Warning: For periodic boundary conditions the atoms in the supplied `geometry.in` must be within the first (central) Wigner-Seitz-cell. For a two layer system it would normally be possible to have one layer within the (central) Wigner-Seitz-cell and the second one sticking out at one side. Periodic boundary conditions will take care. The atoms sticking out would be shifted back into the (central) Wigner-Seitz-cell. This might lead to different ESP-charges and Dipole matrix elements. They are only invariant for collective translations of all atoms. The program will stop if a non-valid geometry is detected.

Tags for general section of `control.in`:

`output` **sub-tag: `esp`** (control.in)

Usage: `output esp`

Purpose: Calculate the ESP-charges at the atomic position from the full density with default settings. The default settings are to use a radial grid with equidistant radii and without mapping back to the unit-cell in case of periodic boundary conditions. The radial multipliers for the exclusion radius are 3 (Minimum) and 8 (Maximum) times the vdw-radius for the cluster case and 1 (Minimum) and 2 (Maximum) times the vdw-radius for pbc. In both cases 5 radial-shells are used. For pbc a cutoff radius for the Ewald summation of 10Å is used. The maximal multiples of the lattice vectors (r_m) and the reciprocal lattice vectors (k_m) used in the summation are 7 (for both). No constraints are applied to the charges.
The keyword can be used multiple times to calculate esp charges with different settings. The optional sub-sub keywords (see below) have to be set after `output esp` for every calculation separately, otherwise defaults are used.

Optional keywords for `output esp` :

- `esp spin` *spin*
  Select *spin* spin state (1 or 2) for the calculation of the esp charges from the transition density ρ_{ij} with states i -> j (default 1) at k-point *kpoint* (default 1).
- `esp state` *state_one state_two*
  Select states i=*state_one* -> j=*state_two* (i,j $\in$ [1,n_states]) for the calculation of the esp charges from the transition density ρ_{ij} with spin *spin* (default 1) at k-point *kpoint* (default 1).
- `esp kpoint` *k-point*
  Select states the `kpoint` (*k-point* $\in$ [1,n_k_points]) for the calculation of the esp charges from the transition density ρ_{ij} states i -> j (default 1) with spin *spin* (default 1).
- `esp radius` *min_radius max_radius*
  Select the minimal (min_radius) and maximal (max_radius) multiple of the vdw radius to select the volume where the potential is evaluated and the esp charges fitted. Defaults: 3 and 8 for clusters and 1 and 2 for pbc. *max_radius* should not be larger than smallest lattice vector for pbc and radial gird.
- `esp n_radius` *n_radius*
  Select the *n_radius* number of of radial shells that are used to create points in the selected volume. Default: 5
- `esp equal_grid_n` *n_grid_x n_grid_y n_grid_z*
  Select the number of points for the cubic grid in x - *n_grid_x*, y - *n_grid_y* and z - *n_grid_z* direction. Default: 10 10 10

- esp rm *r_m*
  PBC only. Maximum number *r_m* of multiples of the lattice vectors that are used in the real space sum of the Ewald summation. Default: 7

- esp km *k_m*
  PBC only. Maximum number *k_m* of multiples of the reciprocal lattice vectors that are used in the reciprocal space sum of the Ewald summation. Default: 7

- esp R_c *R_c*
  PBC only. Real space cutoff radius for the Ewald/Wolf-summation. Default: 20Å.

- esp pbc_method *method*
  PBC only. Switch between methods for periodic boundary conditions. Choices for *method*: *1* - method 1 with Ewald summation (3.69, 3.66); *2* - method 2 with Ewald summation (3.71, 3.66); *3* - method 3 with Wolf summation (3.71, 3.67). Default: 2

- esp grid *type*
  Switch between radial gird - 1, logarithmic radial grid - 2, cubic grid from lattice vectors - 3 and cubic grid from lattice vectors, but truncated in z-direction by maximal vdw-radius (exclude vacuum region for surface slabs) - 4. Default: 3.

- esp output_cube
  Instead of fitting the ESP-charges to the potential, the potential is written to a cube-file (potential_esp_i.cube). A cubic grid is required.

- esp output_fit
  The ESP-charges are fitted to the potential and the potential calculated from the fitted ESP-charges is written to a cube-file (potential_esp_i.cube). A cubic grid is required.

- esp use_dip_for_cube
  The dipole correction is added to the cube output of the (fitted) potential. Only applied if a dipole correction was used during SCF.

Tag: **esp_constraint** (control.in)

Usage: esp_constraint method

Purpose: PBC only. Constrain the calculated ESP charges with periodic boundary conditions for method 1 or 2. The constraints will be used for all calculated ESP-charges. Due to technical reasons (array allocations) you have to specify the method you are using in *control.in* with this keyword. Choices for method are *1* and *2*. The actual constraints have to be specified for each atom in *geometry.in*.

Tag: **compute_esp_charges** (control.in)

Usage: compute_esp_charges E_{min} E_{max} min_vdw_radius grid_type max_vdw_radius n_radius k-point

Purpose: Calculate and output the ESP-charges for the transition states within the energy window [E_{min}, E_{max}] at k-point k-point. Data will be written to file *esp_element_*k-point*.dat*. The grid type grid_type (1 - radial, 2 - logarithmic radial, 3 - cubic, 4 - cubic from radii) will be used. The potential will be calculated on the n_radius radial shells within the volume created by min_vdw_radius and max_vdw_radius times the vdw radius of the atoms or in a cube with n_radiusxn_radiusxn_radius oints. The dipole moment calculated from the fitted charges q_i at atomic positions R_i is also written to file:

$$\mathbf{d_{ij}} = \sum_{k=1}^{N_{at}} q_k \mathbf{R_k} \tag{3.74}$$

for transition states $i \rightarrow j$.

Tags for geometry.in:

Tag: esp_constraint (geometry.in)

Usage: esp_constraint constraint_1 constraint_2 constraint_3

Purpose: PBC only. Define the constraints for the fit of the ESP-charges for each atom. Depending on the chosen method (esp_constraint method in *control.in*). Method 1 needs three parameters (χ, J^{00}, w) 3.69 and method 2 needs two parameters (q_0, β) 3.71 as defined above.

3.27 Large-scale, massively parallel: Memory use, sparsity, communication, etc.

In one way or another, most options available with FHI-aims concern physical algorithms or numerical choices, including those affecting the accuracy and/or efficiency of a given task. As much as possible, FHI-aims attempts to use the exact same code and settings to describe any given system, and on any kind of computer hardware. Usually, this guarantees efficient code on all ends, and improvements for one class of systems immediately benefit all others

However, for very large tasks [meaning here several hundred up to thousands of atoms, depending on the system] and/or on parallel architectures with possibly (ten)thousands of processors, memory and communication constraints may come into play that require specific workarounds not needed (or beneficial) in normal systems. On a practical level, FHI-aims attempts to be as memory-parallel as possible without loss of efficiency. However, if any such modification could affect the performance for normal systems and computer architecture adversely, we recommend to switch it on separately only when needed.

On massively parallel machines, and for very large problems, the following options are particularly important:

- If ScaLapack is used (`KS_method scalapack`), Keyword `collect_eigenvectors` (=.false.) allows to switch off the collection of the full eigenvectors c_{il} on each CPU, saving both communication time and a significant amount of memory.
- In the cluster case, keyword `use_density_matrix_hf` should be used for system sizes of a few hundred atoms and above. (This is the default for all periodic systems anyway).
- For the cluster case, and if ScaLapack is used, keyword `packed_matrix_format` may save a significant amount of memory in the construction of the overlap, Hamiltonian, and density matrices. (This is the default for all periodic systems anyway).
- For the cluster case with more than 200 atoms, the non-periodic Ewald method can be used to accelerate the calculation, see section 3.7.1.
- Keyword `use_local_index` , which ensures that integration grid batches on the same CPU are always close together, requiring only a subset of the packed Hamiltonian matrix as working space for integrals on each CPU. Use `load_balancing` to improve load-balancing in this case to avoid the a negative impact on the efficiency.
- The keyword `distributed_spline_storage` avoids to store the complete multipolar decomposition of the charge density on each processor. Instead, they are only stored for those atoms with local grid points in reach.
- For beyond-GGA: Keyword `prodbas_nb` can be used to enhance the distribution of memory intensive arrays, possibly sacrificing performance.

Finally: There is a new keyword, `use_alltoall` , that allows to switch the communication behaviour of FHI-aims for very large runs when parallel linear algebra (scalapack / ELPA) is used. The default switching point is set at 1024 MPI tasks and affects CPU time (very many cores) and memory use. If you see changes around 1024 cores (especially lack of memory in "normal" LDA/GGA calculations), see there.

Tags for general section of `control.in`:

Tag: `collect_eigenvectors` (control.in)

Usage: `collect_eigenvectors flag`

Purpose: When ScaLapack is used, allows to switch off the collection of all eigenvectors to each CPU, saving memory and computation time. When .true., this ensures that every CPU has a complete and up-to-date copy of every eigenvector for the k-point assigned to it, and when .false., this information is communicated only when absolutely necessary (i.e. matrix multiplications). For compatibility reasons, the default is set to .true., but for "standard" calculations that do not employ post-processing techniques (or employ the post-processing techniques that do support this feature), this imposes an unnecessary overhead. Thus, one should manually set this variable to .false. unless he knows that a desired post-processing feature does not support it, as he will always lessen memory overhead and gain an performance increase for all system sizes.

Restriction: Eigenvectors are needed for some post-processing functionality, e.g., in order to obtain a full Mulliken analysis or a partial density of states.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

Tag: `distributed_spline_storage` (control.in)

Usage: `distributed_spline_storage flag`

Purpose: Request to store the multipolar decomposition of the density only for the atoms needed on a given processor for the corresponding parts of the Hartree potential.

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

This flag is `.false.` by default because the complete splined density might be needed for some postprocessing or output option.

Tag: `force_mpi_virtual_topo` (control.in)

Usage: `force_mpi_virtual_topo flag`

Purpose: Auxiliary option to try to force the MPI library to respect the topology of the nodes used (several tasks within each shared-memory node vs. slower communication between different nodes)

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

If requested, enables to cache the (deduced) topology of the nodes to the default communicator. In principle, the communication layer should then obtain more information on the network topology and organize the communication pattern more efficiently. However, nothing is guaranteed. The standard does not force MPI to respect the cached informa-

tion, and in all decent MPI library implementations, this information should already be provided by the system.

In short: Perhaps try this if a truly strange communication pattern is observed, but probably, there will be no effect.

Tag: `load_balancing` (control.in)

Usage: `load_balancing flag`

Purpose: Using the keyword `use_local_index` has a negative impact on the load-balancing of grid operations. With `load_balancing .true.`, this efficiency drawback can be avoided by explicit reassignment of grid point batches to processors according to timings of test runs.

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

Thanks to the addition of this feature by Rainer Johanni, using `use_local_index` no longer has any negative impact on performance and should always be used when memory is an issue.

Tag: `packed_matrix_format` (control.in)

Usage: `packed_matrix_format type`

Purpose: Allows to use a packed-matrix format for the Hamiltonian, overlap, and density matrices of the real-space basis functions.

`type` is a string that indicates the type of packing used. Default: `none` for the cluster case, `index` for periodic geometries.

The following options exist for `type`:

- `none` - no packing is used
- `index` - matrices are packed by strictly eliminating *all* near-zero elements of all three matrices. Elements ij are eliminated if *both* the overlap matrix element and the initial Hamiltonian matrix element are smaller than a threshold set by keyword `packed_matrix_threshold` .

Packing the overlap, Hamiltonian and density matrices reduces the size of these arrays during matrix integration, at the expense of some small effort to correctly sort intermediate results during the integration appropriately.

From a technical point of view, packing only makes sense if the full Hamiltonian is not required later anyway, during the eigenvalue solution. This is the case:

- In the cluster case: if ScaLapack is used (`KS_method scalapack`).
- In the periodic case: for more than one k-point, and/or if ScaLapack is used.

Tag: `packed_matrix_threshold` (control.in)

Usage: `packed_matrix_threshold tolerance`

Purpose: Tolerance value below which the elements of the overlap / Hamiltonian matrices are eliminated from the `packed_matrix_format` .

`tolerance` : A small positive real numerical value. Default: 10^{-13}.

Tag: `prune_basis_once` (control.in)

Usage: `prune_basis_once flag`

Purpose: Stores the indices of the non-zero basis functions for each integration batch in memory

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

All operations for the integrations and the electron density update are $O(N)$ operations, but verifying which basis functions are non-zero for each integration batch requires to check each basis function, and is thus an $O(N^2)$ operation with a small prefactor. This step can be avoided by checking for the non-zero basis functions once, and then storing their indices in memory for each batch of integration points. For very large systems and restricted memory, it may be worth trying to switch this feature off to save some memory, otherwise this should always be done.

Tag: `store_EV_to_disk_in_relaxation` (control.in)

Usage: `store_EV_to_disk_in_relaxation flag`

Purpose: During relaxation, eigenvectors from a previous geometry can be stored to disk instead of in memory in case the next step is reverted.

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

During relaxation (see `relax_geometry`), geometry steps can be rejected if the total energy increased unexpectedly. In order to revert to a previous step, it is necessary to access the Kohn-Sham eigenvectors used to initialize that step, which must hence be stored for that purpose. In normal calculations, eigenvector storage is not a problem, but their size grows as $O(N^2)$ with system size. For very large system sizes, their storage in memory can become a bottleneck, which is here circumvented by the option to store them to disk.

This keyword is *not* related to the restart functionality of the `restart` keyword, since it is an old, not a current, set of Kohn-Sham eigenvectors that may be needed when reverting a geometry step.

Tag: `use_2d_corr` (control.in)

Usage: `use_2d_corr flag`

Purpose: Allows to switch on or off the two-dimensional distribution of data structures for correlated methods.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`.

Only relevant for correlated beyond-hybrid methods.

Tag: `use_alltoall` (control.in)

Usage: `use_alltoall flag`

Purpose: Allows to switch some communication calls in FHI-aims, depending on the number of tasks.

`flag` is a logical string, either `.false.` or `.true.` Default: `.false.` below 1024 MPI tasks, true for 1024 and above.

Only relevant for parallel linear algebra (use of scalapack or ELPA).

When running on a system with many CPUs, it can be much faster to use "all-to-all" communcation (mpi_alltoallv) than doing n_tasks times a sendrecv call; this is, for example, noticeable on the BlueGene/P with $\approx 10^5$ MPI tasks at at time. For much fewer tasks, the effect is not usually relevant.

On the other hand, "all-to-all" costs a significant amount of memory, and this memory pressure will be felt for much fewer MPI tasks (e.g., Intel architecture with hundreds of CPU cores at a time.

Thus we use "sendrcv" (individual communication) to using mpi_alltoallv when using 1024 CPUs or more. `use_alltoall` can be employed to enforce one or the other type of call throughout. If you see too much MPI-related memory use at 1024 tasks or above, try it—and let us know.

Tag: `use_mpi_in_place` (control.in)

Usage: `use_mpi_in_place flag`

Purpose: Allows in some collective communication calls to switch on or off the use of the "MPI_IN_PLACE" flag.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

Only relevant for parallel runs.

When running on a system with many CPUs, it can be a bit more memory efficient to use "MPI_IN_PLACE" communcation; Then instead of seperate send- and recieve buffers one buffer is used and information is up-dated in place.

However, this can sometimes lead to problems or even erros in the results, depending on the used MPI implementation. It is thus worth trying to switch it off. If you find problems concerning the usage of "MPI_IN_PLACE" please let us know.

Tag: `use_local_index` (control.in)

Usage: `use_local_index flag`

Purpose: Reduces work space size for Hamiltonian / overlap matrix during integrals by storing only those parts that are touched by any grid points assigned to the present CPU.

Restriction: Supported for standard LDA and GGA with ScaLapack and packed matrices, but not for some non-standard options. Requires `prune_basis_once` and a parallel `grid_partitioning_method` .

`flag` is a logical string, either `.false.` or `.true.` (or `if_scalapack`. Default: `.false.`

For very large systems with many CPUs, it makes sense to assign only integration grid batches close to one another to the same CPU, and to store only the Hamiltonian / overlap matrix parts needed for those batches on that CPU. However, the subsequent merge of all results into the ScaLapack infrastructure then becomes more difficult, and some overhead results from the changed load balancing. This option is therefore not used by a standard call to FHI-aims, but must be switched on explicitly if needed.

Tag: `walltime` (control.in)

Usage: `walltime seconds`

Purpose: Can limit the wall clock time spent by FHI-aims explicitly, e.g., to obtain the correct final output before a queuing system shuts down a calculation.

`seconds` is the integer requested wall clock time in seconds. Default: no limit.

In order to reach the postprocessing phase and write information required at the end of a calculation in an organized manner, FHI-aims can force a stop before a certain amount of real time (wall clock time) is exceeded. In order to achieve this safely, FHI-aims uses an internal estimate of the duration of a single s.c.f. iteration within the calculation, and stops if it estimates that the next iteration will take more time than what remains available.

3.28 Fragment molecular orbital DFT calculations

The fragment molecular orbital (FMO or FO) scheme allows the efficient calculation of transfer matrix elements for transport calculations. The two reference states are constructed from isolated calculations of the respective fragments (donor and acceptor). This circumvents the electron delocalistion error, but neglects any interactions between the fragments.

Important: This functionality is not yet available for periodic systems.
Important: The δ+-FODFT scheme is not optimized for large systems.

FODFT flavours

Depending on the approximations done to implement the FODFT method we distinguish between three different flavours of FODFT. Please see [?] for a detailed description and assessment of each. Within FHIaims it is possible to use all different FODFT-schemes.

$H^{2n}@DA$

This is the classic implementation by [?]. The fragment calculations are always done for the neutral fragments and the Hamiltonian is constructed using a wrong number of electrons. $H_{ab} = H_{ba}$ even for hetero dimers.

To use this flavour, simply calculate neutral fragments and continue with the default FODFT options.

$H^{2n-1}@DA$ / $H^{2n+1}@D^{-}A^{-}$

This implementation was first used within the CPMD code[?]. Here, neutral (double anionic) fragment calculations are combined for hole (electron) transfer with a reset of the occupation number in the highest occupied molecular orbital. This leads to the correct number of electrons for the construction of the Hamiltonian.

To use this flavour, calculate neutral (or anionic) fragments and use the `fo_flavour` keyword with the option `reset`.

$H^{2n\pm1}@D^{\pm}A$

This scheme was first implemented within FHIaims and uses charged fragment calculations. This results in slightly different approximations with improved accuracy in the electronic coupling values. Please see [?] for more details.

To use this flavour, calculate appropriately charged fragments and continue with the default FODFT options.

$\delta+$-FODFT

The implementation of the $\delta+$-FODFT scheme is not optimized for large systems and considered *experimental*. In our study[?] we found no significant improvement when using this scheme.

Theoretical Background

The following section intends to give a quick overview over the theory and the implementation within FHIaims. For detailed theory we refer to the before mentioned publications. The following theory is for the charged FODFT flavour ($H^{2n\pm1}@D^{\pm}A$).

In principle, the transfer matrix element for hole transport between the HOMO and the LUMO of two molecules is given by

$$H_{AB} = \langle \Psi^D_{\mathsf{HOMO}} | \hat{H}_{KS} | \Psi^A_{LUMO} \rangle. \tag{3.75}$$

Obtain the fragment wavefunctions

First, two standard-DFT calculations are done to obtain the fragment wavefunctions. Some additional output is needed for the subsequent combination of the fragments to the full system.

Combine the fragment wavefunctions and extract the transfer matrix element

The saved wavefunctions of both fragment calculations are used to obtain the full, non-interacting electron density of the combined system (D+A). Although FHIaims uses atom centered basis functions, it is not generally possible to re-use any restart file. If the ordering of the `atoms` in the *geometry.in* files is identical, it is possible to use a restart file for any geometry with translational symmetry. If the geometry is rotated, it is necessary to rotate the wave function.

For this non self-consistent density the Hamiltonian is calculated and used to determine H_{AB}. We therefore use the non-interacting density, but the interacting Hamiltonian with the correct number of electrons. This is in contrast to other implementations, where only the Hamiltonian of the neutral fragment is used.

The first step is a Loewdin-Orthogonalization of the Kohn-Sham-eigenvector.

$$\mathbf{U}^\top \mathbf{S} \mathbf{U} = \mathbf{s} \tag{3.76}$$

$$\mathbf{X} = \mathbf{S}^{-\frac{1}{2}} = \mathbf{U}\mathbf{s}^{-\frac{1}{2}}\mathbf{U}^\top \tag{3.77}$$

$$\begin{aligned} \mathbf{C}' &= \mathbf{X}^{-1}\mathbf{C} \\ \mathbf{H}' &= \mathbf{X}^\top \mathbf{H} \mathbf{X} \end{aligned} \tag{3.78}$$

In the next step, the Hamiltonian is transformed,

$$\mathbf{H'}_{\text{st.}} = \mathbf{C'}^{\top}\mathbf{H'}\mathbf{C}, \tag{3.79}$$

and the overlap of states is calculated:

$$\mathbf{S}_{\text{st.}} = \mathbf{C}^{\top} \cdot \mathbf{C}. \tag{3.80}$$

In the final step, the transfer matrix element H_{AB} is calculated with

$$H_{AB} = \left(1 - \mathbf{S}_{\text{st.}}(\text{a,b})^2\right)^{-1} \left[\mathbf{H'}_{\text{st.}}(\text{a,b}) - \mathbf{S}_{\text{st.}}(\text{a,b}) \left(\mathbf{H'}_{\text{st.}}(\text{a,a}) + \mathbf{H'}_{\text{st.}}(\text{b,b})\right)/2\right] \tag{3.81}$$

Important technical hints

When creating the input geometries for the calculations, the `geometry.in` must mirror the actual fragmentation. This means, the order of atoms has to be consistent for the dimer and the fragments.

Example: Calculating the H_{AB} for two methane molecules (CH_4^+ – CH_4)

Complete System, CH_4^+–CH_4

```
#fragment 1
atom     12.7490 4.3034  0.0000  C
atom     13.8413 4.3037  0.0000  H
atom     12.3850 4.7804  -0.9128 H
atom     12.3850 4.8558  0.8693  H
atom     12.3851 3.2750  0.0435  H
#fragment 2
atom     -2.6946 4.4740  0.0002  C
atom     -1.6023 4.4740  0.0000  H
atom     -3.0586 3.6144  -0.5671 H
atom     -3.0586 5.3949  -0.4609 H
atom     -3.0585 4.4127  1.0278  H
```

Fragment 1, CH_4^+

```
atom     12.7490 4.3034  0.0000  C
atom     13.8413 4.3037  0.0000  H
atom     12.3850 4.7804  -0.9128 H
atom     12.3850 4.8558  0.8693  H
atom     12.3851 3.2750  0.0435  H
```

Fragment 2, CH_4

```
atom     -2.6946 4.4740  0.0002  C
atom     -1.6023 4.4740  0.0000  H
atom     -3.0586 3.6144  -0.5671 H
atom     -3.0586 5.3949  -0.4609 H
atom     -3.0585 4.4127  1.0278  H
```

Folder structure for FODFT-calculation

At least three separate calculations (fragment1, fragment2 and combination step) are needed for a complete FODFT run. To maintain a clean structure and avoid copying of files, a special scheme is enforced. The folders can have arbitrary names, but need to be in the same root directory. This allows easy reuse of fragment calculations for different dimer calculations.

```
- fodft\_something (main calculation folder)
|
+-- dimer_01
    |
    +-- control.in
    +-- geometry.in
+-- frag1/
    |
    +-- control.in
    +-- geometry.in
+-- frag2/
    |
    +-- control.in
    +-- geometry.in
```

δ^+ FO-DFT - Inclusion of the local potential

In our implementation, an additional option is available to include interactions between the fragments by means of an embedding scheme, using the full local potential (V_{loc}) of the embedded fragment. In contrast to the standard implementation, the fragment wavefunction Ψ_A is polarized by the potential of the second fragment Ψ_B during the SCF cycle.

The general procedure is:

1. Calculation of fragment wavefunctions
   (a) Calculate Fragment A, export potential A
   (b) Import potential A, calculate Fragment B, export new potential B
   (c) Import potential B, calculate Fragment A**
2. Combination of fragment wavefunctions
3. Determination of H_{AB}

**: Please note that this step might not be necessary if fragment A was choosen wisely, depending on the polarizability of the fragments.

The embedding can be used to iteratively converge towards a final solution, but as usual the first steps are the most important ones for the polarization.

IMPORTANT: Since this method requires the reassignment of grid points from the fragment calculations for the local potential, the current implementation is not optimized towards large systems.

Tags for general section of `control.in`:

Tag: `fo_dft` (control.in)

Usage: `fo_dft type`

Purpose: This is the central control keyword for fragment orbital DFT. Its main purpose is the control of the desired calculation step (fragment or combined system) and associated choices. Depending on `type`, further flags or lines are necessary. Those are explained below.

Options: Possible options are `fragment` and `final`.

`fo_dft` **sub-tag:** `fragment` (control.in)

Usage: `fo_dft fragment`

Purpose: This keyword indicates that a fragment for FODFT will be calculated. The restart information (`restart.frag`) and an additional output file (`info.frag`) with information for the recombination of the fragments for the final FODFT run will be written.

`fo_dft` **sub-tag:** `final` (control.in)

Usage: `fo_dft final`

Purpose: This keyword indicates that this is the final FODFT run, where to wavefunctions of both fragments are combined and the transfer matrix element is calculated.
IMPORTANT: This calculation can only be started *after* both fragments have been calculated!

Tag: `fo_orbitals` (control.in)

Usage: `fo_orbitals state1 state2 range1 range2 type`

Required input for the final FODFT step. Determines the actual states of interest for the calculation of the coupling elements.

- `state1`, `state2`: [Integer] – Isolated fragment states for the determination of the respective matrix element.
- `range1`, `range2`: [Integer] – Default 1. Gives a range for the selected states.

  (state1 $= 5$ and range1 $= 2$ will include state 5 and state 6 of fragment 1.)
- `type`: [String] Determines whether the matrix element for the spin-up (`up` or `elec`) or spin-down (`dn` or `hole`) Hamiltonian is calculated. In most cases, spin down means hole transport (excess hole on fragment 1 to fragment 2) and spin up means electron transport (excess electron on fragment 1 to fragment 2).

  Example Zn^+-Zn: `fo_orbitals 15 15 1 1 hole`
  will output the transfer matrix elements for hole transfer between the LUMO of Zn^+ (fragment 1) and the HOMO of Zn (fragment 2).

If not deactivated via `fo_verbosity 0`, the full H_{ab} submatrix will be written to the file `full_hab_submatrix` and any matrix element can be retrieved from this file after the calculation.

Tag: `fo_folders` (control.in)

OPTIONAL Usage: `fo_folders folder_frag1 folder_frag2`

Purpose: This keyword allows the specification of custom names for the folders with the fragment calculation. If not set, standard names (`frag1_00` and `frag2_00`) are used.

Tag: `fo_flavour` (control.in)

OPTIONAL Usage: `fo_flavour flavour`

Purpose: This keyword allows the specification of the FO-DFT flavour to be used for the calculation. See FODFT theory section for details. Options:

- `default` - no occupations are modified. For $H^{2n}@DA$ and $H^{2n\pm1}@D^{\pm}A$ flavours. This is the default.
- `reset` - selects the $H^{2n-1}@DA/H^{2n+1}@D^{-}A^{-}$ scheme.

Note: There is no option yet to calculate H_{ab} and H_{ba} within one FHIaims run. If the system is heterogeneous, two dimer steps with appropriate fragment ordering and charges are necessary!

Tag: `fo_verbosity` (control.in)

Usage: `fo_verbosity level`

Purpose: Control the verbosity of the FO-DFT output. Options:

- 0: Write minimal output to output file. No retrieval of arbitrary couplings without restarting the dimer calculation possible.
- 1: Write calculated couplings to output file and the full H_{ab} matrix to the file `full_hab_submatrix`. This is the default.
- 2: Write all elements for selected states from eq. 3.81 ($S_{st}, H'_{st}, \ldots$) to aims output.

Tag: `fo_deltaplus` (control.in)

Usage: `fo_deltaplus .true.`

Purpose: Enables the polarized δ^+ fragment calculations.

With this keyword, the full local potential of the actual fragment (Hartree + nuclei) will be written to files (`output_potential_0000*`). In addition, if previous potential files are found within the calculation folder, they will be read in and added as an external potential to the current calculation.

Important: In order to use this improved scheme, the (empty) atom positions of the other fragment have to be included in the `geometry.in` (see 3.28).

Important: The number of cores (determined by mpirun -n or similar) has to be the same for both fragments, or the calculation will fail!

Note: Due to differences in the partitioning of the FHIaims grid between these runs it is necessary to reassign the potential to each grid point. Depending on the system size and the number of MPI processes (communication!), this will take some time.

Additional tags for polarized FODFT in `geometry.in` and `control.in`:

The following changes are only necessary for polarized FODFT (`fo_deltaplus`). In order to allow a polarization of the fragment wavefunction, the atom centered integration grids have to be extended to the region of the embedded fragment.

geometry.in:

The atomic positions of the second, embedded fragment must be included as empty sites with the tag `empty` (instead of `atom`). In addition, even the minimal basis functions have to be disabled in the `species_defaults` for this atom type in the `control.in` (see next section). To distinguish between real and empty atoms, the species name has to be changed.

The following example for Zn_2^+ shows the 3 different `geometry.in`-files.

Fragment 1, Zn^+

```
atom 0.00 0.00 0.00 Zn
initial_charge 1
empty 0.00 0.00 5.00 Zn_empty
```

Fragment 2, Zn

```
empty 0.00 0.00 0.00 Zn_empty
#initial_charge 1 #no charge for emtpy atoms
atom 0.00 0.00 5.00 Zn
```

Complete System, Zn_2^+

```
atom 0.00 0.00 0.00 Zn
initial_charge 1
atom 0.00 0.00 5.00 Zn
```

control.in:

For every atom type of the embedded system, the species_data part has to be included and the species name must be changed to *species_empty*. Disable the minimal basis with the keyword `species` `include_min_basis` set to `.false.` and comment **all** tier basis function lines, as showed in the following example for Zn.

```
[...]
  species        Zn_empty
#     global species definitions
    nucleus             30
    mass                65.409
#
    l_hartree           4
#
    cut_pot             3.5          1.5  1.0
    basis_dep_cutoff    1e-4
#
```

```
    include_min_basis .false.

[...]

#  "First tier" - improvements: -270.82 meV to -12.81 meV
     #hydro 2 p 1.7
     #hydro 3 s 2.9
     #hydro 4 p 5.4
     #hydro 4 f 7.8
     #hydro 3 d 4.5
##  "Second tier" - improvements: -3.35 meV to -0.82 meV
     #hydro 5 g 10.8
     #hydro 2 p 2.4
     #hydro 3 s 6.2
     #hydro 3 d 3
```

3.29 Output options

The primary (and most important) output of FHI-aims is written to the standard output channel, and can / should be captured in a file from there. However, FHI-aims provides a host of further output options that can be activated to yield more specialized data not ordinarily required from a standard calculation, but highly useful for specific purposes.

The majority of these output options is activated by invoking the `output` option in file `control.in`. The individual subkeywords to this keyword are therefore listed separately, towards the end of this section. In addition, some particularly important output options are revisited with examples in Chapter 4.

Tags for geometry.in:

verbatim_writeout Usage: verbatim_writeout flag

Purpose: Enables or suppresses the writing of geometry.in to the FHI-aims standard output stream exactly as it is read the first time.

flag is a logical variable (.true. or .false.). Default: .true. .
By default, geometry.in is now written (copied) verbatim into the FHI-aims standard output as it is parsed for the first time, allowing to reproduce exactly any FHI-aims calculation simply by copy-pasting that part to a new geometry.in file.

If verbatim_writeout is set to false anywhere in geometry.in, no writing will occur from that point on forward.

The exact same option (same keyword / syntax) can also be used in control.in, producing the same effect there.

Note that the keyword has the same name in geometry.in and control.in, and is therefore only documented as a clickable link for control.in. Apologies for this omission.

Tags for general section of control.in:

Tag: dos_kgrid_factors (control.in)

Usage: dos_kgrid_factors $n_1\, n_2\, n_3$

Purpose: If set, a post-scf density of states is computed with a denser k-point grid than used in the s.c.f. cycle. Restriction: Works only for periodic systems. Does not work when keyword use_local_index is set.

Only useful in conjunction with the keywords output dos or output postscf_eigenvalues , and only for periodic systems.

In a periodic calculation, one usually specifies the basic k_grid used to obtain the self-consistent electron density, total energy etc. Such k-space grids are usually fairly sparse, and if a density of states (DOS) is calculated directly from the eigenvalues stored at these k-points only, the DOS will either look choppy, or (after significant broadening), smooth, broad, and blurred.

A simple remedy is to use the original k_grid while approaching self-consistency as usual, but then compute the DOS using an auxiliary k-grid that is made *denser* by factors n_1, n_2, n_3, respectively. For example, the settings
k_grid 10 10 10
output dos [...]
dos_kgrid_factors 8 8 8
mean that the s.c.f.-cycle itself is run with a $10\times10\times10$ k-point grid, but subsequently, a density of states is computed with an $80\times80\times80$ k-point grid.

Note that this additional calculation is done using serial lapack solutions of the eigenvalue problems for individual k-points on individual CPU cores. This always works but will create memory problems as the system size increases, simply because local copies of all matrices are kept on single CPUs. For large systems, our usual, more sophisticated parallelization strategies have not yet been copied over to this routine.

Tag: `evaluate_work_function` (control.in)

Usage: `evaluate_work_function`

Purpose: Surface slab calculations only – if true, the work functions of both slab surfaces will be evaluated.

This option requires that a reference z coordinate for the electrostatic potential evaluation deep in the vacuum be provided by hand, through the keyword `set_vacuum_level`. The surface must be parallel to the xy plane.

The output for the "upper' and "lower" surface of the slab (larger and smaller z value, respectively), will be printed separately. Note that for *non-symmetric* slabs, these work functions should generally be different. In practice, this behavior is correctly reproduced only if the `use_dipole_correction` is additionally specified.

If the work function output is requested, keyword `compensate_multipole_errors` is now automatically switched on by default. This will change total energies slightly compared to the uncompensating case, and – we believe – even for the better. It will certainly lead to a better description of the long-range Hartree potential.

However, it must be possible to find a vacuum plane z, where the surface dipole is compensated, that is further than 6 Å away from the nearest atom. Otherwise, the calculation will stop and alert the user.

Specifically, the reference Hartree potential component for the work function evaluation is only the long-range (reciprocal-space) Hartree potential term of the Ewald sum, not the full electrostatic potential. Thus, the vacuum level *must* be specified in a region where all real-space components of the electrostatic potential have safely died away to zero. One may achieve this by increasing the vacuum layer thickness, which can be done at very small overhead cost in FHI-aims.

Tag: `memtrace_summary` (control.in)

Usage: `memtrace_summary mode`

Purpose: This flag controls the level of detail in the final memory accounting table. The available modes are `global`, `host` and `process`. This keyword is independent of `memtrace_use`, which governs the detailed logging of all allocations in seperate logfiles.

To use this feature, you need to compile FHI-aims with the provided python-preprocessor. Detailed instructions on how to use it can be found in Memory Tracing.

Tag: `memtrace_targetdir` (control.in)

Usage: `memtrace_targetdir path`

Purpose: This flag controls the target directory for the logfiles generated by `memtrace_use` . Default: current working directory

Please note that the folder must *exist* at runtime and will not be created by FHI-aims. Furthermore, the logfiles *must not* exist before aims is called. These restrictions are necessary to avoid the use of compiler-specific extensions.
Also note, that this option implicitly set `memtrace_use` to `.true.`.

Tag: `memtrace_use` (control.in)

Usage: `memtrace_use flag`

Purpose: This flag controls the memory tracing option. If enabled, each processor will generate a detailed logfile about all array allocations and deallocations.

`flag` is a logical string, either `.true.` or `.false.` Default: `.false.`

To use this feature, you need to compile FHI-aims with the provided python-preprocessor. Detailed instructions on how to use it can be found in Memory Tracing.
By default, all logfiles are written into the working directory, but you can use the option `memtrace_targetdir` to change the output folder.

Tag: `output` (control.in)

Usage: `output type [further options]`

Purpose: This is the central keyword that controls most of the non-standard output that can be written by FHI-aims.

`type` is a string that specifies the kind of requested output; any further needed options, or possibly additional lines, depend on `type`.

The list of additional output `types` is given as a separate subsubsection below.

Tag: `output_in_original_unit_cell` (control.in)

Usage: `output_in_original_unit_cell flag`

Purpose: Shifts the atoms in a periodic calculation back into the first unit cell before printing them out at the beginning of a new geometry step.

`flag` is a logical string, either `.true.` or `.false.` Default: `.true.`

In some, atoms in FHI-aims can unexpectedly "switch" unit cells during relaxation. This has no effect on the calculation, but the output geometry coordinates (written to the standard output stream) will look strangely detached when visualized. By default, FHI-aims maps its coordinates back to the first unit cell anyway, but this behavior can be

forcibly switched off if so requested (makes nicer movies).

Tag: `output_boys_centers` (control.in)

Usage: `output_boys_centers`

Purpose: Calculates and outputs the maximally localized Boys centers (equivalent of Wannier centers for the isolated molecule case). The maximization procedure follows JCP 135, 134107 (2011). Currently only the cartesian position of the centers is outputted in xyz format in `geometry_boys.xyz`. The transformation matrix is also calculated, but not applied to the KS orbitals.

Tag: `output_level` (control.in)

Usage: `output_level level`

Purpose: Allows to increase the amount of output written to the standard output of FHI-aims.

`level` is a string that determines the amount of output written. Default: `normal` .

If increased to `full`, the Kohn-Sham eigenvalues of every s.c.f. iteration are written to the standard output file. For single-point calculations, this may be quite desirable, but leads to unmanageable file sizes for long relaxation or molecular dymnamics runs. Another option, useful for long molecular dynamics (MD) runs, is `MD_light`. It writes standard output only in the initialization part and at the end of each MD step, while a minimal output is written for the single scf cycle.

Tag: `verbatim_writeout` (control.in)

Usage: `verbatim_writeout flag`

Purpose: Enables or suppresses the writing of `control.in` to the FHI-aims standard output stream exactly as it is read the first time.

`flag` is a logical variable (.true. or .false.). Default: `.true.` .

By default, `control.in` is now written (copied) verbatim into the FHI-aims standard output as it is parsed for the first time, allowing to reproduce exactly any FHI-aims calculation simply by copy-pasting that part to a new `control.in` file.

If `verbatim_writeout` is set to false anywhere in `control.in`, no writing will occur from that point on forward.

The exact same option (same keyword / syntax) can also be used in `geometry.in`, producing the same effect there.

Specific options `output` keyword:

`output` sub-tag: `aitranss` (control.in)

Usage: `output aitranss`

Purpose: Writes Kohn-Sham eigenvectors c_{il} and energies ε_l (where i is a basis function index and l is an eigenstate index) of each spin channel and overlap matrix s_{ij} into separate ASCII-files in a format compatible with AITRANSS (*ab initio* transport simulations) package.

Restrictions: This functionality is available only for *non-periodic* systems. If `KS_method scalapack` is used, `packed_matrix_format` is not supported. Flag `use_local_index` is not supported either.

Please, look at Chapter 5 for a comprehensive description on how to perform transport simulations across nanoscale objects.

`output` sub-tag: `atom_proj_dos` (control.in)

Usage: `output atom_proj_dos Estart Eend n_points broadening`

Purpose: Writes an atom-projected, angular-momentum resolved partial density of states (pDOS).

`Estart` : Lower bound of the single-particle energy range for which the pDOS are given.
`Eend` : Upper bound of the single-particle energy range for which the pDOS are given.
`n_points` : Number of energy data points for which the pDOS are given.
`broadening` : Gaussian broadening applied to obtain a smooth partial density of states based on the peaks produced by individual states.

This option is based on a Mulliken Analysis and shares its syntax with `output dos` and `output species_proj_dos` . See also section 4.4 for more details.

There are two types of output files for each atom:

- `atom_proj_dos_`*`number`*`_raw.dat`, where *number* denotes the atom number in the order of `geometry.in`. This file contains the total and angular-momentum resolved DOS components as a function of eigenvalue energy (first column) as used internally in FHI-aims. The energy zero is then given by the vacuum level (non-periodic systems) or by the $\boldsymbol{G}$=0 component of the long-range Hartree potential (periodic systems).
- `atom_projected_dos_`*`number`*`.dat`, which gives the same information, except that the energy zero is shifted to the Fermi energy (metallic systems) or valence band maximum (insulators), respectively.

Note that projected densities of states such as given here must be based on some kind

of projection orbitals, the choice of which is somewhat arbitrary by necessity. This is thus a tool for *qualitative* analyses.

In FHI-aims, we project directly on the atom-centered angular-momentum components as defined by the *overlapping* basis set. This definition becomes the more arbitrary the larger the basis set, just like a `mulliken` analysis. The closer the full basis comes to completeness, the more ambiguous will a `mulliken` -like analysis become, since it may not be *a priori* clear which electrons should be counted towards the basis functions of one atom vs. those of another atom. Thus, do *not* expect a pDOS to simply converge as the basis set size is increased; use it as a qualitative indicator of trends, but nothing more.

`output` **sub-tag: `band`** (control.in)

Usage: `output band kstart1 kstart2 kstart3 kend1 kend2 kend3 n_points name_start name_end`

Purpose: Plots a band along the line from `<kstart1,kstart2,kstart3>` to `<kend1,kend2,kend3>` at `n_points` equally spaced points. The k-vectors are written in relative coordinates of the reciprocal basis vectors.

Several bands can be plotted; FHI-aims outputs one file per specified `output band` line. These files are named `band1XXX.out` and have the format

`ipoint k1 k2 k3 E1 occ1 E2 occ2 ... EN occN`,

i.e. they specify not only the bands for each k-point but also the occupation number for this particular point. Since this format contains all the important information, but is not particularly useful for actually plotting the band structure, we provide a small script which is described in section 4.4.

Note: the last two input options are technically not needed by the FHI-aims main program, but they are seriously helpful when turning this data into a plot and are used by the band plotting script provided along with this distribution, see Section 4.4 for details.

For periodic calculations, the eigenvectors, overlap matrices, and hamiltonian matrices at each k-point requested by `output band` can be written out using the `output eigenvectors` , `output overlap_matrix` , and `output hamiltonian_matrix` keywords, respectively.

For periodic band structure output, the `exx_band_structure_version` keyword must be set – see the respective Section ?? for a brief explanation of the background.

`output` **sub-tag: `band_during_scf`** (control.in)

Usage: output band_during_scf kstart1 kstart2 kstart3 kend1 kend2 kend3 name_start name_end

Purpose: Plots a band along the line from <kstart1,kstart2,kstart3> to <kend1,kend2,kend3> but only at those k points that are already part of the normal s.c.f. k_grid . The k-vectors are written in relative coordinates of the reciprocal basis vectors.

This keyword allows to get the band structure along a certain reciprocal-space direction, but *only* at those k-points that are already used during the s.c.f. calculation. If there are no appropriate k-points, no band structure is printed. If there are appropriate k-points, they are printed in the same format as the normal band structure from output band , although some additional editing may be required to get a clean plot.

The purpose of this keyword is to allow to extract a band structure even in cases when the normal output band functionality is experimental or, for some reason, not available.

---

output **sub-tag: basis** (control.in)

Usage: output basis

Purpose: Writes radial functions before and after orthonormalization, as well as second derivatives and the basis-defining potentials to separate files.

This output option allows to visualize the basis functions used, as well as some of the other defining pieces of the basis set. Note that the output is written for each grid point of the dense 1-dimensional logarithmic grid, with the radius given in bohr.

Specifically, this option produces the following types of files:

- Ai_j_nl_base.dat : Atomic (minimal basis) radial function $u(r)$ *after* the basis-confining potential was applied, for species number i, atomic-like (minimal) radial function number j, radial and angular quantum numbers nl.
- Ai_j_nl_base_kin.dat : Corresponding kinetic energy expression $[\epsilon - v(r)] \cdot u(r)$
- Ci_j_nl_base.dat : confined free-atom like radial function $u(r)$ number j for species number i, radial and angular quantum numbers nl.
- Ci_j_nl_base_pot.dat : Corresponding basis-defining potential including confining potential
- *El* _base_n_l.dat : Radial function $u(r)$ *after* the basis-confining potential was applied for the species named *El*, radial and angular quantum numbers nl (same as Ai_j_nl_base.dat).
- *El* _base_pot.dat : Basis-defining potential for atomic (minimal) radial functions of the species named *El*, after addition of the confining potential (as defined by cut_pot).
- *El* _base_pot.dat Free-atom density of species named *El* (same as *El* _base_pot.dat).

- *El*_free_n_l.dat: Radial function $u(r)$ *before* the basis-confining potential was applied for the species named *El*, radial and angular quantum numbers nl
- *El*_free_pot.dat Spherical self-consistent free-atom potential of the species named *El* (implicitly confined by the cut_free_atom potential, but this artificial part is here not included)
- *El*_free_rho.dat Free-atom density of the species named *El*.
- Hi_j_nl_base.dat : hydro radial function $u(r)$ number j for species number i, radial and angular quantum numbers nl.
- Hi_j_nl_base_kin.dat : Corresponding kinetic energy expression $[\epsilon - v(r)] \cdot u(r)$
- Ii_j_nl_base.dat : ionic radial function $u(r)$ number j for species number i, radial and angular quantum numbers nl.
- Ii_j_nl_base_pot.dat : Corresponding basis-defining potential including confining potential
- *ty*_i_j_n_l.dat : *After* on-site orthonormalization, radial function $u(r)$ of type *ty* (atomic, confined , hydro , ionic , ...), for species number i, radial function number j, radial and angular quantum numbers n, l.
- kin_*ty*_i_j_n_l.dat : Corresponding kinetic energy expression after on-site orthonormalization.

---

output sub-tag: cube (control.in)

Usage: output cube type

Purpose: Writes a quantity (density, eigenfunction, ...) to a uniform three-dimensional grid, using the ASCII-based cube file format established by the Gaussian code and accepted by numerous visualization tools.

type is a string, indicating the quantity to be plotted.

The "cube" file format originates from the Gaussian code, but publically available descriptions exist, for example here:

http://paulbourke.net/dataformats/cube/

The first two lines serve as comment headers only. The third line contains the number of atoms included, and the origin around which the plot is centered. Lines four, five, and six each contain the number of subdivisions ("voxels") of the x, y, and z axis used for the plot, followed by the length vector of each "voxel" (step along the cube direction). The following lines first summarize the underlying atomic structure, before a the actual volumetric data appear as a block of formatted ASCII output. For more information and an example, see the description in Sec. 3.29.

A complete specification of a cube output consists of up to 10 lines. Of these, all but one have pre-defined settings and can be omitted.

A note on specifying the dimensions of the cube: In some cases, such as separated molecules or surfaces with large amounts of vacuum, the cube dimensions that FHI-aims would silently default to may not be ideal and may result in excessively large files. Presently, the code will stop if the defaults imply cube files with more than 650 MB. Also, many viewers do not implement non-rectangular cube edges, which would result from non-rectangular unit cells by default.

In short: Users are *always* strongly encouraged to specify cube geometries directly.

- `output cube type` This is the only mandatory line, specifying which type of cube file should be produced. FHI-aims presently allows the following options for `type`:

  1. `delta_density` : Writes the difference between the initial (superposition of free atoms) and the final self-consistent density to a file.
  2. `eigenstate` n : Writes the wave function of the $n-th$ eigenstate to a file. n must be an integer number. In periodic calculations, only the real part of the eigenstate is printed. By default, the first spin channel and the first k-point is printed out, see also `cube spinstate` and `cube kpoint`.
  3. `eigenstate_density` n. Writes the electron density of the $n-th$ eigenstate to a file. The eigenstate density is obtained as the square of the wave-function. In periodic calculations, this includes the imaginary part, and it might therefore by more useful to look at the eigenstate density rather than at (the real part of) the wave function. By default, the first spin channel and the first k-point is printed out, see also `cube spinstate`.
  4. `spin_density` : The spin density $n^{\uparrow}(\boldsymbol{r}) - n^{\downarrow}(\boldsymbol{r})$ is written to a file named `spin_density.cube`. Only available for `spin collinear`.
  5. `stm` : Must be followed by a real number V. Calculates 3D tunneling current map (more precisely, the tunneling current is proportional to the printed values) which can be used to plot STM images for a given voltage V (in Volts) in the frame of the Tersoff-Hamann model. This is done by summing up eigenstate densities for all eigenstates between the Fermi level and V (in eV), and the result is multiplied by V. In addition, a file `cube_xxx_stm_z_map.cube` will be printed. It contains values of the z-coordinate at the vertices of the cube, and can be used along with the tunneling current map to color the constant current isosurfaces according to their extent in the z-direction (to mimic the constant current image contrast in STM imaging). The output of stm-cubes works only for periodic systems.
  6. `total_density` : The full electron density is recomputed and written to the a. In case of a periodic calculation, electron density from all unit cells that overlap with the cube output region will be printed.
  7. `long_range_potential` : Prints the long range electrostatic potential of the Ewald summation. This result is useful in regions where no electron density is found and is much faster than the output of the full potential.

8. `hartree_potential`: The whole (i.e, short-range and long-range) electrostatic potential is recomputed on a cube grid and written out. Be careful with keyword `potential`. Please report errors.
9. `xc_potential`: The xc potential is recomputed on a cube grid and written out. **PBE only**.
10. `potential`: Legacy. The whole (i.e, short-range and long-range) electrostatic potential is recomputed on a cube grid and written out. The keyword is considered broken/experimental. Use `hartree_potential`.

Units for densities and eigenstates are $Å^{-3}$ and $Å^{-3/2}$, respectively. The unit for the long range and hartree potential is Hartree [Ha].

- `cube spinstate` *spin*
  This keyword allows to choose whether to print spin-channel 1 or 2. Default:1 An example for how to use the `cube spinstate` keyword to print eigenstates in both spin channels is given below.

- `cube kpoint` *kpoint*
  This keyword allows to choose the k-point to be printed. $kpoint$ is an integer number following the same ordering as the k-points within the SCF-cycle. It is presently not possible to output cube files at a k-point not included in the scf. Keyword `output k_point_list` may be used to print out the entire list of k-points used in the s.c.f. cycle. This will help identify which k-point is printed in a cube file. Default: 1

- `cube state` *spin k-point*
  Deprecated keyword to choose spin and k-point. The `cube spinstate` keyword should be used instead as given in the example below. If nothing else is specified, this keyword defaults to `cube state` $1\ 1$. Note that for cluster calculations, $k-point$ must always be 1.

- `cube filename` *name_of_the_file*
  Allows to customize the name of the cubefile. If this line is not given, FHI-AIMS will default to a file name which contains the number of the cube requested, its type, and, if applicable, the corresponding spin and k-point of the data.

- `cube format` *format*
  Apart from the default cube format, FHI-AIMS also supports output in the formats of the gOpenMol and XCrysden software. This is requested by the line `cube format` $format$. The options for $format$ are cube, gOpenMol, and xsf (the XCrysden format). Default: cube

- `cube divisor` *number*
  This is a technical settings which governs the paralellization of the cube output. The whole cube is divided into smaller, so-called minicubes, which are then treated one after the other. The value governs the number of points in each directed to be used for each minicube, i.e., its size. A larger setting therefore results in less minicubes (and it thus potentially faster), but also a larger demand for memory. Unless there are problems with memory, this setting usually does not need to be

touched, with the exception of output potential, where it should be set to its maximal value (45), independent of the number of processors used. Default: 10

- `cube spinmask` i j
  For total_density cube files, the line `cube spinmask` i j with integer i and j allows to manipulate the spin channels independently according to the following formula: $i \cdot n^{\uparrow}(\boldsymbol{r}) - j \cdot n^{\downarrow}(\boldsymbol{r})$. If $i = 1$ and $j = 1$ (which is the default), the total density will be computed as normal. This keyword replaces the earlier keyword `cube spin` .

- `cube origin` x y z
  Single line which specifies the origin , i.e., the center of the region to be plotted. Values are given in ÅIf omitted, the same origin as for the previous cube file is used. If no origin has been given yet, it defaults to the geometric center of the molecule in the cluster case or (0,0,0) for periodic calculations. For slab type calculations (when using `dipole_correction .true.`), the origin is set to the center of the slab.

- `cube` edge n dx dy dz
  Specifies the edges of the volumetric data to be plotted. Separate lines have to be given for each of the three edges of the cube. In each line, n indicates the number of steps a particular edge (voxel), and dx, dy, dz indicate the length of each individual step [i.e., the full cube edge length is ($n \cdot dx$, $n \cdot dy$, $n \cdot dz$). If omitted, the same edges as for the previous cube file are used. If no edges have been specified yet, FHI-aims defaults (in the cluster case) to orthogonal grids of 0.1 Å length, which span the whole molecule plus 14 Bohr beyond the outermost nucleii. For periodic calculations, the default edges are the same as the lattice vectors, again with 0.1 Å step length. *Note: In some cases, such as separated molecules or surfaces with large amounts of vacuum, the defaults might be far from ideal and result in excessively large files. Presently, the code will stop if the defaults imply cube files with more than 650MB. Users are* always *strongly encouraged to specify cube geometries themselves.*

NOTE:A word of warning here: Historically grown, FHI-aims outputs the density and the wave function in units of $1/\text{Å}^3$, although the cube voxels are in atomic units. This must be accounted for when doing any kind of postprocessing! Although this behaviour might be considered a bug, we decided to leave it this way in order not to break any scripts people are already using. Instead, the keyword `cube_content_unit` was introduced. `cube_content_unit legacy` is the default and gives the output in SI units, while `cube_content_unit bohr` switches to a consistent output in atomic units. (Note that the definition of cube file usually is in bohr, which is why we do not provide a switch to Å).

Example for a spin-polarized non-periodic system:

```
output cube total_density
cube origin 1.59 9.85 12.80
cube edge 101 0.15 0.0 0.0
```

```
cube edge 101 0.0 0.15 0.0
cube edge 101 0.0 0.0 0.15
output cube eigenstate 151
cube spinstate 1
output cube eigenstate 151
cube spinstate 2
output cube eigenstate 152
cube spinstate 1
output cube eigenstate 152
cube spinstate 2
output cube eigenstate 153
cube spinstate 1
output cube eigenstate 153
cube spinstate 2
output cube eigenstate 154
cube spinstate 1
output cube eigenstate 154
cube spinstate 2
```

So this would first request cube output for the total density. See the details above on how to get individual versions of the spin density. Then, the **center** of the cube is specified at (1.59, 9.5, 12.80) Å. Each cube direction has 101 points that are spaced apart by 0.15 Å, giving a total cube edge length of 15 Å in each direction. Finally, output is requested for the Kohn-Sham wave functions associated with eigenstates number 151-154. For each eigenstate, the additional `cube spinstate` *i* line requests first the spin-up channel (i=1), then spin-down (i=2).

NOTE that for periodic systems, the `cube spinstate` lines may have to be followed by `cube kpoint` lines to specify the k-point at which an eigenstate is printed. By default, only k-point number 1 is printed. For unshifted k grids, that is the Γ point.

`overwrite_existing_cube_files` **sub-tag:** `.true./.false.` (control.in)

If set to .false., FHI-aims will check during the output of cube files whether a file with the selected file name already exists. If such a file is found, the file name will be changed by adding a number (1,2,3...) to the end of the file name. This is very useful when relying on default names or when plotting cubes during the SCF cycle. If set to .true. FHI-aims will not check during output whether a file with the selected name already exists. If it does exist, it will be simply overwritten!

`output_cube_nth_iteration` **sub-tag:** `n` (control.in)

Writes all cube files specified in control.in at each nth iteration. n must be an integer. This keyword should be helpful to analyse what is going on during subsequent SCF

cycles. However, the output of cubes is usually quite slow, so choosing this option will slow down the calculation a lot.

---

output sub-tag: density (control.in)

Usage: output density

Purpose: Writes the electron density $n(\boldsymbol{r})$ at each integration grid point $\boldsymbol{r}$ to a file density.dat.

Note that this density output is *not* given on a uniform grid, but simply on the overlapping atom-centered grid used for all internal operations of FHI-aims. For a density on a uniform grid, see the output cube subkeyword.

output density additionally writes the difference between the current density and the superposition-of-free-atom reference density to a file diff-density.dat

---

output sub-tag: dipole (control.in)

Usage: output dipole

Purpose: Calculates and writes the electrical dipole moment of the structure to the FHI-aims standard output as a post-processing step.

This is generally useful, but the dipole moment is particularly needed to compute molecular oscillator strengths for individual vibrational frequencies.

Note that, for charged systems, the electrical dipole is defined with respect to the (possibly arbitrary) origin of the file geometry.in, rather than an origin within the system itself. Thus, charged systems will yield different dipole moments for different choices of origin, but the important dipole *differences* needed to compute, e.g., oscillator strengths via finite diffences remain well-defined.

---

output sub-tag: dos (control.in)

Usage: output dos Estart Eend n_points broadening

Purpose: Writes the density of states (DOS) to an external file for plotting purposes.

Estart : Lower bound of the single-particle energy range for which the DOS is given.
Eend : Upper bound of the single-particle energy range for which the DOS is given.
n_points : Number of energy data points for which the DOS is given.
broadening : Gaussian broadening applied to obtain a smooth density of states based on the peaks produced by individual states.

This keyword shares its syntax with output atom_proj_dos and output species_proj_dos . See also section 4.4 for more details.

Two output files emerge from this option:

- `KS_DOS_total_raw.dat` contains the total DOS components as a function of the eigenvalue energy (first column) as used internally in FHI-aims. The energy zero is then given by the vacuum level (non-periodic systems) or by the $\boldsymbol{G}$=0 component of the long-range Hartree potential (periodic systems).
- `KS_DOS_total.dat` contains the same information, except that the energy zero is shifted to the Fermi energy (metallic systems) or valence band maximum (insulators), respectively.

When followed by the following keyword

`dos_kgrid_factors` n_1 n_2 n_3

where n_1, n_2 and n_3 are integers, the dimensions of the k-point grid along the first, second and third lattice vectors are multiplied by n_1, n_2 and n_3, respectively. New eigenvalues are re-calculated non-selfconsistently on the new denser k-point grid. The new eigenvalues are then used to plot an improved (so-called perturbative) density of states.

If no `dos_kgrid_factors` are specified, the original k-point grid is used.

The unit of output for the density of states is $(eV \cdot V_{\text{unit cell}})^{-1}$, i.e., number of states per energy unit (eV) and unit cell volume.

`output` sub-tag: `eigenvectors` (control.in)

Usage: `output eigenvectors`

Purpose: Writes the actual Kohn-Sham eigenvectors c_{il} into separate files for each spin channel.

Restriction: This functionality is best tested for periodic systems with `KS_method lapack` at present. For periodic systems, it has no effect unless specific band structure output is requested through `output band` .

For *non-periodic* systems, this option causes the Kohn-Sham eigenvectors c_{il} (for basis functions i, eigenstates l) to be written out for each state, whenever the Kohn-Sham eigenvalues are written. However, at present the non-periodic version is mostly of 'debug' character. In particular, `KS_method scalapack` is not supported under all circumstances. Please test carefully.

For *periodic* systems, eigenvector output must be requested together with the `output band` functionality described above. If `output eigenvectors` is set in addition, the Kohn-Sham eigenvectors $c_{il}(\boldsymbol{k})$ will be written into separate files for each spin channel, *only* for each k-point for which band output was requested.

For each state l, the real and imaginary part of $c_{il}(\boldsymbol{k})$ will then be written out for each basis function i. Output file names are assigned automatically based on the number of the output band, and to the specific k-point in that band, to which they pertain, e.g.:

`KS_eigenvectors.band_*.kpt_*.out`

In addition to the eigenvectors themselves, these files also contain as header information:

- the *relative* coordinates of the k-point in question (in units of the reciprocal lattice vectors of the structure in question)
- Information on the identity (atom number and angular momentum) for each basis function
- The Kohn-Sham eigenvalue and occupation number of each state.

The listed eigenvectors pertain to the superimposed, Bloch-like basis functions (with phase factors!) $\chi_{i,k}(\boldsymbol{r})$ as defined through Eq. (22) of the FHI-aims Computer Physics Communications description, Ref. [?].

Note that the `output aitranss` keyword provides one further option to print eigenvectors and other information for any non-periodic system, serial or parallel (scalapack or elpa).

`output` **sub-tag: `elpa_timings`** (control.in)

Usage: `output elpa_timings`

Purpose: Writes timings for the parallel Elpa eigenvalue solver to standard output.

`output` **sub-tag: `grids`** (control.in)

Usage: `output grids`

Purpose: Writes to separate files: (i) the `radial_base` integration grid shells for each species incl. integration weights, and (ii) the full three-dimensional grid point locations incl. integration weights.

`output` **sub-tag: `hessian`** (control.in)

Usage: `output hessian`

Purpose: Writes the initial Hessian approximation for the structure optimizer into the file `hessian.out`.

`output` **sub-tag: `hirshfeld`** (control.in)

Usage: `output hirshfeld`

Purpose: Produces a Hirshfeld analysis of partial charges and moments on each atom.

Restriction: Currently disabled for Hartree-Fock and some hybrid functionals as the underlying spherical free-atom charge density is not produced on a dense 1-dimensional radial grid for these functionals. Support for this can easily be added by commenting out the "stop" line in the source code, except that the Hirshfeld analysis is then based on the DFT-LDA free atom density $n_{\text{at}}^{\text{free}}(r)$.

Defining "atoms-in-molecules" is a classic, intuition based problem; one would like to associate individual (partial) charges or moments with individual atoms in a bonded structure. This process is by necessity not uniquely definable (molecules *are* not atoms, and the are no rigorously defined boundaries between atoms). Nonetheless, much chemical intuition is based on such a concept.

Hirshfeld's [?] "atoms-in-molecules" partitioning relies on the same idea as the partitioning of our charge density for the electrostatic potential (Eq. 3.17), using the free-atom electron density $n_{\text{at}}^{\text{free}}(r)$ as the weight function $g_{\text{at}}(\boldsymbol{r})$ in Eq. (3.13). Since (for a given functional), we know the spherical $n_{\text{at}}^{\text{free}}(r)$ exactly, this analysis remains well-defined even as external circumstances such as the basis set change. However, the resulting values are still qualitative in the sense that Hirshfeld's [?] "atoms-in-molecules" partitioning is just one among many other prescriptions that have been suggested in the literature.

Note that the `output hirshfeld` keyword itself only writes a Hirshfeld analysis for the final geometry of an FHI-aims run, not, for instance, for intermediate molecular dynamics steps. This ensures that the Hirshfeld analysis does not accidentally clutter an output file with large amounts of data if the `output_level MD_light` output level is set. Output for every geometry can be accomplished with the `output hirshfeld_always` keyword below.

`output` **sub-tag:** `hirshfeld_always` (control.in)

Usage: `output hirshfeld_always`

Purpose: Writes out a Hirshfeld analysis at every geometry step of a run.

This keyword ensures that a Hirshfeld analysis is written at every geometry step of a FHI-aims run, not just after the final step. If `output hirshfeld-I` is requested together with `output hirshfeld_always` , results from both the normal and the iterative Hirshfeld analysis are written at every step.

`output` **sub-tag:** `hirshfeld-I` (control.in)

Usage: `output hirshfeld-I`

Purpose: Produces an "iterative Hirshfeld" analysis of partial charges and moments on each atom.

Similar functionality to the normal output hirshfeld Hirshfeld analysis – see the comments for that keyword – except that in the "iterative Hirshfeld" [?] analysis the partition weights are changed. Here, the partitioning densities are not those of neutral atoms but rather those of ions with the same formal charge as the formal charge determined by the "iterative Hirshfeld" analysis.

This keyword is implemented but has not seen much production use. It is therefore not guaranteed that it will always work or that the results will always make sense or even be in line with the original "iterative Hirshfeld" publication.[?] All may be well, but if you do use the functionality, please check very carefully that the results appear to be correct.

---

output sub-tag: k_eigenvalue (control.in)

Usage: output k_eigenvalue number

Purpose: For periodic structures, determines for how many k points FHI-aims will write the electronic eigenvalues $\epsilon_l(\boldsymbol{k})$.

number is an integer number. Default: 1.

Eigenvalues will only be written for the first number k-points by default. For dense $\boldsymbol{k}$-grids, the sheer number of k-points simply gets too large to allow for a full output.

---

output sub-tag: k_point_list (control.in)

Usage: output k_point_list

Purpose: For periodic geometries only, this option writes a complete listing of the reciprocal space coordinates of all k-points in the calculation to the standard output file.

The k-point coordinates are written in units of the reciprocal lattice vectors of the structure. This option is also the default when using output_level full .

---

output sub-tag: h_s_matrices (control.in)

Usage: output h_s_matrices

Purpose: Writes the Hamiltonian and overlap matrices s_{ij} and h_{ij} into separate files. The output format has one line per matrix entry. On this line the first column is the row index of the entry, the second column the column index of the entry and the third column is the value of the entry. Only the upper triangle and the diagonal of the symmetric matrix is written.

This functionality behaves slightly differently in periodic vs. cluster calculations. In the cluster case s_{ij} and h_{ij} are written out in keeping with the packed_matrix_format . In the periodic case only the Gamma-point Hamiltonian is written out and only as a dense matrix regardless of packed_matrix_format settings.

---

output sub-tag: matrices_2005 (control.in)

Usage: output matrices_2005

Purpose: Writes the Hamiltonian and overlap matrices s_{ij} and h_{ij} into separate files. The output format is the legacy format where the entries of the matrix are written out in five-by-five blocks.

Restriction: This functionality is unavailable for periodic systems, or if a packed_matrix_format is used.

---

output sub-tag: matrices_parallel (control.in)

Usage: `output matrices_parallel types [format]`

Purpose: Writes Scalapack/Elpa matrices into separate files.

`types` is a string that determines the type of matrices that are written.

`format` (optional) is a string that determines the format of the output.

Depending on option "types", the upper part of up to three different matrices is written into separate files. If `types` is "`n`" (without quotes), no matrices are written. If `types` is set to "`h`", then the Hamiltonian is written. The choice "`o`" causes the overlap matrix to be output. Finally, "`s`" refers to the system matrix from which the eigenvalues are calculated. The last three options can be combined. For example, "`ho`" means Hamiltonian and overlap.

The format of the files can be controlled with the optional parameter `format`. Possible values are "`asc`" for ASCII output (default) and "`bin`" for binary output.

---

output sub-tag: mulliken (control.in)

Usage: output mulliken

Purpose: Produces a Mulliken analysis of the occupation of each atom and its angular momentum channels in terms of the basis used.

Restriction: This option is available as a per-atom summary only if ScaLapack is used.

Defining "atoms-in-molecules" is a classic, intuition based problem; one would like to associate individual (partial) charges or moments with individual atoms in a bonded structure. This process is by necessity not uniquely definable (molecules *are* not atoms, and the are no rigorously defined boundaries between atoms). Nonetheless, much chemical intuition is based on such a concept.

A classic "atoms-in-molecules" concept is the Mulliken analysis [?], which defines electronic occupations of atoms by projected occupations into the localized basis functions associated with them (see the standard literature for exact definitions and use).

In short, when so requested, FHI-aims will provide a decomposition of the electronic density per atom, angular momentum channel, and possibly spin channel, allowing to deduce approximate partial charges. The summarized Mulliken analysis is written into the standard output stream, while a separate file `Mulliken.out` contains detailed state-by-state projected electron occupations.

Note that a Mulliken analysis is somewhat ill-defined because of basis function overlap; thus electrons can be counted to one atom or another at will. For small basis sets, a Mulliken analysis may still yield qualitatively reasonable numbers, but it becomes increasingly ill-defined as the atom-centered basis sets approach basis set completeness.

`output` **sub-tag:** `nuclear_potential_matrix` (control.in)

Usage: `output nuclear_potential_matrix`

Purpose: Writes the matrix elements of only the bare electron-nuclear potential in the current basis sto to a file

Restriction: This functionality is unavailable for periodic systems, or if a `packed_matrix_format` is used.

This can be useful if the FHI-aims basis functions are needed for a further, separate purpose (Quantum Monte Carlo etc) but please note that the integration accuracy for the Coulomb singularity near the nuclei must be higher than in our standard calculations (where the singularity is cancelled by the kinetic energy), so increasing `radial_multiplier` is in order.

`output` **sub-tag:** `onsite_integrands` (control.in)

Usage: `output onsite_integrands`

Purpose: Writes out onsite integrands for all radial functions on the code's internal 'radial' and 'logarithmic' grids.

Since August 2013, FHI-aims verifies the accuracy of its 'radial' integration grid (the sparse grid of atom-centered radial shells around each atom which is part of the definition of its three-dimensional, overlapping atom-centered integration grids) in comparison to integrals on the dense 'logarithmic' grid which is used to set up the spherical free atom, all radial functions etc. in one dimension.

These integrals take the form

$$\int d^3r \phi_i(\boldsymbol{r}) \hat{H} \phi_i(\boldsymbol{r}) = \int dr \, [f(r)] \times \text{angular integral.} \tag{3.82}$$

With our usual definition of basis functions,

$$\phi_i(r) = \frac{u_i(r)}{r} Y_{lm}(\Omega) \quad , \tag{3.83}$$

we get:

$$f(r) = u_i(r) \cdot \left[-\frac{1}{2} u_i''(r) + \frac{1}{2} \frac{l(l+1)}{r^2} u_i(r) + v(r) u_i(r) \right] \tag{3.84}$$

in the non-relativistic case. In the case of scaled ZORA or atomic ZORA scalar relativity, the kinetic energy part is modified and the integrand reads:

$$f(r) = u_i(r) \cdot \left[\frac{2c^2}{2c^2 - v(r)} \cdot \left(-\frac{1}{2} u_i''(r) + \frac{1}{2} \frac{l(l+1)}{r^2} u_i(r) \right) - \frac{c^2}{(2c^2 - v(r))^2} \cdot v'(r) \cdot \left(u_i'(r) - \frac{u_i(r)}{r} \right) + v(r) u_i(r) \right] . \tag{3.85}$$

These are the integrands to test both the 'logarithmic' grid and the 'radial' grid around each atom, where $v(r)$ is set to be the one-dimensional potential of a spherical free atom as calculated at the outset of each run.

If `output onsite_integrands` is set to be true, the actual integrands $f(r)$ and various of their parts are printed for each radial function. This is mainly useful for debugging purposes, to understand what we are integrating for a given basis function choice. Especially for contracted Gaussian basis functions, $f(r)$ can look quite ugly near the nucleus.

The files that contain the actual integrand $f(r)$ defined above are called `Onsite_r2_phi_h_phi_log.(function).dat` and `Onsite_r2_phi_h_phi_rad.(function).dat` for the logarithmic and radial grids, respectively, with "(function)" indicating the element and the radial function number in the order used by FHI-aims (for instance, the `output basis` keyword uses the same order to output the radial functions used in the code). Units are Å for the radial coordinate, but atomic units (Ha/bohr^3) for the integrand itself.

`output` **sub-tag:** `overlap_matrix` (control.in)

Usage: `output overlap_matrix`

Purpose: Writes out the k-point dependent complex overlap matrices for a periodic system for those k-points for which band structure output was requested.

Restriction: For periodic systems only, and scalapack is not supported. Specific band structure output must be requested through `output band` .

For *periodic* systems, this option allows to write out the k-dependent (Bloch) overlap matrices that correspond to the set of k-points requested with the `output band` keyword. If `output eigenvectors` is set in addition, the Kohn-Sham eigenvectors $c_{il}(\boldsymbol{k})$ will be written into separate files for each spin channel, *only* for each k-point for which band output was requested.

`output` **sub-tag:** `hamiltonian_matrix` (control.in)

Usage: output hamiltonian_matrix

Purpose: Writes out the k-point dependent complex hamiltonian matrix for a periodic system for those k-points for which band structure output was requested.

Restriction: For periodic systems only, and scalapack is not supported. Specific band structure output must be requested through output band .

For *periodic* systems, this option allows to write out the k-dependent (Bloch) hamiltonian matrices that correspond to the set of k-points requested with the output band keyword. Both spin channels are written into the same file.

---

output **sub-tag:** `ovlp_spectrum` (control.in)

Usage: output ovlp_spectrum

Purpose: Writes the non-singular part of the eigenvalue spectrum of the overlap matrix to the FHI-aims standard output.

Restriction: Works only for the cluster case, and only for KS_method `lapack`.

This option can help show if (or if not) the chosen basis set for the full system is close to ill-conditioning (see the comments for keyword basis_threshold).

---

output **sub-tag:** `postscf_eigenvalues` (control.in)

Usage: output postscf_eigenvalues

Purpose: For periodic systems, writes all Kohn-Sham eigenvalues on a potentially dense k-space grid to an ASCII file 'Final_KS_eigenvalues.dat'.

Restriction: Works only for periodic systems. Does not work when keyword use_local_index is set.

If this keyword is set, the eigenvalues and occupation numbers for a periodic system are recomputed and written to a file 'Final_KS_eigenvalues.dat' after the s.c.f. calculation (and, possibly, relaxation, dynamics etc.) is complete. A denser k-space grid than during the original s.c.f. calculation can be specified using the dos_kgrid_factors keyword.

Note that the resulting output file can become very large. See the header of the 'Final_KS_eigenvalues.dat' for details and for units used.

Note that this additional calculation is done using serial lapack solutions of the eigenvalue problems for individual k-points on individual CPU cores. This always works but will create memory problems as the system size increases, simply because local copies of all matrices are kept on single CPUs. For large systems, our usual, more sophisticated parallelization strategies have not yet been copied over to this routine.

---

output **sub-tag:** `quadrupole` (control.in)

Usage: `output quadrupole`

Purpose: Calculates and writes the electrical quadrupole moment of the structure to the FHI-aims standard output as a post-processing step.

`output` sub-tag: `rho_multipole` (control.in)

Usage: `output rho_multipole`

Purpose: Writes the partitioned atom-centered charge multipole components $\delta\tilde{n}_{\mathrm{at},lm}(r)$ to individual files for each atom, l, and m (see Eq. 3.17).

`output` sub-tag: `species_proj_dos` (control.in)

Usage: `output species_proj_dos Estart Eend n_points broadening`

Purpose: Writes an projected, angular-momentum resolved partial density of states (pDOS) averaged over all atoms of each `species` .

`Estart` : Lower bound of the single-particle energy range for which the pDOS are given.
`Eend` : Upper bound of the single-particle energy range for which the pDOS are given.
`n_points` : Number of energy data points for which the pDOS are given.
`broadening` : Gaussian broadening applied to obtain a smooth partial density of states based on the peaks produced by individual states.

This option is based on a Mulliken Analysis and shares its syntax with `output dos` and `output atom_proj_dos` . See also section 4.4 for more details.

Different from the `atom_proj_dos` option, this option writes its output averaged over all atoms of each `species` defined in `control.in` and used in `geometry.in`. This provides a quick handle to obtain averaged pDOS's for well-defined subgroups of individual atoms, e.g., those of a given layer of a slab, by simply defining a separate `species` for the desired atoms in `control.in`.

There are two types of output files for each atom:

- *species* `_l_proj_dos_raw.dat`, where *species* denotes the `species` name used in `geometry.in` and `control.in`. This file contains the total and angular-momentum resolved DOS components as a function of eigenvalue energy (first column) as used internally in FHI-aims. The energy zero is then given by the vacuum level (non-periodic systems) or by the $\boldsymbol{G}$=0 component of the long-range Hartree potential (periodic systems).
- *species* `_l_proj_dos.dat`, which gives the same information, except that the energy zero is shifted to the Fermi energy (metallic systems) or valence band maximum (insulators), respectively.

Note that projected densities of states such as given here must be based on some kind of projection orbitals, the choice of which is somewhat arbitrary by necessity. This is thus a tool for *qualitative* analyses.

In FHI-aims, we project directly on the atom-centered angular-momentum components as defined by the *overlapping* basis set. This definition becomes the more arbitrary te larger the basis set, just like a `mulliken` analysis. The closer the full basis comes to completeness, the more ambiguous will a `mulliken` -like analysis become, since it may not be *a priori* clear which electrons should be counted towards the basis functions of one atom vs. those of another atom. Thus, do *not* expect a pDOS to simply converge as the basis set size is increased; use it as a qualitative indicator of trends, but nothing more.

`output` **sub-tag: `v_eff`** (control.in)

Usage: `output v_eff`

Purpose: Writes the local effective potential $v_{\text{eff}}(\boldsymbol{r})$ at each integration grid point $\boldsymbol{r}$ to a file `v_eff.dat`.

Note that the meaning of this effective potential depends on the `xc` option used. For DFT-LDA, this is simply the full local potential. For gradient-corrected (GGA) functionals, the gradient partial derivatives of the exchange-correlation functional are *not* included in the potential, as they are treated separately by integration by parts (see Ref. [?]). For hybrid functionals or Hartree-Fock, the exchange part of the potential is of course not included.

Note also that this output does *not* happen on a uniform grid. For further processing, a proper visualization tool is needed, and/or an interpolation onto a uniform grid must be done.

`output` **sub-tag: `v_hartree`** (control.in)

Usage: `output v_hartree`

Purpose: Writes the electrostatic (Hartree) potential multipole components $\delta\tilde{v}_{\text{at},lm}(r)$ to individual files for each atom, l, and m.

`output` **sub-tag: `zero_multipoles`** (control.in)

Usage: `output zero_multipoles`

Purpose: Prints out the partial charges associated with the multipole electrostatic potential of each atoms in each s.c.f. iteration.

The resulting values are "atoms-in-molecules" like partial charges assigned to each atom, similar to a `hirshfeld` partitioning (but not identical because a different partitioning function may be used as the `hartree_partition_type`).

3.30 Deprecated keywords

The following section lists a number of keywords in FHI-aims which exist, but which may go away in future versions of FHI-aims. In some cases, this is because the relevant modifications proved successful, and there is no sense in maintaining some old, obsolete extra functionality without any use in production settings. In other cases, the relevant keywords were experiments that did not yield the anticipated success, and / or functionality that may be superseded in a different, more comprehensive way in the future.

Tags for general section of `control.in`:

Tag: `Adams_Moulton_integrator` (control.in)

Usage: `Adams_Moulton_integrator flag`

Purpose: Allows to switch between a simple integrator and the higher-order Adams-Moulton integration scheme to determine the Hartree potential components from classical electrostatics.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

Tag: `batch_distribution_method` (control.in)

Usage: `batch_distribution_method method`

Purpose: Parallel distribution of integration grid batches *only* in the case that the external qhull and METIS libraries are configured.

`method` is a string, the only possible value being `qhull+metis` at this point.

Outsources the distribution of integration grid batches to the external qhull and METIS libraries. Only relevant if these libraries are compiled into the code. However, the associated `grid_partitioning_method` s are less useful than the default `maxmin` algorithm, and the internal work distribution method of FHI-aims usually performs rather well. Therefore, this option is deprecated and kept only for experimental purposes, for now.

Tag: `communication_type` (control.in)

Usage: `communication_type type`

Purpose: Determines the type of calculation / storage of per-atom spline arrays of the Hartree potential for a parallel run.

`type` is a string, see below. Default: `calc_hartree` .

In a parallel run of FHI-aims, each processor holds a certain part of the real-space integration grid, which in turn are each touched by all atom-centered multipole components (splined) of the real-space Hartree potential. So, to construct the electrostatic (Hartree) potential on each grid point, an array of splined atom-centered multipole components $\delta\tilde{v}_{\mathrm{at},lm}(r)$ must be available on every MPI sub-process (see Sec. 3.7 for details regarding the electrostatic potential). The memory use to store these components grows rapidly and with a large prefactor with system size. Thus, keeping a local copy of all the splined multipole components of the Hartree potential on each CPU is not advisable.

The actual handling of these components is instead controlled by keyword `communication_type` . The following choices for `option` are possible:
`calc_hartree` : The default, and usually very efficient. Hartree potential components for each atom are integrated on the fly on each CPU when needed (usually less CPU

time than communication time).
shmem : If compiled with shared memory support (see Section 1.6), each compute node of a parallel run keeps the components of the Hartree potential in a separate shared memory segment, only internode communication is needed. Performance test show hardly any benefet over calc_hartree. Note that the (legacy) keyword distributed_spline_storage should be false for shmem, at least.

Tag: distribute_leftover_charge (control.in)

Usage: distribute_leftover_charge .true./.false.
This keyword is superseded by the compensate_multipole_errors keyword which solves the problem of small residual charge integration errors in slab calculations much better than distribute_leftover_charge .

Keyword distribute_leftover_charge could introduce noticeable total energy inaccuracies especially for "light" integration grid settings.

Tag: force_new_functional (control.in)

Usage: force_new_functional flag

Purpose: For test purposes, allows to switch to an energy functional form that treats the electronic and nuclear electrostatic energy terms separately.

flag is a logical string, either .false. or .true. Default: .true.

See Ref. [?] regarding the correct shape of the energy functional that treats the nuclear and electronic parts of the electrostatic energy together on a per-atom basis.

Tag: force_smooth_cutoff (control.in)

Usage: force_smooth_cutoff tolerance

Purpose: Optionally, enforces smoothness of all basis functions near the cutoff radius.

tolerance is a small positive real number. Default: No check.

If requested, keyword force_smooth_cutoff ensures that the radial function $u(r)$ and its first and second derivatives remain below tolerance at the outermost point of the logarithmic grid where any of them is non-zero at all. The code stops if the onset of the radial function is too abrupt.

It would be a good idea to switch this option on if reducing the width parameter of keyword cut_pot to a very low value (say, below 1 Å).

Tag: grouping_factor (control.in)

Usage: grouping_factor factor

Purpose: Grouping factor for the (experimental, and not recommended!) group grid_partitioning_method .

factor is an integer number, describing how close-by grid points are grouped together. Default: 2.

This keyword is retained for experimental purposes only, for the moment. Since the related grid_partitioning_method group was a proof-of-concept to show that the default maxmin performs better, this keyword is now deprecated. See Ref. [?] for more details if interested.

Tag: hartree_worksize (control.in)

Usage: hartree_worksize megabytes

Purpose: Limits the size of workspace arrays used to construct the Hartree potential on each CPU.

megabytes : The maximum allowed work space size to construct the Hartree potential, in megabytes. Default: 200 MB.

Several large work space arrays across the integration grid are used in the construction of the Hartree potential. Their size can grow quite large, especially when forces are computed for large structures (then, three arrays per atom are required for all atoms).

FHI-aims can circumvent this by computing the final output (integrated energies and forces) in "chunks" of the whole integration grid, limiting the work space used for each chunk. This modification is especially important on low-memory-per-processor architectures such as the BlueGene.

Tag: KH_post_correction (control.in)

Usage: KH_post_correction flag

Purpose: *Under construction – do not use* A way to replace the scaled ZORA post-processing correction for scalar relativity by a Koelling-Harmon type scalar-relativistic correction.

flag is a logical string, either .false. or .true. Default: .false.

This keyword is no longer supported, do not use it. It will be superseded by an improved handling of scalar relativity during the s.c.f. cycle in the future.

Tag: mixer_swap_boundary (control.in)

Usage: mixer_swap_boundary bytes

Purpose: Ignored; never swap. Used to allow to swap the stored density components for Pulay mixing to disk if they exceed a certain memory boundary.

On few-CPU systems and for mid-sized systems (several hundred atoms), the stored

electron density components from past iterations are a large part of the memory used. If this becomes a bottleneck, the stored Pulay arrays can in principle be swapped to disk, instead, to be read only during Pulay mixing.

If anyone has a strong need for this currently unsupported feature, please contact us.

Tag: `multiplicity` (control.in)

Usage: `multiplicity value`

Purpose: If set, specifies the multiplicity of the system.

Restriction: Currently available for non-periodic geometries only. Use `fixed_spin_moment` instead.

`value` : integer number, sets the overall multiplicity as $2S + 1$.

Meaningful only in the spin-polarized case (`spin collinear` in `control.in`). On a technical level, this is a special case of the more general, locally spin-constrained DFT formalism available within FHI-aims (see Sec. 3.14). Note that the underlying `constraint_electrons` keyword can be used to enforce a *non-integer* fixed spin moment, in addition to allowing to fix electron or spin numbers in given subsystems

Also, be sure to check what the Kohn-Sham eigenvalues mean if you need them, do not use them blindly. `multiplicity` shifts the eigenvalues!

Tag: `occupation_thr` (control.in)

Usage: `occupation_thr value`

Purpose: Any occupation numbers below `value` will be treated as zero. `value` is a small positive real number. Default: 0.d0 .

Tag: `recompute_batches_in_relaxation` (control.in)

Usage: `recompute_batches_in_relaxation flag`

Purpose: Allows to switch off the redistribution of atom-centered grid points into new integration batches after a relaxaton step.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

For practical purposes, the integration grid should always be repartitioned after a relaxation step; the associated overhead is low, and the shape of the batches will remain optimal in the face of individual points that "move" with different atoms.

Tag: `squeeze_memory` (control.in)

Usage: `squeeze_memory flag`

Purpose: Used to allow one combined workspace for three different purposes.

This option is no longer necessary due to optimizations by Rainer Johanni. `flag` is a logical string, either `.false.` or `.true.` Default: `.false.`

Tag: `use_angular_division` (control.in)

Usage: `use_angular_division flag`

Purpose: If radial grid shells are used as integration batches, allows to switch off their subdivision into "octant" batches.

`flag` is a logical string, either `.false.` or `.true.` Default: `.true.`

This flag currently only applies to the initialization iteration, in case that self-adapting angular grids are used (not recommended; see keyword `angular_acc` if interested). Even then, switching off the subdivision of radial shells can only decrease the performance.

Subtags for *species* tag in `control.in`:

`species` **sub-tag:** `cut_core` (control.in)

Usage: `cut_core` `type` [`radius`]

Purpose: Can be used to define a separate (tighter) onset of the cutoff potential for all `core` radial functions.

`type` : A string, either `finite` or `infinite`. Default: `finite` .
`radius` : A real number, in Å: Onset radius for the cutoff potential, as defined in the `cut_pot` tag. Default: same as `onset` in `cut_pot` .

Deprecated flag because `basis_dep_cutoff` should supersede this functionality in a more organized way. Having a separate, tighter cutoff for core radial functions sounds like a good idea, but core radial functions are already rather localized anyway. Our experience is that the separate core setting either does not matter for CPU time, or already introduces noticeable total energy changes when it matters.

Chapter 4

Running FHI-aims: Guides to specific tasks

The complete specification of all keywords available in FHI-aims and their capabilities was given in the previous chapter. The present chapter attempts to illustrate these capabilities in a tutorial way. The purpose of this exercise is threefold: (i) to provide clear starting points for the most common types of production calculations; (ii) to provide clear examples that can be modified (but need not be completely reinvented) for the many further capabilitues of FHI-aims; (iii) to describe specific "higher-level" tasks that bind together multiple FHI-aims calculations by way of scripts (e.g., vibrations, or nudged-elastic band calculations to find transition barriers).

4.1 Ground state DFT: Total energies and relaxation

As a first and basic example, we discuss how to set up a simple DFT total-energy calculation for a given structure in FHI-aims. We here expand on the example provided as a testrun: The relaxation of a H_2O molecule towards its minimum-energy structure. The relevant input files `geometry.in` and `control.in` are included in the directory *testcases/H2O-relaxation* (see Chapter 1).

The starting geometry here is badly distorted H_2O, with an initial bond angle of 90°. For any relaxation runs, we strongly recommend a two-step procedure: First, pre-relax the structure with *light* settings down to (say) 10^{-2} eV/Å or so, and only then follow up with a post-relaxation run using *tight* settings or anything else. *light* calculations may easily be cheaper by a factor of 5-10 than *tight* ones, and going down a relaxation trajectory of any length can then be a trememdous waste of computational resources.

The test case below only includes the quick but safe prerelaxation step, leading to an improved geometry that is the optimum using *light* settings. Based on this resulting geometry, the postrelaxation step with *tight* settings should be simple follow-up exercise.

Input files

Turning first to `geometry.in`, we see that the basic geometry input for FHI-aims is very simple in most cases: `atom` lines that contain nuclear coordinates in Å, together with the appropriate `species` designation (in this case, H and O). For a spin-polarized calculation, one might additionally want to specify initial spin moments for selected atoms using the `initial_moment` keyword, in order to define a good initial spin density guess for the s.c.f. procedure.

The input file `control.in` contains all necessary computational information regarding the desired run. Most importantly, the `xc` keyword is required to specify the exchange-correlation functional; FHI-aims will not proceed without this information. The further "physical" specifications – `spin`, `relativistic`, and `charge` – are all at their default settings (no spin-polarization, no relativity, and no charge), but are listed explicitly to make them visible at a quick glance. This is especially important for the `relativistic` keyword, where the `none` setting would not be justified for heavier elements (see Sec. 4.2).

The next few keyword concern the technicalities of obtaining self-consistency: a low `occupation_type` broadening by 0.01 eV of occupation numbers around the Fermi level (this has no physical impact in a molecule with a HOMO-LUMO gap), specifications for the `pulay` `mixer`, and convergence criteria for the s.c.f. cycle. The criteria referring to energies and the density are here set tightly, in order to have forces that are already mostly converged when the (more expensive) force computation is first done.

The last general setting, `relax_geometry`, specifies a geometry relaxation using the `bfgs` algorithm, together with a standard convergence criterion for the forces in the final geometry: No force component for any atom of the relaxed structure should exceed 10^{-2} eV/Å. This criterion may well be set tighter for sensitive cases, such as a

starting geometry to obtain vibrational frequencies, but not orders of magnitude tighter (for example, a setting of 10^{-4} eV/Å might end up just probing some residual numerical noise, for example from the finite integration grids, with no noticeable geometry or total energy changes resulting at all).

The version of the BFGS optimization algorithm used here is a trust-radius enhanced version (as compared to a straight, textbook-like BFGS implementation which could alternatively be used, see the description of the `relax_geometry` keyword). By default, the convergence of the relaxation is additionally sped up by an intelligent guess for the Hessian matrix used in the initial BFGS step. This is done by way of a slightly modified version of the general purpose model matrix proposed by Lindh and coworkers [?], see keyword `init_hess` .

In `control.in`, it remains to set the `species` information for H and O, the elements included in `control.in`. Normally, these settings should be obtained by copy-pasting the relevant information from the *species_defaults* directory, for example the choices in the *light* or *tight* subdirectory located there. Once this is done, the `species` defaults may still be adapted for the purpose in question.

Output stream

We next analyze some significant parts of the standard output produced by FHI-aims, also provided in the file *H2O.reference.out*. We emphasize that this output is kept as human-readable as possible; it pays to actually *look* into the output, especially when something does not appear to have gone correct. Often, a simple warning in in the initial input section or elsewhere in the file may already tell you what is going on. Warnings can also be identified by "grepping" for asterisks in the file.

The standard output stream is structured as follows:

- A summary of the setup (nodes used, required fixed dimensions, information in `control.in` and `geometry.in`) and, importantly, default values inserted for parameters that were *not* explicitly specified in `control.in`.
- Preparation of fixed parts of the calculation – most importantly, information regarding the setup of the per-species basis
- Initialization – information on the setup of all three-dimensional integrations, and solution of the first Kohn-Sham eigenvalue problem for the initial superposition-of-free-atoms electron density.
- Process and total energy information for each successive s.c.f. iteration.
- Upon convergence of the s.c.f. cycle, the Kohn-Sham eigenvalues are also included, as well as final total energies and forces in a long format for reading by external utilities (scripts etc.)
- This is followed by information on the geometry optimization, up to the coordinates predicted for the next step, based on the converged forces obtained from the previous iteration.

- Reinitialization, s.c.f., and geometry optimization information is repeated for each successive geometry step until the optimum geometry is found within the force tolerance specified by `relax_geometry` .

Key information that should be paid attention to is the following:

Eigenvalue and total energy information:
This should be checked for roughly realistic total energy values, a reasonable eigenvalue spectrum: Make sure that the expected core states, the correct number of eigenstates, and possibly the expected HOMO-LUMO gap are all in place.

As in all other iterations, *three* variants of the total energy are given (here quoted from the *initialization* step, just prior to the start of the s.c.f. cycle for the initial geometry):

```
| Total energy                  :         -76.44577662 Ha       -2080.19542099 eV
| Total energy, T -> 0          :         -76.44577662 Ha       -2080.19542099 eV
| Free energy                   :         -76.44577662 Ha       -2080.19542099 eV
```

For systems with a HOMO-LUMO gap, these values should all be the same, but for systems with fractional occupation numbers (metallic systems, large `occupation_type` setting, or degenerate levels at the Fermi level), this is not generally the case.

For molecular systems, the meaning of fractional occupation numbers is questionable, and we recommend to always rely on the first value, the "Total energy".

For metallic systems, fractional occupation numbers are unavoidable. However, it is possible to estimate the total energy for zero occupation broadening ("Total energy, $T -> 0$") based on an entropy expression associated with the fractional occupation numbers. This estimate is based on a free electron gas argument; do not trust it for atoms or small molecules. However, bulk metals or metallic surfaces benefit from this correction, and their total energy may be estimated from the extrapolated total energy ("Total energy, $T -> 0$").

The "Free Energy" simply sums the total energy together with the full entropy correction from the fractional occupation numbers. This quantity has no physical meaning, but it is strictly *this* free energy to which our calculated forces correspond. The reason is that fractional occupation numbers may carry an implicit derivative of their own with respect to the nuclear coordinates. This would enter the total energy, but this derivative is not available in simple analytical terms in FHI-aims.

Geometry information:
After each relaxation step, the convergence of the present geometry is checked (by monitoring the maximum remaining force component on any atom in the structure), and the updated geometry to be treated next is printed. For instance, the final (converged) geometry can be found near the end, in a format that is directly suitable for a follow-up run:

```
------------------------------------------------------------
  Final atomic structure:
                          x [A]             y [A]             z [A]
```

```
            atom          0.00000000       -0.07326360        0.00000000  O
            atom          0.76740642       -0.67036820        0.00000000  H
            atom         -0.76740642       -0.67036820        0.00000000  H
------------------------------------------------------------
```

By default, a version of this information is also written into a file `geometry.in.next_step`, which can be used to continue a relaxation run (see below).

Timing information:
Finally, we note that FHI-aims also provides detailed timing information for each s.c.f. iteration, and as a summary at the end of each run. For example, the provided test run reads similar to this:

```
------------------------------------------------------------
          Leaving FHI-aims.
          Date     :  20110113, Time     :  160442.031

          Computational steps:
          | Number of self-consistency cycles          :           39
          | Number of relaxation steps                 :            4

          Detailed time accounting                     :  max(cpu_time)    wall_clock(cpu1)
          | Total time                                 :        4.028 s          10.275 s
          | Preparation time                           :        0.067 s           3.245 s
          | Grid partitioning                          :        0.097 s           0.214 s
          | Preloading free-atom quantities on grid    :        0.046 s           0.108 s
          | Free-atom superposition energy             :        0.022 s           0.048 s
          | Total time for integrations                :        1.412 s           1.946 s
          | Total time for solution of K.-S. equations :        0.147 s           0.180 s
          | Total time for EV reorthonormalization     :        0.007 s           0.011 s
          | Total time for density & force components  :        1.563 s           2.216 s
          | Total time for mixing                      :        0.068 s           0.265 s
          | Total time for Hartree multipole update    :        0.142 s           0.174 s
          | Total time for Hartree multipole sum       :        0.558 s           1.576 s
          | Total time for total energy evaluation     :        0.013 s           0.008 s

          Have a nice day.
------------------------------------------------------------
```

The date and time at the end are in the `ddmmyyyy` and `hhmmss.mmm` formats of a wall-clock time, *not* in seconds; i.e., the above calculation did *not* take 222254 s, but rather *ended* at 16:04:42 h, one fine January 13.

In addition, detailed timing is provided both for as elapsed CPU time (on individual CPUs during the run), and actual elapsed wall clock time. One interesting detail to note is that, in the present example, there is a large difference between the reported CPU times and the wall clock times. In a normal production run, such significant discrepancies should *not* occur, and could indicate serious problems with load balancing or communication in a parallel run. For example, the run quoted above was run from within a virtual machine

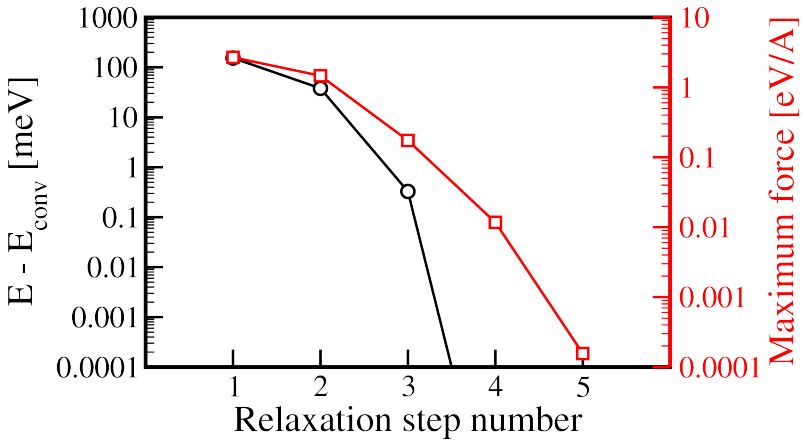


Figure 4.1: Total energy convergence (left y axis) and force convergence (right y axis) during the relaxation test run of H_2O. A total of five steps is taken (initial geometry plus four relaxation steps). The total energy of the final step is taken as the reference here, and the total energy in the second-to-last step is already identical within the numerical accuracy of the calculation. The final step is only taken to bring the residual forces (total energy gradients) to practically zero as well.

on a laptop computer whose host operating system had other things to do. As a result, a large amount of real (wall clock) time was spent not doing any computations. On a dedicated production machine, this should not happen.

Further analysis

Some statistics on the complete relaxation run can be obtained using the script *get_relaxation_info.pl*, located in the *utilities* subdirectory of the FHI-aims distribution. This script is invoked as

```
> get_relaxation_info.pl < H2O.reference.out > statistics.dat
```

or with any other output file. The script searches the FHI-aims output file for total energies (no entropy correction) and maximum force components at the end of each relaxation step. The development of the total energy, total energy difference with respect to the starting geometry, and maximum force component can then be visualized as a function of the relaxation step number, using standard tools such as xmgrace. Fig. 4.1 visualizes the progress of the relaxation run based on the data obtained from *get_relaxation_info.pl*.

Likewise, much other information can and should be extracted from the standard output using similar scripts. For example, the development of the geometry can be visualized in the format of a `.xyz` file using the *create_xyz_movie.pl* script, using standard visualization tools such as `jmol` or `vmd`.

To continue a relaxation run (for instance, for a post-relaxation step with tight settings, see below), use the file `geometry.in.next_step` that is created by default during a relaxation. It contains the atomic structure of the current relaxation step, as well as

the current estimate of the Hessian matrix as created by the `bfgs` relaxation algorithm. This output can be fine-tuned (or prevented) using the `write_restart_geometry` and `hessian_to_restart_geometry` keywords.

Obviously, all intermediate geometries (but not the Hessian) are also part of the standard output stream by default, in the right format for a restart.

Next steps

As with any electronic structure code, **monitoring the convergence of the basis set for each element is an important user task**, to make sure any physical conclusions are accurate. For instance, the present example could be continued as follows:

- A "prerelaxation" such as the present test run should not employ a huge basis set, simply for efficiency's sake. The *light* settings used for the present elements use a *tier* 1 basis set. Very often, geometries obtained with *light* are already rather close to converged.
- For these same elements, you will find that the *tight* settings actually employ *tier* 2, which is significantly larger and very close to basis set convergence at least for DFT methods.
- Obviously, one might extend this to *tier* 3 or higher for test purposes, possibly even based on *really_tight* settings otherwise. Comparing the changes made between *light*, *tight*, and *really_tight* settings, and different basis sets, is an interesting exercise. For the H_2O molecule shown here, it should hopefully reveal that there is not much to be gained beyond what is prescribed as "*tight*".

Finally, we note that "*tier* 1" does not necessarily mean the same level of convergence accuracy for all elements. For light elements, *tier* 2 may often be needed, and we set them by default in our *tight* settings. For significantly heavier elements (transition metals in particular), *tier* 1 is already well converged for ground-state DFT calculations, which is therefore mostly the default in our *tight* settings.

4.2 Heavy elements ($Z \gtrsim$30): Modifications for scalar relativity

With H and O, the preceding, simple H_2O testcase involves only elements so light that relativistic effects on their total energies can be safely ignored in production calculations.

The rule of thumb that relativistic effects do not matter much for quantities derived from total energies (binding energies, geometries) holds up to elements number Z=20-30 (Ca or Zn), depending on the reqired accuracy. For example, the DFT-LDA lattice parameter for fcc Cu is 3.54 Å in the non-relativistic case, but 3.52 Å if relativistic effects are included. In any case, relativistic effects should be accounted for in accurate calculations *beyond* these elements. As described in more detail in Ref. [?], this process is handled by FHI-aims at different levels of approximation, with some overhead resulting compared to the simple non-relativistic case.

In a nutshell, the physical impact of relativity for heavy elements *is* noticeable not just as a total-energy offset, but importantly in total energy differences, and in particular impacts also geometries. The underlying reason is that core and valence orbitals "see" the near-nuclear region (where relativistic effects are most important) differently, depending on their angular momentum l. Somewhat simplistically put, the increased relativistic mass of the electron near the nucleus results in a tendency for all orbitals to "shrink" compared to the non-relativistic case, but to a different degree for different orbitals. The shrinkage thus changes not just atom sizes, but also the nature of bonding itself. A detailed discussion of relativistic effects is beyond the scope of this manual (and can be found in many excellent reviews, e.g., Ref. [?]) – but bear in mind that relativistic effects should not simply be shrugged off!

As outlined in Ref. [?] and Section 3.8, FHI-aims provides two levels of approximation to scalar relativistic effects: the "atomic" and "scaled" zero-order regular approximation (ZORA). Both are invoked by the `relativistic` keyword. In our experience, energy differences and geometries from *both* approximations closely match relativistic benchmark calculations performed using the full-potential linearized augmented plane wave method. While the absolute total energies from scaled ZORA are significantly closer to the fully relativistic limit than atomic ZORA, atomic ZORA appears to be slightly closer to other benchmark codes for energy differences, and the implementation of total energy *gradients* is much simpler in the atomic ZORA case.

We therefore recommend the following strategy to perform standard relativistic calculations with FHI-aims:

- For forces, geometry changes (structure optimization, molecular dynamics, vibrations), energy differences, and valence eigenvalues, the simple recommended option is
  " `relativistic atomic_zora scalar`".
- Optionally (for nicer-looking total energies and core eigenvalues), perform a follow-up calculation (fixed geometry) using
  " `relativistic zora scalar threshold`"
  (where threshold is a small number, e.g., 10^{-12}).

Note that this reflects a change from earlier recommendations in this manual; for many years, we were uncertain how (if at all) to qualify the very small differences between atomic ZORA and scaled ZORA compared to other benchmark approaches. Following the "Delta Project" (https://molmod.ugent.be/deltacodesdft), it appears that atomic ZORA is slightly more reliable – but most importantly, it is simpler.

If, for any reason, you do choose to follow up with "scaled ZORA" anyway, we emphasize two important points for the practical use of the "scaled ZORA" approximation:

1. **Only use the *final* total energy from a converged scaled-ZORA run**, given after the scaled ZORA post-processing step is complete. During the s.c.f. cycle, the Kohn-Sham wave function and all intermediate total energies are obtained at the simple ZORA level (*not* the same as atomic ZORA) only, so *any intermediate total energy is not reliable*.

2. **For energy differences between different structures, use the same `radial_base` integration grids**. Relativistic effects near the nucleus need to be integrated accurately, but residual integration errors cancel on an atom-by-atom basis. In energy differences, the remaining errors for the same `radial_base` and `radial_multiplier` settings thus cancel out.

We illustrate the practical use of of atomic and (if needed) scaled ZORA for the Au dimer in DFT-LDA, found in the *testcases/Au_dimer* directory.

In the subdirectory *relax_light*, a quick but sufficiently accurate prerelaxation is set up, at the `atomic_zora` level. The `control.in` file is set up to use `relativistic atomic_zora scalar` and `relax_geometry bfgs 1.e-2` to converge forces down to 10^{-2} eV/Å. Since this is intended to be a quick prerelaxation run (starting with an arbitrary separation of 3 Å of both atoms), the "light" species default settings for Au are used for the `species` subsettings. Compared to the much more accurate "tight" settings,

- the radial and angular integration grids are significantly reduced,
- the Hartree potential expansion is capped at `l_hartree` =4,
- the cutoff potential onset and width in `cut_pot` are reduced to a minimum that we still consider safe (3.5 Å / 1.5 Å, respectively),
- the basis set is the *spdf* section of *tier* 1 only.

. Among these settings, the reduction of the basis set to *spdf* has the biggest impact on the resulting equilibrium geometry, d=2.465 Å. Just adding the g function of *tier* 1 would already reduce the error by 0.013 Å.

After this relaxation run is complete, a file `geometry.in.next_step` is written out by FHI-aims. This file contains the final, relaxed geometry from the "light" prerelaxation run just performed, as well as the Hessian matrix estimate created by the `relax_geometry bfgs` relaxation algorithm. This file can serve as an improved guess for `geometry.in` in the next, typically more expensive "tight" post-relaxation step.

In the subdirectory *postrelax_tier1*, the final geometry from the previous step is used as the starting point for an accurate post-relaxation using the "tight" *species_defaults* settings. Note that, while all other settings are tightly converged at this level, the basis set convergence should normally still be tested explicitly. The *tier* 1 level used here for Au is quite sufficient for an accurate result. The binding distance converges after two additional relaxation steps, at d=2.449 Å.

Optionally: In the subdirectory *final_scaled_zora*, the fixed post-relaxed equilibrium geometry is used to compute the Au_2 binding energy at the scaled ZORA level. The computation is a single-point run with the full *tier* 1 basis set, using the rather safe grid and Hartree potential settings from the "tight" species default file. FHI-aims output files using the same `control.in` file are given for both Au_2 and a single Au atom, yielding a binding energy of 3.263 eV for the dimer. Note that this deviates from a possible `atomic_zora` result by less than 0.02 eV, matching our experience that energy *differences* usually agree very well between `atomic_zora`, scaled `zora`, and independently with results other scalar relativistic all-electron results obtained using other codes.

In conclusion, this section illustrates how a quick pre-relaxation followed by a safe calculation of the relevant energy differences can be combined. The key point in this endeavour is to ensure that the final relaxed geometry is accurate with as little computational overhead as possible.

4.3 k-point sampling in the Brillouin zone for semiconductors

In a semiconductor, the choice of the k-grid is crucial for accurately sampling the Brillouin zone, because the valence band maxima (VBM) and conduction band minima (CBM) are usually located at high symmetry points. Here, we describe three different choices for a k-grid offset:

- a Γ-centered grid,
- the grid suggested by Monkhorst and Pack [?] and
- an off-Γ grid.

They are defined as

$$u_r = \begin{cases} \frac{r-q}{q} & \Gamma\text{-centered}, \\ \frac{2r-q-1}{2q} & \text{Monkhorst-Pack}, \\ \frac{r-1}{q} + \frac{1}{2q} & \text{off-}\Gamma \end{cases} \tag{4.1}$$

where $r = 1 \ldots q$ and q is the total number of points. In Fig. (4.2, 4.3 and 4.4) the three different shifts are illustrated. The Monkhorst-Pack grid[1] agrees with the Γ-centered grid for an uneven number of grid points and with the off$-\Gamma$ grid for an even number of points. The Γ-centered grid and sequentially the Monkhorst-Pack grid with an uneven number of grid points include the Γ-point.
To achieve the relevant settings in the `control.in` file, two keywords have to be changed. For the number of grid points the keyword `k_grid` needs to be specified:
`k_grid n1 n2 n3`,
where the numbers n_1, n_2, n_3 are integers defining the number of splits along the reciprocal vectors.
For the off-Γ and Monkhorst-Pack shift, a second keyword has to be given in the `control.in` file:
`k_offset f1 f2 f3`,
where `fi` are fractional coordinates between zero and one. The offset can be defined according to Eqn. (4.3).

$$\text{k_offset} \begin{cases} 0.0 \quad 0.0 \quad 0.0 & \Gamma\text{-centered}, \\ \frac{1}{2} - \frac{1}{2n_1} \quad \frac{1}{2} - \frac{1}{2n_2} \quad \frac{1}{2} - \frac{1}{2n_3} & \text{Monkhorst-Pack}, \\ \frac{1}{2n_1} \quad \frac{1}{2n_2} \quad \frac{1}{2n_3} & \text{off-}\Gamma \end{cases} \tag{4.2}$$

[1] One could argue that Monkhorst and Pack never intended their formula to be used for for odd q. Still, for the sake of this section, "Monkhorst-Pack" refers to this definition.

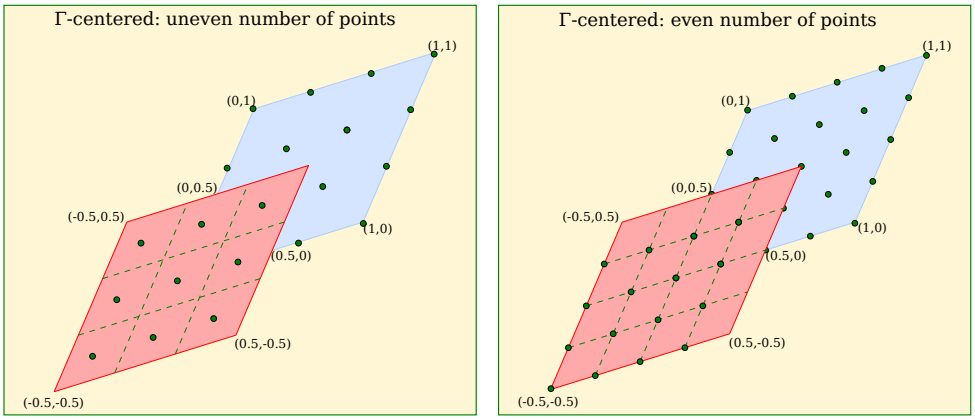


Figure 4.2: The Γ-centered grid in two dimensions for even and uneven numbers of grid points. In both cases the Γ-point is included in the grid.

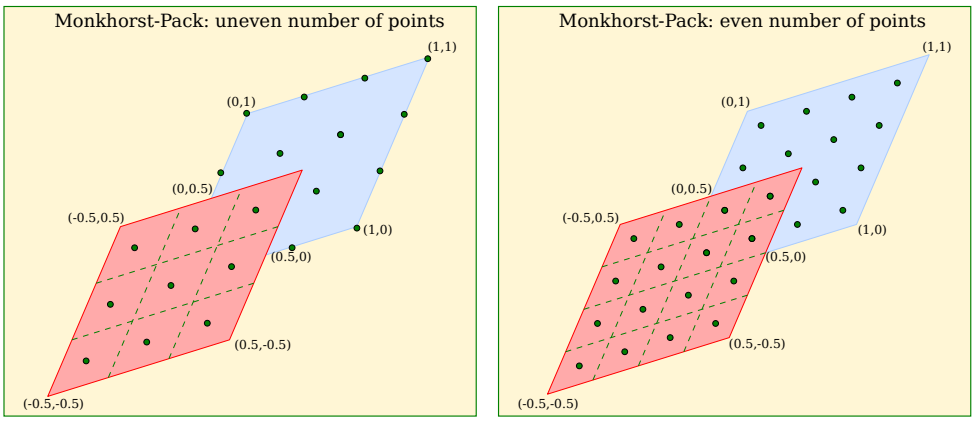


Figure 4.3: The Monkhorst Pack grid [**?**] in two dimensions for even and uneven numbers of grid points. In the left picture (uneven number of grid points) the Γ point is included in the grid.

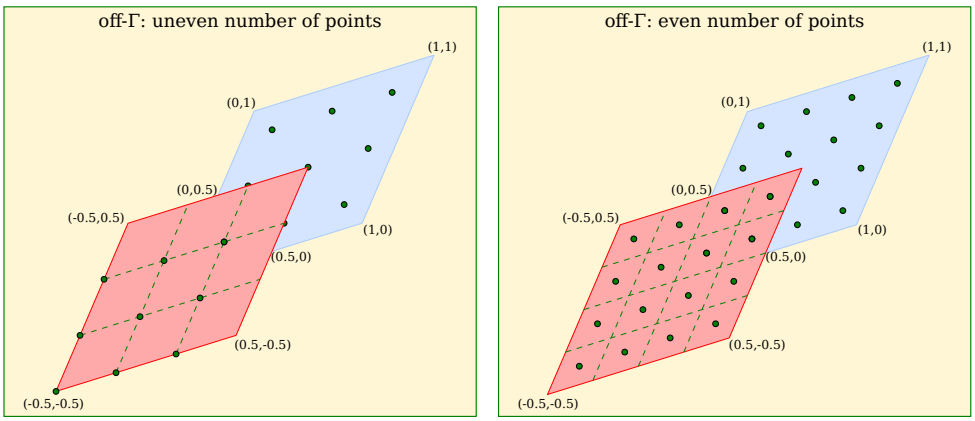


Figure 4.4: The off-Γ grid in two dimensions for even and uneven numbers of grid points.

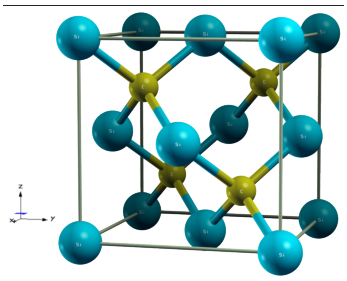


Figure 4.5: The crystal structure of silicon carbide in its cubic zinc blende structure (3C-SiC)

Here,we present a concrete example. Cubic silicium carbide (3C-SiC) is a semicoductor featuring an indirect band gap between the Γ-point and the X-point [see Fig. (4.7)]. In Fig. (4.5) the crystal structure of 3C-SiC is shown. The crystal is similar to the zinc-blende structure. The lattice constant was found to be $4.36\,\text{Å}$ [?].The k-grid was tested in terms of grid density in the Brillouin zone and the three different shifts discussed above.
As a reference energy the total energy was taken with a k-grid of $25 \times 25 \times 25$ k-points. In Fig. (4.6) the energy differences $|E(k = 25) - E(k)|$ are plotted on a logarithmic scale versus the number of k-points.
The oscillatory behaviour in Fig. (4.6) of the Monkhorst-Pack grid reflects the agreement with the off-Γ grid in the case of an even number of grid points and the Γ-centered for uneven number of grid points respectively.

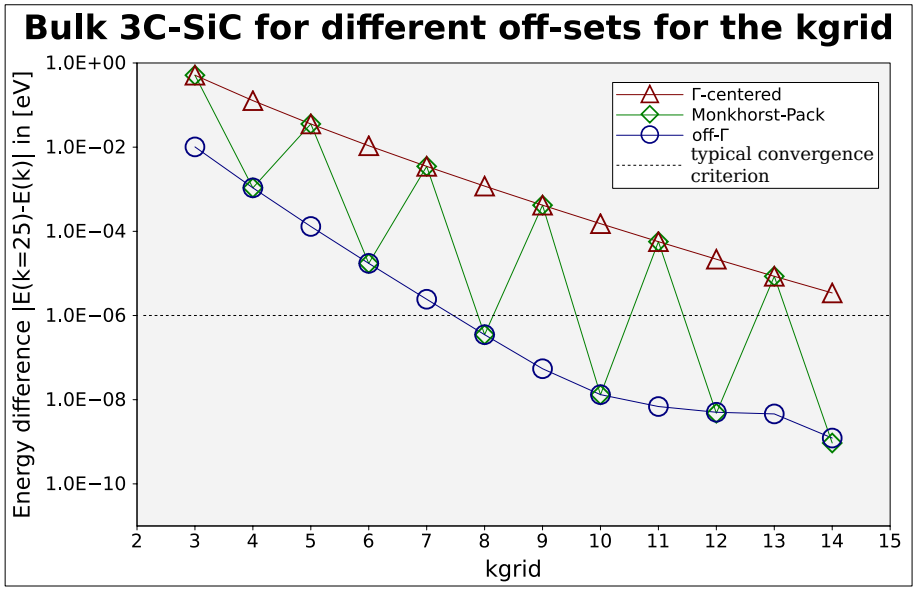


Figure 4.6: The k-grid was tested with respect to the grid density and the shift of the grid in reciprocal space. The shifts are defined in Eqn. (4.1).

A typical convergence criterion for the total energy is a value of 10^{-6} eV. In Fig. (4.6) the dashed line marks the convergence criterion. In the case of an off-Γ shift the calculation is well converged with a k-grid of $8\times8\times8$. For a Γ-centered grid the calculation did not reach convergence with a grid of $14\times14\times14$. As mentioned before, in 3C-SiC the VBM is in the Γ-point. Therefore the convergence of the total energy with respect to the density of the k-grid is slow for every grid including the Γ-point, because the contribution of the energy in the Γ-point is over-weighted. In this case the off-Γ shift gives best convergence.

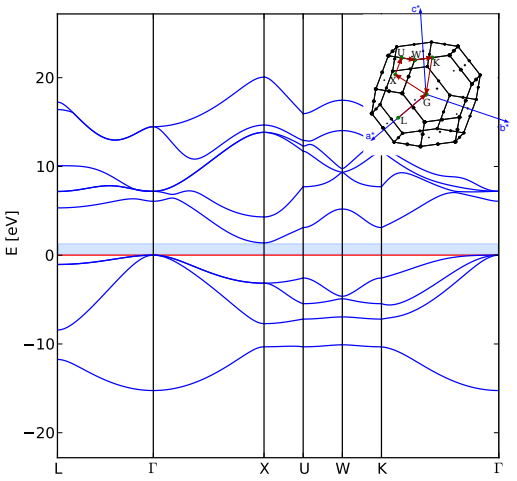


Figure 4.7: The band structure of 3C-SiC with tight, Tier 2 basis settings.

We note the above conclusion is mainly applicable for semiconductors. In a metal, the sampling of the Fermi-surface is crucial, this demands a high k-grid density rather than a special off-set.

4.4 Plotting the band structure and density of states of a solid

An example for plotting band structures and the density of states is contained in the example for bulk silicon (diamond lattice) in the testcases directory. Likewise, the fcc Al test case contains the necessary lines for control.in (just commented out).

This section contains a walk-through of the necessary steps and keywords to obtain this band structure and density of states. First, let us look at the relevant lines in `control.in` as given in the test case.

```
# output DOS
  output dos -18.  0.  200  0.05
  dos_kgrid_factors 8 8 8

# output band structure
  output band 0.5   0.5   0.5    0.0   0.0   0.0    50  L     Gamma
  output band 0.0   0.0   0.0    0.0   0.5   0.5    50  Gamma X
  output band 0.0   0.5   0.5    0.25  0.5   0.75   50  X     W
  output band 0.25  0.5   0.75   0.375 0.375 0.375  50  W     K
```

To get an output of the band structure, use the keyword `output band`. Each '`band`' line covers a single line in reciprocal space, from a starting point to an end point and with a given number of points inbetween.

The total density of states of the system is obtained by the '`output dos`' line, from a starting single-particle energy to an ending single-particle energy, with a specified number of points inbetween and for a given broadening to create a smooth line (see section 3.29 for details).

The density of states of a solid is a continuous function, but originates from a Brillouin zone integral,

$$g(\epsilon) = \sum_n \int_{\mathrm{BZ}} d^3k\, \delta(\epsilon - \epsilon_n(\boldsymbol{k}))\,. \tag{4.3}$$

The index n runs over all bands, and $\boldsymbol{k}$ is the crystal momentum.

As it stands, this expression for the density of states creates a problem. We obtain it from a discrete set of k-points, rather than as a continuous integral. The density of states from a discrete k-mesh would thus be a sequence of δ-function peaks.

In practice, the δ function is replaced by a Gaussian function with a finite width. The chosen width in our test case is 0.05 eV (typically, 0.1 eV would also be acceptable, although giving a somewhat more washed-out density of states).

If we just applied our expression Eq. 4.3 to a k-space mesh typically used to obtain a reasonably converged total energy in an s.c.f. calculation (in our example, '`k_grid` 12 12 12'), horrible things would happen to our DOS. With the broadening chosen here, a rather wiggly line would result. It turns out that, e.g., a '96 96 96' k-mesh would yield a much nicer result ... except that this choice would make any s.c.f. calculation extremely expensive.

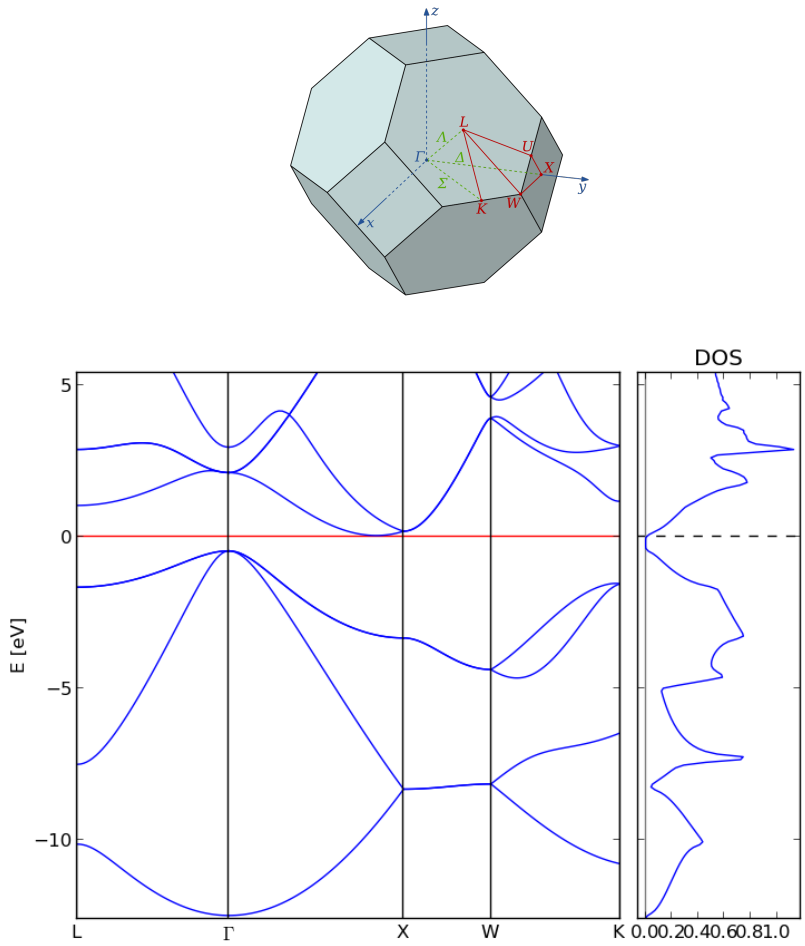


Figure 4.8: Band structure and density of states of the bulk Si test case as generated by the 'aimsplot.py' script, which is located in the "utilities" directory. A schematic visualization of the Brillouin zone of the fcc Bravais lattice including some high-symmetry k-points is also shown (taken from Wikipedia).

What to do?

Here is where the ' `dos_kgrid_factors` ' line comes in. The factors given here mandate a the use of a denser k-space mesh *only* for the evaluation of the density of states, after the s.c.f. calculation is already complete. The effective k-mesh used in our example is 'k_grid(i)·dos_kgrid_factor(i)' (i=1,2,3), in our case 12·8=96 in each direction. However, the extra k-points are only evaluated for the fixed self-consistent Kohn-Sham potential that emerged from the original s.c.f. calculation.

With the lines for band structure output, DOS output and `dos_kgrid_factors` in `control.in`, we can run the example calculation in the testcases directory.

Once this is done, a number of new files have appeared (in addition to the usual standard output stream): The files `band1XXX.out`, `KS_DOS_total.dat`, and `KS_DOS_total_raw.dat`.

These files contain the necessary information in column formats that can be manipulated for further output with standard plotting tools. The result of such a manipulation is

shown in Fig. 4.8.

However, it would be nice to not have to personally massage the output data every time to get this kind of output. The script 'aimsplot.py', located in the utilities directory, should do this for you. Simply run it in the output directory, and a nice-looking band structure and DOS should automatically appear.

As there are various conventions for plotting the density of states, each possible DOS plot produces two different output files. The first one, ending in "_raw.dat", contains the exact density of states, using exactly the same eigenvalue energies and energy zero as internally in FHI-aims (in cluster systems, the energy zero is simply the vacuum level; in periodic systems, the energy zero is set by the $\boldsymbol{G}$=0 component of the long-range electrostatic potential).

The file `KS_DOS_total.dat` contains the same information, but the energy zero is shifted to the Fermi-level of the system. Note that, for metals, the Fermi level is usually well defined. For insulators, it is not well defined, and may end up anywhere in the gap (at some location where FHI-aims finds the normalization criterion for the number of electrons to be sufficiently fulfilled).

If, for a semiconductor, you wish to have the energy zero of a DOS shifted to the top of the valence band, you will have to do this as an extra step. Identify the location of the valence band edge (highest occupied level) either from the FHI-aims output or from the band structure, and shift the x-axis of `KS_DOS_total.dat` and other files by the appropriate amount.

Except for the *total* density of states, there are also a few other options to obtain densities of states with some local, spatial resolution. In particular:

- an atom-projected density of states, using `output atom_proj_dos`
- an angular-momentum component projected density of states averaged over all atoms *of a single species*, using the keyword `output species_proj_dos`. This is useful because different atoms of the same element may be grouped together as a separate species in `control.in`, e.g., all atoms in the first layer of a slab.

Note that there are two useful projections of the DOS, both of which are based on a Mulliken analysis. `output species_proj_dos` can be used to get a species-dependent angular momentum projection, while `output atom_proj_dos` resolves the states into the various atomic angular-momentum contributions.

As mentioned above, the `species_proj_dos` keyword can also be used to get the DOS contribution from a whole group of atoms at once. Just duplicate the species defaults for the element(s) in question in `control.in`, give the species a new name, and then use that new species (say) for all surface atoms in `geometry.in`. This will get you the density of states contribution from all surface atoms.

4.5 Visualizing charge densities and orbitals

Charge densities, orbitals, etc. can be written onto a cartesian grid in the "cube" file format (see below, as well as Section 3.29, specifically the `output` keyword and the `cube` subkeyword described there). The visualization described in this section pertains to a simple H_2O molecule, and can be repeated for example for the relaxed geometry obtained in the relaxation testrun for H_2O in this distribution.

After running FHI-aims for the desired target geometry with the appropriate output flags (see Section 3.29), some files will be generated, all of which have the extension "*.cube". In the H_2O example shown in this section, the calculation was run with *tier* 2 basis set, which should produce a more accurate charge density, and the parameters entered in the control.in file were:

```
output cube total_density
  cube origin 0.0  0.0  0.0
  cube edge 29  0.15 0.0  0.0
  cube edge 29  0.0  0.15 0.0
  cube edge 29  0.0  0.0  0.15
output cube eigenstate 5
output cube eigenstate 6
```

The "Gaussian CUBE" files are written in a file format originally defined by the "Gaussian" program package, but now implemented as a *de facto* standard by many visualization programs. In this section, an example of how to visualize them using jmol program [?] is presented.

The CUBE files are composed of a header and the volumetric data. The header is divided in the following manner:

- 1st and 2nd lines: text ignored by visualization programs
- 3rd line: number of atoms in the file followed by the origin around which the volumetric data will be plotted.
- 4th, 5th, and 6th lines: number of points to be plotted, followed by the axis vector.
- 7th line on, until the end of the header: one line for each atom of the system, consisting of atomic number, followed by the atom xyz position.

An example of such a header with the beginning of the volumetric data is given below:

```
CUBE FILE written by FHI-AIMS
*****************************
   3   -2.100000   -2.100000   -2.100000
  29    0.150000    0.000000    0.000000
  29    0.000000    0.150000    0.000000
```

```
 29    0.000000    0.000000    0.150000
  8    0.000000    0.000000    0.000000    0.000000
  1    0.000000    0.707000   -0.707000    0.000000
  1    0.000000   -0.707000   -0.707000    0.000000
0.12810E-04  0.18208E-04  0.25394E-04  0.34819E-04  0.46974E-04  0.62322E-04
0.81196E-04  0.10365E-03  0.12928E-03  0.15704E-03  0.18520E-03  0.21135E-03
0.23277E-03  0.24687E-03  0.25180E-03  0.24687E-03  0.23277E-03  0.21135E-03
.
.
.
```

In order to visualize these files in jmol, open the program and open the script console. Load a cube file, for example the total electronic density, by typing the following command:

```
> load "total_density.cube"
```

This will show your molecule. In order to visualize the surfaces, one must use the "isosurface" command. It has a myriad of options, all of which are explained in the jmol documentation [?], as of this writing located at http://chemapps.stolaf.edu/jmol/docs/#isosurface. As an example, in order to plot a volume, the command is:

```
> isosurface cutoff <cutoff> "total_density.cube"
```

The field <cutoff> specifies a radius for the surface, since all values equal too or less than this one will be plotted.

Although these files are written by default in Å, some programs (including jmol), read them in atomic units (bohr) by default. In the *utilities/* folder, one can find the `angstrom_to_bohr.pl` script that converts the CUBE files to atomic units.

Plotting planes is also very easy, and the command is, for example:

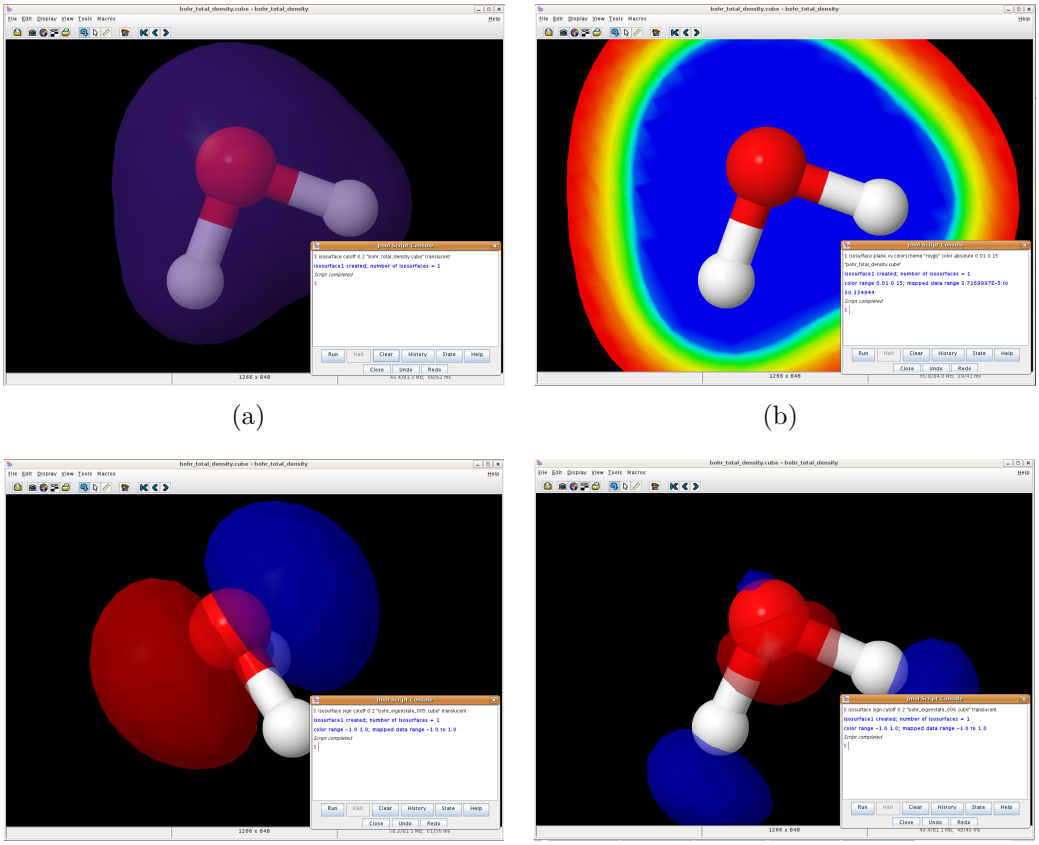
```
> isosurface plane <plane_position> <other_options> "total_density.cube"
```

The field <plane_position> can either specify the axis that define your plane (e.g. xy) or three atoms, whose centers will specify the plane (syntax: (atomno=1) (atomno=2) (atomno=3)). The field <other_options> may contain all sorts of color scheme and/or cutoff specifications.

One can plot as many isosurfaces simultaneously as one wishes. The command to delete them is simply:

```
> isosurface delete
```

Some example images and their respective commands are shown in Figure 4.9. The CUBE files shown were all converted to atomic units.



(a) (b)

(c) (d)

Figure 4.9: Jmol plots of charge densities and orbitals.

4.6 Calculation of vibrational and phonon frequencies

Vibrations (for non-periodic structures) and phonons (for periodic structures) can be computed in FHI-aims via script-based finite difference approaches. Each tool first does all necessary DFT calculations and then calls a routine to set up and diagonalize the Hessian matrices.

We support vibrations in non-periodic systems and phonons in periodic systems using two different types of infrastructure:

- Infrastructure for vibrations in non-periodic systems (clusters or molecules). The relevant perl scripts and source code for this step for the entire finite-difference calculation are included with FHI-aims. To compile the tools, change into the source directory from which you also compiled the original FHI-aims. With the same Makefile settings as for the main program, type either `make vibrations` or `make vibrations.mpi`.
- Infrastructure for phonons in periodic systems, based on the GPL `phonopy` tool (not distributed with FHI-aims itself). Phonopy is a python-based utility to generate and process finite-diffence generated phonon-related quantities from a variety of codes. The infrastructure for FHI-aims support in phonopy was mainly written by Jörg Meyer (now at Technical University of Munich) and contributed to the GPL'd phonopy project. We note that `phonopy` and its dependencies must be installed separately (in addition to FHI-aims) on your system.
- A third level of infrastructure for vibrations and phonons in non-periodic and periodic systems based on density functional perturbatio theory (DFPT) is nearing readiness and usable to some degree. However, parts of this implementation are not yet optimized. The DFPT implementation should therefore still be considered experimental at this time.

Note that the former, native phonon infrastucture in FHI-aims is no longer needed. The previous make targets `phonons` and `phonons.mpi` still exist, but we no longer recommend them. Phonopy is, simply, more stable and general.

The infrastructure for non-periodic vibrations is covered in the next section, followed by a section on phonons in periodic systems using phonopy.

Vibrations by finite differences within FHI-aims

For the FHI-aims vibrations target, we recommend the `vibrations.scalapack.mpi` make target. This will create two files in the FHI-aims binary directory. One is the actual diagonalization subprogram, the other one the `perl`-wrapper. **You will need to edit the header of the `perl`-wrapper scripts** and give three key parameters: The absolute location of the FHI-aims binary directory (`$AIMS_BINDIR`), the aims executable to be used (`$EXE`), and the calling command for the executable (`$CALLING_PREFIX` and

`$CALLING_SUFFIX`). The `$CALLING_PREFIX` is required for specification of parallel runs, for example

```
$CALLING_PREFIX = mpirun -np 2
```

for a two-core parallel run. In addition, some parallel environments (most notably IBM's poe require that the number of cores is specified after the executable is called, for those instances you can set `$CALLING_PREFIX`. Note that the postprocessing routines for the vibrations are also parallel.

Having compiled the vibrations script, running it is very simple. We use the same water molecule that was relaxed in the example section 4.1. This test case is contained in the directory `testcases/H2O-vibrations`.

We are here calculating a second derivative of the energy from the numerical change on the forces arising from small finite displacements δ, in the following way:

$$\frac{\partial E}{\partial x_i \partial x_j} = \frac{\boldsymbol{F}_i(x_j + \delta) - \boldsymbol{F}_i(x_j - \delta)}{2\delta} \tag{4.4}$$

Here, E is the total energy as a function of all atomic coordinates x_i (i=1,...,3M for M individual atoms), and $\boldsymbol{F}_i$ are the components of the analytical first derivatives (forces) on each atom (again, i=1,...,3M) due to a displacement of coordinate another x_j by an amount plus or minus δ.

Since we are interested in the Hessian in a local minimum of the potential energy surface (ideally, $\boldsymbol{F}_i(\{x_j\})$=0 for all i at the local minimum geometry $\{x_j\}$), this expression is the difference of two force values that are themselves already near zero. The smaller the displacement δ, the smaller the absolute magnitude of the forces to be subtracted, giving greater weight to any residual numerical noise in the calculation. The larger the displacement δ, the larger does the difference become, but at the same time, we will also move out of the region of the potential energy surface that is exactly harmonic, and introduce systematic errors into the Hessian. We are thus faced with a tradeoff between choosing a value δ that is large enough to be free of any potential numerical noise, yet small enough to avoid large systematic errors due to real anharmonicities. The default value for δ is 0.0025 Å, but this value should be checked explicitly in any practical calculation.

Since the vibrational frequencies depend strongly on the accuracy of the forces, and on any near-zero residual forces on the fully relaxed geometry itself, we apply the following changes to the earlier file `control.in`:

- We use the *tight* species defaults for elements H and O. Among other things, these defaults prescribe denser integration grids than *light* and thus lead to more accurate forces. Obviously, the previous optimum geometry from the *light* settings should be postrelaxed to the exact minimum for the modified *tight* settings, and our script does this automatically (but check the output just in case).
- We increase the accuracy to which the forces are converged in the self-consistency cycle, `sc_accuracy_forces` , to 1E−5. *However*, be sure to check that the remaining settings, especially `sc_accuracy_eev` are accurate enough. You

do *not* want your FHI-aims calculation to spend more than 1-2 s.c.f. cycles per geometry on actually converging the forces, because each s.c.f. iteration with forces can be up to a factor 10 more expensive than an s.c.f. iteration during which the forces are not yet checked.

- We change the `relax_geometry bfgs` convergence criterion to `1E-4`. This is, again, a rather harsh requirement, and one should verify (especially for large molecules) that the relaxation algorithm is not spending large amounts of time on failed relaxation steps near the minimum, probing any residual numerical noise of integration grids etc. rather than following an actual, smooth potential energy surface.

These new settings contain all that is necessary to get close enough to the actual minimum. Again, the convergence settings quoted above should normally be fine, but in case of doubt, check. If, for some reason, those stringent criteria can not be reached exactly, it is still preferable to obtain a slightly less converged Hessian within an amount of CPU time that is actually finite.

In order to run the calculation of the vibrations, change to the directory containing the input files `control.in` and `geometry.in` and run the aims.vibration.*.pl script in the directory. That's it. The calculation should take no more than a few minutes to complete. It first relaxes the structure to its minimum configuration and then applies six finite displacements to each of the atoms: 18 single-point calculations in total for the water molecule. The result (using the default settings for δ) can be visited in subdirectory `test_default`.

The output stream contains all of the important data, which is additionally saved in a number of temporary and output files. The file basic.vib.out contains the output of the normal mode analysis, while the file basic.xyz contains the actual eigenmodes (vibrations), which can be read by a number of molecular viewers.

A short note on the actual physical output basic.vib.out: The frequencies are given in cm^1, the corresponding zero point energies in eV, and their cumulative total. It is important to ensure that the first six eigenmodes are close to zero, which means that the structure in question actually corresponds to a minimum on the potential energy surface. FHI-aims does not do an *a priori* reduction of the translational and rotational eigenmodes, which allows an explicit check on the quality of the strucure.

In the default version (no command line options specified), the vibration script automatically assumes a jobname `basic`. This name, as well as the finite difference displacement δ, can be changed on the command line, leading to the following complete calling form of the script:

`aims.vibrations.*.pl` ⟨`jobname`⟩ `delta`

⟨jobname⟩ is a name of choice that will be appended to all output files produced during the run. One can thus run the same job with different settings δ multiple times in the same directory, starting from the same input files.

The second parameter, `delta`, is the finite displacement δ used for calculating the Hessian matrix. Its default is 0.0025 Å, which has been doing a good job for all the

cases we are aware of, but still, please verify that it works for your particular system. (See the brief discussion above).

For example, for our test case, subdirectory `test_delta_0.001` contains a second example run with the following command line:

```
aims.vibrations.*.pl test_0.001 0.001
```

For the default settings, our resulting frequencies look like this:

Mode number	Frequency [cm^(-1)]	Zero point energy [eV]	IR-intensity [D^2/Ang^2]
1	-12.66044778	-0.00078485	1.89406419
2	-0.15559990	-0.00000965	0.00266735
3	-0.00081379	-0.00000005	0.00000000
4	0.04885082	0.00000303	0.00002821
5	6.58934967	0.00040849	0.00000000
6	7.01167236	0.00043467	5.41560478
7	1592.96295435	0.09875111	1.63341394
8	3708.86739272	0.22992045	0.04298854
9	3813.74446188	0.23642200	1.07616306

In contrast, the smaller value δ=0.001 Å produces the following results:

Mode number	Frequency [cm^(-1)]	Zero point energy [eV]	IR-intensity [D^2/Ang^2]
1	-5.13719038	-0.00031847	1.89407621
2	-0.06616895	-0.00000410	0.00323346
3	-0.00073155	-0.00000005	0.00000000
4	0.01938144	0.00000120	0.00002697
5	2.47172391	0.00015323	0.00000000
6	2.70760068	0.00016785	5.41498492
7	1593.00852915	0.09875393	1.63340808
8	3708.82166272	0.22991761	0.04299176
9	3813.70936996	0.23641982	1.07615430

So, in this case, the rather tight relaxation and force convergence settings do produce a set of translations and rotations that are closer to zero for δ=0.001 Å than the default.

However, once again be warned that for larger molecules the same harsh convergence criteria may not be applicable, and a too small value δ can in fact inflate any residual noise. So, a small value δ=0.001 Å or even less should not be blindly expected to improve numerical accuracy.

It should also be noted that the "physical" vibrational frequencies 7-9, above the six translations and rotations, remain completely unimpressed by the change of δ, either way. This is generally true, and would even remain true for a yet larger value δ=0.005 Å, which would also be a reasonable default choice.

In the following you find the description of tags defining the output of free energies.

Tag: `vibrations` (control.in)

Usage: `vibrations [subkeywords and their options]`

Purpose: allows to choose temperature/pressure range for output of free energies. It has the subkeywords `free_energy`, `trans_free_energy`, as detailed below.

`vibrations` sub-tag: `free_energy` (control.in)

Usage: `vibrations free_energy Tstart Tend Tpoints`

Purpose: governs the calculation of the free energies of vibration and rotation (rigid rotor approximation)

The calculation of the free energy of vibration and rotation (rigid rotor approximation) can be requested with this keyword. They are calculated between the temperatures `Tstart` (K) and `Tend` (K) with a total of `Tpoints` steps. The implemented equations read:

$$F_{\text{vib}}(T) = \sum_{i}^{3N-6} \left[\frac{\hbar\omega_i}{2} + k_B T \ln\left(1 - e^{\frac{-\hbar\omega_i}{k_B T}}\right) \right] \quad , \tag{4.5}$$

with N being the number of atoms in the molecule and ω_i depicting the vibrational frequencies.

$$F_{\text{rot}}(T) = -\frac{3}{2} k_B T \ln \left[\frac{2k_B T}{\hbar^2} (I_A I_B I_C)^{1/3} \pi^{1/3} \right] \quad , \tag{4.6}$$

where I_A, I_B, and I_C depict the moments of inertia of the molecule.

`vibrations` sub-tag: `trans_free_energy` (control.in)

Usage: `vibrations trans_free_energy pstart pend ppoints`

Purpose: governs the calculation of the translational free energy

The calculation of the free energy of translation can be requested with this keyword. It only works if `vibrations free_energy` is used as well. The free energy of translation is calculated within the temperature range given with `vibrations free_energy` and between pressures `pstart` (Pa) and `pend` (Pa) with `ppoints` steps. The equation reads:

$$F_{\text{trans}} = -k_B T \left[\ln \frac{k_B T}{p} + 1 + \ln\left(\frac{m k_B T}{2\pi\hbar^2}\right)^{3/2} \right] \quad , \tag{4.7}$$

with p denoting the partial pressure.

Vibrations by density-functional perturbation theory (DFPT) within FHI-aims

---

Tag: `DFPT vibration` (control.in)

Usage: `DFPT vibration [subkeywords and their options]`

Purpose: Allows to calculate vibrations using density-functional perturbation theory, use Acoustic Sum Rule (ASR) to get Hessian matrix, do not use moving-grid-effect.

Usage: `DFPT vibration_with_moving_grid_effect [subkeywords and their options]`

Purpose: give the results for vibrations with moving-grid-effect, do not use ASR for Hessian matrix.

Usage: `DFPT vibration_without_moving_grid_effect [subkeywords and their options]`

Purpose: give the results for vibrations without moving-grid-effect, ONLY served as comparison with vibration_with_moving_grid_effect.

Polarrizability by density-functional perturbation theory within FHI-aims

---

Tag: `DFPT polarizability` (control.in)

Usage: `DFPT polarizability [subkeywords and their options]`

Purpose: Allows to calculate polarizability for cluster systems using density-functional perturbation theory.

Phonon (gamma) by density-functional perturbation theory within FHI-aims

---

Tag: `DFPT phonon_gamma` (control.in)

Usage: `DFPT phonon_gamma [subkeywords and their options]`

Purpose: Allows to calculate phonon for PBC systems using density-functional perturbation theory.

Phonon(real space) by density-functional perturbation theory within FHI-aims

---

Tag: `DFPT phonon` (control.in)

Usage: `DFPT phonon [subkeywords and their options]`

Purpose: Allows to calculate phonon (real space method) for PBC systems using density-functional perturbation theory. This method could get force constant using real space.

Phonon(reciprocal space) by density-functional perturbation theory within FHI-aims

---

Tag: `DFPT phonon_reduce_memory` (control.in)

Usage: `DFPT phonon_reduce_memory [subkeywords and their options]`

Purpose: Allows to calculate phonon (reciprocal space method) at q point for PBC systems using density-functional perturbation theory. This method get dynamic matrix at q. This feature is under developing.

Phonon calculations in general

To calculate the phonons, we use the so-called *direct* or *supercell* method, based on the approximations that the interactions necessary to calculate the phonons are finite [see e.g. Ref. [?] and references therein for details of the method].

A phonon calculation proceeds as follows. A single (user-specified) unit cell is extended a number of times in the direction of all three basis vectors. A finite displacement technique as for the vibrations is then used to gather the necessary force response, which then makes up a basic force constant matrix. All finite displacement structures are investigated for their symmetry (within the octahedral group), such as to minimize the computations required and also to obtain properly symmetrized forces where applicable. Finally, the force constant matrix is diagonalized for a number of selected reciprocal lattice points (i.e. along the requested bands) and the eigenvalues give the phonon spectra.

Three things are really important:

- **Converge your supercell size.** The minimum sensible cell is $2 \times 2 \times 2$. It might be tempting to restrict a calculation to this size, but the results will almost certainly be insufficient.

- **The file geometry.in must contain only the primitive 1x1x1 cell of your lattice.** `phonopy-FHI-aims` constructs its own supercell. If, by accident, your `geometry.in` already contains a $3 \times 3 \times 3$ supercell and you request phonons for a $3 \times 3 \times 3$ supercell on top of that, you will end up with a $(9 \times 9 \times 9)$ supercell calculation.

- Do not forget to run the actual FHI-aims calculations with the right `k_grid` for the supercell – not for the primitive cell. If your primitive cell requires a $(20 \times 20 \times 20)$ k-space grid, a $(5 \times 5 \times 5)$ supercell only requires `k_grid 4 4 4`. If, by accident, you use a $(20 \times 20 \times 20)$ k-grid for the $(5 \times 5 \times 5)$ supercell instead, you will pay with a disproportionately large amoung of computer time, 125 times more time than what you should have been using.

Notes: (i) FHI-aims does not (yet) provide support for long-range electrostatic fields ionic compounds, which leads to splittings of the LO and TO branches at the Γ point. Phonopy supports this, but providing the necessay derivatives within FHI-aims is a work in progress. (ii) Tight s.c.f. convergence (`sc_accuracy_rho` *etc.)* is a strong requirement for phonons. If you see, e.g., non-zero frequencies at the Γ point, check your s.c.f. convergence.

phonopy and FHI-aims

There are roughly four steps to a calculation with phonopy:

- On a convenient machine, e.g., your desktop, prepare an input directory with the usual `control.in` and `geometry.in` files. You should only specify the primitive cell in `geometry.in`, all the rest may be handled by the `phonon` keyword in `control.in` .

- Change to that directory and run the `phonopy-FHI-aims` command. This creates the supercell input geometry files with displacements for the force calculations in FHI-aims.

- Copy over the supercell input geometries and the `control.in` file to your favorite production computer and run FHI-aims. Do not forget to specify a k-grid that is appropriate for the supercell – else, if you specify an unnecessarily dense k-grid, you will pay a high price. Run the individual FHI-aims calculations.

- Copy the FHI-aims output files for each displacement back to the first machine. Run `phonopy-FHI-aims` again. This should produce the output you were looking for.

To use the `phonopy-FHI-aims` infrastructure,

python $(>= 2.5)$ http://www.python.org

numpy http://numpy.scipy.org

and a C compiler, e.g., gcc

are required. The necessary symmetry analysis routines, wich are taken from *spglib* (http://spglib.sourceforge.net/), are already included in the *phonopy* package. To obtain plots via the integrated plotting functionality,

matplotlib http://matplotlib.sourceforge.net/

has to be installed. The latter dependency is, however, not compulsory for running a calculation.

Since `phonopy-FHI-aims` completely decouples FHI-aims calculations and phonon pre- and post-processing operations, all the necessary packages can easily be installed via the package management system of a normal Linux desktop machine, e.g., for Ubuntu linux: *apt-get install python python-numpy python-matplotlib*

The details:

`phonopy-FHI-aims` supports the `phonon` sub-keywords and output styles discussed above. When called from the command line in a directory containing a correct `geometry.in` and `control.in` file, it generates $n = 1, .., N$ subdirectories *phonopy-FHI-aims-displacement-n*.
Each of these directories contains correct input files for one specific displacement. The calculations for the generated inputs need to be done manually (and concurrently if desired) on any available computing machinery. The output has to be stored in the respective subdirectory in a file named *basename.out*, e.g., *phonopy-FHI-aims-displacement-01/phonopy-FHI-aims-displacement-01.out*.

Once the FHI-aims output files are available (at the right places), rerunning `phonopy-FHI-aims` in the same directory as before will produce the output (band structure, DOS, thermodynamic properties) which has been requested in control.in. The calculated data is written to ASCII files which include comments describing their contents. Some plots are also generated in .pdf files if matplotlib is available.

Additionally, `phonopy-FHI-aims` supports some advanced features that are still experimental at this point:

- Corrections for the long-range electrostatic fields in ionic crystals (Γ point splitting of optical modes) can be accounted for. The required Born effective charges will be available from FHI-aims soon, but not yet as of this writing (version 081213). Please check back with us if needed.
- The force constants for the whole supercell can be written out (a prerequisite to use the thermodynamic integration routines for anharmonic free energy calculations).

Additional information can be found in the README text file provided at http://www.fhi-berlin.mpg.de/aims/utilities.html .

The usual `control.in` and `geometry.in` files are read by both `phonopy-FHI-aims` and `FHI-aims` itself.

The phonon keyword

We finally document the phonon keyword that is part of the usual input file control.in.

Tag: phonon (control.in)

Usage: phonon [subkeywords and their options]

Purpose: Main driver keyword for the phonon calculation script. has the subkeywords supercell , band , dos , displacement , as detailed below.

---

phonon sub-tag: band (control.in)

Usage: phonon band kstart1 kstart2 kstart3 kend1 kend2 kend3 npoints startname endname

Purpose: To define a phonon band to be plotted explicitly. The remaining syntax is kept the same as for the output option band : the first six parameters describe the starting and ending point in RELATIVE k-coordinates, npoints is the number of points in the band, followed by the names of the special points for starting and ending the band.

---

phonon sub-tag: displacement (control.in)

Usage: phonon displacement delta

Purpose: To define the absolute displacement of each atom for the finite difference calculation.

Default: 0.001 Å

---

phonon sub-tag: dos (control.in)

Usage: phonon dos fstart fend fpoints broad qdensity

Purpose: governs the output of the phonon density of states.

This keyword defines the DOS output along a frequency axis in THz. fstart, fend, and broad give the starting and ending frequency as well as the broadening in units of THz, while fpoints is the number of points. Unlike the output option dos , here we also have to specify a k-point density qdensity, which does the BZ integration on a qdensity×qdensity×qdensity grid. Normally, the dynamic matrix only contains a very small number of elements (3 in the example described below), which means that

one can chose qdensity quite high in order to get a well-converged density of states. Notice that this value might be changed by the program if the DOS and c_v calculations are requested at the same time. In that case, the higher of the two values for qdensity specified for the two calculations is chosen in order to minimize the computational effort. See also the free_energy description.

---

phonon sub-tag: free_energy (control.in)

Usage: phonon free_energy Tstart Tend Tpoints qdensity

Purpose: governs the calculation of the phonon free energy, the internal energy, and the specific heat for a given unit cell.

The phonon free energy and some derived quantities can be requested with this keyword. They are calculated between the temperatures Tstart and Tend with a total of Tpoints steps. A k-point density qdensity is also required, as the calculation involves an integration over the Brillouin zone. Notice that this value might be changed by the program if the DOS and free energy calculations are requested at the same time. In that case, the higher of the two values for qdensity specified for the two calculations is chosen in order to minimize the computational effort. See also the dos description.

Please refer to phonopy for the meaning of the exact quantities that are written out.

---

phonon sub-tag: frequency_unit (control.in)

Usage: phonon frequency_unit unit

Purpose: Allows to specify the frequency output for the phonons by setting unit to either cm^-1 or to THz. Default is cm^-1

---

phonon sub-tag: supercell (control.in)

Usage: phonon supercell n1 n2 n3

Purpose: To define a $n1 \times n2 \times n3$ supercell in which to approximate the calculation of the dynamic matrix, based on the lattice vectors provided in geometry.in.

The supercell keyword can also be used to prescribe a general (rotated) supercell in matrix notation. For instance,

```
phonon supercell -1 1 1 1 -1 1 1 1 -1
```

corresponds to a supercell matrix

$$\begin{pmatrix} -1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & -1 \end{pmatrix}.$$

---

phonon sub-tag: symmetry_thresh (control.in)

Usage: `phonon symmetry_thresh thresh`

Purpose: To define the maximally allowed coordinate difference (in Å) for the phonon symmetry checker to decide that two atoms are at the same location. Default: 10^{-6}

This keyword should not have to be changed from its default.

4.7 Restarting FHI-aims calculations

This section gives background information on restarts in FHI-aims.

4.7.1 General restart procedure

Calculations can be restarted from previous wavefunctions using the keyword `restart`. In general, restarting the same calculation on the same computer using the same number of MPI-tasks will always work.

Important — Rotated systems: FHI-aims wavefunctions are not symmetry-aware. While the same restart file can be used for molecules *translated* by a vector, this does not work for *rotated* molecules. For a way to restart systems with rotated geometries or superpositions of molecular densities see Section 4.7.3.

FHI-aims has two main means of saving the wavefunction for a later restart, depending on the used keywords.

Kohn–Sham eigenvector (*"wavefunction"*): This is the default case always used for calculations using the LAPACK solver. The saved file contains some information on the system (number of basis functions, states, spin, k-points), the Kohn–Sham eigenvector, eigenvalues and occupations.

Kohn–Sham density matrix (*"density matrix"*): This variant is only used for calculations using the scaLAPACK or ELPA solver. The saved file contains the density matrix ($\hat{n}_{ij} = \sum_l f_l c_{il} c_{jl}$), either in full or sparse storage.

The corresponding keywords determining which version of storage is used are `KS_method` and `density_update_method`. See Fig. 4.10 for possible options and outcomes.

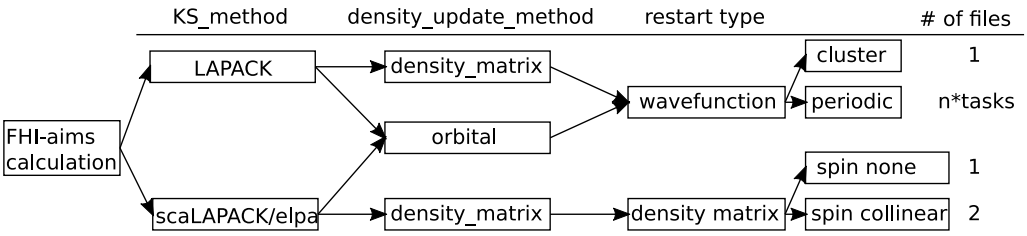


Figure 4.10: Scheme of different restart types depending on used methods.

For technical reasons there is not one single way, but the above mentioned variants with again, variants.

The *density matrix* based restart always writes one file per spin-channel, no matter how many parallel MPI tasks are used. The *wavefunction* based restart generally writes a single restart file for all cluster calculations and one file per MPI-task for periodic

calculations. Due to this periodic calculations using the *wavefunction* restart need to be restarted with the same number of MPI-tasks.

4.7.2 Mixing variants - the "force_single_restartfile" option

For a subset of possible calculations the above described differences are not valid and it is possible to restart different calculations using one type of restart file. This is the case for all possible cluster calculations (using either the LAPACK or scaLAPACK/elpa solver) and for periodic calculations with only 1 k-point, Γ-only (using either the LAPACK or scaLAPACK/elpa solver).

This is implemented via the keyword `force_single_restartfile` . Using this functionality, it is for example possible to profit from the parallel ELPA solver for huge systems and still get a file containing the KS-wavefunction for post-processing.

4.7.3 Rotating the FHI-aims wavefunction

This feature is new and tested for the case of weakly interacting organic molecules as can be found in organic crystals. If you encounter any problems or strange behaviour, please let us know!

In many molecular systems a rotation does not change the electron density (or wavefunction) of a system. Therefore, the same wavefunction can be used again to restart such a calculaction.

This is especially true for weakly interacting systems such as van-der-Waals bound organic crystals. In these cases, the wavefunction of each single molecule in the crystals is not too different from the isolated wavefunction of a single molecule. This can be exploited to start a FHI-aims calculation of huge molecular systems from a *superposition of molecular densities* instead of the usual *superposition of atomic densities*.

This functionality is implemented as an external tool using Python and can be found in the FHI-aims repository itself (Python-package *aimsutils*) or at `https://gitlab.lrz.de/theochem/aimsutils`.

If you use this functionality, please cite *C. Schober, K. Reuter, H. Oberhofer, J. Chem. Phys., 144:054103, 2016*[?].

Important pre-requisites

This functionality only works for restart files containing the wavefunction of the system, not the density matrix. Therefore, please make sure you have wavefunction restart files or use the option `force_single_restartfile` . The FHI-aims output file of the calculation is also necessary.

Rotations can be defined via Euler angles in *zyz* convention or quaternions. To avoid any confusions with rotational conventions (especially with Euler angles), the input geometry will be rotated and saved using the same rotation matrix. Please be sure to check

if your expected and the actual rotation are the same (or directly use the generated `geometry.in`).

The final rotated restart file is again in the wavefunction format.

```
- example_calc
|
+-- control.in
+-- geometry.in             # necessary
+-- aims.out                # necessary
+-- restart.wavefunction    # name can be anything, will be parsed from outfile
```

Examples

The two examples are shown using pre-defined scripts, but it is totally possibly to use the Python package directly. The full Python API is available via the Sphinx-documentation of the *aimsutils* package.

Rotation of a single system: The script `RotateSingle.py` can be used to create the rotated geometry and restart for a single calculation.

```
$: # RotateSingle alpha beta gamma -t x y z
                   '-----.-----'    '---.---'
                   Euler angles   translation vector
$: RotateSingle 45 0 123 5.4 2.3 6.3

$: # RotateSingle x_i x_j x_k x_l -t x y z
                   '------.------' '---.---'
                     quaternion   translation vector
$: RotateSingle 0.33 -0.12 0.44 0.84 5.4 2.3 6.3
```

This will create a subfolder *rotated* with the new `restart.combined` and `geometry.in`.

Rotation and combination of wavefunctions The script `RotateMany.py` reads a instruction file (`rotations.in`) with the following format:

```
# Comments or empty lines are ignored

# title will also be foldername
title project_x
# lattice vector for new cell if any
lattice_vector 11.2 0.00 0.00
lattice_vector 0.00 10.5 0.00
lattice_vector 0.00 0.00 10.7
# now all rotated molecules
```

```
 # Euler angles (zyz)
 # alpha beta gamma x y z parent_molecule
 33.4 12.4 167.0   10.  0.  0.  calc_2
  0.  45.    0.     3.5 2.1 5.4 calc_1
130.  90.   23.     0.  2.6 12. calc_3
 # OR Quaternion
 # x_i x_j x_k  x_l   x y z
 -0.4 0.15 0.33 -0.84 4 0 0 calc_2
 -0.2 0.45 0.13  0.84 0 2 0 calc_1
  0.0 0.00 0.00 -0.32 4 4 5 calc_3
```

The necessary converged FHI-aims calculations must be in subfolders with the appropriate name:

```
- calc_something/ (main calculation folder)
|
+-- rotations.in
+-- calc1/
    |
    +-- ...
+-- calc2/
    |
    +-- ...
+-- calc3/
    |
    +-- ...
```

This will create a subfolder `project_x` with the rotated wavefunction assembled from the individual calculations defined in the script (`restart.combined`) and geometry (`geometry.in`).

Theory

The following should give a summary of how the rotation of wavefunctions can be done with FHI-aims. For details on the used methods and mathematics, please have a look at the references.

In FHIaims, a basis function is defined as

$$\Phi_{i,lm} = \frac{u_i(r)}{r} \cdot S_{l,m}(\theta, \phi), \tag{4.8}$$

with $S_{l,m}(\theta, \phi)$ being real valued spherical harmonics. These are obtained from the

complex spherical harmonics Y_{lm} via

$$S_{l,m}(\theta,\phi) = \begin{cases} \frac{(-1)^m}{\sqrt{(2)}}(Y_{lm} + Y^*_{lm}) & m > 0 \\ Y_{l0} & m = 0 \\ \frac{(-1)^m}{i\sqrt{(2)}}(Y_{l|m|} - Y^*_{l|m|}) & m < 0 \end{cases} \tag{4.9}$$

A wavefunction is then given by

$$\Psi_k(r) = \sum_{i=1}^{\text{n_basis}} c_i^k \Phi_i(r) \tag{4.10}$$

with the coefficients c_i^k.

A rotation of the molecule (rotation matrix $\mathbf{R}$ leads to the same set of YLMs $\mathcal{Y}$ (as they are fixed with respect to the xyz-coordinate system), but with different coefficients $\mathbf{c}$ (new linear combination of basis functions):

$$\mathbf{c}' = \mathbf{R}\mathbf{c} \tag{4.11}$$

The rotation of complex YLMs can be done with Wigner D matrices ($D^l_{mm'}$). For real YLMs different schemes are available[?, ?, ?]. Due to the non-default YLM convention in FHIaims we construct our rotation matrices for different l from the complex Wigner D matrices via the transformation matrix $\mathbf{C}^l$ (reference: [?]):

$$S_{l,m} = \mathbf{C^l} Y_{l,m} \tag{4.12}$$

The matrix $\mathbf{C}$ is constructed according to [?] with the constraint of the different sign convention in FHIaims. The real rotation matrix $\Delta^l(R)$,

$$\Delta^l(R) = \left(\mathbf{C}^l\right)^* \mathbf{D}^l(R) \left(\mathbf{C}^l\right)^t, \tag{4.13}$$

is then used to obtain the rotated coefficients for each l, m for each basis function in the system.

$$\mathbf{c}'^l = \Delta^l(R)\mathbf{c}^l \tag{4.14}$$

4.8 Finding Transition States: the aimsChain

This project aims to provide an all-in-one package for various flavours of the chain of states methods for finding the minimum energy path(MEP). Currently the nudged elastic band method (NEB)[?, ?], the string method[?], and the growing string method[?] are included.

Please direct any questions or suggestions to *yaoyingyu@hotmail.com*, we need your help to improve the package.

The aimsChain code is distributed under the Lesser General Public License as published in Version 3, 29 June 2007, at http://www.gnu.org/licenses/lgpl.html . Some of the optimizer routines in the code originated within the Atomic Simulation Environment (ASE). We wish to give full credit to the developers of these routines. The aimsChain code can also be found in a separate distribution maintained by Yingyu Yao. The reason we distribute them directly within the FHI-aims repository is for the convenience of all FHI-aims users, but again: We wish to give full credit to the work and the license of the original authors within ASE.

4.8.1 Installation

The package is located in `src/aimsChain`.

In general, any computer/cluster that is capable of running FHI-ams and has numpy/scipy support can be used. To install, user should follow these simple steps:

1. Copy the package to the directory where you normally install softwares, e.g. `~/local/bin` or your home directory.

2. Modify your `~/.bashrc` to include the following lines

   ```
   export PATH=$PATH:<package path>/tools
   export PYTHONPATH=$PYTHONPATH:<package path>
   module load python/scipy/2012.10
   ```

   These modifications are recommended but not necessary. You can also add them to your job script instead if you do not want to pollute your `.bashrc`.

   The last line serves to load scipy into your system. On clusters you can normally find the corresponding keyword using `module avail`, and this could also be done by including `scipy` and `numpy` in your `$PYTHONPATH`.

   Although unlikely, future updates of numpy/scipy may cause compatibility issues. If you observe anything strange in this regards, please let me know.

3. You are done.

To test your installation, source the `.bashrc` file and start a python shell. The package is successfully linked if executing

```
import aimsChain
import scipy
import numpy
```

reports no error. Otherwise please double check your configurations and search for installation problems.

4.8.2 A Quick Start

While offering an extensive list of keywords, the packaged is aimed to work with most systems out of the box. Often very little configuration is required to conduct a successful path search. The keywords are available, on the other hand, to make the package as adjustable as possible, so that it can be finely tuned to tackle the less conventional systems.

In the package directory, a few sample jobs have been prepared, imitating several different possible use cases. Please modify `chain.in` in each sample (other than sample 1) so that `run_aims` links to your own aims executable.

- sample 1: This is a graphical example on a 2D analytic surface that demonstrates the principles behind the chain-of-states methods. You can skip it if you have used NEB/SM elsewhere.

  You can and should run this demo in your personal computer instead of the cluster, since it is not actually calling FHI-aims for calculations.

  To start, you should go to `samples/sample1/run` and execute `job.sh`. This will execute the MEP finding algorithm on the 2D LEPS potential energy surface. To see the resulting process, proceed to `samples/sample1` and run `gnuplot plot.gnu` to see the evolving path on the surface.

  This is also the perfect test bench for testing with different configurations. However keep in mind that performance in 2D is very different from the 3N-D space in reality, and therefore this example should not be used for performance optimization.

- sample 2: The inversion of an ammonia molecule. This example surveys the MEP with 8 images, using an very tight convergence setting(`force_thres` =0.01), which is too tight for most of the more complex systems, and will require tremendous amount of computation for non trivial systems in general. The resulting path would represent a very accurate representation of the MEP.

- sample 3: This example measures the energy barrier to transfer a methyl group from one chlorine atom to another. It is using 6 images, coupled with a relatively loose convergence setting(`force_thres` =0.2). The resulting MEP from this is in general not accurate enough for a quantitative analysis. However, climbing image is turned on with a relatively tight convergence setting(`climb_thres` =0.05). The result is an accurate energy barrier calculation.

Note that in this example we are also utilizing `constrain_relaxation` in our input geometries to limit the movement of atoms. In this particular case, one of the chlorine atom is fixed at the origin, and the others are constrained to move along the x-axis.

- sample 4: In this example we demonstrate how to start from an external set of initialization images . We are calculating the MEP for a non-trivial isomerization. Geometries from an previous calculation is provided as the starting geometries.

- sample 5: This sample provides an insight into MEP searching with periodic systems. It involves only the `interpchain.py` provided in `aimsChain/tools`, which should be already in your $PATH if you followed the installation process. This tool will generate an initial path exactly the same way as the actual job script does. The result is printed into a directory named `interpolation`. A multi-frame *.xyz* file is written, as well as one *.in* geometry file for each image.

  Sample 5.1 and 5.2 are the same geometry, the only difference is that periodic interpolation is set to `true` in the former, and `false` in the latter. Run `interpchain.py` in each directory and inspect the resulting path with your favourite visualization software. In sample 5.1 we acquire a smooth interpolation, but in sample 5.2 with periodic interpolation turned off, one row of atoms had to travel across the entire cluster.

  Another way to do a correct interpolation is shown in sample 5.3. You can manually adjust the coordinate of the atoms so that they are set at the correct coordinate with respect to the initial geometry. `fin.in` is adjusted in this case to reflect the change. This way, the interpolated path is again correct, even though `periodic_interpolation` is turned off. This is the preferred method of dealing with periodic systems, because the automatic interpolation finds the shortest path, but not necessarily the correct path. When working with periodic systems you should always do a `interpchain.py` first and inspect the result before running the actual job.

- sample 6: Here we show when and why the growing string(GSM) method should be used. Here the isomerization process involves rotation of in dihedral angles, and linear interpolations would result in overlapping atoms, which can be checked using the `interpchain` tool.

  The traditional approach dealing with these cases is to rotate the atoms manually and generate a non-overlapping initial path. GSM provides an alternative to this approach by growing the string from end points, which eliminates the need for manually generated initial path.

- sample hydra: It is essentially the same as sample 1, but prepared for the hydra cluster. This demonstrates the required modifications when transferring between different platforms.

The samples include the following files.

- ini.in: initial geometry in the standard aims format

- fin.in: final geometry in the standard aims format
- control.in: standard control file for aims. This control will be used for all aims calculations.
- job.sge: job submission script, which you submit into the cluster.
- chain.in: control file for aimsChain, see below for a detailed description

If everything is set up properly, you can simply `qsub job.sge` to get the job running. When running on clusters not using SGE, the job scripts must be changed accordingly.

4.8.3 Configuration

aimsChain control file

The chain control file is the file that governs the evaluation of MEP. It follows the same configuration convention set by the aims `control.in`, namely a list of key value pairs. The control file should be named `chain.in`.

This is a sample control file.

```
#A sample chain.in
run_aims mpiexec -ppn 8 -n $NSLOTS ~/bin/aims.062812.scalapack.mpi.x
initial_file ini.in
final_file fin.in
n_images 6
force_thres 0.2
use_climb true
climb_thres 0.05
```

All the available keywords for the control are listed below:

Tag: `run_aims` (chain.in)

Usage: `run_aims` your command
Purpose: Provide the command that is used to call aims in your environment.
Default value: mpiexec -ppn 8 -n $NSLOTS ~/bin/aims.081912.scalapack.mpi.x

You will need to change the default value for whatever command you are using.

There is no need to quote your command, simply type it as you would in a bash shell. Everything after the keyword are concatenated into a single string.

Tag: `initial_file` (chain.in)

Usage: `initial_file` filename
Purpose: Provide the initial geometry file for the path.
Default value: ini.in

The format should be the same as the aims `geometry.in`. Flags such as constraints are interpreted, and will affect the evaluation of MEP. All the intermediate images will have the same constraints as the initial geometry.

Atoms in the initial and final geometry must establish a one to one correspondence. The ith line in the initial geometry and the final geometry must correspond to the same atom. Often there are many possible combinations, it is the user's responsibility to use the most physical one and/or check different combinations. You can perform a test interpolation with the `interpchain.py` tool provided.

Tag: `final_file` (chain.in)

Usage: `final_file` filename
Purpose: The final geometry for the path. Refer to `initial_file` .
Default value: fin.in

Tag: `n_images` (chain.in)

Usage: `n_images` value
Purpose: The number of images used for MEP calculation.
Default value: 5

Suitable value for this variable is highly dependent on the desired degree of resolution and the complexity of the system. A simple single barrier reaction will require 5 images to achieve a good result, while more complex potential energy surfaces require roughly 3 or more images per hill/trough.

If the geometry is not computationally expensive(tens of seconds per calculation with a reasonable # of CPUs), the simple principle of "the more the merrier" can always be applied.

Tag: `external_geometry` (chain.in)

Usage: `external_geometry` file_name
Purpose: If this variable is set, the initial set of images will be obtained from a external source instead of a linear interpolation.
Default value: None

This flag should be set to the filename of a text file listing all of the intermediate geometries in order, e.g.

```
./geo/1.in
./geo/2.in
./geo/3.in
```

It is advised that if you have anything better than a direct linear interpolation, you should use it as the initial guess. Linear interpolation can be highly inefficient, and would result in cases which never converges. When in doubt, you can always use the growing string method.

Tag: `periodic_interpolation` (chain.in)

Usage: `periodic_interpolation` true/false
Purpose: Whether or not interpolation is done while considering periodic boundary.
Default value: False

This setting affects periodic systems only. If set to true, each atom's final geometry will be checked against all periodic counterparts to find the shortest path between the initial and final geometry. This can resolve some problems caused by the periodic boundary condition. Consult sample 4 for detailed usage. Always use the provided `interpchain.py` tool to check the resulting interpolation and confirm that it's physical before running.

Tag: `resample` (chain.in)

Usage: `resample` true/false
Purpose: Resample the input path into arbitrary number of images.
Default value: False

This tag is effective only when `external_geometry` is set. When `resample` is set to true, the package will resample the path you provided and interpolate it to have # of images equal to `n_images` . This can be very useful if you find your calculation to have an inadequate number of images. You can extract the geometries from the final iteration, and start a new job by resampling them.

Tag: `aims_restart` (chain.in)

Usage: `aims_restart` value
Purpose: Reuse the wave function restart file to speed up the calculation. This should set to be the same as `restart` in `control.in`.
WARNING! This function is not compatible with aims as of July 2013, and only works occasionally.
Default value: None

Value should be the same as `restart` in your `control.in`. The wave function restart file will be copied from iteration to the next, so that the scf cycles can start from an existing configuration. When working correctly, this can cut the computation time required by half or more.

However, at this stage `restart` has very limited check on the input restart file, resulting in an error when the wave functions don't match. Do not use this keyword unless you are sure that your version of aims will start a new scf initialization instead of reporting errors when the wrong restart file is provided. If certain of which, however, the keyword should always be used to speed up the calculation.

Setting `restart` will consume a decent amount of disk space. You may want to remove the restart files if you are planning to store the calculation permanently.

Tag: `restart` (chain.in)

Usage: `restart` true/false
Purpose: Whether the package will allow restarts. When set to true, you can simply submit the job again to continue from the previous calculations.
Default value: true

When set to true, a restart file will be written as the path iterates. When the job is submitted/resubmitted, it will first check for the restart file to see if it's possible to start from a previous calculation. This is useful, when, for example, your job is killed due to time limitations on the cluster. Please refrain from changing settings between restarts, which is always error prone. If you are planning to change some settings, it's always safer to extract the most recent geometries and use them as the initial path for a new calculation.

There is still the risk that the job is killed while the restart file is been written, in which case anything could happen when the job is resumed. However, considering the rarity of such event, it should not be a problem in practice.

Tag: `method` (chain.in)

Usage: `method` neb/string
Purpose: Pick the method to use for the MEP calculation.
Default value: string

Currently only string method and NEB are supported. In our testing, string method have shown to be slightly more stable than NEB, and therefore is our recommended method.

Tag: `neb_spring_constant` (chain.in)

Usage: `neb_spring_constant` value
Purpose: This sets the spring constant used in NEB calculation, and has no effect if `method` is string.
Default value: 20.0

It should be set to have the same magnitude as the force felt by the geometry, which is hard to determine before hand. However a generic value of 20 is good in general. Please try `method` string first if you suspect that the spring constant have to be changed for the system to converge.

Tag: `force_thres` (chain.in)

Usage: `force_thres` value
Purpose: The threshold for convergence.
Default value: 0.2

Optimization will stop when the residual forces in the system is smaller than this preset

value. If use_climb is true, climbing-image calculation will start after this threshold has reached. $0.2eV$ is always a good starting point, and for more rigorous calculations $0.1eV$ or $0.05eV$ can be used. Do not set it too small, since some numerical noises in the forces will always exist even when the path is well converged.

Also note that adequate value is dependent on the system. A reaction with a $3eV$ barrier can be much more well converged at $0.05eV$ threshold than a reaction with a $0.3eV$ barrier at the same threshold, because the former, very often, has a steeper barrier and hence larger force by nature.

Tag: `optimizer` (chain.in)

Usage: optimizer dampedBFGS/BFGS/LBFGS/trm/CG/FIRE
Purpose: This picks the optimizer used for optimization.
Default value: dampedBFGS

BFGS is a textbook implementation of BFGS optimizer, which was observed to have wild behaviour is some situations. The fact that BFGS series of optimizer can be used for this type of calculations is in fact purely coincidental. The series of force projections involved in the chain of states methods can severely hamper the effectiveness of quasi-Newton optimizers. Please resort to FIRE optimizer whenever you observe wild behaviour in the optimization process.

dampedBFGS damps the original BFGS optimizer using several techniques, and is slightly more stable in those situations. (But they still get stuck from time to time!) This is the default setting that should always be tried first.

LBFGS is the limited memory version of BFGS, which uses less memory for extremely large systems. However, given that in general FHI-aims is applied to systems with hundreds of atoms at most, this optimizer has little advantage in this front. However, due the fact that LBFGS uses only recent iterations for approximation, the results can be quite different from BFGS in a some PES. Therefore it's worth trying if BFGS fails.

trm is the same trust-radius method ported from FHI-aims. In our tests it has proven to be reasonably stable and efficient. We would recommend this as another alternative along with FIRE when dampedBFGS fails.

CG is the textbook implementation of conjugate-gradient algorithm using finite difference scheme. In our tests it has not shown any advantage over other algorithms, ans is provided for the sake of completeness.

FIRE is the Fast Inertial Relaxation Engine, one of the better non-Newton type optimizers. It is slower than BFGS series of optimizers in general, but is immune to the aforementioned instability because it does not approximate the Hessian. If dampedBFGS fails, try FIRE with global_optimizer off as an alternative.

Tag: `global_optimizer` (chain.in)

Usage: global_optimizer true/false
Purpose: Whether all images are optimized as a single object, or if individual images are optimized as separated object.
Default value: true

The global version, when coupled with BFGS-series of optimizer, is often faster than the non-global optimizer. However, this combination is less stable, and can lead to wild results. The non-global optimizer coupled with FIRE seems to be the most stable one in our tests, but is much slower than the former. Non-global optimizer combined with BFGS, on the other hand, slows the calculation significantly due to overestimation, and should be avoided.

Tag: `xyz_lattice` (chain.in)

Usage: xyz_lattice a b c
Purpose: This key only affects the *.xyz* file written in `paths` directory. It governs how the lattice is repeated in each lattice vector. The default is a standard 2 2 1 setting, valid for most surfaces.
It has no effect on clusters.
Default value: 2 2 1

Tag: `map_unit_cell` (chain.in)

Usage: map_unit_cell true/false
Purpose: This key only affects the *.in* file written in `paths` directory. It determines whether atoms in these geometries are mapped back to the central unit cell.
It has no effect on clusters
Default value: false

Tag: `use_climb` (chain.in)

Usage: use_climb true/false
Purpose: This will turn on the climbing image[?] feature.
Default value: false

The points with highest energies will move toward a higher energy location along the path until the saddle point is reached. Please consult climb_mode for a detailed explanation of this process. If you are only interested in finding the energy barrier, it's possible to set force_thres to some larger value, such as 0.2, and set climb_thres smaller(e.g. 0.05) for a reasonably tight convergence. In this case decreasing force_thres will not affect the accuracy of climbing image, so long as the climbing image converges. Transition state finding can be sped up significantly this way by reducing the number of single point calculations required.

Tag: `climb_thres` (chain.in)

Usage: climb_thres value
Purpose: Set the convergence criterion for the climbing image.
Default value: same as force_thres

The climbing image will stop when the residual forces in the system is smaller than this value. It is normally, although not necessarily, set to a value smaller or equal to force_thres . In simple systems, the climbing image is capable of pushing the system done to $meV/\text{Å}^2$ range, but that's not as plausible in larger systems.

Tag: `interpolated_climb` (chain.in)

Usage: interpolated_climb true/false
Purpose: Determine whether the climbing image is chosen from one of the existing images or interpolated from known energies and geometries.
Default value: true

When set to true, the current energies and geometries will be fitted with a cubic spline to identify the point with highest energy. If this geometry is sufficiently far from existing nodes, then the interpolated geometry is used. If the geometry is close to one of the existing images, than that image is set to be the climbing image.

When set to false, the image with highest energy is set to be the climbing image.

Tag: `climb_mode` (chain.in)

Usage: climb_mode 1/2/3
Purpose: This determines the "mode" of the climbing image.
Default value: 2

Increasing mode corresponds to increasing stability and decreasing efficiency. The main difference lies in the number of images that are allowed to move during the climbing process.

climb_mode 1: Only the image with highest energy is allowed to move. This setting can manage many circumstances, provided that force_thres is small enough so that the converged path provides a stable basin.

climb_mode 2: The image with highest energy and its two neighbouring images are allowed to move. The two additional image provides an evolving tangent estimate as the central image climbs. This mode can tackle nearly all cases where climb_mode 1 fails under the same force_thres setting. It is safe, in general, to set force_thres to 0.3 if you are only interested in energy barrier.

climb_mode 3: All of the images excluding the end points are allowed to move. This is the most stable, but rarely necessary setting. The efficiency drop is highly dependent on the system and the force_thres setting. In general this should be used as a last resort when force_thres can be converged to a tight setting but climbing image on other modes fails.

Tag: `climb_global_optimizer` (chain.in)

Usage: climb_global_optimizer true/false
Purpose: The same as global_optimizer , except this keyword is dedicated for climbing image.
Default value: true

The tag has no effect when climb_mode is 1, in which case the global and non-global optimizer are equivalent. Unlike global_optimizer , BFGS+Non-global setting can be a good combination for climb_mode 2 if the default setting fails, mainly due to the well behaving local basin for a roughly converged string.

Tag: `climb_optimizer` (chain.in)

Usage: climb_optimizer dampedBFGS/BFGS/LBFGS/CG/TRM/FIRE
Purpose: The same as optimizer , but dedicated for climbing image.
Default value: fire

Tag: `climb_control` (chain.in)

Usage: climb_control file_name
Purpose: Specify a different `control.in` for the climbing image.
Default value: control.in

It's possible, for example, to converge the entire path with a light setting, and then converge the climbing image with tight setting, which consumes far less computational power.

However, there are a few things to note if this feature is utilized. First, force_thres must be set smaller than normal, perhaps even the same value as climb_thres . Since a different control file can produce a radically different PES, you would like to be as accurate as possible when converging the string, so that the error is not amplified with a tighter setting. Secondly, it's better to use climb_mode 2 or 3 for these kind of computations, since the single image climbing mode can rarely recover from a bad starting point.

This feature is best suitable for production works where a tighter setting which will dramatically increase the time required for single point calculations.

As an alternative, you can also kill your running process and change the `control.in` file manually before restarting the process.

Tag: `use_gs_method` (chain.in)

Usage: use_gs_method true/false
Purpose: This controls whether the growing string method is used or not.
Default value: False

The growing string method(GSM) is explained in sample 6. It is best suitable for cases where it is certain that linear interpolations between initial and final images will not lead to a correct path. This can be caused by movements such as rotations. It is also useful

when doing large-scale automated scanning, where the user does not have the time to look at geometries on a case by case basis.

When set to true, the path will start from the two end point and slowly grow inward to generate a physical path. When the path is completely grown, it will be passed onto NEB/SM for further calculations.

Beware that when direct interpolation can lead to correct paths, e.g. surface dispersion, using growing string method may reduce efficiency.

Tag: `gs_thres` (chain.in)

Usage: `gs_thres` true/false
Purpose: The GSM counterpart for `force_thres` .
Default value: `force_thres` *1.5

When the forces in the system goes below this preset value, a new node will be added to the path. It should not be set too small, since the purpose of GSM is to generate a physical path, not a well-converged path. The default value gives a good guideline for this key.

Tag: `gs_n_images` (chain.in)

Usage: `gs_n_images` value
Purpose: The GSM counterpart for `n_images` .
Default value: `n_images`

In some circumstances, you may want to specify a different number of images for the growing stage of the calculation. This key serves this purpose. After the growing process, the path will be re-sampled to match the value of `n_images`

Tag: `gs_optimizer` (chain.in)

Usage: `gs_optimizer` dampedBFGS/BFGS/LBFGS/trm/CG/FIRE
Purpose: The GSM counterpart for `optimizer` .
Default value: trm

For GSM, dampedBFGS, trm, and FIRE are optimizers worth trying.

Tag: `gs_global_optimizer` (chain.in)

Usage: `gs_global_optimizer` true/false
Purpose: The GSM counterpart for `global_optimizer` .
Default value: false

We have not done enough testings to determine conclusively the better set of optimizers. The default provided here have performed well in our benchmarks, but feel free to try other combinations.

Tag: `lbfgs_alpha` (chain.in)

Usage: `lbfgs_alpha` value
Purpose: Set the curvature used to initialize the Hessian(in $eV/\text{Å}^2$) in LBFGS. In general any value that is not magnitudes off are acceptable.
Default value: 120.0

Tag: `lbfgs_memory` (chain.in)

Usage: `lbfgs_memory` value
Purpose: Set the number of past iteration that LBFGS is going to remember. A larger value will increase memory consumption, and a small value will decrease accuracy.
Default value: 30

Tag: `lbfgs_maxstep` (chain.in)

Usage: `lbfgs_maxstep` value
Purpose: The maximum step in Å that an atom can take in a single iteration. This is similar to `max_atomic_move` , but defaulted to a much smaller value due to the increasing complexity.
Default value: 0.04

Tag: `bfgs_alpha` (chain.in)

Usage: `bfgs_alpha` value
Purpose: Same as `lbfgs_alpha` , but used by BFGS, trm, and dampedBFGS optimizers.
Default value: 120

Tag: `bfgs_maxstep` (chain.in)

Usage: `bfgs_maxstep` value
Purpose: Same as `lbfgs_maxstep` , but used for BFGS and dampedBFGS optimizers.
Default value: 0.04

Tag: `fire_dt` (chain.in)

Usage: `fire_dt` value
Purpose: The initial time step used by the FIRE optimizer.
Default value: 0.02

It is set to a very conservative value because FIRE is intended to be used as a fall back when BFGS fails. Values up to 0.1 can be used to speed up the calculation, provided

that the PES is smooth enough. Internally, the time step is dynamically adjusted, and this key only serves to initialize the value.

Tag: `fire_maxstep` (chain.in)

Usage: `fire_maxstep` value
Purpose: Same as `lbfgs_maxstep` , but used for FIRE optimizer.
Default value: 0.04

4.8.4 Preparation before running

Creating a project directory

You should create a directory for each and every aimsChain calculation you are planning to run. It should contain the following files.

```
-Project directory
|
|--chain.in
|
|--control.in
|
|--ini.in*
|
|--fin.in*
|
|--extgeo.lst*+
|
|--images*+
      |
      |-image1.in*+
      |
      |-image2.in*+
      |
      |-...
*The filename can be set by the user
+The files are only necessary for starting from external geometry.
```

geometry file

The initial and final geometry should be in the standard aims input format. Keywords such as constraints will be interpreted. The `constrain_relaxation` tag should be used with caution–it acts as a double-edged sword in terms of MEP evaluation. You can try to remove the constraint tag from sample 2 and observe the efficiency boost for example. In other cases, setting it may help convergence by reducing degree of freedoms.

In addition, you should pre-align your initial and final geometries to remove any rotational/translational component in the geometry, this will reduce the computational effort required.

The ordering of atoms in the geometry file is very crucial. The atoms in the initial and final geometry must establish a one to one correspondence, so that the ith line in the initial and final geometry represents the same atom. Changing the order can change the result from the evaluation dramatically.

For example, consider the diffusion of benzene on a surface. By changing the ordering of atoms you may force the diffusion to be done via rotational or translational motion, which would result in very different MEP. If you can visualize more than one possible combination, try all of them to find the lowest barrier. It is a matter of fact that there often exists more than one MEP between two geometries, but the overall barrier is only determined by the MEP with lowest barrier. Chain of states methods will evolve toward the MEP that's closest to the initial path, but not necessarily the lowest in energy.

aims control file

The aims control file should contain two lines.

```
compute_forces .true.
final_forces_cleaned .true.
```

This will turn on the force evaluation for aims, which is required for any chain of states method. `relativistic` , when required, must be set to `atomic_zora scalar`, since we require force evaluations.

Sometimes additional configurations can require more than the two lines listed above. For example, when using `b3lyp` you need to set

```
RI_method lvl_fast
```

for a correct force evaluation. Generally you can find these information in the aims output. A rule of thumb is to relax the geometry using your configurations and skim through the aims output for any warnings. This will force aims to provide all warnings related to the calcualtion of forces. When you have confirmed that the configuration if warning-free, just comment out the `relax_geometry` for aimsChain calculation.

You can also set `sc_iter_limit` to a lower value, which should still be much higher than the normal number of cycles that your system consumes. As the path evolves, it can be the case that during a few iterations the geometry become non-physical, and its scf cycle will never converge. Such cases often recover itself within a few iteration, and will not affect the final result. However, a default limit of 1000 will consume lots of computational power in these cases, which is a complete waste especially for larger systems.

The control file should not have `relax_geometry` , any molecular dynamic keywords, and etc. The geometry should not be altered by aims. It is advised that you always use

a light setting for the first run, so that tunning settings and confirming convergence can be done efficiently. If a more accurate result is desired, you can submit a tight run using the resulting geometries from the light run, which is always faster than a direct tight run from linear interpolation.

To ensure efficiency, you should configure your control file so that each single point calculation takes at most few minutes to finish. If a tighter setting is desired, consider using `climb_control` for a different control file during climbing image.

aimsChain control file

It is unlikely that you will need a long list of keywords in your control. Tags should be added gradually if you find that the default settings is not sufficient for your purpose. Keys that should be included on the first run are `run_aims`, `initial_file`, `final_file`, `n_image`, `force_thres`, `restart`, and `external_geometry` if so desired. Simply grabbing the control from the samples will also work most of the times.

If you believe that your system follows a rather straightforward path geometrically(i.e. diffusion, small rotation, etc.), you can set the climbing image on your first run and see if it works out correctly. For any complex paths, such as non-trivial isomerization, fine tunning of the control file can be required for the system to converge.

job script

The job script should contain a call to `runchain.py`, as well as loading the necessary modules if that is not done in `.bashrc`. No post processing should be included unless you are certain that the system will converge and terminate within the time limit.

external geometry

If you have any information about the path you are trying to find, please include them here. This can be a guess for transition state, geometry reported by papers, or your own interpolation by tweaking atoms in a visualization software. Coupled with `resample`, any number of external geometries can help speed up the calculation process, making the process very flexible.

It is a common practice to take the intermediate path from a previous calculation (perhaps before it has gone wild) and use them as the initialization path for a new job. This is perfectly fine, and aimsChain outputs intermediate paths just for this purpose. However, when doing so please remember to remove the first and last image from the path, which are the same geometry as the initial and final states. The list of external geometries should only contain intermediate images, entering initial/final states in there will most likely lead to a dead end.

growing string method

GSM offers another possibility for no-trivial calculations. If you believe your geometry will result in a geometrically complex reaction path, then using GSM will be your best bet.

GSM works by starting from the two end point, and gradually adding images toward the centre of the path. Whenever an image is added, it will be evolved until its forces falls below a threshold. This way we are not requiring any a priori knowledge on the intermediate path, where the standard SM/NEB method approximates with a linear interpolation.

test the interpolation

One of the most important step you can take to ensure a successful MEP evaluation is to ensure that the initial path is physical. This is especially true for periodic systems where the periodic boundary condition plays an unwanted part in this.

We have provided a tool in `aimsChain/tools`, the `interpchain.py`. This gadget performs the resampling and interpolation process exactly the same way that the actual script is doing it, and therefore is a good way to check if the initial path is reasonable.

When ran, the program will create the `interpolation` directory in your project directory (and will clear that folder if it already exists!).

```
-Project directory
|
|--interpolation
      |
      |-image001.in
      |
      |-image002.in
      |
      |-...
      |
      |-path.xyz
```

There will be `n_images` +2 aims geometry created, including initial and final geometry. A multi-frame *.xyz* file is also created in the directory for use with visualization softwares that doesn't support multi-file animation. If the geometry is periodic, the *.xyz* file is repeated in each lattice vector according to `xyz_lattice` to make visualization easier. (because the standard xyz does not encode periodic information) Always check the interpolation if you are working with a new project, which can save you lots and lots of time if you happen to spot a bad interpolation(as shown in sample 4).

a note on periodic systems

Running aimsChain on periodic systems requires extra precaution, because initial files from a periodic system can often provide misleading initial path.

The default interpolation algorithm when `periodic_interpolation` works as follows. For each atom in the final geometry, the coordinates are offset by each lattice vector in both the positive and negative direction. The results are compared with the same atom in the initial geometry, and the coordinate with the shortest distance is used as the actual coordinate for the final atom. This method is good in general.

However, it's still possible to imagine cases where a longer path is the actual path. For example, when trying to simulate the effect of an edge on one end. The shortest path may corresponds to climbing over the edge to reach the other end, while the true path, moving in the other direction, is to simply walk away from the edge.

In these cases, the coordinate of the final geometry must be adjusted manually, so that its coordinate corresponds to its true coordinate after the move, and does not involve any periodic boundary condition. `periodic_interpolation` should be turned off, and the interpolated path should be looked in details to ensure that the atoms in the base are also correctly interpolated.

4.8.5 Running the script

When the script is running, it will first generate a few directories and files in the project directory.

```
-Project directory
|
|--forces.log
|
|--climbing_forces.log+
|
|--growing_forces.log*
|
|--iterations
|     |
|     |-iteration0000
|     |     |
|     |     |-aims-chain-node-0.00000
|     |     | |
|     |     | |-aims-chain-node-0.00000.out
|     |     | |
|     |     | |-control.in
|     |     | |
|     |     | |-geometry.in
|     |     |
```

```
|     |     |-aims-chain-node-0.10000
|     |     |     |
|     |     |     |-...
|     |     |
|     |-...
|
|--paths
      |
      |-iteration0000
      |     |
      |     |-image001.in
      |     |
      |     |-image002.in
      |     |
      |     |-...
      |     |
      |     |-path.xyz
      |     |
      |     |-ener.lst
      |     |
      |     |-path.lst
      |
      |-iteration0001
      |     |
      |     |-...
      |
      |-...
```

```
+only generated if climbing image is used
*only generated if growing string method is used
```

Warning! Directories named `optimized`, `paths`, and `iterations` may be cleared/changed if they already exist. Don't leave useful information there!

The `iterations` directory is where all the aims single point calculations are done. Each iteration has its own directory, where each single point calculation is again put in its own directory. They are labelled by the unique id of the particular images as well as the iteration it is in. This is also the place where aimsChain stores optimizer and restart related files.

The `paths` directory contains useful information for you while aimsChain is running. A directory is created for each iteration, containing all the necessary information you might be interested in. The `geometry.in` for each image is written, including the initial and final state. This is the perfect place to extract intermediate path if you would like to start a new job from there. Remember not to include the initial and final geometry in your external geometry list. A *.xyz* animation is written, governed by `xyz_lattice`, which is handy for visualizing the current stage. `ener.lst` stores the current energy of the system, which is extracted from the aims outputs. The energies are offset so that

the initial state is at the zero point. This will give a rough idea of the energy landscape along the path at the current iteration. `path.lst` lists the corresponding path in the `iterations` directory, in case you want to check the aims output for diagnostics.

`ener.lst` and `path.lst` also labels the state each image is in. A "FIXED" label indicates that the images is not been moved from iteration to the next, which can be the case for the initial and final geometry, as well as during climbing image calculation. A "CLIMB" label indicates that this image is the climbing image, and when converged, represents the geometry of the saddle point. Normal nodes are simply labelled "Normal".

There are also files named `forces.log`, `growing_forces.log`, and `climbing_forces.log` in your project directory, which records the residual forces in the system for each iteration. They are the most straightforward way to check the current state of the system.

When the path is converged, the result will be put into a directory named `optimized`. They will have the same geometry as the last iteration in the `paths` directory, but they are copied from `iterations` directory so that the aims outputs are included. The same is down for growing string method. Once the growing process is completed all the relevant info are write to the `grownstring` directory, so you don't have to re-grow your geometries for different calculations.

4.8.6 Tips & Guides On Running

It is important to inspect the calculation once in a while when it's running, so that it can be stopped when going astray.

If the forces are going to extremely big values (several hundreds) after reaching a relatively small value, please have a look at the most recent geometry in `paths` to see if it's physical. Any non-physical path is most likely to have been caused by the BFGS optimizer. You should go backward and find the last iteration at which the geometries are physical (most likely corresponding to a small residual forces), and use that as the initial geometry for a new calculation with non-global FIRE optimizer or trm optimizer.

If the forces goes up and down repeatedly, try switching off the global optimizer. If that doesn't help, try FIRE or trm.

When climbing image is not converging, your `force_thres` might be too large for the system. If you are using `climb_mode` 1, consider extract the converged path before climbing started, and experiment with mode 2. Otherwise start a new calculation with the converged path and a higher threshold.

If you are focusing on achieving a very precise calculation, and your system is not computationally intensive, you may want to do the following. Increase number of images used. Utilize `climb_control` for tighter convergence at climbing image. Use `climb_mode` 2 or 3 for climbing image. Start with non-global FIRE in the first place to avoid potential problems with BFGS.

If you are working with systems that are computationally expensive for aims, try these. Decrease the number of images, but use at least 3. (or more, if you have a complex reaction) First converge a path to a reasonable value.(below 1 $eV/\text{Å}^2$ for example)

Extract the path and try `aims_restart`. Your system may be one of the few that has a consistent wave function configuration as the string evolves to MEP. If the job stops after one iteration, then that has failed, simply re-submit the job after turning the flag off. Set `restart` in both `control.in` and `chain.in`, even if you are not using `aims_restart`. This will save time when your calculation is stopped when exceeding the time limit. Try to use the lightest setting possible for the calculation. Only switch to tight settings after a good result is achieved on the light setting.

When the climbing image is converged, the image that's labeled "CLIMB" in `ener.lst` is the transition state. It may be the case sometime that when using `climb_mode` 1, the converged transition state has a lower energy than its neighbour. This is not a problem in general, its neighbour was not well converged to the path. If you are worried about it, set a tighter convergence threshold or using a different climb mode will help.

4.9 Transition state search: Nudged Elastic Band method

The functionality described in the present section is no longer the preferred option to calculate transition states in FHI-aims. We here keep it as legacy only. Please try the 'aimschain' functionality of the previous section first. Let us know if you are still using the old NEB.f90 version. Else, it will be removed in the near future.

4.9.1 Theory and methods

4.9.2 Usage

The transition state search in AIMS at present is implemented in form of a PERL-script `NEB.pl` that calls a FORTRAN-90 code `NEB.f90`, both are located in the `src`-subdirectory `NEB`. The Fortran code still has to be compiled with `LAPACK` and `BLAS` routines linked in.

The job description of a TS-search is contained in a separate control file with a number of keywords defined below. That control file should be called `control_NEB.in`. You may change the name to anything else you like, just specify the name of the new control file as an argument when calling `NEB.pl`. Note that (at present) a lot of keywords are simply assumed to be there, and not enough testing is done to capture any missing options. This might lead to unexpected runtime behaviour. Please check to make sure everything you (think you) need is contained in the file `control_NEB.in`.

The interface to this program is via multi-frame `xyz` geometry files, which are described under the keyword `input_template`. You also need a control file template for the AIMS calls (see keyword `aims_control_file_template`), which must contain a line of the form
`restart < aims_restart_string >`
where the restart file for the different images can be specified and passed on. Towards the end of a run, this functionality greatly accelerates the DFT calculations. Also make sure that the AIMS control file template contains the two lines
`compute_forces .true.`
`final_forces_cleaned .true.`

The keywords used in the NEB-control file are:

Tag: **`aims_control_file_template`** (control_NEB.in)

- Usage: `aims_control_file_template filename`
- Location of the AIMS template file to be used for the TS search.

Tag: **`aims_restart_string`** (control_NEB.in)

- Usage: `aims_restart_string template_string`
- Marker in the control file to be replaced by the actual restart file of a given iteration.

Tag: `dft_exe` (control_NEB.in)

- Usage: `dft_exe filename`
- `filename` is the AIMS executable including the full path.

Tag: `force_convergence_criterion` (control_NEB.in)

- Usage: `force_convergence_criterion value`
- `value` is the convergence force component on any image coordinate after it has been corrected by the transition state search.

Tag: `input_template` (control_NEB.in)

- Usage: `input_template filename`
- The location of the initial guess for the TS-search. The file name contains the number of the iteration, followed by `.xyz`, while `input_template` should be without those two points.
- Example: An initial guess called `space_dog_0.xyz` would be described with
  `input_template space_dog_`
- This file has to be a multi-frame xyz file with the following format:

```
<n_atoms>
start
<species1> <xstart> <ystart> <zstart>
...
<n_atoms>
image 1
<species1> <x1> <y1> <z1>
...
<n_atoms>
image <n_images>
<species1> <x1> <y1> <z1>
...
<n_atoms>
END
<species1> <x1> <y1> <z1>
...
```

Tag: `max_atomic_move` (control_NEB.in)

- Usage: `max_atomic_move value`
- The maximally allowed displacement during a single NEB step.
- This value should be relatively small (default is 0.1Å) as the algorithm might literally tear apart some of the images if the force constants are not set ideally and the transition path is not extremely close to the actual path.
- Same as keyword `max_atomic_move` in AIMS

Tag: `line_step_reduce` (control_NEB.in)

- Usage: `line_step_reduce value`
- for BFGS: factor by which a line step will be reduced if the object function increased more than `object_function_tolerance` between two successive iterations.
- at present only useful for `method` PEB, as there is no implementation of an object function for the other methods.
- This is the same as the original AIMS keyword `line_step_reduce`

Tag: `method` (control_NEB.in)

- Usage: `method flag`
- `flag` can be either PEB or NEB for the pure elastic band and the nudged elastic band methods respectively.

Tag: `min_line_step` (control_NEB.in)

- Usage: `min_line_step value`
- `value` is the minimal line step for the BFGS solver below which the code simply executes a step in order to get out of any numerical noise.
- same as the keyword `min_line_step` in the original AIMS code.

Tag: `n_atoms` (control_NEB.in)

- Usage: `n_atoms value`
- `value` is the number of atoms in each image.

Tag: `n_images` (control_NEB.in)

- Usage: `n_images value`
- `value` is the total number of images, not counting the start and end points.

Tag: `n_iteration_start` (control_NEB.in)

- Usage: `n_iteration_start value`
- Default `n_iteration_start = 0`
- If there is data available from a previous run of the same system, it might start at iteration `value` rather than from the beginning.
- Careful that you use the proper `save_data` file if using this feature!

Tag: `n_max_iteration` (control_NEB.in)

- Usage: `n_max_iteration value`
- `value` is the maximal number of iteration of the search.

Tag: `object_function_tolerance` (control_NEB.in)

- Usage: `object_function_tolerance value`
- An iteration is rejected when the object function increases more than `value` between two successive iteration.
- This only works for the pure elastic band `method` as there is no object function in the current NEB implementation.
- Same function as keyword `energy_tolerance` in the main AIMS code.

Tag: `save_data` (control_NEB.in)

- Usage: `save_data filename`
- Gives a place to store BFGS and other information between successive iteration.
- Be careful to delete this file when starting a new NEB run.

Tag: `spring_constant` (control_NEB.in)

- Usage: `spring_constant value`
- `value` is the single fixed spring constant throughout a single NEB run.
- Using this is either default or it should be done in conjunction with `use_variable_spring_constants .false.`

Tag: `spring_constant_max` (control_NEB.in)

- Usage: `spring_constant_max value`
- Maximal value in a range of possible spring constants
- can only be used together with the option `use_variable_spring_constants .false.`

Tag: `spring_constant_min` (control_NEB.in)

- Usage: `spring_constant_min value`
- Minimal value in a range of possible spring constants
- can only be used together with the option `use_variable_spring_constants .false.`

Tag: `start_BFGS` (control_NEB.in)

- Usage: `start_BFGS value`
- Force convergence criterion below which the algorithm switches from a steepest-descent type approach to BFGS algorithm.

Tag: `switch_to_CI-NEB` (control_NEB.in)

- Usage: `switch_to_CI-NEB value`
- will switch to using the climbing image nudged elastic band method after reaching a NEB force convergence of `value`

Tag: `use_variable_spring_constants` (control_NEB.in)

- Usage: `use_variable_spring_constants .true./.false.`
- If `.true.` a range of spring constants that depend linearly on the energy [?] is used.
- `.true.` Requires setting of the keywords `spring_constant_min` and `spring_constant_max`

4.10 Plugin for free-energy calculations with molecular dynamics: PLUMED

Molecular dynamics based free-energy calculations can be performed with the aid of the external plugin PLUMED.
Methods included are metadynamics [?], well-tempered metadynamics [?], umbrella sampling [?, ?, ?], Jarzynski-equation based steered molecular dynamics [?, ?]. A large and nearly exhaustive set of collective variable (CV) is accessible through a simple input script.

PLUMED is a free package that, after registration, can be downloaded from http://merlino.mi.infn.it/~plumed/PLUMED/Home.html Currently, a copy of the PLUMED library is kept in the *external* directory of the FHI-aims source code, and must be compiled separately using the makefile Makefile.meta (see section 1.6). In the future a patch for modifying FHI-aims in order to compile it with PLUMED will be available on the PLUMED webpage.

4.10.1 Usage

The actual use of the plugin is switched on by this single line in `control.in`:

```
plumed .true.
```

With `plumed .false.` (default) or nothing, the code would behave exactly as compiled without this plugin. It is implied that some MD scheme must be used in `control.in`, in order to see PLUMED acting. What PLUMED does, in facts, is to modify the molecular dynamics forces according to the selected scheme.
All the specific controls of the free energy calculation are contained in the file `plumed.dat` (which must be in the working directory, together with `control.in` and `geometry.in`, if `plumed .true.` is set). For all the details on `plumed.dat`, we defer to PLUMED manual which can be found on the project website.
Here we report a minimal example for metadynamics:

```
PRINT W_STRIDE 10
DISTANCE LIST 1 <g1> SIGMA 0.35
g1->
2 3 4
g1<-
HILLS HEIGHT 0.003 W_STRIDE 10
ENDMETA
```

This script would make PLUMED deposit Gaussians (HILLS) of HEIGHT 0.003 hartree, every W_STRIDE timesteps. The (only) CV that will be biased by metadynamics is a distance between atom '1' and the center of mass of atoms '2', '3', and '4'. The number

labelling the atoms follows their order of appearance in geometry.in. The results will be printed (see below) every PRINT W_STRIDE time steps. The width of the Gaussian for the distance CV is specified by SIGMA

A note on the units: the units in plumed.dat and in the output(s) are the internal ones in FHI-aims, i.e. energies in hartree, distances in bohr, forces in hartree/bohr.

When using PLUMED, some extra output files are created. In log.dat the specifics of the run are given. COLVAR contains the trajectory of the selected CVs. Notably, if no biasing method is selected, but one or more CVs are defined in plumed.dat, PLUMED prints nonetheless the trajectory of those CVs in COLVAR (one can also explicitly switch off the biasing of *some* CVs via the NOHILLS directive).
In case metadynamics is used, then also HILLS is generated, which contains the informations for reconstructing the free energy profile. This is done with the postprocessing tool, "sum_hills", which is given with the distribution.
For umbrella sampling a powerful tool for reconstructing the free-energy from COLVAR, can be downloaded from: http://membrane.urmc.rochester.edu/Software/WHAM/WHAM.html.

4.11 Script based parallel tempering (a.k.a. replica exchange)

A script based parallel tempering implementation is available. Part of the script is dependent on the particular batch-queueing system in use; with the distribution, we provide a solution that has been tested on linux machines with SGE batch-queueing system. Whereas the overall structure of the batch script would not change by changing the batch-queueing, few crucial lines might need intervention.

4.11.1 Usage

In order to run the parallel tempering the batch script "`submit.rex`" must be submitted to the queueing system. The batch script:

1. creates a subdirectory "rex_??" for each replica,
2. copies the files needed for the FHI-aims run and runs them
3. manages the swaps between replicas.
4. prints outputs

The files that have to be present in the working directory are:
`control.in.basic`
`control.in.rex`
`geometry.in.basic`
optional: `list_of_geometries`
`rex.AIMS.pl`
`submit.rex`
The last two files are provided with the distribution and are contained in the subdirectory `utilities/REX`.

- `control.in.rex`, it must contain the following lines:
  `n_rex` number of replicas
  `temps` list of target T separated by a space; the number of T's must agree with the above line
  `freq` time interval between rex swaps, in ps, as in control.in
  `MAX_steps` maximum number of replica exchange steps (i.e., the whole simulation will contain MAX_steps*freq ps per replica)

- `control.in.basic`, as in FHI-aims. Note, though, that the script will delete any keywords about geometry relaxation and MD, with the exception of MD_time_step, and appends at the end of each control.in in each subdirectory the MD_settings for the replica exchange. In detail, the following are the lines which are managed

by the script:
```
MD_run $t NVT_parrinello $temp[$i+1] 0.1
MD_MB_init $temp[$i+1]
MD_restart .true.
MD_clean_rotations .true.
output_level MD_light
```
where `$t` is a multiple of the "`freq`" keywords in `control.in.rex`, updated at each MD substep between swaps, and `$temp[$i+1]` is the target temperature for the particular replica and parallel tempering step. These lines are hard coded in the perl script `rex.AIMS.pl`.

- `geometry.in.basic`, written in the geometry.in format. It will be copied into each subdirectory, so that each replica would start form the same geometry.

- optional: `list_of_geometries` If present, it must contain a list of geometry files (each in the geometry.in format), one line each, that must be present in the working directory. The script will copy the file in the first line into the first subdirectory (i.e. related to the first temperature in `control.in.rex`), and so on. In case `list_of_geometries` contains less lines than the defined number of replicas, the "exceeding" replicas will start with the geometry contained in `geometry.in.basic`.

- `rex.AIMS.pl`, managing perl script. Nothing to be done here, in principle. If invoked as
  `perl rex.AIMS.pl stat <log_file>`
  in a directory that contains a log_file created by rex.AIMS.pl itself (see next section), it provides useful statistics (even on the fly).

- `submit.rex` is the batch script. Some attention form the user is required here, too.

  - select the total number of slots with the keyword "`# $ -pe impi`", according to the number of replicas. For performance reasons only, it is a good idea to have the number of slots be a multiple of the number of replicas (`n_rex` in `geometry.in.rex`). Informations and warnings concerning this issue will be written to `log_rex`.
  - give the variable `type` the value '`init`' or '`restart`', according to the kind of run. Note that by running a '`restart`', the script will complete the possibly interrupted parallel tempering steps (also only in some of the subdirectories) and then will continue with the replica exchange algorithm.
  - set the proper name and path for the aims binary
  - set the number of slots per node (host) with `ncpupn=<#SlotsPerNode>`. This is particularly important for the right distribution of available slots. For performance reasons only, it is a good idea to have the number of slots per replica be a multiple of the number of slots per node (`ncpupn`) or vice versa. Informations and warnings concerning this issue will be written to `log_rex`.

Below, the relevant area for the settings is reported:

```
################### to be taken care of by the user ###############
binary='<binary path and name>'
# put type='init', if initializing, 'restart' if restarting
type='init'
# type='restart'
# number of CPU per node (host)
ncpupn=<#SlotsPerNode>
#####################################################################
```

- `run_rex.sh` is a bash script in order to run locally
  - serves as a substitution for `submit.rex` if the SGE is not available
  - if possible, use `submit.rex` because of performance reasons due to the more sophisticated distribution of jobs over the available slots (CPU)

4.11.2 Output

- in each of the subdirectories `rex_??` there are the files:
  - `temp.out` full FHI-aims output for the parallel tempering tempering step
  - `control.in` and `control.in`, the usual FHI-aims input files. They will change at each parallel tempering step, managed by the script.
  - `energy.trajectory`. Cumulative (i.e. appended after each attempted swap) energy trajectory for the replica.
  - `out.xyz`. Cumulative geometry trajectory, in `xyz` format.
- in the working directory: `log_rex`. It contains useful information on the swapping process. Below there is a commented example for a four replicas run.

`> Mon Apr 5 03:51:05 CEST 2010`
The time at the attempeted swap
`> Tt 100.0 200.0 150.0 250.0`
The list of the running target temperatures, first place for `rex_00`, and so on
`> map 1 3 2 4`
Map of the temperatures in the "`Tt`" line, into the original list given in `control.in.rex`
`> TE -6963471.3877 -6963471.2516 -6963471.4951 -6963471.3286`
Total Energy ("`Total energy (el.+nuc.)`") in each replica (first item in `rex_00` and so on)
`> swapping 3 1 @T 150.0 100.0 accepted`
`> swapping 4 2 @T 250.0 200.0 rejected`
Detail of attempeted swaps, with outcome
`> temp 150.0 200.0 100.0 250.0`
List of running target temperatures, after swaps.
`> vfact 1.04880884817015 1 0.9534625892455937218 1`

Rescaling coefficients for the velocities in each replica, for the next step
> ####### End of rex step #################

WARNING: when wall-clock ends in the middle of a prallel tempering step, it will always be printed the message:
  WARNING: rex_??/temp.out Not converged?
Please check this problem before continuing.
If the reason that any of temp.out's does not reach not the end of the parallel tempering step is the end of the wall-clock time, then the run can be safely restarted by putting 'type=restart' in submit.rex

- It is also possible to restart (prolong) a job that has been completed successfully, i.e. after the desired number of Replica Exchange steps has been performed. In order to do so, set 'type=restart' in submit.rex, set 'MAX_steps <#MaxSteps>' in control.in.rex according to the (new) desired maximum number of steps, and replace the third number in rex_par with that same number '<#MaxSteps>.

- in the working directory: out.????, where ???? is a temperature, in 4 digits. Constructed by appending the temp.out temporary outputs at the same temperature, each out.???? contains the full FHI-aims output at the given temperature.

4.12 Formation energies of charged defects

The Gibbs free energy of formation of a defect is given by

$$\Delta G_f^D = E_{tot}^D - E_{tot}^{perf} - \sum_i n_i \mu_i^{ref} + q\varepsilon_F^{ref} - \sum_i n_i \Delta\mu_i + q\Delta\varepsilon_F, \qquad (4.15)$$

where E_{tot}^D and E_{tot}^{perf} are the total energies of the defected and the perfect sytem, n_i is the number of atoms of type i added (> 0) or removed (< 0), $\Delta\mu_i$ are the corresponding atomic chemical potentials referenced to μ_i^{ref}, $\Delta\varepsilon_F$ is the Fermi level referenced to ε_F^{ref} and q is the charge of the system.
A common choice as a reference for the electron chemical potential ε_F^{ref} is the valence band maximum (VBM), so that the Fermi level can be assumed in the range between the VBM and the conduction band minimum (CBM), but in principle the choice of references for the chemical potentials is arbitrary.
FHI-aims uses the Ewald summation technique to calculate the electrostatic Hartree potential for a periodic system. For a charged periodic system (specified by the keyword `charge` in `control.in`) a neutralizing homogeneous background charge density is introduced to remove the divergent $\mathbf{G} = 0$ component of the long-range part of the electrostatic potential.
This scheme is not suitable for periodic surface models, because the background charge density would be spread over the whole unit cell including the vacuum region. Instead charged surface defects can be treated within a virtual crystal approach (VCA), which corresponds to distributed doping of the material. The following scheme can be used for an insulating system with a localized defect level in the bandgap. By modifying the charge of the atomic nuclei (using the keyword `nucleus` in `control.in`), while keeping the system neutral, additional delocalized states can be introduced at the top of the valence band or at the bottom of the conduction band. The occupation of the defect levels can thus be tuned by the amount of charge distributed on the cations or anions in the system. To ensure that the defect has the desired charge q, the sum of the modified nuclear charges Z'_i should differ from the sum of the original nuclear charges Z_i by the value of q:

$$\sum_i^{N_{atoms}} Z'_i = \sum_i^{N_{atoms}} Z_i - q.$$

Note that for calculating the formation energy of a charged defect within the VCA the reference system should be the doped undefected system, not the perfect undoped system. Since doping pins the Fermi level $\Delta\varepsilon_F$ vanishes for this method.
For example, a way to model a positively charged oxygen vacancy at a metal oxide Me_xO_y surface is to distribute the charge uniformly on the metal atoms Me by changing their nuclear charge from Z(Me) to

$$Z(Me^{VCA}) = Z(Me) - \frac{q}{N(Me)},$$

where N(Me) is the number of metal atoms in the system. This introduces vacant states at the VBM which in the defected system will be occupied by electrons from the defect level.

When using the neutralizing background method for bulk systems the additional term may introduce an arbitrary shift, so that it is necessary to find a common energy reference for the charged and the neutral periodic system to which the respective potentials can be aligned. For example alignment of the core levels of an atom far away from the defect can be done according to

$$\Delta\varepsilon_{\mathsf{F}} = (\varepsilon_{\mathsf{F}} - \varepsilon^{\mathsf{D}}_{\mathsf{core}}) - (\varepsilon^{\mathsf{perf}}_{\mathsf{VBM}} - \varepsilon^{\mathsf{perf}}_{\mathsf{core}}).$$

Plot the atom projected density of states (output option `output atom_proj_dos` in `control.in`) for this atom for the charged defected and the neutral perfect system to visualize changes in the core states. (Be aware that in an all-electron approach the deeply lying core states are sensitive to local changes in electron density due to relaxation and charge redistribution, so that their shift in a defected system with respect to the perfect host system may not include only the average potential shift.)
Due to spurious electrostatic interaction as a result of the employed periodic boundary conditions the formation energy of a charged defect depends on the dimensions of the supercell. The formation energy scales as $\Delta\mathsf{G}^{\mathsf{D}}_{\mathsf{f}}(L) \approx \mathsf{a}\frac{1}{L} + \mathsf{c}$ for sufficiently large supercells [?]. For a simple cubic unit cell L corresponds to the supercell lattice constant and can take up integer multiples of the unit cell lattice constant $\mathsf{L}^{(0)}$. For differently shaped unit cells with lattice constants $\mathsf{L}^{(0)}_1$, $\mathsf{L}^{(0)}_2$, $\mathsf{L}^{(0)}_3$ set for example $\mathsf{L}{:=}\mathsf{L}_1$ and build supercells $L_1 = \mathsf{n}\cdot\mathsf{L}^{(0)}_1$, $L_2 = \mathsf{n}\cdot\mathsf{L}^{(0)}_2$, $L_3 = \mathsf{n}\cdot\mathsf{L}^{(0)}_3$ with integer n. The desired formation energy of a single defect in an infinite supercell $\Delta\mathsf{G}^{\mathsf{D}}_{\mathsf{f}}(L\to\infty)$ can then be obtained by extrapolation. Note, that the convergence of the extrapolated energy with respect to the supercell size should be tested carefully. Taking into account geometric relaxation can improve the convergence significantly. Alternatively postprocessing correction schemes that allow to remove the spurious interaction terms have been suggested in literature [?, ?].

Chapter 5

The AITRANSS package

The AITRANSS (*ab initio* transport simulations) package is a project under continuous development at the Institute of Nanotechnology of the Karlsruhe Institute of Technology (KIT), Germany, since 2002. In brief, when combined with FHI-aims, AITRANSS provides a post-processor module that enables, e.g., calculation of the electron transport characteristics of molecular junctions based on a Landauer formalism in a (non-equilibrium) Green's function formulation.

Currently, the version of the code accessible to FHI-aims users is limited to computation of the ballistic (Landauer-Büttiker) transmission function and partial atom-projected density of states. According to current planning advanced options, e.g., out of equilibrium transport response, will be available in the future releases.

A discussion of the underlying physical formalism and details of the implementation are described in the references [?, ?, ?]:

> A. Arnold, F. Weigend, and F. Evers, "Quantum chemistry calculations for molecules coupled to reservoirs: Formalism, implementation, and application to benzenedithiol." J. Chem. Phys. **126**, 174101 (2007).
>
> J. Wilhelm, M. Walz, M. Stendel, A. Bagrets, and F. Evers, "Ab initio simulations of scanning-tunneling-microscope images with embedding techniques and application to C58-dimers on Au(111)." Phys. Chem. Chem. Phys. **15**, 6684 (2013).
>
> A. Bagrets, "Spin-polarized electron transport across metal-organic molecules: a density functional theory approach." J. Chem. Theory Comput. **9**, 2801 (2013).

Please, cite the above works together with FHI-aims publications, when using AITRANSS.

For questions and bug reports, contact Alexej Bagrets (`Alexej.Bagrets@kit.edu`).

5.1 Source code and supporting materials

The source code and supporting material of the AITRANSS-module for the FHI-aims package is placed in the subdirectory `aitranss/`. This directory contains subdirectories:

- `source/` : with the Fortran90 code and the example `Makefile` ;
- `tcontrol.script/` : contains a script `tcontrol.aims.x`, which is served to prepare a mandatory input file `tcontrol` for the transport-type calculation ;
- `electrodes.library/` : contains a library of representative gold (Au) clusters (xyz-files) which should be linked, via anchoring groups, to your molecular system to create an "extended molecule": its electronic structure (Kohn-Sham molecular orbitals and energies) is a prerequisite to compute transport characteristics ;
- `examples/` : contains examples, with input and output files of the FHI-aims and AITRANSS; `README` files found in this subdirectory contain also short guidelines on how an input for a particular transport calculation has been created.

5.2 Compiling the AITRANSS module

Please, use a template of the `Makefile` found in the directory `source/`, and adjust variables referring to compiler (`FC` and `LD`), compiler's options (`FLAGS`) and a path to libraries (`LIBS`) at your computer system. A mandatory prerequisite to build the code is a Fortran 90/95 capable compiler and a compiled version of LAPACK and BLAS (for example, Intel's MKL). A binary (`aitranss.x`) built by the `Makefile` will go to the `bin/` directory of the FHI-aims.

In contrast to FHI-aims, the current release of AITRANSS is not yet based on MPI. However, you are encouraged to use a fortran compiler option(s), aka `"-openmp"` and `"-O2"` for Intel's `ifort`, to enable the auto-parallelizer to build a multithreaded code based on OpenMP directives.

According to our experience, a generated code can be safely executed in parallel within a single compute node with multiple processors, and with a significant gain in computation time.

We advise you as well to copy a script `tcontrol.aims.x` found in the directory `tcontrol.script/` to the directory `bin/` of the FHI-aims installation, and to make files in that directory accessible for the execution from a command line by adjusting your shell variable `PATH`.

5.3 How to set-up and run transport calculations

5.3.1 FHI-aims run: input and output

Having your molecule "at hands", use your favorite modeling and visualization tools/-software, and prepare an extended structure by linking the molecule via anchoring groups to two atomic clusters, representing parts of macroscopic *source* and *drain* electrodes. Consult the `electrodes.library/` directory, and use predefined Au clusters found there. A typical example of an "extended molecule", which you are requested to build, is shown in Fig. 5.1.

Include a line

```
output   aitranss
```

into your `control.in` file. Furthermore, following settings are recommended for the self-consistent DFT calculation:

```
occupation_type    gaussian 0.1
mixer              pulay
  n_max_pulay        10
  charge_mix_param   0.2

sc_accuracy_rho    1E-4
sc_accuracy_eev    1E-2
sc_accuracy_etot   1E-6

relativistic zora scalar 1.0e-10
```

Invoke FHI-aims exploiting a cluster type (non-periodic) calculation. After the FHI-aims run is finished, you'll find in your directory three ASCII files: `basis-indices.out`, `omat.aims` and `mos.aims`. These files contain: some limited information on basis functions; overlap integrals; and data on Kohn-Sham molecular orbitals & energies of the "extended molecule", respectively. If spin channels of your system are not identical, `mos.aims` will be substituted by two other files called `alpha.aims` and `beta.aims`.

5.3.2 What to be aware of before running AITRANSS module

The AITRANSS module should be run from the same directory, where output files of FHI-aims are placed. A file `geometry.in` is mandatory and should also be there.

A file `control.in` is not used. Instead, another mandatory file for the transport calculation is `tcontrol`. Please, *always* use a script `tcontrol.aims.x` to create this file. Executing a script `tcontrol.aims.x` without arguments outputs a help information:

```
[...]
-----------------------------------------------------------------
"tcontrol.aims.x"   script creates a mandatory file "tcontrol"
                    which is required to run the "aitranss"
                    post-processing module after FHI-aims
-----------------------------------------------------------------
USAGE: tcontrol.aims.x [ -option <argument> ] ...

where options & arguments are:

                        ! electrodes geometry:
 -lsurc <atom1>         three atoms which define an outermost
 -lsurx <atom2>         LEFT surface layer of the extended
 -lsury <atom3>         molecule

 -rsurc <atom4>         three atoms which define an outermost
 -rsurx <atom5>         RIGHT surface layer of the extended
 -rsury <atom6>         molecule

 -nlayers <number>      number of atomic layers coupled to
                        reservoirs via a self-energy

                        ! energy window, in Hartree [H], to
                        ! output transmission function T(E) :
 -ener  <E1[H]>         initial energy point, E1
 -estep <dE[H]>         energy step, dE
 -eend  <E2[H]>         final energy point, E2

                        ! output :
 -outfile <file_name>   output file name for T(E) [default: TE.dat]
```

When executed with options and arguments, a script `tcontrol.aims.x` checks for the `geometry.in` file and other mandatory FHI-aims output files (`basis-indices.out`, `omat.aims`, `mos.aims` or `alpha.aims` & `beta.aims`) in your directory, reads from these files information on a system size and the Hamiltonian H and overlap matrix dimension, and exports this information together with your arguments to an ASCII file `tcontrol`. Options and arguments are used: (i) to provide information on the self-energy construction; (ii) to introduce an energy window for the calculation of the transmission function $T(E)$, and (iii) (optionally) to introduce an output file name for $T(E)$.

Comment on the self-energy. When an "extended molecule" is contacted to macroscopic reservoirs, a propagation of an electron with energy E within a subspace limited by the "extended molecule" is described by the Green's function: $G^{-1}(E) = E - H - \Sigma(E)$, where a self-energy $\Sigma(E)$ accounts for the interaction between a finite system and macroscopic reservoirs. As argued in Refs. [**?**, **?**], if atomic clusters introduced to model parts of metallic electrodes are large enough, the reservoirs can be modeled by absorbing

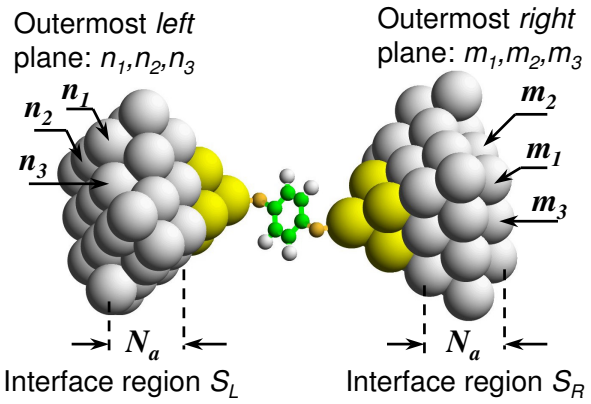


Figure 5.1: A schematic view of the "extended molecule". The shaded regions are interfaces to the two reservoirs. Within the interface regions absorbing boundary conditions are active and the self-energy Σ is introduced. A user defines interface regions by specifying the outermost *left* and *right* atomic planes (introducing indices of three different atoms that form a triangle, n_1, n_2, n_3, and m_1, m_2, m_3, respectively) and the amount of atomic layers, N_a.

boundary conditions which become active at the interface regions $\mathcal{S}_L$ and $\mathcal{S}_R$ (labeled by a gray color in Fig. 5.1) where the "extended molecule" is coupled to reservoirs. Within this model, the self-energy is approximated by an energy-independent, diagonal matrix,

$$\Sigma_{nn'}^{\mu\mu'} \approx -i\eta_n \delta_{nn'} \delta_{\mu\mu'},$$

where indices n and n' label atoms, and μ and μ' label corresponding internal degrees of freedom (i.e., atom-centered basis functions). Here absorption rates η_n are allowed to have non-zero weights only within the interface regions $\mathcal{S}_L$ and $\mathcal{S}_R$. A user defines interface regions by specifying the outermost *left* and *right* atomic planes (introducing indices of three different atoms forming a triangle, n_1, n_2, n_3, and m_1, m_2, m_3, respectively) and the amount of atomic layers, N_a.

5.3.3 How to create a mandatory file `tcontrol`

To create a `tcontrol` file, a `tcontrol.aims.x` script should be launched with the following options and arguments:

`tcontrol.aims.x -lsurc` n_1 `-lsurx` n_2 `-lsury` n_3 `-rsurc` m_1 `-rsurx` m_2 `-rsury` m_3 `-nlayers` N_a `-ener` E_1 `-estep` dE `-eend` E_2

- where integer numbers n_1, n_2, n_3 are indices of three different atoms fixing the outermost *left* atomic plane of the "extended molecule" (see Fig. 5.1); atoms are numbered according to their appearance in file `geometry.in` ;
- integer numbers m_1, m_2, m_3 are indices of three different atoms fixing the outermost *right* atomic plane of the "extended molecule" (see Fig. 5.1);

- an integer N_a indicates the number of atomic layers defining the interface regions $\mathcal{S}_L$ and $\mathcal{S}_R$ (see Fig. 5.1); users are strongly advised to take Au clusters of similar size from the directory `electrodes.library/` and use a parameter N_a suggested in the header of a library file. Using Au clusters of similar size insures that N_a can be consistently chosen to be the same for both *left* and *right* interface regions;
- real numbers E_1, E_2 and a positive real number dE define the energy window $[E_1, E_2]$ and energy step dE to calculate transmission function $T(E)$;
- optionally, you can launch the script with "`-output` *filename*" where *filename* is a string, specifying a name of the output file for $T(E)$.

An example of the transport calculation set-up for the Au-benzene-dithiol-Au junction can be found in the directory `examples/au-c6h6-au/`. A script `tcontrol.aims.x` has been executed there with the following options and arguments:

```
tcontrol.aims.x -lsurc 34 -lsurx 30 -lsury 36 -rsurc 35 -rsurx 33
 -rsury 31 -nlayers 2 -ener -0.4000 -estep 0.0001 -eend 0.0000
```

to create the following file `tcontrol`:

```
#input data for the aitranss module
$aims_input on
$landauer on
$coord   file=geometry.in
$natoms  50
$basis   file=basis-indices.out
$read_omat file=omat.aims
$scfmo   file=mos.aims
$nsaos   2268
$ecp     on
$lsurc   34
$lsurx   30
$lsury   36
$rsurc   35
$rsurx   33
$rsury   31
$nlayers 2
$s1i     0.1d0
$s2i     0.05d0
$s3i     0.025d0
$ener   -0.4000
$estep   0.0001
$eend    0.0000
$output  file=TE.dat
$testing off
$end
```

Detailed description of keywords in file `tcontrol` are given in the paragraph 5.4.

5.3.4 How to submit a transport calculation and its output

Assuming a binary file `aitranss.x` is placed in the directory `bin/` of the FHI-aims installation which is referenced in your shell variable `PATH`, you can run the transport type calculation from a command line, like

```
> aitranss.x | tee my.transport.calc.out
```

or

```
> nohup aitranss.x > my.transport.calc.out &
```

FHI-aims output files together with information provided from files `tcontrol` and `geometry.in` will be used by the AITRANSS module to reconstruct a Kohn-Sham Hamiltonian (H) of the "extended molecule", to supplement H with the self-energy Σ, and to compute a ballistic (Landauer-Büttiker) transmission function, exploiting Green's function formalism.

After calculation is finished, you'll get two files. The first file with a default name `TE.dat` contains information on the transmission function, $T(E)$. Data in this file are arranged in columns as indicated in a file's header: (i) first column is energy in Hartree (atomic) units; (ii) second column is energy in eV units given with respect to the Fermi energy, which is also stated in the file's header; (iii) third column contains data on ballistic transmission per spin. If spin channels of your system are different, there will be present third and forth columns, referring to transmission in up-spin (α) channel and down-spin (β) channel, respectively. A contribution to conductance due to spin σ electrons is given by transmission at the Fermi energy, $T_\sigma(E_F)$, in units e^2/h.

The second file has a name `self.energy.in`, and contains information about the model self-energy construction. A format of this file is similar to the format of `geometry.in`, where each line corresponds to one atom in the structure, and atom's specific data are arranged in columns: (i) columns from 1st till 5th are atom index n, its x, y and z-coordinates (in Å), and atomic symbol, respectively; (ii) 6th column can contain entries `left`, `right` or `empty`, depending on whether a given atom n is a part of the left interface region $\mathcal{S}_L$, right interface region $\mathcal{S}_R$, or does not belong to none of them (see Fig. 5.1 for details); (iii) last column contains a real number in a "double precision" `fortan`-like format: this number is a local leakage rate η_n, in Hartree units, at given atom n (for details, please see a comment on the model self-energy construction, section 5.3.2).

Default leakage rates η_n's, used for construction of the model self-energy, have extensively been tested in numerous previous transport studies with Au-electrodes. These rates are referenced also in the `tcontrol` file and controlled by the keywords (tags) `$s1i` , `$s2i` and `$s3i` as explained in detail in the following paragraph 5.4.

Warning: If you are not a transport-expert user and employ the AITRANSS module as a "black-box" for routine transport calculations, we advise you against modifying the default parameters. This may easily lead to misleading or even completely incorrect results.

If the post-processor transport calculation is resubmitted, a previously created file `self.energy.in` will be used to initialize the self-energy matrix, as controlled by the keyword `$self_energy` in file `tcontrol`, see section 5.4.

5.3.5 Further option: local density of states

Besides a calculation of the ballistic transmission function, you may use AITRANSS to output local density of states projected on groups of atoms forming a molecular junction. This option is controlled by a keyword `$ldos` and described in detail in the subsequent section 5.4.

5.4 Keywords of file `tcontrol`

All keywords (tags) of `tcontrol` file begin with a special symbol $. Lines starting from # are comment lines. All lines after a keyword `$end` are ignored.

Tag: `$aims_input` (tcontrol)

Usage: `$aims_input on`

Purpose: mandatory flag, sets up the FHI-aims like format of input/output files.

Tag: `$landauer` (tcontrol)

Usage: `$landauer flag`

Purpose: mandatory keyword; `flag` should take values either `on` or `off`. If `flag` is `on`, ballistic transmission function is computed. If `flag` is `off`, calculation of the transmission function is not performed. In this case you, however, may wish to compute atom projected local density of states that is controlled by the keyword `$ldos` .

Tag: `$ldos` (tcontrol)

Usage: `$ldos flag`

Purpose: optional keyword; `flag` can take values `on` or `off`. If `flag` is `on`, atom projected local density of states (LDoS) is computed; otherwise (`flag` is `off`) calculation of LDOS is not performed.

On output, the (energy dependent) density of states of the system is projected onto atomic orbitals of atoms marked by the same chemical symbols and is redirected to corresponding files, which have the self-explanatory names, e.g. `ldos.au.dat`, `ldos.c.dat`, `ldos.h.dat` and `ldos.s.dat` in the case of benzene-dithiol molecular junction shown in Fig. 5.1. For example, the file `ldos.c.dat` would contain LDoS summed up over all six C atoms of the molecule. Furthermore, only LDoS projected onto those electrodes' atoms of the "extended molecule" which are not part of the interfaces to the reservoirs (shown in grey color in Fig. 5.1) is redirected to the file `ldos.au.dat`.

Data in the output files are arranged in columns as indicated in the file's header, namely:

energy in Hartree (atomic) units; energy in eV units given with respect to the Fermi energy; LDoS (in 1/eV units) in the α spin channel; if open shell calculation is chosen, next column contains LDoS (in 1/eV units) in the β spin channel; last column contains LDoS summed up over two spin channels.

There is a possibility to arrange atoms having the same chemical symbol in groups by using the sixth field of the line staring from `'atom'` as appears, e.g., in the `geometry.in` file (see also a keyword `$coord`). For example, LDoS projected on the two Au atoms indicated by the mask 'apex' as shown in the example below,

```
atom     0.1424445     0.1134435     4.4557346     Au    apex
...
atom    -0.1689195     0.0042317    -4.4601905     Au    apex
```

would appear in the file `ldos.au_apex.dat`. *Caution:* maximum 16 characters can be used to designate a group of atoms.

Tag: **`$coord`** (tcontrol)

Usage: `$coord` `file=`*geo-filename*

Purpose: mandatory keyword; sets up a file name with atomic positions (in FHI-aims format); *geo-filename* is a text string without spacings, e.g., `geometry.in`

A specified file should be present in your directory. The whole string, `file=`*geo-filename*, should not contain any spacings.

Tag: **`$natoms`** (tcontrol)

Usage: `$natoms` `n`

Purpose: mandatory keyword, specifies number of atoms in the "extended molecule"; `n` is a positive integer number.

Tag: **`$basis`** (tcontrol)

Usage: `$basis` `file=`*basis-filename*

Purpose: mandatory keyword; sets up a file name with information on basis functions quantum numbers as written out by the FHI-aims; *basis-filename* is a text string without spacings, default file name is `basis-indices.out`

A specified file should be present in your directory. The string `file=`*basis-filename* should not contain spacings.

Tag: **$read_omat** (tcontrol)

Usage: $read_omat file=*overlap-filename*

Purpose: mandatory keyword, sets up a file name with overlap matrix elements as written out by the FHI-aims; *overlap-filename* is a text string without spacings, default file name is omat.aims.

A specified file should be present in your directory. The string file=*basis-filename* should not contain spacings.

Tag: **$scfmo** (tcontrol)

Usage: $scfmo file=*mos-filename*

Purpose: mandatory keyword in case of non-spin-polarized calculation; sets up a file name with self-consistent-field molecular orbitals (e.g., Kohn-Sham wave functions) as written out by the FHI-aims; *mos-filename* is a text string without spacings, default file name is mos.aims.

A specified file should be present in your directory. The string file=*mos-filename* should not contain spacings.

Tags: **$uhfmo_alpha** (tcontrol)
$uhfmo_beta (tcontrol)

Usage: $uhfmo_alpha file=*alpha-filename*
$uhfmo_beta file=*beta-filename*

Purpose: mandatory keywords in case of spin-polarized calculation; set up file names with self-consistent-field molecular orbitals (e.g., Kohn-Sham wave functions) as written out by the FHI-aims for α (up-spin) and β (down-spin) electrons, respectively; *alpha-filename* and *beta-filename* are text strings without spacings, default file names are alpha.aims and beta.aims.

Specified files should be present in your directory. The strings file=*alpha-filename* and file=*beta-filename* should not contain spacings.

Tag: **$nsaos** (tcontrol)

Usage: $nsaos N

Purpose: mandatory keyword, specifies dimension N of the overlap matrix and a single-particle Hamiltonian of the "extended molecule".

N is a positive integer number; its value can be found in the header line of default output FHI-aims files omat.aims and mos.aims (or alpha.aims and beta.aims).

Tag: `$ecp` (tcontrol)

Usage: `$ecp flag`

Purpose: optional keyword, `flag` can take values `on` or `off`. Default (and recommended) value is set to `on` and serves to substantially accelerate electron transport calculations.

In that case, exploiting Green's function formalism, "core" electronic states of atoms with $Z \geq 19$ are integrated out and projected on a subspace of the remaining ("valence") electronic states. Thus, dimension of the effective Hilbert space of the "extended molecule" is reduced (see `$valence_electrons`), and only valence states are coupled to macroscopic reservoirs via model self-energies. For atoms from the n-th period of the periodic table, core states are associated with those "atomic" basis functions, which have the principle quantum numbers up to $n - 2$.

Tag: `$valence_electrons` (tcontrol)

Usage: `$valence_electrons` N_{val}

Purpose: optional keyword, N_{val} is integer number, which is evaluated automatically and specifies amount of valence states in the "extended molecule" when a keyword `$ecp` is switched on.

Tags: `$lsurc` (tcontrol)
`$lsurx` (tcontrol)
`$lsury` (tcontrol)

Usage: `$lsurc` n_1
`$lsurx` n_2
`$lsury` n_3

Purpose: mandatory keywords; integer numbers n_1, n_2 and n_3 are indices of three different atoms according to their appearance in file `geometry.in`, which define in a unique way an outermost *left* atomic surface of the "extended molecule" (see Fig. 5.1 for details).

Tags: `$rsurc` (tcontrol)
`$rsurx` (tcontrol)
`$rsury` (tcontrol)

Usage: $rsurc m_1
$rsurx m_2
$rsury m_3

Purpose: mandatory keywords; integer numbers m_1, m_2 and m_3 are indices of three different atoms according to their appearance in file geometry.in, which define in a unique way an outermost *right* atomic surface of the "extended molecule" (see Fig. 5.1 for details).

Tag: $nlayers (tcontrol)

Usage: $nlayers N_a

Purpose: integer number N_a specifies amount of atomic layers within interface regions at the boundaries of "extended molecule" where absorbing boundary conditions are active and self-energy matrix elements (leakage rates) are non-zeros, see Fig. 5.1 for details.

An integer number N_a should be taken from a header line of library files for Au electrodes which are placed in electrodes.library/ directory.

When using tcontrol.aims.x, the number N_a is passed to a script by the option: -nlayers N_a.

Tags: $s1i (tcontrol)
$s2i (tcontrol)
$s3i (tcontrol)

Usage: $s1i η_1
$s2i η_2
$s3i η_3

Purpose: positive real numbers η_1, η_2 and η_3 define local leakage rates (in Hartree units) which parametrize self-energy matrix elements.

Default values of leakage rates for Au clusters, written by the script tcontrol.aims.x to the file tcontrol, reflect a gradual switching of perturbation and are given by: $\eta_1 = 0.1$ for the outermost atomic layer of the "extended molecule" (see Fig. 5.1 for details); $\eta_2 = 0.05$ for the next-to-the-outermost atomic layer; and $\eta_3 = 0.025$ for the rest of atomic layers within the interface regions of the "extended molecule".

See also a keyword $self_energy .

Disclaimer: 'Black-box'-users of AITRANSS are strongly advised to work with the Au-electrodes listed in the library and use default parameters for the self-energy, only. The transport code will print out a warning message if other chemical elements are employed as electrodes material. Unexpected modification of electrodes or self-energy settings will, in general, lead to misleading or incorrect scientific results.

Tags: **$ener** (tcontrol)
$estep (tcontrol)
$eend (tcontrol)

Usage: $ener E_1
$estep dE
$eend E_2

Purpose: real numbers E_1, dE and E_2 should be given in Hartree units and define the energy window $[E_1, E_2]$, and the energy step dE for calculation and output of the transmission function $T(E)$.

If any of above mentioned keywords is missing, only conductance at the Fermi energy is calculated.

Tag: **$self_energy** (tcontrol)

Usage: $self_energy file=*self-energy-file*

Purpose: sets up a name of the file with atom specific values parameterizing the self-energy matrix; a format of the self-energy file is explained in section 5.3.4.

self-energy-file is a text string, a default file name is `self.energy.in`. The string `file=`*self-energy-file* should not contain spacings.

If a keyword $self_energy is present in `tcontrol`, the diagonal elements of the self-energy matrix are read from the referenced file. In that case, parameters and values given by keywords $lsurc, $lsurx, $lsury, $rsurc, $rsurx, $rsury, $nlayers, $s1i, $s2i, and $s3i do not have an effect.

Tag: **$testing** (tcontrol)

Usage: $testing flag

Purpose: optional keyword, reserved for testing purposes; `flag` can take values `on` or `off`. Default value is set to `off`.

If `flag` is set to `on`, several internal checks are performed to insure that employed numerical procedures give correct results, e.g.: (i) eigenvalues of the reconstructed Kohn-Sham Hamiltonian H coincide with values stored in files `mos.aims`, `alpha.aims` or `beta.aims`, and eigenvectors of H are orthogonal to each other; (ii) eigenvalues of the overlap matrix are positive, and the square-root of the overlap matrix multiplied by itself gives back the overlap matrix; (iii) a matrix B of eigenvectors of the complex valued operator $H + \Sigma$ multiplied by the inverse B^{-1} gives a unitary matrix, $BB^{-1} = 1$, etc. Furthermore, there appear many `.tmp` files: one of them called `zmos.tmp` (or `zalpha.tmp` and `zbeta.tmp`) contains information on the complex poles $E_n = \varepsilon_n - i\delta_n$ of the Green's function $G^{-1}(E) = E - H - \Sigma$.

Tag: `$end` (tcontrol)

Usage: `$end`

Purpose: mandatory keyword, indicates the last line of file `tcontrol`, which is read by the transport module AITRANSS. All lines below this one are ignored.

Acknowledgment: A. Bagrets and F. Evers acknowledge the help of Richard Korytár in writing a manual on AITRANSS.

Appendix A

Trouble-shooting

We sincerely hope that FHI-aims will largely "do the job" for you as it comes; in fact, a large amount of work has gone into ensuring sane responses and understandable output from code when something was requested that was not safe or reasonable to do. Nonetheless, as with every piece of software, non-trivial issues can happen that are not immediately obvious to the user. In this appendix, we provide a list of known conditions that have taken unwary users by surprise, how to detect and how to fix them.

If you know of an issue that is not discussed below, but that should be included because it presents an easy stumbling block especially for new/inexperienced users, please let us know, and we will address the issue here.

A.1 Format flags required by some compilers

FHI-aims contains source code files in different formats (.f or .f90), and sometimes containing rather long lines in the .f90 versions.

The Makefile therefore contains two different versions of the compiler flags, `FFLAGS` and `F90FLAGS`, which can be the same for some compilers, but do not have to be the same.

For specific compilers, flags that must be added to account for the diffent file formats properly are:

- xlf90 compiler (IBM): `FFLAGS` must contain the option "-qfixed" in addition to all other options specified with the `F90FLAGS`.

- g95 compiler: `F90FLAGS` should contain the option -ffree-line-length-none in addition to all other options found in `FFLAGS`.

A.2 FHI-aims aborts with a segfault at the beginning of the first test run.

We here repeat the information given already in Chapter 1:

If you are not familiar with Unix or Unix-like operating systems, the following will perhaps clarify what is going on. In Unix, the operating system *kernel* will allow a program to allocate / deallocate the variables it requires on the so-called *heap*. For small quick variables needed at runtime, this is not always the most efficient procedure (you do not want to allocate / deallocate very single loop counter in your code, for example). Such small variables can instead be requested from a *stack*, available per process, and also controlled by the kernel.

In principle, using the stack is not a great problem on current computers available for scientific computing, because you will rarely ever find more than a few processes at the same time that make excessive use of the stack. So, technically it should not matter whether you get your memory from the heap, or from the stack.

Unfortunately, for reasons unbeknownst to us, some opertaing system vendors limit the default user stack size to ≈5 MB in a time when typical available RAM per processor is 2 GB or more. For some purposes, FHI-aims *requires* that the execution stack size available to you be large enough for some initial internal operations. If too little stack is available, your FHI-aims run will *segfault* shortly after the command was launched. In that case, type:

```
> ulimit -s unlimited
```

(when using the bash shell or similar), or

```
> limit stacksize unlimited
```

(when using the tcsh or similar).

```
> echo $SHELL
```

will tell you which shell you are using. Ideally, this same setting should be specified in your .profile, .bashrc, or .cshrc login profiles. If "unlimited" does not work, try setting a large value instead, e.g., `ulimit -s 500000`.

If any of these steps are not allowed, you will have to contact the system manager of your computer in order to modify the stack size limits set by the kernel of your operating system.

FHI-aims prints at startup the settings of the stacksize as they are found on your system. As mentioned, here "unlimited" or a large value should be reported.

A.3 Use of FHI-aims with multithreaded BLAS (e.g., Intel's mkl)

The performance of FHI-aims depends critically on the basic linear algebra subroutine (BLAS) library used to perform matrix operations. Such libraries are highly CPU-specific, and should be provided and optimized by yourself or your computer vendor for your particular computer.

Unfortunately, with the advent of multi-core CPUs for PCs, some computer vendors (Intel, IBM) have decided that their proprietary BLAS implementations will, by default, use *all* available CPU's by way of *threads*, since they do not expect a user to know how to create parallel code.

In contrast, FHI-aims makes great efforts to distribute its workload evenly itself (much more efficient than leaving the task up to the BLAS, which are used for some, but by no means all operations in the code). Thus, FHI-aims invokes the correct number of sub-processes via the message-passing interface (MPI), then distributing any further basic numeric operations (matrix multiplications) using BLAS routines correctly itself.

If the default settings provided by a vendor are to use *all* CPUs for *every single* call or the BLAS operations, on a system with n CPUs this will lead to $n \times n$ tasks running in parallel – not good at all for efficiency.

The problem is easily fixed by setting the system variable `OMP_NUM_THREADS` (number of threads invoked by OpenMP-parallelized libraries, e.g., BLAS) to 1:

```
export OMP_NUM_THREADS=1
```

This syntax is correct for the bash shell). When using Intel's mkl, you may likewise wish to set `MKL_NUM_THREADS` to 1. On top of this, Intel will still ignore your choice unless you set the less well documented variable `MKL_DYNAMIC` to `FALSE`.

Another, much simpler and equally well performing option is to use the freely available Goto BLAS subroutines that can be downloaded and compiled on standard architectures.

Some versions of Intel's mkl are known to have an error in a function called "pdtran". Thus at startup FHI-aims tests the version of pdtran it is currently using for correctness. Should FHI-aims abort with an error message "pdtran test failed! Aborting..." you will have to use a different version of Intel' mkl or replacements like Goto Blas.

A.4 Parallel runs across different file systems

In parallel runs on distributed computers (clusters), FHI-aims expects its input files in the directory from which it is invoked. Its standard output can be redirected by hand to any given location, and other (optional, see keyword `output`) output files are again written in the same directory from which FHI-aims is invoked.

This procedure works well on most standard cluster and/or high-performance computing architecture available today, but you must make sure that the directoried for input and output are visible and readable / writable on all the nodes across which FHI-aims is parallelized for a given run.

A.5 I'm running a calculation with a large system, and it is abruptly exiting. What's going on?

It is likely that you are running out of memory. See the next section, "What do I do if I run out of memory?"

A.6 What do I do if I run out of memory?

Like all other electronic structure codes, the current bottleneck for FHI-aims when performing calculations for large systems is memory usage. A number of flags exist to alleviate memory usage, but as many of them have an associated trade-off of performance overhead, poor scaling for small system sizes, or breaking post-processing techniques, they have not been enabled by default. For large systems where memory usage becomes an issue, however, enabling them will be worth it. This list of flags may be found at in section 3.27, "Large-scale, massively parallel: Memory use, sparsity, communication, etc."

A.7 Nearly singular basis sets: Strange results from small-unit-cell periodic calculation with many k-points

We have observed numerous times that periodic bulk calculations with small unit cells and many k-points are apparently much more prone to ill-conditioning of the basis set than any other type of calculation. The symptom is that, with the usual accuracy and grid settings, large but still affordable basis sets (e.g., tier 3) will show reasonabl convergence behavior at the outset, but then suddenly show a large jump and unphysical total energies at some point in the s.c.f. cycle.

The underlying reason is that the basis sets used by FHI-aims are overlapping and non-orthogonal. As the basis functions located at each atom of the structure approach completeness, the basis set as a whole becomes overcomplete. The result may be that certain linear combinations of basis functions are approximately expressable as linear combinations of some others. The eigenvalue problem Eq. (3.25) becomes *ill-conditioned*, and small amounts of numerical noise in the Hamiltonian / overlap matrix elements can group together to produce large unphysical effects in the eigenvakue spectrum.

If this happens, a number of strategies are available to deal with this situation. These

are summarized in the following. Note, however, that ill-conditioning does indicate that your chosen basis set is already closer to completeness than even your computer can handle, and a smaller basis set for production calculations should be equally sufficient (and much faster) for high-quality results.

- Employ the `basis_threshold` keyword. This allows to identify the near-linear dependent components of the overlap matrix and eliminate them from the calculation. The successful threshold value depends on your chosen basis set and system, so test different choices (typically, 10^{-4} or 10^{-5}). Note, however, that a large `basis_threshold` value may also impact the the total energy found at a level of a few meV/atom.

- In addition, the keyword `override_illconditioning` must be set in order to run with a basis set that is reduced by `basis_threshold`. This should serve as an indicator that extra care is required in this situation—in particular, a detailed convergence analysis of the behaviour of the problem with increasing basis set size, and (separately!) with increasing cutoff radius, up to the value you are using. In most cases, it should turn out that either the basis set, or the cutoff radius, or both, were chosen to be far overconverged.

- Increase the accuracy of the integration grids via `radial_base` and `angular_grids`. This is an expensive strategy (use for proof-of-principle only!), but it will serve to reduce the numerical noise in your calculations and thus increase the validity range of the eigenvalue solution, Eq. (3.25).

Again, note that we do not usually observe any ill-conditioning related problems for large periodic structures (e.g., surface slabs) or even very large molecules, even when employing very large basis sets.

A.8 No convergence of the s.c.f. cycle even after many iterations

One first thing first: If you encountered unexpected s.c.f. convergence issues, did look at your exact `geometry.in` file in a viewing program, such as jmol? One of the most common reasons to find unexpectedly bad s.c.f. convergence for apparently harmless structures are issues such as the wrong lattice parameter for the structure in question, an atom in the wrong location, or the wrong structure altogether. Structures that are chemically unstable are often not happy at all when it comes to the stability of their electronic structure, and slow or no s.c.f. convergence can be an indicator of a simple geometry mistake.

Successful strategies for s.c.f. convergence in standard electronic structure problems have been developed in the field for a long time. Still, there remain some particular pathological classes of calculations, and even some of particular physical interest: large metallic slabs, where charge oscillations can occur; spin-polarized systems with closely

competing spin states; systems near the crossing of two Kohn-Sham eigenvalues at the Fermi level; etc.

The standard s.c.f. convergence strategy within FHI-aims is to use `mixer pulay`, with a configurable `charge_mix_param` and number of mixed iterations `n_max_pulay`. These, possibly together with a `preconditioner`, should be modified first in order to see whether the problem can be contained.

Typical settings that work for wide varieties of systems are as follows:

- Semiconducting or insulating solids or molecules (i.e., insulating systems with an appreciable band gap):

```
mixer pulay
charge_mix_param 0.2
occupation_type gaussian 0.01
```

are usually enough to work for such systems.

- Metallic systems with no appreciable band gap, slabs to model surface phenomena, etc.:

```
mixer pulay
charge_mix_param 0.05
occupation_type gaussian 0.05
```

or similar should usually work. Note that `charge_mix_param 0.05` is *not* a small value for a mixing parameter, since this is employed with a Pulay mixer. After several iterations, the Pulay mixer has usually figured out what the right mixing parameters should be and thereby tends to undo the slowing effect of a small mixing parameter.

- Periodic slab models that are still problematic:

In periodic systems, FHI-aims uses a Kerker preconditioner by default, in addition to the usual Pulay mixer. For very anisotropic metallic systems (think graphene sheet with a 10x10 unit cell and a large vacuum), the Kerker preconditioner may not be appropriate – switching it off may help:

```
preconditioner kerker off
```

- Slow convergence for spin polarized systems:

Only ever run a spin polarized calculation if there is a good reason to do so. Running a clearly non-spin-polarized system with `spin none` just for aesthetic reasons is a bad idea: It will double your computer time usage in the best of cases, *and* it will require you to begin the calculation from some finite spin initialization that may not be close to

the self-consistent electronic structure you are seeking. Thus, spin polarization may add additional s.c.f. cycles *and* cost extra time in any case.

Obviously, please do run spin polarized calculations if there is a good physical reason to do so. Just do not underestimate the cost. It is also wise (= essential!) to think about the initial moments chosen for the initialization of the calculation. The closer the initial moment distribution matches the expected self-consistent result, the better (faster) the convergence.

In contrast, beginning every spin-polarized calculation with a high spin state for every atom may be a highly bad idea. In particular, do not run molecular or condensed phase calculations (i.e., anything other than single free atoms) with `default_initial_moment hund`. Hund's rules, which can be read up on Wikipedia and a host of other sources, apply to free atoms. Condensed systems have very different spin moment distributions.

Beyond this, s.c.f. convergence issues can be highly system-specific in our experience, and general guidelines are hard to give. Things that will always work to some extent are:

- a `linear` `mixer` with a (very!!) small `charge_mix_param` , which in the limit will guarantee convergence, albeit at the expense of excessively many s.c.f. cycles to reach convergence
- A increased broadening specified with `occupation_type` . This is essential especially for metalic systems, but for small clusters, the quality of the obtained total energies will deteriorate somewhat as a result, since these suddenly correspond to fractional (very high temperature) occupation numbers around the Fermi level.

For particularly hardy cases, we strongly recommend to review in detail all the options available in Sec. 3.10.

While trying out all these options, however, we strongly suggest to use the `output_level full` keyword, in order to have the actual eigenvalue spectrum printed across different s.c.f. iterations, and then to *visualize* the behaviour of the eigenvalue spectrum as a function of s.c.f. iteration. In many cases, this step may yield some critical physical insight into the nature of the problem. For example, Kohn-Sham density functional theory may sometimes be forced to place competing electronic levels (d and f in rare earth elements are a good example, but there are many others!) *at* the Fermi level in order to ensure a given (ground-state) fractional occupation. The search for the correct occupation of these levels will then oscillate between different iterations, and could be the source of the instability. Stabilizing such a problem is still not easy, but at least, looking at the electronic structure as it develops may give some critical hint as to what is happening, instead of leaving the user groping in the dark.

Appendix B

Structure of the code

The bulk of this manual is concerned with the available options to build and run FHI-aims for a particular purpose. In our experience, most users will probably remain at this level in their use of the code, already due to the time constraints of a normal research schedule.

Nonetheless, we strongly believe that running a "physics" code as a complete black-box package is not a good idea. Our branch of physics is based on (mostly) well-understood differential equations, and the solution technique applied, including its limitations, must be well understood, or at least understandable, to gauge the outcome of a particular calculation. In order to achieve this, the source code to the method used must be available. While we do not expect most users to go through the code in its entirety with a fine-tothed comb, we *do* encourage any user to look into the source code and try to understand in which way FHI-aims produces its exact solution to a problem.

The purpose of this appendix is to facilitate this understanding, by outlining the overall structure of the code, specifically, the high-level subdivision of physical tasks.

B.1 Flow of the program

The uppermost level of FHI-aims is the subroutine `main.f90`, the structure of which is shown as a structogram in Fig. B.1. As a subroutine, `main.f90` can also be called by external code as a library subroutine, with the restriction that, for parallel execution, this can happen only once [by definition of the message passing interface (MPI) for parallel communication, there can only be one call to `mpi_init` per program run].

The purpose of `main.f90` is to provide a logical separation of groups of computational tasks by way of high-level wrapper subroutines (listed in `typewriter` font in Fig. B.1). With the exception of global convergence checks, no outright physical quantities are directly manipulated in `main.f90`. All physically relevant quantities are handled inside the lower-level structure of the code and, if necessary, are passed between them by way of specific modules. For example, the module `physics.f90` handles all variables of tangible physical importance (Hamiltonian and overlap matrices, Kohn-Sham wave function, electron densities, potentials, ...). The geometry information for a given electronic

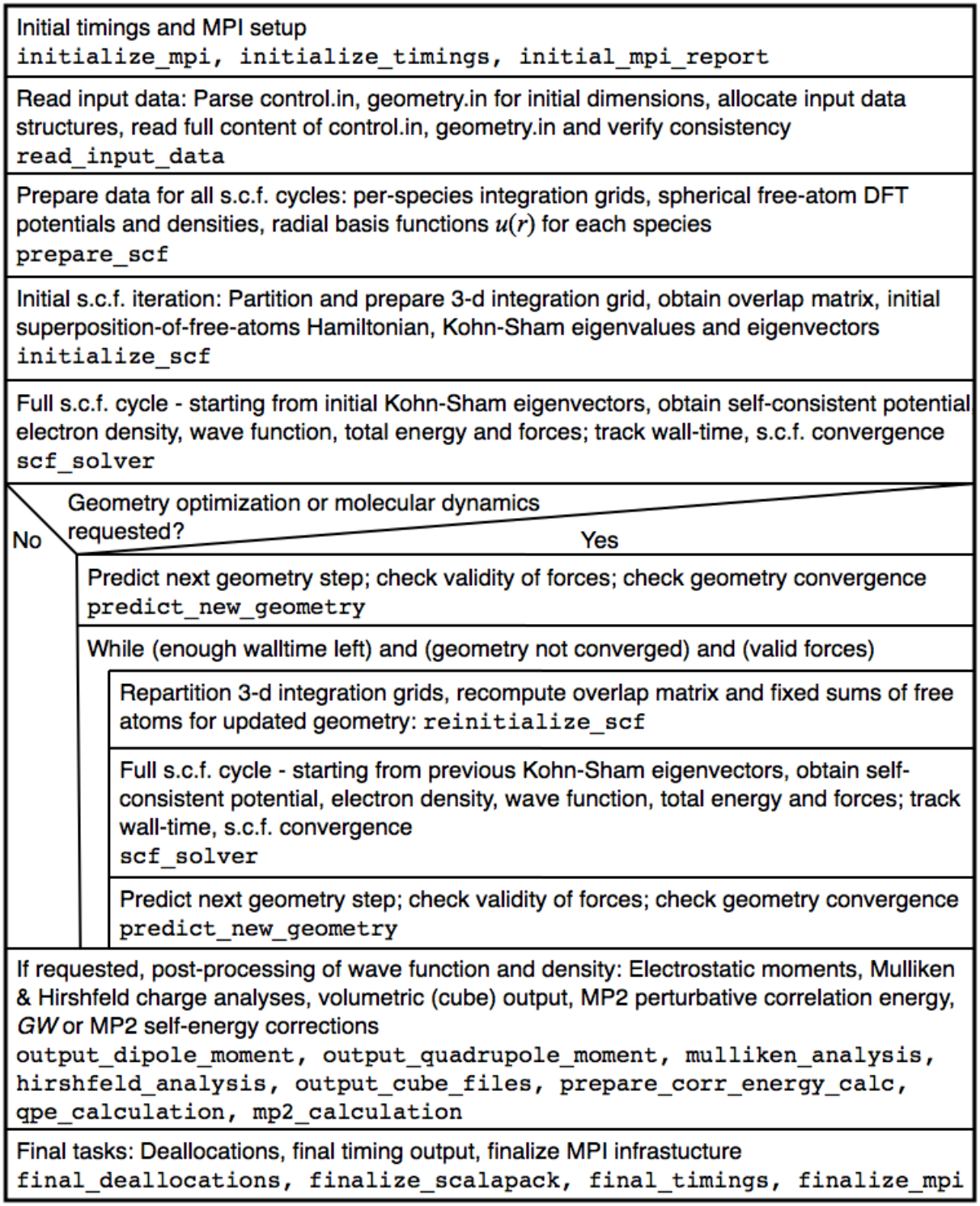


Figure B.1: High-level program flow of FHI-aims

structure cycle (coordinates) are found in module `geometry.f90`; etc.

More details regarding these and other modules are included with the code distribution as a separate document. The point here is that `main.f90` should never need to use any but the highest-level modules explicitly, i.e.:

- `dimensions.f90` : Inclusion of array dimensions for consistent allocations across the code, and wrapper flags to prevent access to unallocated variables which are not needed for a given task.
- `localorb_io.f90` : Wrapper module for consistent writing of output in parallel runs
- `mpi_utilities.f90` : Wrapper module for MPI initialization, finalization (first and last tasks of a run, respectively), and task distribution
- `timing.f90` : Wrapper module including all timing and accounting information, including the count of s.c.f. iterations, relaxation or MD steps.

The first task of `main.f90` is to initialize any accounting (timing etc.) information, the infrastructure required for MPI (or, to silently switch off the use of MPI entirely in the case of non-parallel runs), and to record all this information (initial time stamps, code version, number of parallel tasks and computer system layout) in the standard output.

The obvious next task is to read and process all input information given in `control.in` and `geometry.in`. Internally, this is handled in three steps (see the wrapper subroutine `read_input_data.f90`): First, both input files are parsed once, while extracting only the dimension information needed to set up any necessary arrays / array dimensions needed to house the following input data. Organizing this information is the task of module `dimensions.f90`. Next, the information in `control.in` is read and checked for consistency, using `read_control.f90` for all general information, and repeated calls to `read_species_data.f90` for all `species` -related information. Finally, subroutine `read_geo.f90` reads the input data of `geometry.in`, and verifies its consistency with the data contained in `control.in`.

At the end of this step (subroutine `read_input_data.f90`), *all* input data from all input files should have been read and processed. It is important that any known conflicts, or incomplete settings, should have been verified at this stage, stopping the code with an error message if outrightly conflicting input information is detected. For completeness, we mention that any technical input settings of global interest (e.g., the handling of spin, relativity, or exchange-correlation) are collected and accessible through the top-level module `runtime_choices.f90`.

The following steps are the "household" steps of electronic structure theory.

Wrapper subroutine `prepare_scf.f90` sets up all structure-independent, fixed pieces of the calculation, and stores them for easy access in the actual self-consistency cycle. This includes all free-atom quantities (densities and potentials for the initialization), radial basis functions for all `species` , one-dimensional logarithmic and three-dimensional radial and angular integration grids, and fixed coefficients for the analytic long-range part of the Hartree potential.

Wrapper subroutine `initialize_scf.f90` performs the initial s.c.f. cycle of the electronic structure calculation. In this step, the full three-dimensional integration grid is filled with fixed initial quantities (superposition of free-atom densities, potentials, and partition functions), the overlap and Hamiltonian matrix integrals are performed for the first time, and the initial Hamiltonian and overlap matrices are used to determine the storage requirements in the event of sparse matrix storage. If a two-electron Coulomb operator is needed (hybrid functionals, Hartree-Fock, MP2, GW etc.), the three-center overlap matrix elements $(ij|\mu)$ of Eq. (3.35) (see Sec. 3.20 for details) and the Coulomb matrix of the auxiliary "resolution of the identity" basis set [denoted $V_{\mu\nu}$ in Eq. (3.35)] are precomputed. The most important task in `initialize_scf.f90` is the initial solution of the Kohn-Sham equations Eq. (3.25), providing a first solution of the wave function coefficients c_{jl}. These are the starting point of every iteration of the s.c.f. cycle in the following step, subroutine `scf_solver.f90`.

With all preliminary information available, the task of `scf_solver.f90` is to produce a self-consistent electron density, wave function, and all associated observables for a given, fixed nuclear geometry. The order of the cycle until convergence is reached is:

1. Calculation of the Kohn-Sham electron density associated with the current wave function, c_{jl}
2. electron density mixing and preconditioning, to produce the *input* electron density for the next set of Kohn-Sham equations
3. decomposition of the electron density into atom-centered multipole fragments $\delta\tilde{n}_{\mathrm{at},lm}(r)$ [see Eq. (Eq;mp)], and construction of the multipole components of the Hartree potential, $\delta\tilde{v}_{\mathrm{at},lm}(r)$
4. construction of the full electrostatic potential $v_{\mathrm{es}}(\boldsymbol{r})$ on all points $\boldsymbol{r}$ of the three-dimension integration grid
5. Integration of the updated Hamiltonian matrix elements, h_{ij}
6. possible addition of two-electron exchange matrix elements to h_{ij}
7. solution of the Kohn-Sham equations Eq. (3.25), to produce an updated wave function c_{jl}
8. computation of updated total energy components, and check of all convergence criteria.

After convergence is reached, `scf_solver.f90` also performs some inevitable post-processing steps, including "scaled ZORA" perturbative corrections for the appropriate relativistic treatment, and band structure and density of states data output.

With a converged self-consistent solution at hand, the code can now perform any number of similar calculations for updated geometries, e.g., for a geometry optimization, molecular dynamics, etc. If so, an updated geometry is first produced by subroutine `predict_new_geometry.f90`. Note that this simple subroutine should also serve as the starting point for any other calculations involving multiple geometries, such as the

calculation of "serial" geometries along a given set of coordinates, etc. For the updated geometry, all geometry-related storage arrays in the calculation and the overlap matrix must be recomputed in subroutine `reinitialize_scf.f90`. Following this, subroutine `scf_solver.f90` is invoked again, and a new self-consistent solution is obtained.

The final step of the code is to produce, by post-processing, any information that can be obtained from the converged self-consistent wave function or electron density, including electrostatic moments, charge analyses, etc. Beyond this, only necessary cleanup tasks follow, most notably the deallocation of all storage arrays and the MPI infrastructure, and the final time accounting information.

B.2 Commenting and style requests

Generally, no *strong* style conventions are enforced within FHI-aims, recognizing the fact that most programmers follow their own style conventions and preferences when it comes to details. There are, however, some conventions that should be followed in order to keep the code as a whole legible, and maintainable. When writing additional code, please adhere to these, using existing modules or subroutines as models where appropriate.

1. Please comment your work. Any necessary functionality should be accompanied with comments, enough so that *at least* someone familiar with the underlying mathematics can follow your code.

2. Please provide regular comment headers for your work. Every module and subroutine comes with comment headers in the style used by the *Robodoc* code management system (see, e.g., `http://www.xs4all.nl/~rfsber/Robo` for details and a manual). Beyond the possible use of Robodoc, we follow this convention because it provides a clear and unambiguous laundry list of items that are needed in any subroutine header: a description of the subroutine *purpose*, possibly its *input* and *output* data, and most importantly a *copyright* statement that is needed for every file in the code distribution.

3. Please keep your usage of Fortran conservative. Some ambitious constructs, while desirable when following formal techniques such as fully object-oriented programming, can still expose compiler bugs when too "un-Fortran-like" syntax is used.

4. In particular, compiler-dependent bugs are the reason why the use of pointers is strongly discouraged in FHI-aims. Even when following the textbook, allocating and deallocating pointers works differently with different compilers, opening the possibility of unexpected memory leaks outside our control. Please don't do it — this problem *has* bitten us before.

Appendix C

Memory tracing - how to compile FHI-aims using the preprocessor

FHI-aims features a preprocessor which can be used to add a memory tracing feature at compile time. In its normal operation mode, it gives you a overview of the memory c consumption at the end of the FHI-aims run, if you want also per host (node) and per process. In its second operation mode, the tool acts as debug utility, because it gives a very detailed account of all (de)allocations, including date, time, arraysize, current memory consumption, peak memory consumption, the arrayname and its origin (source file and line) for each process.
To use this feature, you will need in addition to the usual requirements:

- a python installation (tested for Python 2.7 and 3.2)
- the *regex* module for python (available at https://pypi.python.org/pypi/regex) (a user-local installation is possible)
- a compiler with the *SIZEOF* extension which yields the byte size of an object

If all these requirements are fulfilled, you can compile FHI-aims with the memory tracing feature by following the steps listed below.

1. adjust your makefile and set the variable *MEMTRACE_USE* to "yes"
2. set *MEMTRACE_KEEP* to "yes" if you want to keep the interim files generated by the preprocessor (suffix *.i90). The *.itypes and *.imod files will be kept regardless of your choice.
3. compile FHI-aims as usual

You now should have a version of FHI-aims that is capable of tracing the memory consumption. The usage of this functionality is covered in the documentation of the keywords `memtrace_summary` , `memtrace_use` and `memtrace_targetdir` .

Appendix D

Debug Manager - a centralized debugging facility for developers

FHI-aims features a centralized utility for controlling debug output. This utility keeps track of all registered modules (in a physical sense, e.g. the DFPT features are a module in this context) and can enable their debug output selectively via the *control.in* file.

The main idea behind this utility is to provide a simple tool to generate debug-flags that can be set in the *control.in* without having to register them manually in the input parsing routines. Furthermore, it also provides a central debug interface to the user, instead of having to use different flags, flags that can only be toggled in the sourcecode or even no toggles at all.

You can enable debugging for any registered module in the input file by adding a `debug_module my_module`. The list of currently supported modules can be found in the file *init_debug.f90*.

To register your module, you only need to add a single line to the *init_debug.f90*:

```
register_debugmodule("your_tag")
```

Afterwards, you can conveniently activate debugging in the *control.in* by using the flag:

```
debug_module your_tag
```

This tells the code to enable debugging for your code. Just call

```
debugprint(message, your_tag)
```

in your code to only print the message if debugging for your module is enabled. For more complicated debug functionality, there also is

```
module_is_debugged(your_tag)
```

This function returns a logical value and thus can be used as condition in an enclosing if-block. Both functions are located inside the `debugmanager` module, which you can import like any other module with a use-statement.

Appendix E

XML output

The FHI-aims can print results of the calculations in an XML format. This functionality is provided by the `xml_write` module. The module is quite general and capable of printing any XML, but also has convenient functions for printing scalars, vectors, matrices and n-dimensional arrays. The formatting of the data in the XML elements is both human-readable and machine-readable. The obvious advantage is that such an output from the code is extremely easy and fast to parse.

There are two ways how the XML module can be used. There is a global XML file which can be turned on with `xml_file <filename>` in `control.in`. Printing to this file is as simple as loading the module and calling one of its routines. Note however that if the block of code that prints to this global file calls another code also printing to the global file, the XML output of these two codes might get intertwined, resulting in a valid but nonsensical XML file.

A second way how to use the XML module is to create a new "local" XML file with the `open_xml_file` routine and use that one by calling the module printing routines with the optional argument `file=<xml_file_instance>`.

Appendix F

Optional Libraries to be Linked into FHI-aims

A core principle for FHI-aims is that it should be possible to build all essential functionality with only a minimal set of dependencies, typically a Fortran compiler, BLAS and LAPACK libraries, an MPI library, and the scaLAPACK library. These packages are essential for performance and usually available on every relevant supercomputer architecture.

We note that while it is often easy to fulfil other, far more complex dependencies on standard Linux type systems with household tools, this is not always true for highly specialized supercomputers. Yet, that computer may be precisely one the multi-million dollar machine which would help solve an advanced scientific problem. It is essential that a mere human be able to compile FHI-aims with the appropriate performance on such a machine.

That said, it may be beneficial to connect other libraries to FHI-aims for special purposes, including libraries which require cross-compilation with a C compiler. This section is intended to be developed into a list of such libraries. Please also see Section 1.5 for some basic information.

F.1 Adding Optional Libraries into FHI-aims: Stubs

The optional presence or absence of certain libraries from FHI-aims at compile time means that the main FHI-aims code must be able to deal with the possible absence of certain subroutine or function calls to those libraries at compile time.

In many codes, this is handled by adding preprocessor statement. In FHI-aims, preprocessor flags are **NOT** the way forward. The simple reason is that a single preprocessor flag may remain understandable for an ordinary user; the proliferation of ten or more preprocessor flags will create a serious obstacle for others to obtain a reasonably compiled code. Code that introduces preprocessor statements into FHI-aims will be removed from the code base.

In FHI-aims, the recommended way to optionally add or circumvent a given library is by

creating a "stub" file for those library calls – essentially, empty subroutines that inform a user that they should not have ended up here with a given version of the code (i.e., without linking to a certain external library). At compile time, an appropriate flag in the Makefile and Makefile.backend should be created that switches between the real external library (if available) or the "stub" file. Please follow the example of "Spglib" below in the Makefile and in Makefile.backend for more information. The principle is simple and easy to apply for any other optional library.

F.2 Spglib

Spglib is a library for finding and handling crystal symmetries, written by Atsushi Togo (**http://spglib.sourceforge.net**). FHI-aims can be interfaced to the spglib.

So far, FHI-aims only supports the determination of the crystal symmetry for a given (periodic) geometry file, but does not make use of this information otherwise.

Prerequisites:

spglib is written in C and therefore needs a C compiler with appropriate options in addition to the usual Fortran compiler with which FHI-aims is built. Please enable a C compiler and add the appropriate compiler flags to the `Makefile` or `make.sys` file as described in Section 1.5.

Specifically for spglib, include in `Makefile` or `make.sys`:

```
USE_SPGLIB = yes
```

Tag: `use_symmetry_analysis` (control.in)

Usage: `use_symmetry_analysis .true. / .false.`

Purpose: This flag activates the interface to the spglib to obtain symmetry information for the provided geometry file once. The symmetry information is directly written after the geometry printout in the FHI-aims standard output. spglib needs to be compiled with FHI-aims to get the output.

Default: `.true.`

Tag: `sym_precision` (control.in)

Usage: `sym_precision value`

Purpose: This value determines the accuracy for identifying symmetric atoms. The lower the value, the more strictly the atomic positions must overlap.

Default: 10^{-5}

Appendix G

Split large cluster allocation to multiple aims instances

Specific tasks need to perform many small jobs rather than a few big ones. The job submission of such tasks on large compute facilities which typically require a few 100 MPI processes can be cumbersome and depend on the given job submitter. Aims allows for running multiple instances of itself and split a global MPI communicator int smaller, independent parts. This allows for a unified approach to supmit one large job and let the splitting happening in aims. To use this feature, ou need to compile the multiaims executable by compiling the multiaims target, e.g. by typing make multi.scalapack.mpi. In addition to the usual requirements you need to specify a C-compiler.

The multiaims environment needs its jobs distributed in subdirectories labeled 1 to N, where N is the total number of jobs. In the main directory one additional control file *multiaims.in* needs to be specified. This file contains two keywords: You now should have a version of FHI-aims that is capable of tracing the memory consumption. The usage of this functionality is covered in the documentation of the keywords:

Tag: `tasks_per_subjob` (multiaims.in)

Usage: `tasks_per_subjob value`

Purpose: Specifies the number of processes working on one subtasks. Should be a divisor of total number of processes.

Tag: `tasks_per_subjob` (multiaims.in)

Usage: `start_id value`

Purpose: Specifies the first job folder to start with.

Bibliography

[1] C. Adamo and V. Barone. *J. Chem. Phys.*, 110:6158, 1999.

[2] C. Van Alsenoy. *J. Comput. Chem.*, 9:620, 1988.

[3] C. Ambrosch-Draxl and J. O. Sofo. Linear optical properties of solids within the full-potential linearized augmented planewave method. *Computer Physics Communications*, (175):1–14, 2006.

[4] H.C. Andersen. *J. Chem. Phys.*, 72:2384, 1980.

[5] V. I. Anisimov, editor. *Strong Coulomb correlations in electronic structure calculations*. Gordon and Breach, New York, 2000.

[6] R. Armiento and Ann E. Mattsson. *Phys. Rev. B*, **72**:085108, 2005.

[7] A. Arnold, F. Weigend, and F. Evers. Quantum chemistry calculations for molecules coupled to reservoirs: Formalism, implementation, and application to benzenedithiol. *J. Chem. Phys.*, 126:174101, 2007.

[8] G. Aubert. An alternative to Wigner d-matrices for rotating real spherical harmonics. *AIP Adv.*, 3(6):062121, June 2013.

[9] A. Bagrets. Spin-polarized electron transport across metal-organic molecules: a density functional theory approach. *J. Chem. Theory Comput.*, 9:2801, 2013.

[10] J. Baker, J. Andzelm, A. Scheiner, and B. Delley. *J. Chem. Phys.*, 101:8894, 1994.

[11] A. Barducci, G. Bussi, and M. Parrinello. Well-tempered metadynamics: A smoothly converging and tunable free-energy method. *Phys. Rev. Lett.*, 100:020603, 2008.

[12] C. I. Bayly, P. Cieplak, W. D. Cornell, and P. A. Kollman. A well-behaved electrostatic potential based method using charge restraints for deriving atomic charges: The resp model. *J. Phys. Chem.*, 97:10269–10280, 1993.

[13] A.D. Becke. *J. Chem. Phys.*, 88:1053, 1988.

[14] J. Behler, B. Delley, S. Lorenz, K. Reuter, and M. Scheffler. *Phys. Rev. Lett.*, **94**:036104, 2005.

[15] J. Behler, B. Delley, K. Reuter, and M. Scheffler. *Phys. Rev. B*, **75**:115409, 2007.

[16] B. H. Besler, K. M. Merz, and P. A. Kollman. Atomic charges derived from semiempirical methods. *J. Comput. Chem.*, 11:431–439, 1990.

[17] Miguel A. Blanco, M. Flórez, and M. Bermejo. Evaluation of the rotation matrices in the basis of real spherical harmonics. *Comp. Theor. Chem.*, 419(1–3):19–27, December 1997.

[18] V. Blum, R. Gehrke, F. Hanke, P. Havu, V. Havu, X. Ren, K. Reuter, and M. Scheffler. *Ab initio* molecular simulations with numeric atom-centered orbitals. *Comp. Phys. Comm.*, **180**:2175, 2009.

[19] S.D. Bond, B.J. Leimkuhler, and B.B. Laird. The Nose-Poincare method for constant temperature molecular dynamics. *J. Comp. Phys.*, 151(1):114–134, 1999.

[20] A. Bondi. van der waals volumes and radii. *J. Phys. Chem.*, 68:441–451, 1964.

[21] S.F. Boys and I. Shavitt. *University of Wisconsin Rept.*, WIS-AF-13, 1959.

[22] C. M. Breneman and K. B. Wiberg. Determining atom-centered monopoles from molecular electrostatic potentials. the need for high sampling density in formamide conformational analysis. *J. Comput. Chem.*, 11:361–373, 1990.

[23] P. Bultinck, C. Van Alsenoy, P.W. Ayers, and R. Carbo-Dorca. Critical analysis and extension of the hirshfeld atoms in molecules. *J. Chem. Phys.*, **126**:144111, 2007.

[24] G. Bussi, D. Donadio, and M. Parrinello. Canonical sampling through velocity rescaling. *J. Chem. Phys.*, 126:014101, 2007.

[25] C. Campana, B. Mussard, and T. K. Woo. Electrostatic potential derived atomic charges for periodic systems using a modified error functional. *J. Chem. Theory Comput.*, 5:2866–2878, 2009.

[26] Mark E. Casida. Time-depenent density-functional response theory for molecules. *Theoretical and Computational Modeling of NLO and Electronic Materials*, 1996.

[27] D. M. Ceperley and B. J. Alder. Ground state of the electron gas by a stochastic method. *Phys. Rev. Lett.*, **45**:566–569, 1980.

[28] Michele Ceriotti, Giovanni Bussi, and Michele Parrinello. Langevin equation with colored noise for constant-temperature molecular dynamics simulations. *Phys. Rev. Lett.*, 102:020601, Jan 2009.

[29] Michele Ceriotti, Giovanni Bussi, and Michele Parrinello. Colored-noise thermostats à la carte. *Journal of Chemical Theory and Computation*, 6(4):1170–1180, 2010.

[30] Michele Ceriotti, Joshua More, Michele, and David Manolopoulos. i-pi: A python interface for ab initio path integral molecular dynamics simulations. *Comp. Phys. Comm.*, 185:1019–1026, 2014.

[31] Michele Ceriotti, Michele Parrinello, Thomas E. Markland, and David E. Manolopoulos. Efficient stochastic thermostatting of path integral molecular dynamics. *The Journal of Chemical Physics*, 133(12):124104, 2010.

[32] D.J. Chadi and M.L. Cohen. *Phys. Rev. B*, **8**:5747, 1973.

[33] D.-J. Chen, A. C. Stern, B. Space, and J. K. Johnson. Atomic charges derived from electrostatic potential for molecular and periodic systems. *J. Phys. Chem. A*, 114:10225–10233, 2010.

[34] L. E. Chirlian and M. M. Francl. Atomic charges derived from electrostatic potentials: A detailed study. *J. Comput. Chem.*, 8:894–905, 1987.

[35] L. A. Constantin, E. Fabiano, and F. Della Sala. *Phys. Rev. B*, 86:035130, 2012.

[36] S. R. Cox and D. E. Williams. Representation of the molecular electrostatic potential by a net atomic charge model. *J. Comput. Chem.*, 2:304–323, 1981.

[37] G. Crooks. Nonequilibrium measurements of free energy differences for microscopically reversible markovian systems. *J. Stat. Phys.*, 90:1481, 1998.

[38] B. Delley. *J. Chem. Phys.*, 92:508, 1990.

[39] B. Delley. *J. Comp Chem.*, 17:1152, 1995.

[40] M. Dion, H. Rydberg, E. Schröder, D. C. Langreth, , and B. I. Lundqvist. *Phys. Rev. Lett.*, 92:246401, 2004.

[41] R. M. Dreizler and E. K. U. Gross. *Density Functional Theory*. Springer, Berlin, 1990.

[42] Brett I. Dunlap, Notker Rösch, and S. B. Trickey. Variational fitting methods for electronic structure calculations. *Mol. Phys.*, 108(21):3167, 2010.

[43] C. Eckart. *Phys. Rev.*, **47**:552, 1935.

[44] K. Eichkorn, O. Treutler, H. Öhm, M. Häser, and R. Ahlrichs. *Chem. Phys. Lett.*, 240:283, 1995.

[45] F. Evers and A. Arnold. Molecular conductance from ab initio calculations: Self energies and absorbing boundary conditions. *arXiv:cond-mat/0611401v1*, 2006.

[46] E. Fabiano, L . A. Constantin, and F. Della Sala. Generalized gradient approximation bridging the rapidly and slowly varying density regimes: A pbe-like functional for hybrid interfaces. *Phys. Rev. B*, **82**:113104, 2010.

[47] D. Frenkel and B. Smit. *Understanding Molecular Simulation: From Algorithms to Applications*. Academic Press, second edition, 2002.

[48] Christoph Freysoldt, Jörg Neugebauer, and Chris G. Van de Walle. Fully ab initio Finite-Size corrections for Charged-Defect supercell calculations. *Phys. Rev. Lett.*, 102:016402, 2009.

[49] C.-L. Fu and K.-M. Ho. *Phys. Rev. B*, **28**:5480, 1983.

[50] Martin Fuchs and Matthias Scheffler. Ab initio pseudopotentials for electronic structure calculations of poly-atomic systems using density-functional theory. *Computer Physics Communications*, 119(1):67–98, 1999.

[51] M.J. Gillan. *J. Phys.:Condens. Matter*, 1:689, 1989.

[52] B Grabowski, L Ismer, T Hickel, and J Neugebauer. *Phys. Rev. B*, 79:134106, 2009.

[53] S. Grimme. *J. Chem. Phys.*, **20**:9095, 2003.

[54] Andreas Grüneis, Martijn Marsman, Judith Harl, Laurids Schimka, and Georg Kresse. Making the random phase approximation to electronic correlation accurate. *J. Chem. Phys.*, 131:154115, 2009.

[55] A. Gulans, M. Puska, and R. Nieminen. Linear-scaling self-consistent implementation of the van der waals density functional. *Phys. Rev. B*, 79:201105(R), 2009.

[56] G. Mills K. W. Jacobsen H. Jonsson. *Nudged Elastic Band Method for Finding Minimum Energy Paths of Transitions*. World Scientific, 1998.

[57] A. J. S. Hamilton. Uncorrelated modes of the non-linear power spectrum. *Mon. Not. R. Astron. Soc.*, 312(2):257–284, 2000.

[58] Andrew J. S. Hamilton. Fftlog, 2000.

[59] B. Hammer, L.B. Hansen, and J.K. Nørskov. *Phys. Rev. B*, 59:7413, 1999.

[60] Myung Joon Han, Taisuke Ozaki, and Jaejun Yu. O(N) LDA+U electronic structure calculation method based on the nonorthogonal pseudoatomic orbital basis. *Physical Review B*, 73(4):045110, January 2006.

[61] V. Havu, V. Blum, P. Havu, and M. Scheffler. *J. Comput. Phys.*, **228**:8367, 2009.

[62] L. Hedin. *Phys. Rev.*, 139:A796, 1965.

[63] G. Henkelman, B. P. Uberuagga, and H. Jonsson. A climbing image nudged elastic band method for finding saddle points and minimum energy paths. *J. Chem. Phys.*, **113**:9901, 2000.

[64] Graeme Henkelman and Hannes Jonsson. Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points. *Journal of Chemical Physics*, 113(22):9978–9985, 2000.

[65] John M. Herbert and Martin Head-Gordon. Accelerated, energy-conserving Born-Oppenheimer molecular dynamics via fock matrix extrapolation. *Phys. Chem. Chem. Phys.*, 7:3269–3275, 2005.

[66] Jochen Heyd, Gustavo E. Scuseria, and Matthias Ernzerhof. *J. Chem. Phys.*, 118:8207, 2003.

[67] Jochen Heyd, Gustavo E. Scuseria, and Matthias Ernzerhof. *J. Chem. Phys.*, 124:219906, 2006.

[68] F.L. Hirshfeld. *Theor. Chim. Acta (Berl.)*, 44:129, 1977.

[69] P. Hohenberg and W. Kohn. *Phys. Rev. B*, 136:864, 1964.

[70] H. Hu, Z. Lu, and W. yang yang yang yang. Fitting molecular electrostatic potentials from quantum mechanical calculations. *J. Chem. Theory Comput.*, 3:1004–1013, 2007.

[71] Arvid Conrad Ihrig, Jürgen Wieferink, Igor Ying Zhang, Matti Ropo, Xinguo Ren, Patrick Rinke, Matthias Scheffler, and Volker Blum. Accurate localized resolution of identity approach for linear-scaling hybrid density functionals and for many-body perturbation theory. *New Journal of Physics*, 17(9):093020, 2015.

[72] H. Ishida, Y. Nagai, and A. Kidera. *Chem. Phys. Lett.*, 282(2):115, 1998.

[73] A. Itoh and H. Matsunami. Single crystal growth of sic and electronic devices. *Critical Reviews in Solid State and Materials Sciences*, 22(2):111–197, 1997.

[74] C. Jarzynski. Nonequilibrium equality for free energy differences. *Phys. Rev. Lett*, 78:2690, 1997.

[75] Jmol. An open-source java viewer for chemical structures in 3D. http://www.jmol.org/.

[76] J. Junquera, O. Paz, D. Sanchez-Portal, and E. Artacho. *Phys. Rev. B*, 64:235111, 2001.

[77] G.P. Kerker. *Phys. Rev. B*, **23**:3082, 1981.

[78] A. Klamt and G. Schüürmann. COSMO: a new approach to dielectric screening in solvents with explicit expressions for the screening energy and its gradient. *J. Chem. Soc., Perkin Trans. 2*, (5):799–805, January 1993.

[79] L. Kleinman and D. M. Bylander. Efficacious form for model pseudopotentials. *Phys. Rev. Lett.*, 48:1425–1428, May 1982.

[80] Franz Knuth, Christian Carbogno, Viktor Atalla, Volker Blum, and Matthias Scheffler. All-electron formalism for total energy strain derivatives and stress tensor components for numeric atom-centered orbitals. *Computer Physics Communications*, 190:33–50, 2015.

[81] W. Kohn and L.J. Sham. *Phys. Rev.*, 140:A1133, 1965.

[82] Jiří Kolafa. Numerical integration of equations of motion with a Self-Consistent field given by an implicit equation. *Mol. Simul.*, 18:193, 1996.

[83] G. Kresse and J. Furthmüller. Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set. *Comp. Mat. Sci.*, **6**:15–50, 1996.

[84] Aliaksandr V. Krukau, Oleg A. Vydrov, Artur F. Izmaylov, and Gustavo E. Scuseria. *J. Chem. Phys.*, 125:224106, 2006.

[85] Thomas D. Kühne, Matthias Krack, Fawzi R. Mohamed, and Michele Parrinello. Efficient and accurate Car-Parrinello-like approach to Born-Oppenheimer molecular dynamics. *Phys. Rev. Lett.*, 98:066401, 2007.

[86] S. Kumar, J. M. Rosenberg, D. Bouzida, R. H. Swendsen, and P. A. Kollman. Multidimensional free-energy calculations using the weighted histogram analysis method. *J. Comput. Chem.*, 16:1339, 1995.

[87] A. Laio and M. Parrinello. Escaping free energy minima. *Proc. Natl. Acad. Sci.*, 20:12562, 2002.

[88] V.I. Lebedev. *Zh. Vychisl. Mat. mat. Fiz.*, 15:48, 1975.

[89] V.I. Lebedev. *Zh. Vychisl. Mat. mat. Fiz.*, 16:293, 1976.

[90] V.I. Lebedev and D.N. Laikov. *Doklady Mathematics*, 59:477, 1999.

[91] C.L. Lee, W. Yang, and R.G. Parr. *Phys. Rev. B*, 37:785, 1988.

[92] C. Lessig, T. de Witt, and E. Fiume. Efficient and accurate rotation of finite spherical harmonics expansions. *J. Comput. Phys.*, 231(2):243–250, January 2012.

[93] Sergey V. Levchenko, Xinguo Ren, Jürgen Wieferink, Rainer Johanni, Patrick Rinke, Volker Blum, and Matthias Scheffler. Hybrid functionals for large periodic systems in an all-electron, numeric atom-centered basis framework. *Computer Physics Communications*, 192:60 – 69, 2015.

[94] R. Lindh, A. Bernhardsson, G. Karlström, and P.-Å. Malmqvist. *Chem. Phys. Lett.*, 241:423, 1995.

[95] P.O. Loewdin. *J. Chem. Phys.*, 19:1936–1401, 1951.

[96] Steven G. Louie, Sverre Froyen, and Marvin L. Cohen. Nonlinear ionic pseudopotentials in spin-density-functional calculations. *Phys. Rev. B*, 26:1738–1742, Aug 1982.

[97] G. Makov and M. C. Payne. *Phys. Rev. B*, 51:4014, 1995.

[98] M. Manninen, R. Nieminen, and P. Hautojärvi. *Phys. Rev. B*, 12:4012, 1975.

[99] M. Mantina, A. C. Chamberlin, R. Valero, C. J. Cramer, and D. G. Truhlar. Consistent van der waals radii for the whole main group. *J. Phys. Chem. A*, 113:5806–5812, 2009.

[100] Miguel A. L. Marques, Micael J. T. Oliveira, and Tobias Burnus. Libxc: a library of exchange and correlation functionals for density functional theory. *Computer Physics Communications*, 183(10):2272–2281, 2012.

[101] B. Mennucci, J. Tomasi, R. Cammi, J. R. Cheeseman, M. J. Frisch, F. J. Devlin, S. Gabriel, and P. J. Stephens. Polarizable continuum model (pcm) calculations of solvent effects on optical rotations of chiral molecules. *J. Phys. Chem. A*, 106(25):6102–6113, 2002.

[102] N.D. Mermin. *Phys. Rev.*, 137:A1441, 1965.

[103] M. Methfessel and A. T. Paxton. *Phys. Rev. B*, **40**:3616, 1989.

[104] F. A. Momany. Determination of partial atomic charges from ab initio molecular electrostatic potentials. application to formamide, methanol, and formic acid. *J. Chem. Phys.*, 82:592–601, 1978.

[105] H.J. Monkhorst and J.D. Pack. *Phys. Rev. B*, **13**:5188, 1976.

[106] R.S. Mulliken. *J. Chem. Phys.*, 23:1833, 1955.

[107] D. Nabok, P. Puschnig, and C. Ambrosch-Draxl. *Phys. Rev. B*, 77:245316, 2008.

[108] R. Nieminen. *J. Phys. F*, 7:375, 1977.

[109] Anders M. N. Niklasson, C. J. Tymczak, and Matt Challacombe. Time-Reversible Born-Oppenheimer molecular dynamics. *Phys. Rev. Lett.*, 97:123001, 2006.

[110] Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. Springer, 2. edition, 2006.

[111] H. Oberhofer and J. Blumberger. Revisiting electronic couplings and incoherent hopping models for electron transport in crystalline c60 at ambient temperatures. *Phys. Chem. Chem. Phys*, 14:13846–13852, 2012.

[112] Jochen Heyd Aliaksandr V. Krukau Oleg A. Vydrov and Gustavo E. Scuseria. *J. Chem. Phys.*, 125:074106, 2006.

[113] K. Parlinski, Z. Q. Li, and Y. Kawazoe. First-principles determination of the soft mode in cubic zro_2. *Phys. Rev. Lett.*, **78**:4063–4066, 1997.

[114] J. P. Perdew, K. Burke, and M. Ernzerhof. Generalized gradient approximation made simple. *Phys. Rev. Lett.*, **77**:3865–3868, 1997.

[115] J. P. Perdew, J. A. Chevary, S. H. Vosko, K. A. Jackson, M. R. Pederson, D. J. Singh, and C. Fiolhais. Atoms, molecules, solids, and surfaces: Applications of the generalized gradient approximation for exchange and correlation. *Phys. Rev. B*, **46**:6671–6687, 1992.

[116] J. P. Perdew and Y. Wang. Accurate and simple analytic representation of the electron-gas correlation energy. *Phys. Rev. B*, **45**:13244–13249, 1992.

[117] J. P. Perdew and A. Zunger. Self-interaction correction to density-functional approximations for many-electron systems. *Phys. Rev. B*, **23**:5048–5079, 1981.

[118] J.P. Perdew, A. Ruzsinszky, G.I. Csonka, L.A. Constantin, and J. Sun. Workhorse semilocal density functional for condensed matter physics and quantum chemistry. *Phys. Rev. Lett.*, 103:026403, 2009.

[119] J.P. Perdew, A. Ruzsinszky, G.I. Csonka, L.A. Constantin, and J. Sun. Erratum: Workhorse semilocal density functional for condensed matter physics and quantum chemistry. *Phys. Rev. Lett.*, 106:179902(E), 2011.

[120] J.P. Perdew, A. Ruzsinszky, G.I. Csonka, O.A. Vydrov, G.E. Scuseria, L.A. Constantin, X. Zhou, and K. Burke. *Phys. Rev. Lett.*, 100:136406, 2008.

[121] Baron Peters, Andreas Heyden, Alexis T. Bell, and Arup Chakraborty. A growing string method for determining transition states: Comparison to the nudged elastic band and string methods. *The Journal of Chemical Physics*, 120(17):7877–7886, 2004.

[122] A. G. Petukhov, I. I. Mazin, L. Chioncel, and A. I. Lichtenstein. Correlated metals and the LDA+U method. *Physical Review B*, 67(15):153106, April 2003.

[123] R. Peverati and D.G. Truhlar. Improving the accuracy of hybrid meta-gga density functionals by range separation. *J. Phys. Chem. Lett.*, 2(21):2810, 2011.

[124] R. Peverati and D.G. Truhlar. M11-l: A local density functional that provides improved accuracy for electronic structure calculations in chemistry and physics. *J. Phys. Chem. Lett.*, 3:117, 2011.

[125] William K. Press, Saul A. Teukolsky, William T. Vetterlin, and Brian T. Flannery. *Numerical Recipes*. Cambridge University Press, 3 edition, 2007.

[126] P. Pulay. Convergence acceleration of iterative sequences. the case of scf iteration. *Chem. Phys. Lett.*, **73**:393–398, 1980.

[127] Peter Pulay and Géza Fogarasi. Fock matrix dynamics. *Chem. Phys. Lett.*, 386:272–278, 2004.

[128] P. Pyykkö. *Chem. Rev.*, 88:563, 1988.

[129] X. Ren, P. Rinke, V. Blum, J. Wieferink, A. Tkatchenko, A. Sanfilippo, K. Reuter, and M. Scheffler. Resolution-of-identity approach to Hartree-Fock, hybrid density functionals, RPA, MP2 and GW with numeric atom-centered orbital basis functions. *New J. Phys.*, **14**:053020, 2012.

[130] X. Ren, P. Rinke, G. E. Scuseria, and M. Scheffler. in preparation.

[131] X. Ren, A. Tkatchenko, P. Rinke, and M. Scheffler. Beyond the random-phase approximation for the electron correlation energy: The importance of single excitations. *Phys. Rev. Lett.*, **106**:153003, 2011.

[132] Norina A. Richter, Sabrina Sicolo, Sergey V. Levchenko, Joachim Sauer, and Matthias Scheffler. Concentration of vacancies at metal-oxide surfaces: Case study of mgo(100). *Physical Review Letters*, 111:045502, 2013.

[133] H. N. Rojas, R. W. Godby, and R. J. Needs. *Phys. Rev. Lett.*, **74**:1827, 1995.

[134] B. Roux. The calculation of the potential of mean force using computer-simulations. *Comput. Phys. Comm.*, 91:275, 1995.

[135] R. S. Rowland and R. Taylor. Intermolecular nonbonded contact distances in organic crystal structures: Comparison with distances expected from van der waals radii. *J. Phys. Chem.*, 100:7384–7391, 1996.

[136] A. Sayvetz. *J. Phys. Chem.*, **7**:383, 1939.

[137] C. Schober, K. Reuter, and H. Oberhofer. A critical analysis of fragment orbital dft schemes for the calculation of electronic coupling values. *J. Chem. Phys.*, 144:054103, 2016.

[138] G.E. Scuseria and V.N. Staroverov. Progress in the development of exchange-correlation functionals. In C.E. Dykstra, G. Frenking, K.S. Kim, and G.E. Scuseria, editors, *Theory and Applications of Computational Chemistry: The First 40 Years*, chapter 24. Elsevier, Amsterdam, 2005. The parameters for "VWN"-LDA as implemented in the Gaussian code are given in Table 1.

[139] K. Senthilkumar, F. C. Grozema, F. M. Bickelhaupt, and L. D. A. Siebbeles. Charge transport in columnar stacked triphenylenes: Effects of conformational fluctuations on charge transfer integrals and site energies. *J. Chem. Phys.*, 119(18):9809–9817, 2003.

[140] P. Sherwood, A. H. de Vries de Vries de Vries de Vries, M. F. Guest, G. Schreckenbach, R. A. Catlow, S. A. French, A. A. Sokol, S. T. Bromley, W. Thiel, A. J. Turner, S. Billeter, F. Terstegen, S. Thiel, J. Kendrick, S. C. Rogers, J. Casci, M. Watson, F. King, E. Karlsen, M. Sjøvoll, A. Fahmi, A. Schäfer, and C. Lennartz. *J. Mol. Struct (Theochem)*, **632**:1, 2003.

[141] E. Sigfridsson and U. Ryde. Atomic charges from the electrostatic potential and moments. *J. Comput. Chem.*, 19(4), 1998.

[142] U. C. Singh and P. A. Kollman. An approach to computing electrostatic charges for molecules. *J. Comput. Chem.*, 5:129–145, 1984.

[143] R.E. Stratmann, G.E. Scuseria, and M.J. Frisch. *Chem. Phys. Lett.*, 257:213, 1996.

[144] Jianwei Sun, Adrienn Ruzsinszky, and JohnâĂŁP. Perdew. Strongly constrained and appropriately normed semilocal density functional. *Phys. Rev. Lett.*, 115:036402, 2015.

[145] James D. Talman. Numerical methods for multicenter integrals for numerically defined basis functions applied in molecular calculations. *Internat. J. Quant. Chem.*, 93(2):72–90, 2003.

[146] J.D. Talman. NumSBT: a subroutine for calculating spherical bessel transforms numerically. *Comput. Phys. Comm.*, 180(2):332–338, February 2009.

[147] J.M. Tao, J.P. Perdew, V.N. Staroverov, and G.E. Scuseria. Climbing the density functional ladder: Nonempirical meta-generalized gradient approximation designed for molecules and solids. *Phys. Rev. Lett.*, 91:146401, 2003.

[148] A. Tkatchenko and M. Scheffler. *Phys. Rev. Lett.*, 102:073005, 2009.

[149] Alexandre Tkatchenko, Alberto Ambrosetti, and Robert A DiStasio Jr. Interatomic methods for the dispersion energy derived from the adiabatic connection fluctuation-dissipation theorem. *J. Chem. Phys.*, 138:074106, 2013.

[150] Alexandre Tkatchenko, Robert A DiStasio Jr, Roberto Car, and Matthias Scheffler. Accurate and efficient method for many-body van der waals interactions. *Phys. Rev. Lett.*, 108(23):236402, 2012.

[151] G. Torrie and J. Valleau. Nonphysical sampling distributions in monte carlo free energy estimation: Umbrella sampling. *J. Comput. Phys.*, 23:187, 1977.

[152] O. Vahtras, J. Almlöf, and M.W. Feyereisen. *Chem. Phys. Lett.*, 213:514, 1993.

[153] E. van Lenthe van Lenthe van Lenthe van Lenthe, E.J. Baerends, and J.G. Snijders. *J. Chem. Phys.*, 101:9783, 1994.

[154] S.H. Vosko, L. Wilk, and M. Nusair. *Can. J. Phys.*, 58:1200, 1980.

[155] L. Vočadlo and D. Alfè. *Phys. Rev. B*, 65:214105, 2002.

[156] F. Wagner, Th. Laloyaux, and M. Scheffler. *Phys. Rev. B*, 57:2102, 1998.

[157] E Weinan, Weiqing Ren, and Eric Vanden-Eijnden. Simplified and improved string method for computing the minimum energy paths in barrier-crossing events. *J. Chem. Phys.*, 126:164103, 2007.

[158] J. Wilhelm, M. Walz, M. Stendel, A. Bagrets, and F. Evers. Ab initio simulations of scanning-tunneling-microscope images with embedding techniques and application to c58-dimers on au(111). *Phys. Chem. Chem. Phys.*, 15:6684, 2013.

[159] R. Winkler. *Spin-Orbit Coupling Effects in Two-Dimensional Electron and Hole Systems*. Springer, 2003.

[160] Igor Ying Zhang, Xinguo Ren, Patrick Rinke, Volker Blum, and Matthias Scheffler. Numeric atom-centered-orbital basis sets with valence-correlation consistency from h to ar. *New Journal of Physics*, 15:123033, 2013.

[161] I.Y. Zhang, N. Su, A.G. Bremond, C. Adamo, and X. Xu. Doubly hybrid density functional xdh-pbe0 from a parameter-free global hybrid model pbe0. *J. Chem. Phys.*, 136:174103, 2012.

[162] Y. Zhang and W. Yang. *Phys. Rev. Lett.*, 80:890, 1998.

[163] Y. Zhao and D.G. Truhlar. Density functional for spectroscopy: No long-range self-interaction error, good performance for rydberg and charge-transfer states, and better performance on average than b3lyp for ground states. *J. Phys. Chem. A*, 110:13126, 2006.

[164] Y. Zhao and D.G. Truhlar. The m06 suite of density functionals for main group thermochemistry, thermochemical kinetics, noncovalent interactions, excited states, and transition elements: Two new functionals and systematic testing of four m06-class functionals and 12 other functionals. *Theor. Chem. Acc.*, 120:215, 2006.

[165] Y. Zhao and D.G. Truhlar. A new local density functional for main-group thermochemistry, transition metal bonding, thermochemical kinetics, and noncovalent interactions. *J. Chem. Phys.*, 125(19):194101, 2006.

[166] Y. Zhao and D.G. Truhlar. Exploring the limit of accuracy of the global hybrid meta density functional for the main-group thermochemistry, kinetics, and noncovalent interactions. *J. Chem. Theor. Comp.*, 4(11):1849, 2008.

Following pages: Index

(this page inserted to enforce proper hyperlink to index)